

AIとつくり、仕組みを理解し、人に届ける。

Vibe Coding

からはじめる

アプリ開発の 教科書

デモで終わらせないための

企画・設計・API・データベース・テスト・公開

5枚の地図で、開発の全体を見渡す。

01

価値

02

構造

03

変更

04

品質

05

運用

「動いた」を、「使える」に育てる。

目次

はじめに	3
序章 まずは20分でアプリを作ってみよう	6
第1章 Vibe Codingとアプリ開発は何が違うのか	15
第2章 機能から考えず、困りごとから考える	27
第3章 「こんな感じ」をアプリの仕様に变える	40
第4章 AIへの指示を「プロンプト」から「仕事の依頼」に変える	52
第5章 アプリはどこで動いているのか	64
第6章 コードを全部書かなくても、読めるようになる	78
第7章 API——アプリ同士がお願いし合う仕組み	93
第8章 データはどこに保存されるのか	103
第9章 利用者、ログイン、権限を理解する	118
第10章 プロジェクトの中身と変更履歴を管理する	130
第11章 エラーをAIに丸投げせず、原因を切り分ける	143
第12章 「動いた」から「確かめた」へ——テストで根拠を残す	157
第13章 セキュリティ、プライバシー、権利を後回しにしない	170
第14章 アプリを公開し、使い続けられる状態にする	186
第15章 卒業制作——自分の課題を解決するアプリを作る	201
おわりに——理解は外注できない	215
付録A ツール別スタートガイド	217
付録B つまづき別チートシート	222
付録C 用語ミニ辞典	227
付録D 授業・研修シラバス	232
付録E 卒業制作テーマカタログ	238

はじめに

「作れた」の、その先へ

文化祭の模擬店。注文を聞きながら、頭の中で合計とお釣りを計算する。後ろには、次のお客さんが並んでいます。「商品を押すだけで計算できたら」。その小さな願いから、この本のアプリ開発は始まります。

AIに頼めば、ボタンが並び、合計が表示される画面を作れます。自分の考えが、触れるものになる。その喜びを、まず序章で味わってください。最初の題材は3つのボタンだけの気分記録アプリ。環境の準備ができていれば、20分を目安に試せます。

ところが、使ってみると新しい問いが生まれます。再読み込みしても記録は残るか。割引を重ねたとき、金額は合っているか。自分が席を外しても、次の人が使えるか。画面ができたからこそ、確かめたいことが見えてきます。

本書は、Vibe Coding(バイブコーディング)を入口に、その問いへ答えられるようになる本です。AIと対話して作る楽しさを出発点に、企画し、仕組みを読み、検証し、人に渡すところまで進みます。「API」「データベース」という言葉も、その場面で何の役に立つから説明します。

目指すのは、「動いた」と喜んだあとに、「ここを確かめよう」と一歩進めること。その一歩を、章ごとに積み上げていきましょう。

コードはAIに。判断はあなたに。

本書を貫く考え方は、一文で書けます。

コード生成はAIに任せられる。だが、何を作るか、正しいか、公開してよいか、誰が守るかは任せきれない。

コードをすべて手で書けるようになる前に、試作品を動かして学べます。ただし、文法や仕組みを学ぶ意味がなくなったわけではありません。本書では、目の前のアプリを理解するために必要なところから学びます。

誰の困りごとを解決するのか。何を確認できたら人に渡してよいのか。壊れたとき、どこへ戻るのか。AIから提案をもらいながら、現場の事情と実際の結果をもとに決めていく。これが、本書で練習する仕事です。

そのために、2つの道具を手に入れます。何がどこで動いているかを示す地図と、AIや専門家と話すための語彙です。プログラミング経験は前提にしません。第6章から短いコードも読みますが、暗記より「この行は何の仕事か」を確かめることに重きを置きます。

この本の進み方

本書では、文化祭の模擬店で使う会計アプリを、AIと一緒にゼロから作り、公開するところまで進みます。金額計算と現金でのお釣りを扱う教材で、オンライン決済は含みません。社会人の方は、小さな店舗や部署の道具に置き換えて読んでください。実務で使う際は、その現場の会計・権限・運用の条件を別途確かめます。

進み方の型は、作る → 壊す・困る → 直す → 説明する。記録が消える、計算が合わない、見えてはいけないものが見える。そんな場面を練習用のアプリで確かめながら、仕組みと専門用語を学びます。大切なデータが入った本番アプリを、演習のために壊す必要はありません。

もうひとつ、大事な約束をしておきます。あなたの画面は、この本と同じにはなりません。AIの返事は毎回すこしずつ違うからです。本書はそれを前提に書いてあります。各演習には「AIの出力がどうであれ、これが確認できれば先に進んでよい」という条件が付いています。本と違うことは、失敗ではありません。

章はクエスト、学びの記録は「冒険の書」。ゲームのような言葉は、進んだ距離を見渡すための目印です。変更前にセーブする、失敗したら記録を読む。そうした習慣が、開発でも役に立ちます。

各章の「文化祭まで、あと〇日」は、物語上の時間です。学習の期限ではないので、ご自身のペースで進めてください。会社勤めの方なら「リリースまで」と読み替えてもよいでしょう。次の担当者へアプリを渡す日を、少しずつ具体的にしていきます。

本書の文化祭の出来事とAIとの会話は、学習のための作例です。AIの実際の返事、生成時間、画面は環境によって変わります。会話の成功宣言を結果の証拠にせず、演習のクリア条件を自分の画面で確かめてください。

各章の歩き方

序章と第1～15章は、同じ順番で進みます。順番を知っておくと、いま自分がどこを読んでいるかを見失いません。

1. この章のねらい——章を終えたときにできるようになることを、3つ示します
2. 今回のクエスト・ボス出現——会計アプリで、実際に困る場面から始めます
3. 本文(n.1、n.2…)——困りごとを解くための仕組みと用語を、1つずつ手に入れます
4. 演習——自分の画面で試します。クリア条件が確かめられれば、本と画面が違って先へ進めます
5. この章のまとめ——持ち運べる原則だけを、短く言い切ります
6. 理解度チェック——3問に答えて、理解の抜けを確かめます。解答のめやすには、読み直す節を示しました
7. 60秒でクエスト報告・レベルアップ——人に説明できる形にして、冒険の記録に日付を書きます

理解度チェックで迷った問いは、失敗ではなく「戻る場所の目印」です。示された節を読み直してから、次の章へ進んでください。本文中の「※厳密に言うと」は、初めて読むときは飛ばしてもかまいません。2回目や実務で使うときに役立つ補足です。

この本の使い方

ひとりで学ぶ方へ。序章から順に進めると、同じアプリに必要な仕組みが少しずつ加わります。1章60～90分は目安です。時間がない日は、本文を読み、図を1枚説明して、翌日に演習へ戻ってもかまいません。わからなかった点を残しておくと、再開しやすくなります。

大学の授業で使う方へ。序章+全15章は、90分×15回の半期授業にそのまま対応しています。各章の理解度チェックは、授業冒頭の振り返りや小テストにも使えます。文系向けの8回版、評価基準(ルーブリック)、最終口頭試問の5問は付録Dにまとめました。AIがコードを書く時代の成績評価——完成画面ではなく、説明と検証の記録を評価する方法——も、そこで提案しています。

企業研修で使う方へ。2日間の研修カリキュラムを付録Dに用意しました。ゴールは「本番システムを2日で作る」ことではなく、「業務課題を検証できる安全な試作品と、エンジニアに渡せる仕様書を作る」ことです。

使う環境がまだ決まっていなければ、付録Aから始めてください。必要な機能の探し方、スマートフォンで開けない場合の確認先、練習用データの用意のしかたをまとめています。操作名が変わっても、「何を確かめるための操作か」がわかれば、公式ヘルプで探し直せます。

卒業の条件

卒業制作では、次の5つを自分のアプリで示します。

説明できる。変更できる。壊れても戻せる。正しさを確かめられる。人に渡せる。

たとえば「戻せる」なら、戻し方の説明だけでなく、練習用コピーで実際に戻した記録を残します。5つすべてが最初からできる必要はありません。できることが1つ増えるたび、次の変更を少し安心して試せるようになります。

それでは、はじめましょう。まずは20分で、最初のアプリを作ります。

序章 まずは20分でアプリを作ってみよう

クエスト0: 旅立ちの村を出る

文化祭まで、あと30日。

この章のねらい

- 日本語で頼んで小さなアプリを作り、ボタンで記録が増えることを確かめられる
- 具体的な頼み方と曖昧な頼み方を試し、結果の違いを説明できる
- 再読み込みで記録が消える理由を、表示と保存の違いで説明できる

今回のクエスト

この章のミッションは、ひとつだけです。

20分を目安に、小さなアプリを作り、動くところを確かめること。

プログラミング経験は要りません。使う環境へのログインとプレビューの準備は、付録Aで先に済ませます。準備に時間がかかった場合は、その分を20分に含めなくてかまいません。

そして章の終わりに、小さな事件が起きます。その事件が、この本全体の出発点になります。

0.1 AI開発環境を開く

最初に、作業場を用意します。

AI開発環境とは、AIとチャットしながらアプリを作れるサービスのことです。画面の片側にチャット欄があり、あなたが日本語で頼むと、AIがアプリを作って、もう片側にその場で表示してくれます。

環境選びと始め方は付録Aへ。まず、日本語で依頼できること、生成したアプリを実行できることを確認します。一般的なチャット画面にコードが表示されただけなら、まだ実行環境の準備が必要です。

本書で順に使う道具は、次の6つです。1つの製品に全部そろってなくても、必要な章で別の道具を組み合わせられます。

図 0-1



図0-1 AI開発環境の6つの道具

6つ全部は、まだ覚えなくて大丈夫です。今日使うのはチャットとプレビュー(作ったものがその場で映る画面)の2つだけ。残りの4つは、この本を進むうちに、ひとつずつ「使える道具」になっていきます。

0.2 「気分を記録する3つのボタン」を作る

最初に作るのは、世界最小の日記です。ボタンを押すだけで、今日の気分が記録されていくアプリ。それでは、チャット欄にこう打ち込んでみてください。

あなた → AI

気分を記録する練習用アプリを作ってください。画面に「よい」「ふつう」「つかれた」の3つのボタンを置き、押すと時刻付きの記録が1行ずつ増えるようにします。この演習では記録を画面のメモリだけに置き、再読み込みすると消える仕様にしてください。ログインや外部送信は不要です。

AIから、たとえば次のような返事が返ってきます。生成にかかる時間は環境によって異なります。

AI → あなた

気分記録アプリを作りました。3つのボタンを押すと、下の一覧に時刻つきで記録が追加されます。プレビューで確認してみてください。

プレビューを見てください。3つのボタンが並んでいます。「よい」を押すと、下に「12:34 よい」のような記録が増えます。何回か押して、記録が積み上がっていくのを見てみましょう。

動きましたか。

動かなければ、まず「何を押したら、何が起きたか」を記録します。エラー文があれば、自分で要約せず、そのまま添えてAIへ相談してください。貼る前に、APIキー(第7章で扱う秘密の鍵)、パスワード

ド、個人情報が混ざっていないかを確認、該当部分だけ伏せます。詳しい切り分け方は第11章で練習します。

ここで、立ち止まって確認したいことがあります。

あなたはまだ、コードを一行も書いていません。

それでも、アプリは動いています。これが、AIに日本語で頼んで作る開発——のちほど第1章で正式に紹介する「Vibe Coding」と呼ばれるスタイルの入口です。

ひとつ、この本全体の約束をしておきます。あなたの画面は、この本と同じにはなりません。AIの返事は毎回すこしずつ違います。ボタンの色も、文章も、この本の例と違っていいのです。「3つのボタンが押せて、記録が増える」が確認できていれば、先に進んでください。

0.3 色や文章を自然言語で変更する

次は、できあがったものに注文をつけてみます。

あなた → AI

「つかれた」のボタンだけ紫色にしてください。あと、記録の文章を「17:03 つかれた」のように短くしてください。

変わったら、色だけでなく、ボタンを押して記録が増えることも確認します。会話を重ねながら、自分の好みに近づけていきましょう。

もうひとつ、今度はわざと曖昧に頼んでみましょう。

あなた → AI

全体を、もっとやさしい雰囲気にしてください。

AIは「やさしい雰囲気」を自分なりに解釈して、色を淡くしたり、角を丸くしたりします。あなたのイメージと合っていれば当たり。違っていたら、もう一度言い直せばいいだけです。

2つの頼み方を比べてみましょう。「紫色」は結果を見比べやすく、「やさしい雰囲気」は人によって解釈が分かれます。曖昧な部分は、AIが仮定で埋めます。その仮定を言葉にして確かめる方法を、第2～4章で学びます。

0.4 スマートフォンで表示してみる

共有できるプレビューURLがあれば、スマートフォンのブラウザで開いてみます。開けないときは、PCでブラウザの幅を狭くして表示を確認、実機での確認は後に回してもかまいません。環境によっては、サービスへのログインや同じネットワークへの接続が必要です。確かめ方は付録A.5にまとめました。

自分のスマートフォンに、自分の作ったアプリが映る。ボタンを押すと、ちゃんと記録が増える。

すこし、誰かに見せたくありませんか。隣の人に渡して押してもらってください。その「見せたい」感じは、ものを作った人だけが知っている感覚です。

ただし、ひとつだけ。いま見ているURLは仮のもので、環境を閉じると消えることがあります。世界に向けてちゃんと公開する方法は、第14章でやります。

0.5 ボス出現——再読み込みしたら、記録が消えた

さて、事件です。次のことを試してください。

1. ボタンを何回か押して、記録を3行以上ためる
2. ページを再読み込みする(スマートフォンなら画面を下に引っばる)

記録は消えましたか。0.2で頼んだとおりに作られていれば、一覧は空になります。

壊れたのではありません。もう一度押せば、また記録は増えます。でも、再読み込みするたびに、ゼロに戻ります。日記としては、致命的です。

この演習では、記録をメモリにだけ置きました。メモリは、画面が開いているあいだけ使える一時的な置き場所です。再読み込みで画面が作り直されると、その記録も失われます。黒板のメモを消すようなものです。

残したければ、ノートのように、あとで読み戻せる場所へ保存する必要があります。その仕組みを第8章で加えます。

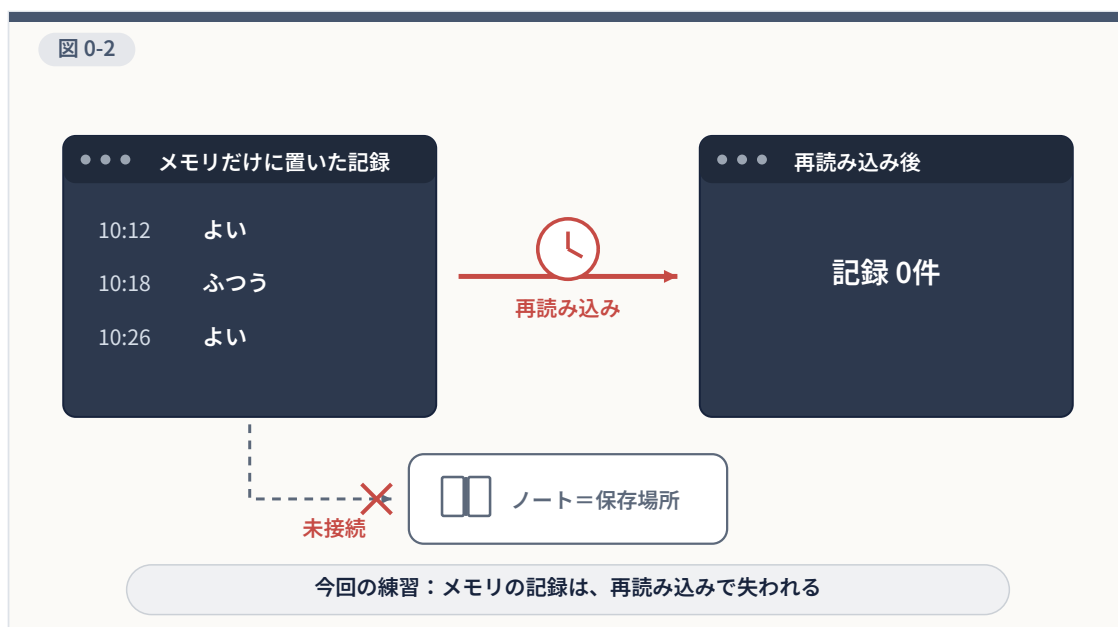


図0-2 表示しただけでは保存されない

図0-2では、黒板にあたる画面の記録が再読み込みで0件に戻ることと、ノートにあたる保存場所がまだつながっていないことを見比べてください。

※厳密に言うと: 再読み込みしても記録が残った場合は、AIが保存の仕組みを付け足した可能性があります。「どこに記録していますか」と聞き、この演習用には保存なしの版を作ってもらって比べてください。

実は、これと同じ種類の「足りないもの」が、ほかにもあります。いまのアプリは——

- 記録を残す仕組みがない(今回は練習のため、あえてそうした)
- URLがいつまで、誰に使えるかをまだ確認していない
- ほかの人と使うなら、誰の記録を誰が見られるかを決める必要がある
- 壊れたとき、どこを見ればいいのかわからない
- 安全かどうか、確かめる方法を知らない

いま作ったものも、小さなアプリです。ただ、日記として使い続けるには、保存などの条件が足りません。作りたいものの目的によって、必要な仕組みが変わります。

「画面が動く」と「アプリとして使える」のあいだには、距離があります。

AIは保存や公開のコードも作れます。そこで必要になるのは、何を残し、誰に渡し、どう確かめるかという判断です。その判断の根拠を持てるようになることが、この本の目的です。

0.6 デモからアプリへの「5つの壁」と、本書の「5枚の地図」

試作品を使い続けられるものへ育てるときの迷いを、本書では5つの壁に整理します。いま感じている疑問に近いものは、どれでしょうか。

壁	そのときの実感
1. 伝わらない	言った通りに作ってくれない
2. 中が見えない	どこで何が動いているのか、見当もつかない
3. 変更が怖い	昨日は動いていたのに。もう戻れない
4. 確かめられない	動いてはいる。でも「正しい」のかわからない
5. 渡せない	自分の環境でしか動かない。渡した後も不安

壁は、才能で越えるものではありません。地図があれば越えられます。この本は、5つの壁に対応する5枚の地図を、1枚ずつあなたに渡していきます。

地図	答えられるようになる問い	対応する部
①価値の地図	誰の、どんな困りごとを、どう解決するのか	第Ⅰ部(1～4章)
②構造の地図	画面、処理、データは、どこにあるのか	第Ⅱ部(5～9章)
③変更の地図	AIは、どのファイルの何を变えたのか	第Ⅲ部(10章)
④品質の地図	正しく安全に動くと、どう証明するのか	第Ⅲ部(11～13章)
⑤運用の地図	公開した後、誰がどのように守るのか	第Ⅳ部(14～15章)

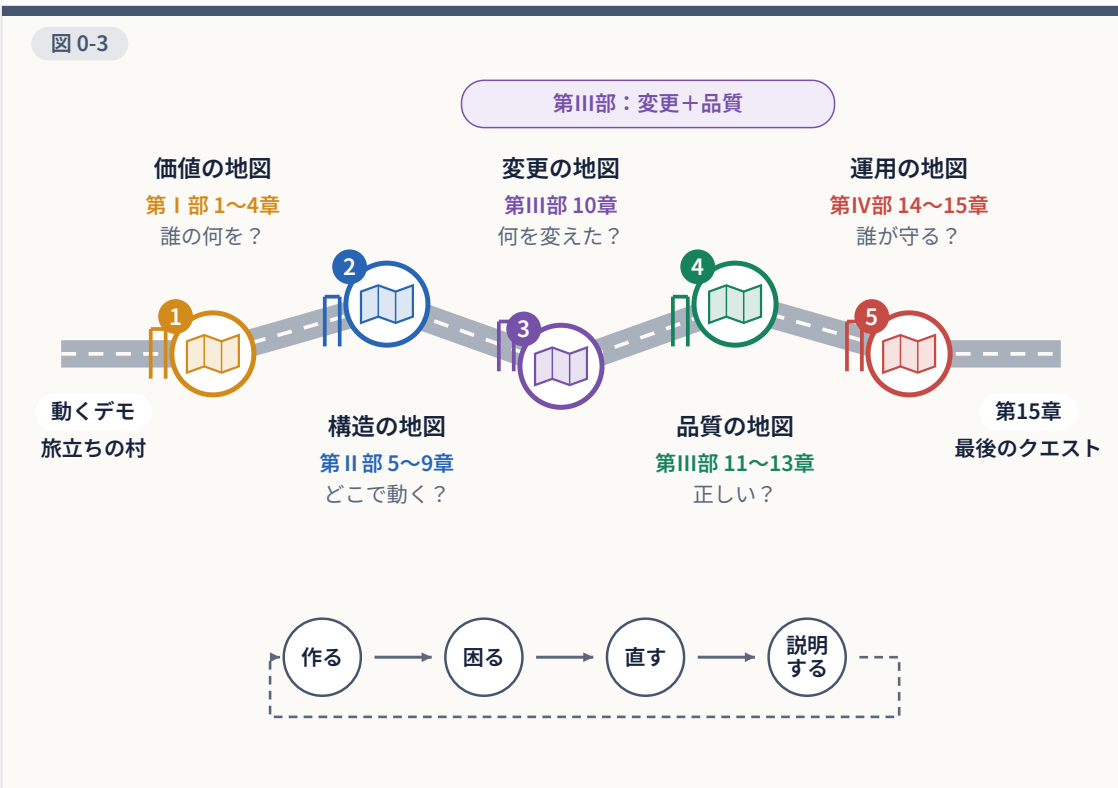


図0-3 アプリ開発の冒険地図

図0-3の上側は本全体の道順、下側は各章で繰り返す学び方です。困ったときにどの地図へ戻ればよい、章が進むたびに見直してみてください。

そして章の終わりに、その章の成果物を冒険の書に記録します。冒険の書とは、この本でいう開発ポートフォリオのことです。仕様書、構成図、テスト記録——第15章が終わるころには、「なぜ作ったか、どう作られているか、どう確かめたか」を丸ごと語れる一冊になっています。記録しなければ、冒険も学びも消えます。それは、さっき再読み込みで消えた記録が教えてくれた通りです。

最後に、旅の道具をもうひとつ渡します。冒険の記録です。章をひとつクリアするたび、この表に日付を書き込んでください。

Lv.	章	称号	クリアした日
0	序章	旅立った人	月 日
1	第1章	開発責任者(見習い)	月 日
2	第2章	困りごとハンター	月 日
3	第3章	仕様を書ける人	月 日
4	第4章	パーティリーダー	月 日
5	第5章	構成図の描き手	月 日
6	第6章	コードの読み手	月 日

Lv.	章	称号	クリアした日
7	第7章	窓口と話せる人	月 日
8	第8章	記録者	月 日
9	第9章	門番	月 日
10	第10章	セーブの達人	月 日
11	第11章	バグハンター	月 日
12	第12章	検証者	月 日
13	第13章	守り手	月 日
14	第14章	届けた人	月 日
15	第15章	開発者	月 日

日付の横に「次に確かめたいこと」を1行添えてもよいでしょう。空欄は遅れの印ではなく、戻ってくる場所の目印になります。

演習0 旅支度

- ミッション: 気分記録アプリを完成させ、「分からなかったことリスト」を作る
- 時間の目安: 20分(アプリ)+10分(リスト)
- クリア条件:
 1. 3つのボタンで記録が増える
 2. 見た目のどこか1か所を、日本語で頼んで変えられた
 3. スマートフォンで開けた。または狭い画面幅で確認し、実機で開けない理由と次の確認先を記録した
 4. 「分からなかったこと・気になったこと」を3個以上書き出した
- つまづいたら:
 - エラーが出たら、秘密情報を伏せ、文面と「何をしたら起きたか」をAIに伝える
 - ぐちゃぐちゃになったら、新しいチャットで最初から頼み直してよい。作り直しは失敗ではなく、立派な技のひとつ
 - スマートフォン表示の方法は付録Aへ
- 裏ダンジョン: ボタンを4つに増やし、4つめは自分の言葉(「ねむい」「絶好調」など)にする
- 冒険の書に記録: アプリの画面写真と、「分からなかったことリスト」

「分からなかったことリスト」は、この本でいちばん大事な持ち物です。「保存ってどこにされるの?」「URLって何?」「AIは何をしたの?」——分からないことは恥ではなく、地図の空白です。空白は、

これから章がひとつ進むごとに埋まっていきます。リストの1行が消える瞬間が、あなたのレベルアップの瞬間です。

この章のまとめ

- AI開発環境では、コードを書かなくても、日本語で頼むだけで動くアプリを作れる。見た目も会話で変えられる
- 「紫色に」のような頼み方は結果を確かめやすく、「やさしい雰囲気」のような頼み方は解釈が分かれる。曖昧な部分は、AIが仮定で埋める
- AIの返事は毎回すこし違う。本と同じかではなく、頼んだ動き(ボタンが押せて記録が増える)ができていくか確かめる
- 画面に表示されていることと、保存されていることは別。メモリにだけ置いた記録は、再読み込みで消える
- 「画面が動く」と「アプリとして使える」のあいだには距離がある。保存、URLを誰がいつまで使えるか、安全の確かめ方などが残っている

理解度チェック

1. 0.3で試した「紫色にして」と「やさしい雰囲気にして」のうち、AIの解釈が分かれやすいのはどちらですか。そのとき、AIは曖昧な部分をどう扱いますか。
2. 0.2の依頼どおりに作った気分記録アプリで、記録を5行ためたあと、スマートフォンで画面を下に引っばりました。一覧はどうなりますか。理由も教えてください。
3. 画面に記録が10行並んでいます。これだけで「記録は保存されている」と言えますか。確かめ方も教えてください。

解答のめやす

1. 「やさしい雰囲気にして」です。曖昧な部分は、AIが仮定で埋めます(色を淡くする、角を丸くするなど)(→0.3)
2. 空になります。画面を下に引っばるのは再読み込みで、記録はメモリにだけ置いているため、画面が作り直されると失われます(→0.5)
3. 言えません。画面に表示されていることと、保存されていることは別だからです。再読み込みしても残るかを試し、AIに「どこに記録していますか」と聞いて確かめます(→0.5)

60秒でクエスト報告

この章を人に説明するなら、こうなります(自分の言葉で言い直せたらクリアです)。

AIに日本語で頼んで、気分を記録するアプリを作りました。コードは一行も書いていません。ボタンを押すと記録が増えますが、再読み込みすると全部消えます。記録は画面に表示させていただいて、どこにも保存されていなかったからです。保存のしくみはまだ知らないで、これから学びます。

レベルアップ!

Lv.0 称号: 旅立った人(冒険者登録、完了。レベルは第1章から、章の番号と一緒に上がっていきます)

- 手に入れた道具: AI開発環境、プレビュー、エラー文と「何をしたら起きたか」を伝える作法
- 冒険の書: 最初のミニアプリ+分からなかったことリスト
- 冒険の記録: Lv.0の行に、今日の日付を書き込もう。最初のひとマスです

次は、この小さなアプリを誰かに渡す場面を考えます。「作れた」と「任せられる」は、どこが違うのでしょうか。第1章で、これから引き受ける仕事を見渡します。

第1章 Vibe Codingとアプリ開発は何が違うのか

クエスト1: 敵と味方の正体を知る

文化祭まで、あと29日。

この章のねらい

- デモ・プロトタイプ・PoC・MVPを作る目的で見分け、本番かどうかと分けて説明できる
- AIに任せられる作業と、人が引き受ける判断・責任を見分けられる
- 序章のアプリの現在地を判定し、人に渡す前に確かめることを挙げられる

今回のクエスト

序章では、日本語の依頼が、ボタンを押せるアプリになりました。けれど本書の作例では、再読み込みひとつで記録が消えました。この章では、いったん手を止めて、あの20分に何が起きていたのかを確かめます。

ミッションは、「動くデモ」と「人に渡せるアプリ」の違いを、自分の言葉で説明することです。今回、新しく作るものではありません。その代わり、この先のすべての章で使う「物差し」を手に入れます。

会社勤めの方へ、ひとこと。この本の題材は学生の文化祭ですが、「小さな店舗」「社内の業務ツール」と読み替えれば、そのまま職場の話です。この章の内容は、社内で「AIでツールを作ってみよう」と提案する場面で、そのまま使えます。

まずAIに聞いてみよう

今日は作らせる代わりに、AIに質問をぶつけてみます。序章の気分記録アプリを開いて、チャットにこう打ってみてください。

あなた → AI

このあいだ一緒に作った気分記録アプリのことです。これを、私のサークルのメンバー20人で、毎日の気分を共有するボードとして本番で使えますか。問題があれば、遠慮なく全部挙げてください。

AI → あなた

率直にお答えすると、このままで本番利用はおすすめできません。理由は次の通りです。

1. 記録がどこにも保存されず、再読み込みすると消えます
 2. いまのURLは一時的なもので、この作業環境を閉じると開けなくなります
 3. 20人で使うと、誰の記録なのか区別できません
 4. 不具合が起きたとき、原因を調べて直す手立てがありません
 5. 気分という個人的な記録を預かるのに、安全面の確認をしていません
- デモとしては十分ですが、毎日使うには、保存・利用者の区別・安全の確認が必要です。

返事がこの例と違ってもしかたありません。挙がった問題を、自分のアプリと照らしてみてください。逆にAIが「使えます」と答えた場合も、保存、利用者の区別、公開範囲を確かめます。返事の口調より、根拠を見るための練習です。

ボス出現——20分で動いた。でも、人に渡せない

この返事、どこかで見た覚えがありませんか。序章の終わりに並べた「足りないもの」の一覧と、ほとんど同じです。保存されない。複数人で使えない。壊れても直せない。安全かわからない。

作るときには出てこなかった問題が、使う場面を伝えた途端に並びました。「画面を作る依頼」と「20人が毎日使えるかを点検する依頼」では、確認する範囲が違うのです。

※厳密に言うとは: これで「AIは最初から全部わかっていた」とは言えません。AIの指摘にも、見落としや思い違いはあります。

「動いたから完成だ」と思いたくなるのは自然です。最初の一步がうまくいったことと、その先に仕事が残っていることは、両立します。できたものを足場にして、次に何を確かめるかを決めましょう。

敵の正体を知らずに戦うことはできません。この章では、あなたが序章で体験した開発スタイルに付いている「名前」から調べていきます。

1.1 Vibe Codingの誕生と本来の意味

序章であなたがやったこと——日本語で頼み、AIの提案をそのまま受け入れ、コードをほとんど読まずに進める——には、名前があります。Vibe Coding(バイブコーディング)です。

この言葉は、著名なAI研究者のアンドレイ・カルパシー(Andrej Karpathy)が名付けたものです。2025年2月、X(SNS)への投稿でした。その投稿で彼が描写したのは、おおよそ次のようなスタイルでした。

- 雰囲気(vibe)に身をまかせ、コードの存在すら忘れて進める
- AIが提案する変更を、中身を細かく読まずに「すべて受け入れる」
- エラーが出たら、メッセージをそのままコピーしてAIに貼り付ける
- コードが自分の理解を超えて育っても、気にせず先へ進む

出典: [Andrej Karpathyの投稿（2025年2月）](#)。本節の箇条書きは投稿の要旨です。

3つめのやり方は、序章でも紹介しました。エラーメッセージを手がかりにAIとやり取りし、結果を見て次へ進む。コードを書けなくても、まず試してみられる入口が開かれたのです。

この投稿は、週末に気軽なプロジェクトを作る体験を語ったものでした。本書では、その中でも「コードの理解や確認を後回しにして、対話で試作を進める」という特徴に注目します。

速さと引き換えに、理解と確認を後回しにする。それがこのスタイルの、正直な設計です。

※厳密に言うとは: 日本語でAIと開発しても、確認まで省くとは決まっていません。本書でVibe Codingと呼ぶのは、理解と確認を後回しにして進める場合です。

図 1-1

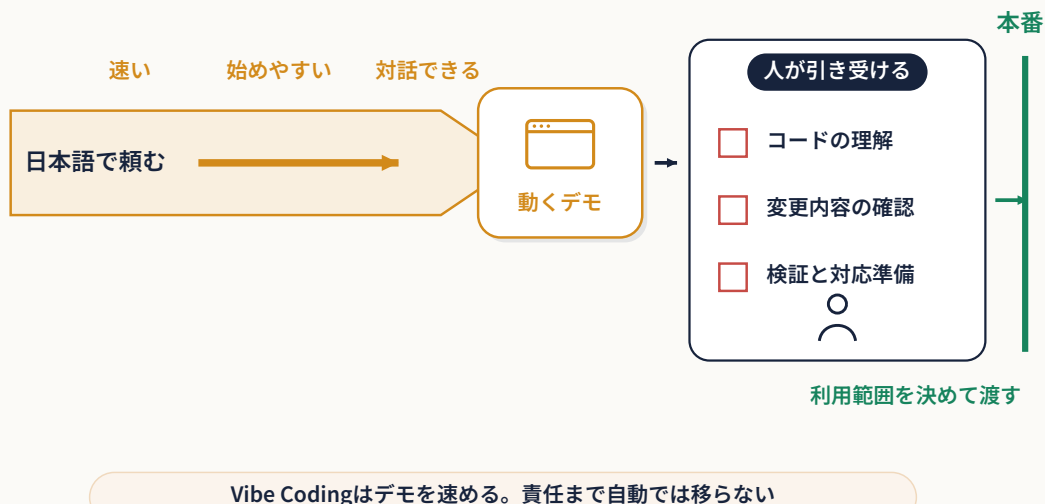


図1-1 Vibe Codingの利得と責任

図1-1では、日本語で頼んでから動くデモまでは速く進み、本番の手前に人が引き受ける確認が残ることを見てください。「20分で作れたのに、そのまま人に渡せない」のは、試作の速さと運用の準備が別の仕事だからです。保存先を確かめる、利用者を区別する、壊れたときの戻し方を用意する。この先で、その仕事を一つずつ足していきます。

では、Vibe Codingは敵なのでしょうか。違います。Vibe Codingは、この本を通じてあなたの味方です。20分でデモが作れる力は、開発の歴史の中でも革命的な追い風です。

敵は別にいます。デモと「使えるアプリ」のあいだに横たわる距離——序章で見た5つの壁のほうです。味方の性格を知って使う。それがこの章の目的です。

1.2 コーディング、プログラミング、アプリ開発の違い

「AIがコードを書いてくれるなら、自分は何を学ぶのだろう」。その答えを探すために、ふだん混ぜて使われている3つの言葉を分けてみます。

コーディングは、決まった内容をコードとして書き下ろす作業です。日本語で言うと「清書」に近い仕事です。

プログラミングは、その一歩外側です。どういう手順で動かすかを考え、組み立て、動かして確かめる。コーディングは、プログラミングの一部分です。

アプリ開発は、さらにその外側の営みの全体です。誰のどんな困りごとを解くかを決め、作り、正しさを確かめ、人に届け、使われ続ける状態を保つ。プログラミングでさえ、アプリ開発の一部分なのです。

図 1-2

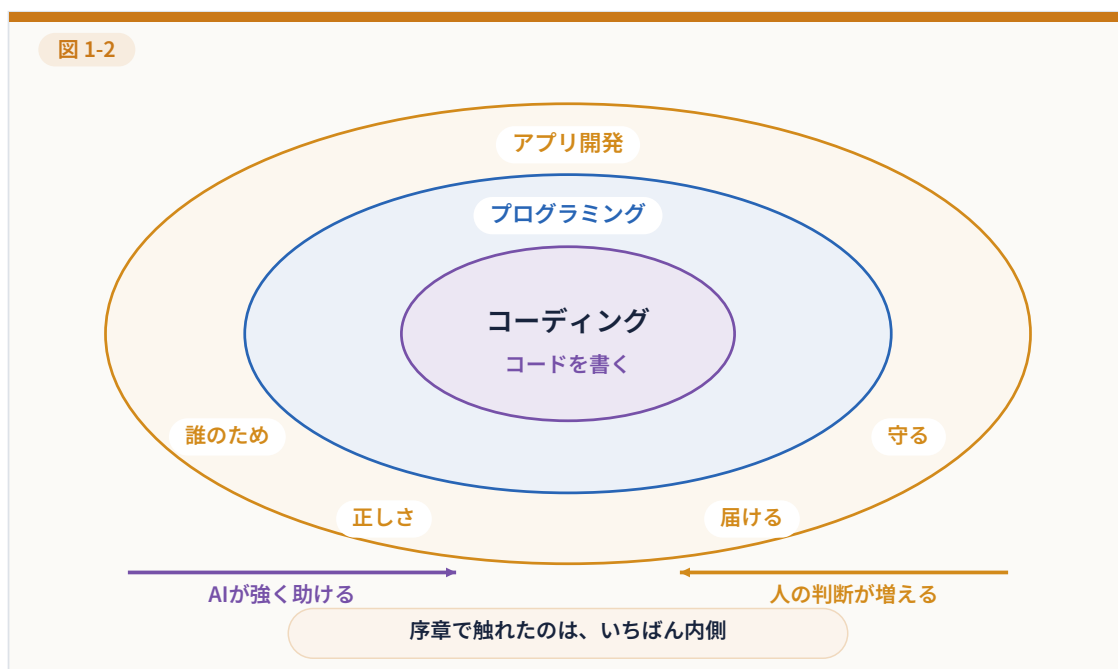


図1-2 コードからアプリ開発へ

図1-2では、内側の輪ほどAIが強く助け、外側の輪ほど人の判断が増えることを見てください。

AIは、コードの下書きや修正、処理手順の提案を助けてくれます。外側の輪にあたる企画や検証にも使えます。ただし、「自分たちの店では何を優先するか」「この結果で人に渡してよいか」は、現場の事情と結び付けて判断する必要があります。

序章の20分を、この円に重ねてみましょう。コーディングは100パーセントAIがやりました。プログラミングもほぼAIです。

では、いちばん外側の輪はどうだったか。誰のためのアプリかも、正しさの確かめ方も、消えたときに誰が困るのかも——手つかずのままでした。「アプリにならない」の正体は、この外側の輪の空白です。

※厳密に言うと: コーディングとプログラミングの線引きには諸説あり、現場では区別せずに使う人も多い言葉です。本書では「AIの得意領域がどこか」を考えるための整理として、この3層を使います。

1.3 デモ、プロトタイプ、PoC、MVP、本番アプリ

試作品を見せるとき、「できました」の一言だけでは、相手はどこまで使ってよいかわかりません。そこで、作る目的と使われ方を伝える5つの言葉を使います。どれも、完成度順に並んだ段階ではありません。

デモ(demo)は、見せるための試作です。決まった場面が動いて見えればいい。裏側がどれだけ張りばてでもかまいません。

プロトタイプ(prototype)は、試して学ぶための試作です。人に触ってもらい、「ここが使いにくい」「この機能は要らない」を学ぶために作ります。学んだら捨てても、役目は果たしています。

PoC(Proof of Concept)は、実現できるかの確認です。「この技術で、この処理は本当に可能か」という一点を証明するために作ります。見た目は二の次です。

MVP(Minimum Viable Product)は、利用者についての仮説を確かめる、最小限の製品です。機能数の少なさそのものより、「使ってもらって何を学ぶか」が大事です。本書では、模擬店の部員に試してもらい、会計ミスと操作の負担が減るかを確かめる小さなアプリを目指します。

本番アプリは、実際の業務や生活で使われているアプリです。「本番」は完成品という称号ではなく、利用状態の名前です。MVPを本番で使うこともあります。人が頼りにする以上、機能が少なくても、データや不具合への対応が必要になります。



図1-3 試作の目的と本番の違い

図1-3では、5つの呼び方が目的で分かれていて、必ず順番に通る段階ではないことを見てください。自分のアプリに引きつけると、確かめる問いはこうなります。

呼び方	自分のアプリへの問い
デモ	動く場面に人に見せられるか
プロトタイプ	これに触ってもらって、何を学びたいのか言えるか
PoC	「実現できるか」を確かめたい一点は何か
MVP	最小限の機能で、利用者について何を確かめるか

呼び方	自分のアプリへの問い
本番アプリ	実際に誰が使い、困ったときは誰が対応するか

この物差しで測れば、序章のアプリの所在地は明らかです。デモです。

デモには、使う場面を相手に想像してもらう価値があります。そこから操作のしやすさを知りたければプロトタイプとして試し、技術的な不安があればPoCで確かめる。同じアプリでも、目的に応じて使い方が変わります。

「いま何を確かめているか」と「実際に使ってよい範囲」を伝える。この二つをセットにすると、相手も安心して試せます。

出典: MVPの説明は、Eric Ries [What Is an MVP?](#)を参照。各用語の使い方は組織によって異なるため、名前に加えて目的を説明しましょう。

▼コーナー「先輩エンジニアならここを見る」

「技術を確かめるためのPoCです。実際の売上はまだ記録しません」。こう説明すれば、試してよい範囲が伝わります。「本番です」と答えるなら、利用者と、困ったときの連絡先まで言えるかも確かめます。

1.4 AIに任せられる仕事、任せきれない仕事

開発の3層と、試作の目的を整理したところで、仕事の分担表を作ります。序章の20分と、先ほどのAIの返事を思い出しながら見てください。

AIに任せられる仕事	任せきれない仕事
コードを書く	何を作るかを決める
決まった計算を実装する	計算のルールそのものを決める
エラーの原因の候補を挙げる	どの直し方が自分の現場に正しいか決める
問題点や心配ごとを列挙する	どれを重く見て、どれを捨てるか決める
説明の下書きを作る	その説明に責任を持つ
公開の手順を教える	公開してよいと判断する
直し方を提案し続ける	壊れたとき、最後に対応する人になる

左の列を眺めると、あることに気づきます。どれも「作業」です。右の列は、すべて「判断」と「責任」です。

AIは右の列についても候補や判断材料を出せます。しかし、店の事情を確認し、採用した結果に対応するのは、作り手や運用する組織です。相談はできても、採用を決める人と、結果に対応する人は省けない、という分担です。

この表を一文に圧縮したものが、本書の中心メッセージです。この先、迷子になったらいつでもここに戻ってきてください。

コード生成はAIに任せられる。だが、何を作るか、正しいか、公開してよいか、誰が守るかは任せきれない。

「何を作るか」は壁1(伝わらない)に、「正しいか」は壁4(確かめられない)に対応します。「公開してよいか」「誰が守るか」は壁5(渡せない)。序章の壁を、ここでは具体的な仕事として言い換えています。

1.5 人間は「発注者」ではなく「開発責任者」になる

では、任せきれない仕事を引き受けるあなたは、何者になればいいのでしょうか。

コードをAIに書いてもらおうと、自分は完成を待つ「発注者」のように感じるかもしれません。けれど、依頼する側にも、条件を決め、納品物を確かめる仕事があります。AIとの開発では、その役割を意識して引き受けます。

人間のチームで仕事を頼むときは、誰が検査し、誰が不具合に対応するかを決めます。外部に発注する場合も、役割や契約の確認が必要です。

AIから「できました」と返事が来ても、検査と運用の担当者が決まったことにはなりません。たとえば模擬店の会計アプリで、AIが「合計を出せるようにしました」と返してきたとします。実際の注文で金額が合うかを誰かが確かめるまでは、部員に任せられません。テストをAIに手伝ってもらおうとしても、何を確認し、結果を誰が受け止めるかは決めておきます。

発注者気分のまま進むと、責任だけが宙に浮きます。宙に浮いた責任は、公開したあと、いちばん困る形であなたに戻ってきます。

だからこの本では、あなたを最初から開発責任者として扱います。開発責任者は、コードを書く人ではありません。決めて、頼んで、確かめて、直させて、届ける人です。

手を動かす主力はAIでかまいません。しかし、何を作るか決めるのも、できたものを検査するのも、公開してよいと最後に決めるのも、あなたです。

図 1-4

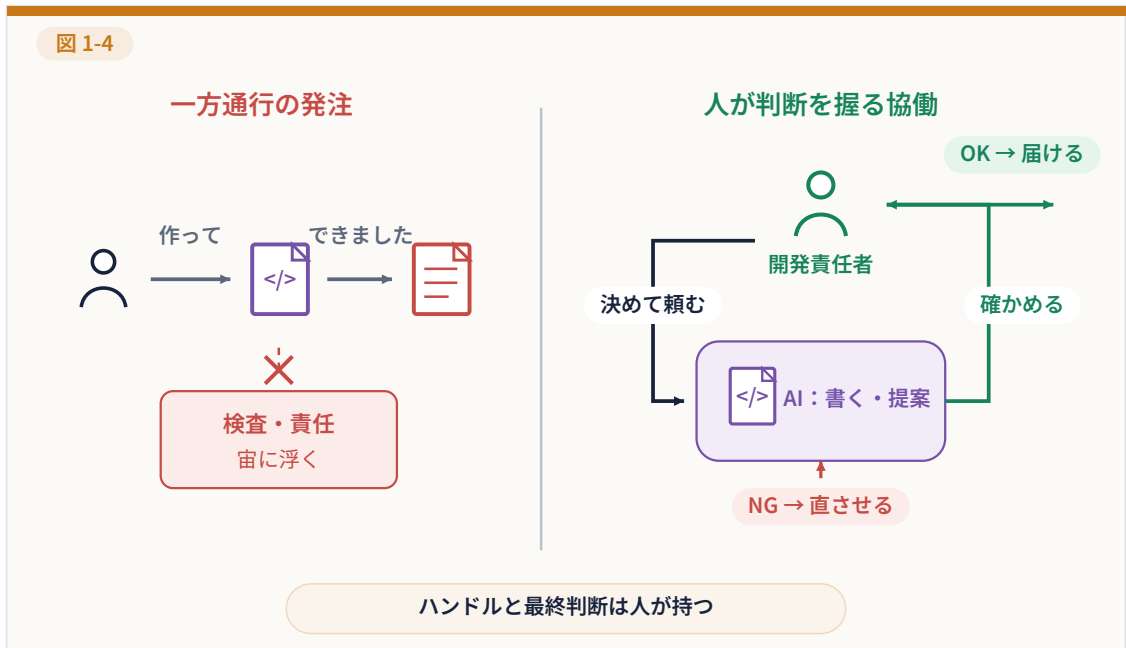


図1-4 発注者から開発責任者へ

図1-4の2つの形で、「できました」のあとの検査と責任を誰が持っているかを見比べてください。

期待と違うものができたら、依頼が曖昧だったのか、明示した条件をAIが見落としたのか、実装に不具合があるのかを分けて調べます。すべてを自分の頼み方のせいにする必要はありません。「期待した結果」と「実際の結果」を並べることが、責任者の最初の仕事です。

肩書きに構える必要はありません。「記録が消えた。次は保存先を確かめよう」と次の一手を決められれば、その仕事はもう始まっています。開発責任者の仕事には、難しい判断を一人で抱えず、詳しい人に相談することも含まれます。

1.6 AIが作ったコードの所有者は誰か

開発責任者を名乗るなら、一度は考えておくべき問いがあります。AIが書いたコードは、いったい誰のものなのでしょうか。

ここでは、「誰のものか」を三つの問いに分けます。生成物に著作権が生じるか、サービスの規約でどのように使えるか、他者の権利を侵害していないか。これらは別の問題です。

1つめの「著作権が生じるか」について、日本の文化庁の整理では、人の創作意図や創作的な寄与などを踏まえて、AI生成物が著作物にあたるかを判断します。2つめの「規約でどう使えるか」については、サービスが利用を認めていても、それで法律上の問題がすべて解消するわけではありません。3つめの「侵害していないか」は、生成物に著作権が認められる場合でも、既存の著作物との関係で別に確かめます。

最初にできることは具体的です。利用規約を確認し、コードに組み込んだ部品(ライブラリ。第10章で扱います)の利用条件を記録する。公開や業務利用で判断に迷う点があれば、所属先の窓口や専門家へ確認しましょう。

出典: 文化庁 [AIと著作権についての「AIと著作権に関する考え方について」](#) およびチェックリスト（2026年9月確認）。

もうひとつ、コードの権利とは別の水脈もあります。AIが書いたコードには、世界中の誰かが作った部品が組み込まれることがよくあります。その部品にはそれぞれ利用の条件が付いています。

この話は、第13章でじっくり扱います。いまは「所有者は誰か、は公開前に確認する問いだ」とだけ、装備欄にメモしておいてください。

1.7 本書における「開発できる」の定義——説明できる・変更できる・戻せる・確かめられる・渡せる

最後に、この本のゴールを正確に決めます。「開発できるようになる」とは、何ができることなのか。

この本の目標は、すべてのコードを暗記して書くことではありません。AIの助けを使いながら、次の5つができる状態を目指します。

1. 説明できる——このアプリが誰の何を解決し、どういう仕組みで動いているかを、自分の言葉で言える
2. 変更できる——直したい場所を狙って、意図した通りに変えられる
3. 戻せる——変更で壊れても、壊れる前の状態に帰れる
4. 確かめられる——「動いた」ではなく「正しく動く」と、根拠を持って示せる
5. 渡せる——自分がいなくても、人が使い続けられる状態で手渡せる

図 1-5

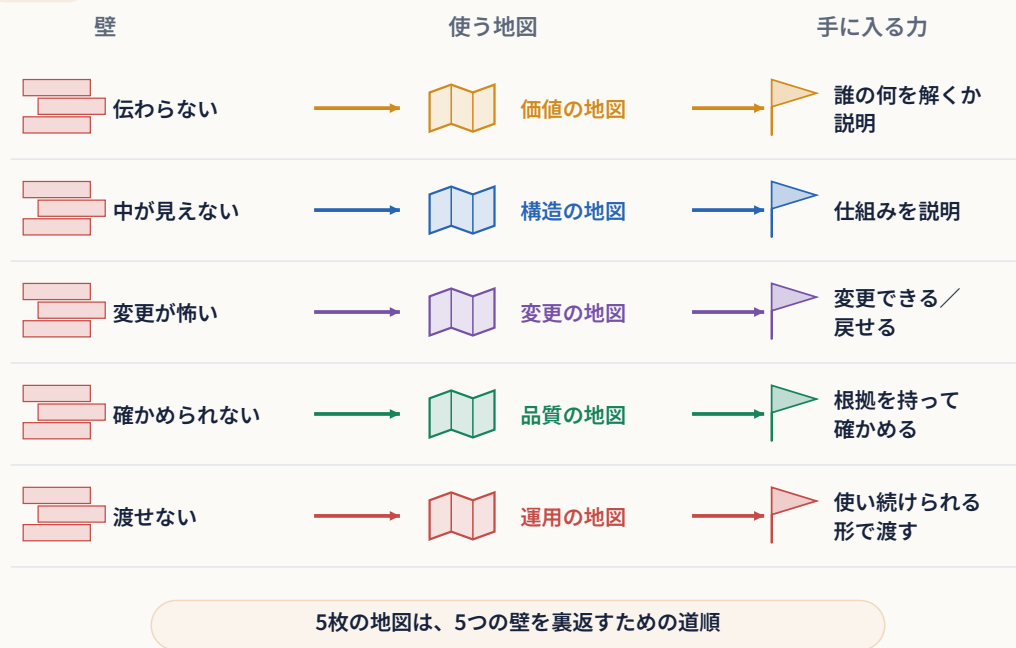


図1-5 5つの壁を5枚の地図で越える

気づいたでしょうか。この5条件は、序章の5つの壁をひっくり返したものです。壁を1枚越えるたびに、条件がひとつ手に入る。

5枚の地図は、そのための道順です。そして第15章の口頭試問で問われる5つの質問も、この5条件をそのまま尋ねるものになっています。

序章で、すでに説明できるようになったこともあるでしょう。一方で、まだ自信のない条件も残っています。5条件に「できる・練習中・未確認」の印を付けておいてください。第15章でもう一度見ると、画面の変化だけでは見えない、自分の成長がわかります。

別の条件でも動くか試す

この章で手に入れた問いは、自分のアプリ以外にも使えます。身の回りのソフトウェアを2つ3つ選び、「何のために作られ、誰が実際に使っているか」を考えてみてください。毎日頼っている地図アプリは、本番で使われている例です。

では、職場の誰かが作って、その人しか直せない集計マクロは？ 実業務で使っているなら本番です。ただし、作った人が異動すると誰も手入れできない、という弱点を説明できます。「本番」という名前と、安心して使えるかの評価は分けて考えます。

同じアプリを「見せるためのデモ」と呼ぶ人と、「操作を学ぶプロトタイプ」と呼ぶ人がいてもかまいません。名前の違いが、目的の違いを教えてください。

大事なのは名前を当てることより、「次に何を確かめれば、人に渡せるか」を言えることです。この感覚は、次の演習でそのまま使います。

演習1 現在地を測る

- ミッション: 序章のアプリを何のために使うかを、デモ・プロトタイプ・PoC・MVPの言葉で説明し、本番利用の有無と分けて書く
- 時間の目安: 15分
- クリア条件:
 1. 作った目的を表す呼び方を選び、実際に誰かが日々使っているかも書いた
 2. 「人に渡す前に確かめること」を2つ以上、自分の言葉で挙げた
 3. 同じ判定をAIにも頼み、自分の判定と見比べた(一致でも不一致でもよい。不一致なら、どちらの理由がもっともらしいか考えた)
- つまづいたら:
 - 呼び方に迷ったら、1.3の表から、いまの目的に最も近い問いを選ぶ。上から順に合格する必要はない
 - AIの判定がこの本の説明と食い違っても気にしない。AIの返事は毎回変わる。理由を読み、納得できるほうを採ればよい
 - 序章の「分からなかったことリスト」を開く。あのリストの1行1行が、そのまま「足りないもの」の候補になっている
- 裏ダンジョン: 家族や職場で使われているソフトを3つ選び、同じ物差しで測って、いちばん意外だった結果を人に説明する
- 冒険の書に記録: 現在地判定メモ(選んだ段階+理由1行+足りないもののリスト)
- (授業向け)ペアで判定メモを交換し、選んだ段階と理由の違いを2分で話し合う

この章のまとめ

- Vibe Codingは、コードの理解や確認を後回しにして、AIとの対話で試作を速く進めるスタイル。速さは手に入るが、確認の責任までは自動でAIに移らない
- アプリ開発は、コーディングとプログラミングを内側に含み、誰のために作るかを決め、正しさを確かめ、人に届け、使われ続ける状態を保つまでの全体を指す
- デモ・プロトタイプ・PoC・MVPは、作る目的を表す呼び方で、完成度順の階段ではない。本番は完成度の称号ではなく、実際に使われているという利用状態の名前
- AIはコードを書き、候補や判断材料を出せる。だが、何を作るか、正しいか、公開してよいか、誰が守るかの判断と責任は人に残る
- 本書でいう「開発できる」とは、説明できる・変更できる・戻せる・確かめられる・渡せるの5つがそろった状態

理解度チェック

1. PoCとMVPは、それぞれ何を確かめるために作りますか。
2. 模擬店の会計アプリを作る場面で、AIに任せられる仕事と、任せきれない仕事を1つずつ挙げてください。
3. PoCで「この技術で会計の処理ができる」と確かめられました。これで、そのまま文化祭当日の本番で使ってよいと言えますか。理由も教えてください。

解答のめやす

1. PoCは「この技術で、この処理は本当に可能か」という一点を確かめます。MVPは、最小限の製品を使ってもらい、利用者についての仮説を確かめます(→1.3)
2. 任せられる仕事は、コードを書く、決まった計算を実装する、などから1つ。任せきれない仕事は、計算のルールそのものを決める、公開してよいと判断する、壊れたときに最後に対応する人になる、などから1つ(→1.4)
3. 言えません。PoCは実現できるかを確かめる試作で、完成度の段階ではないからです。本番で使うなら、実際に誰が使い、困ったときに誰が対応するか、データや不具合への対応ができるかを確かめ、公開してよいと人が判断します(→1.3、1.5)

60秒でクエスト報告

序章では、AIとの対話で動くデモを作りました。次は、使う人と場面を決め、記録がどこに残るかを確かめます。デモやPoCは作る目的を表す言葉で、本番かどうかとは別です。コードの生成はAIに助けてもらえますが、採用する変更を決め、結果を確認し、人に渡す準備をする仕事が残ります。説明できる・変更できる・戻せる・確かめられる・渡せる。この5つを、これから一つずつ身に付けます。

——この報告は、この先のどの章の報告にも土台として埋め込まれます。第15章の口頭試問5問すべての、最初の一步です。

レベルアップ!

Lv.1 称号: 開発責任者(見習い)

- 手に入れた装備: Vibe Coding、デモ、プロトタイプ、PoC、MVP、本番アプリ
- 冒険の書: 序章アプリの現在地判定メモ
- 見習いの「見習い」が外れるのは、5条件がそろった第15章。長い旅ですが、地図はもう手の中にあります

次の第2章では、開発責任者の最初の仕事に取りかかります。「何を作るか」を決める仕事です。

——ところで、あなたの頭にはもう「あったらいい機能」がいくつか浮かんでいませんか。その機能、誰がいつ使うのか、いま答えられますか。

第2章 機能から考えず、困りごとから考える

クエスト2: 宝(困りごと)の探索

文化祭まで、あと27日。

この章のねらい

- ・ 困りごとを、課題・原因・解決策に分けて見分けられる
- ・ 〈誰〉〈場面〉〈困りごと〉〈解決〉の型で、価値仮説を一文で書ける
- ・ 「作らない」案と比べて、アプリを作るかどうかの理由を説明できる

今回のクエスト

第1章で、あなたは「開発責任者になる」と学びました。責任者の最初の仕事は、AIに頼むことではありません。何を作るかを決めることです。

この章のミッションは、アプリを「機能」からではなく「困りごと」から考えられるようになること。章の終わりには、課題定義書——「誰の、どんな困りごとを解決するか」を1枚にまとめたシート——が手に入ります。

文化祭の会議で並んだ「欲しい機能」を出発点に、店番の困りごとへ視点を移してみましょう。なぜ会計アプリを作るのか。その理由まで、あなた自身の題材に持ち帰れる章です。

まずAIに作らせてみよう

こんな場面から始めます。あなたの部は、秋の文化祭で模擬店を出すことになりました。チャイやコーヒーなどの飲み物と、焼き菓子を出す小さな店です(会社勤めの方は「小さな店舗」と読み替えてください。以降もずっと同じです)。

準備の会議で、誰かが言いました。「せっかくだから、アプリ作らない?」あなたは序章で、アプリを作れるようになっています。会議は盛り上がり、ホワイトボードに「欲しい機能」が並びました。それをそのままAIに渡してみます。

あなた → AI

文化祭の模擬店で使う多機能アプリを作ってください。機能は、シフト表、メニュー紹介、お知らせ掲示板、売上メモ、天気表示、写真ギャラリー、来場者向けのスタンプラリーです。

AI → あなた

模擬店サポートアプリを作りました。画面上部のボタンで、シフト表・メニュー・掲示板・売上メモ・天気・ギャラリー・スタンプラリーの7つの機能を切り替えられます。プレビューで確認してください。

プレビューには切り替えボタンが7つ並び、押すとそれぞれの画面が出てきます。序章の3ボタンのアプリより、ずっと大きなものができました。次の会議で見せたら、部員も身を乗り出してくれそうです。

実際、歓声は上がりました。問題は、そのあとです。

ボス出現——「で、これ、誰がいつ使うの?」

ひとしきり盛り上がったあと、去年も店番をした先輩が、ぼつりと聞きました。

「で、これ、誰がいつ使うの?」

あなたは答えに詰まります。質問は続きます。「シフト表は、もうLINEのノートにあるよね」「天気は、みんな天気アプリを見るんじゃない?」「スタンプラリーの台紙って、誰が作って誰が配るの?」——ひとつも答えられません。

ホワイトボードには機能が並んでいますが、使う人の動きはまだ描かれていません。シフトを決める部長、値段を知りたい来場者、会計をする1年生。それぞれ違う場面で、違うものを必要としています。

職場のツールでも、「誰の、今のどの仕事を置き換えるのか」が曖昧だと、使う人は使い始めるきっかけをつかめません。

画面は増えたのに、誰の困りごとにもまだ確かめていない。これがこの章のボスです。作っているあいだの楽しさと、使う人の役に立つこと。その二つを結ぶ材料を探しに行きます。

序章で、「やさしい雰囲気にして」という曖昧な頼みをAIが想像で埋めたのを覚えていますか。今回は、もっと大きなものが想像で埋まっています。誰が、いつ、何に困って使うのか——アプリの存在理由そのものが、全員の思い込みで埋まっていたのです。

このボスを倒す武器は、機能リストの反対側にあります。困りごとです。

2.1 「こんなアプリが欲しい」だけでは足りない

「こんなアプリが欲しい」という言い方には、隠れた主語があります。欲しがっているのは、たいいてい作る側です。使う側が本当に困っているのかどうかは、その時点では誰も確かめていません。

機能は、解決策の部品です。何を解決するかが決まる前に部品を並べるのは、行き先を決める前に切符を買うようなものです。

切符は本物なので、買えてしまいます。同じように、機能リストをAIに渡せば、アプリは本当にできてしまいます。できてしまうことと、役に立つことは、別のことです。

試作を短時間で作れると、思いついた機能を次々に足したくなります。そこで、機能を足す前に「これがあると、誰のどの手間が減るか」を一度書いてみます。たとえば冒頭の天気表示は、「みんな天気アプリを見る」なら、減る手間はほとんどありません。

序章の20分を思い出してください。作るコストが下がった時代は、「何を作るか」を決めることの価値が上がった時代です。

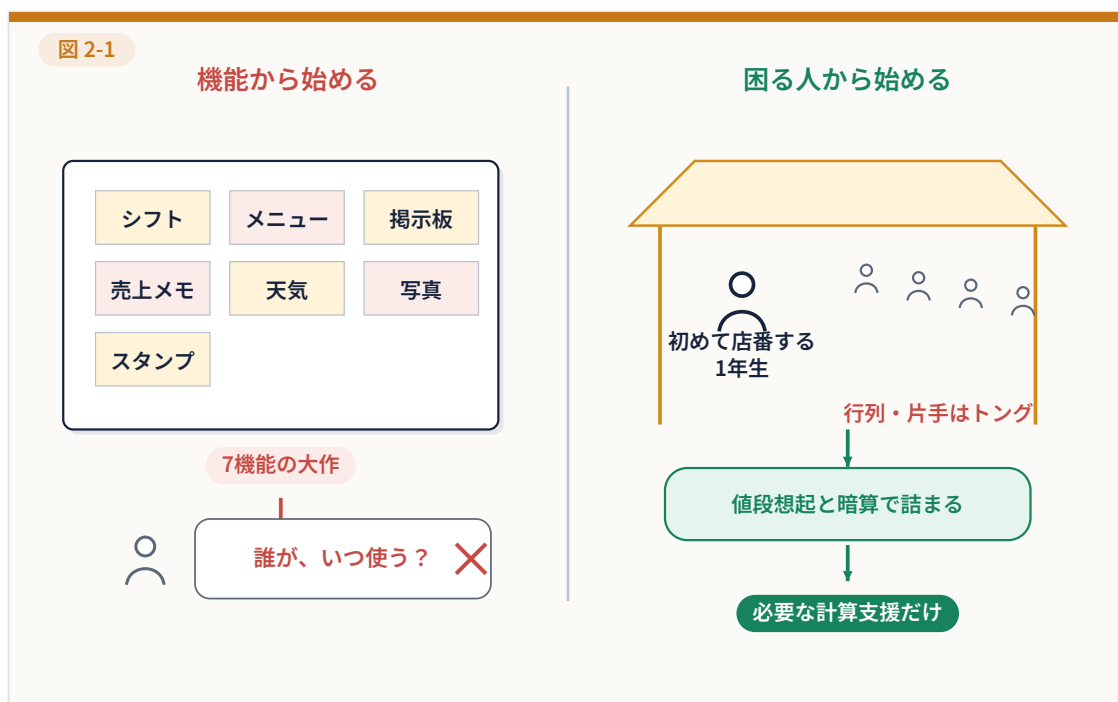


図2-1 機能ではなく困りごとから始める

この章では、右側の道をたどります。使う場面から逆に考えると、必要な機能を選ぶ理由が見えてきます。

2.2 ユーザーとは誰か——利用場面と利用前後を考える

困りごとから考えるための、最初の道具は言葉です。まず2つ、言葉の解像度を上げます。

ユーザー(user)とは、そのアプリを使う人のことです。日常語で言うと、そのまま「使う人」。当たり前に見えて、実はいちばん間違えやすい言葉です。

7機能アプリのユーザーは誰でしたか。「部員全員と来場者」。対象は広いままですかまいませんが、設計するときは、最初に助けたい人と場面を一つ選ぶと考えやすくなります。

判定の目安をひとつ。ユーザーを、顔が浮かぶ一人で言えるか。「部員全員」ではなく、「初めてシフトに入る1年生の後輩」。顔が浮かべば、その人の当日の動きを具体的に考えられます。あなたのアプリのユーザーは、名前で呼べる誰かですか。

次が利用場面です。日常語で言うと、いつ・どこで使うか。同じ「店番の部員」でも、場面が変われば状況はまるで違います。模擬店の店番なら——屋外のテント、立ちっぱなし、片手はトングでふさがり、目の前には行列。スマートフォンを両手で持って、落ち着いて操作する余裕は、たぶんありません。

そして、使う瞬間の前と後ろまで見ます。会計という行為の前には「注文を聞く」「値段を思い出す」があります。後ろには「お金を受け取る」「お釣りを渡す」「商品を渡す」が続きます。一連の流れを並べてみましょう。

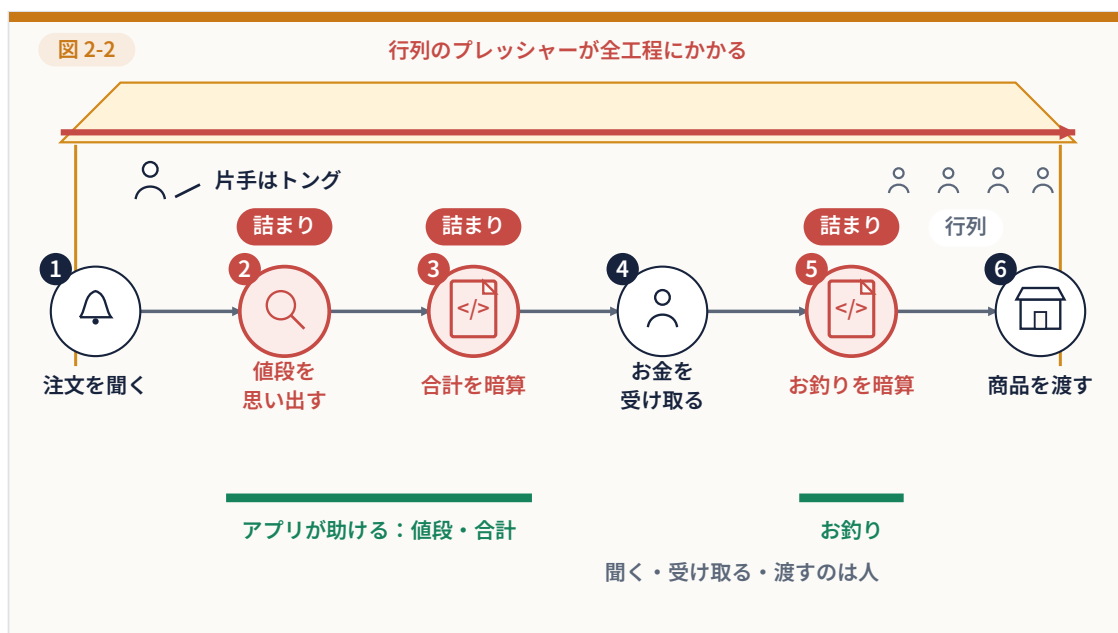


図2-2 店番1回の詰まりどころ

前後を並べると、困りごとが「どの瞬間に」起きているのかが見えてきます。同時に、アプリが手伝える場所と、手伝わない場所(商品を渡すのはアプリにはできません)も見えてきます。

2.3 課題、原因、解決策を分ける

2つめの道具は、3つの言葉をきちんと分けることです。

課題とは、解決したい困りごとのことです。日常語で言うと、そのまま「困りごと」。「会計でミスが起きる」は課題です。一方、「会計アプリを作る」は課題ではなく、解決策です。

この2つを混ぜると、別の解決策を比べにくくなります。「アプリが欲しい」に「何に困っているから?」と続けると、課題の話へ戻れます。

さらに、課題と解決策のあいだには原因が挟まっています。会計ミスという課題の原因は、何でしょうか。候補はいくつもあります。

- 暗算だから? 2品3品の合計と、お釣りの計算を頭の中でやっている
- 手書きの値段表が見にくいから? 遠くて小さくて、とっさに読めない
- 行列のプレッシャーで焦るから? 急かされると、誰でも間違える

原因によって、効く解決策は変わります。暗算が原因なら、計算を機械に任せるのが効きます。値段表が原因なら、大きく印刷し直すだけで済むかもしれません。プレッシャーが原因なら、列の整理や2人体制のほうが効くかもしれません。

原因を飛ばして解決策に跳ぶのは、診察せずに薬を出すようなものです。

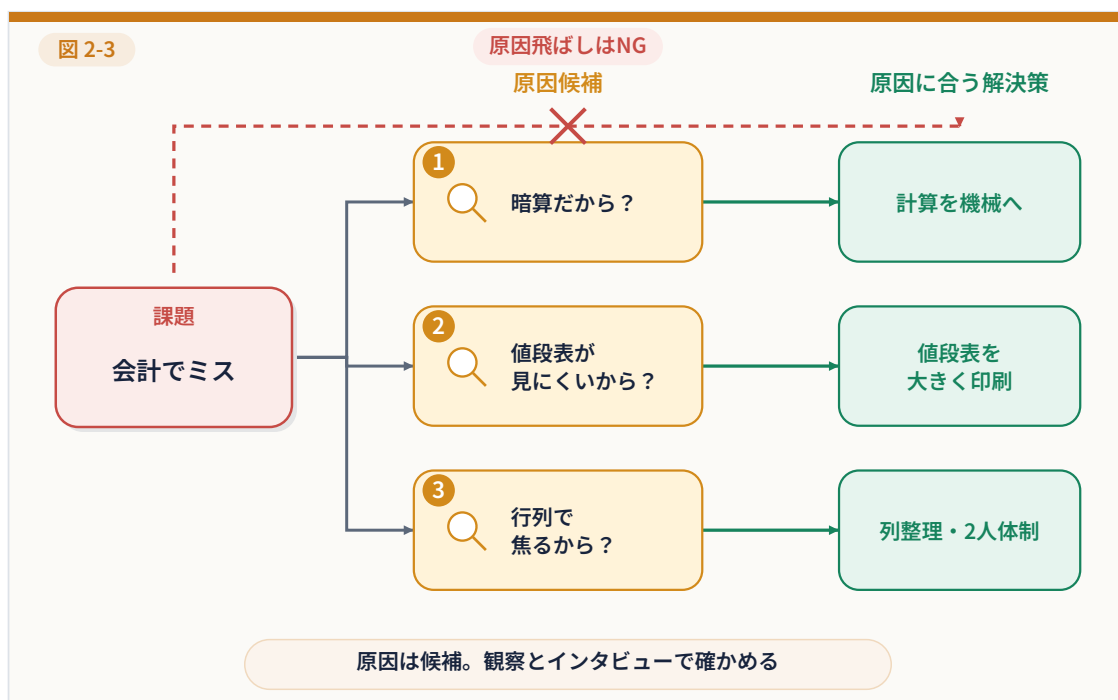


図2-3 課題・原因・解決策を分ける

図2-3では、1つの課題から原因候補が3本に分かれ、原因ごとに合う解決策が違うことを確かめてください。

習慣にしてほしいことは、ひとつだけです。困りごとを見つけたら、「それは、なぜ起きているんだろう」と一度だけ聞き返す。この一往復があると、次の節の「確かめに行く」が生きてきます。

※厳密に言うとは：原因を掘る手法には「なぜを5回くり返す」という有名な型があります。ただし日常の題材なら、1回聞き返すだけで景色は十分に変わります。

2.4 観察と短いインタビュー

では、原因はどうやって突き止めるのか。机の上で当てるものではありません。現場で確かめます。確かめ方は2つ。見ること(観察)と、聞くこと(インタビュー)です。どちらも、大げさな準備は要りません。

模擬店の場合、最高の情報源がすぐ近くにいます。昨年、実際に店番をした先輩です。15分だけ時間をもらいましょう。聞くことは、実質ひとつで足ります。

「去年の店番で、何が一番イライラした？」

短いインタビューのコツは3つです。

1. 具体的な出来事から聞く。「こんなアプリがあったら使う？」だけでは、何を解決すべきかわかりません。「去年、困ったとき何をしていた?」「どう乗り切った?」と聞くと、行動をたどれます。記憶違いもあり得るので、観察できることは後で確かめます。
2. 「一番」を聞く。困りごとを全部挙げてもらうより、一番を聞くほうが、痛みの序列がわかります。

- 最後に「ほかには?」を添える。こちらが想像していなかった困りごとがないか、相手が付け足せる余地を残します。

質問を考える段階では、AIがよい壁打ち相手になります。

あなた → AI

昨年の文化祭で模擬店の店番をした先輩に、15分だけインタビューします。店番まわりの困りごとを聞き出す質問を8個作ってください。未来の希望ではなく、過去に実際に起きたことを聞く形にしてください。

AI → あなた

過去の事実を聞く形で8個作りました。抜粋します。

- ・店番の1回のシフトの中で、一番忙しかったのはいつごろでしたか
- ・お客さんを待たせてしまった場面で、直前に何をしていましたか
- ・会計で間違えた、または間違えそうになったことはありましたか。そのとき何が起きましたか(以下略)

AIの挙げる質問は、あなたの画面では違うものになるはずです。それでかまいません。「過去の事実を聞く形になっているか」を自分で点検して、使えるものだけを拾ってください。

さて、インタビュー本番です。昨年、店番をした先輩の「一番」は、こうでした。

「ピークの30分、列が途切れないの。2品3品の合計を暗算して、お釣りも暗算して、絶対どこかで間違えてた。実際、閉店後にレジのお金が合わなくてさ。誰のミスかわからないから、犯人探しみたいな空気になったのが一番最悪だった」

この話には、課題(会計ミス)、原因の有力候補(行列の中での暗算)、その影響(レジのお金が合わず、部員同士も気まづくなる)が含まれています。ただし、お金が合わなかったのが暗算だけのせいかは、まだわかりません。聞いた事実と、そこから考えた原因候補を分けてメモするのがコツです。

観察もできます。放課後に友達5人に客の役を頼み、会計の予行演習をしてみるのです。1品の注文と複数品の注文を用意し、かかった時間、手が止まった場所、計算のやり直しを記録します。数値は測ってから書きます。アプリを試した後も同じ注文で確かめれば、何が変わったかを比べられます。

いまは、解決策を選ぶ前です。「よくないですか?」と賛成を求めるより、困った場面と、その場で使った工夫を聞いてみましょう。アイデアを見せて感想を聞くことは、解決策の候補ができてから役に立ちます。

2.5 ペルソナを作り込みすぎない

ユーザー像を考える道具として、ペルソナという言葉聞いたことがあるかもしれません。日常語で言うと、代表ユーザーの人物像です。名前や年齢まで決めた架空の一人を作り、その人に向けて設計するという手法です。

本書では、これを1行で止めることをすすめます。「初めてシフトに入る、1年生の部員」。以上です。

会計アプリの操作を考えるなら、学年よりも「店番は初めて」「片手がふさがる」「メニューの値段を覚えていない」といった情報が役に立ちます。名前や趣味を細かく決めるのは、それが設計の判断に必要なときに十分です。

人物像に書いた内容は、観察や聞き取りでわかったことか、まだ仮定なのかを区別します。AIに人物像を考えてもらっても、それだけでは利用者を調べたことにはなりません。

先輩に話を聞いたあとなら、「会計ミスだけでなく、閉店後の照合作業にも困っている」と人物像を書き直せます。本書の1行ペルソナは、そうした発見を設計中に忘れないためのメモです。

▼コーナー「先輩エンジニアならここを見る」

「誰の何を解決するの?」に続けて、「それを何で確かめたの?」とも聞いてみます。利用者の一言や観察メモを添えられると、企画を検討する人も同じ場面を思い浮かべられます。

2.6 価値仮説を一文で表す

ここまでに集めた材料——使う人、場面、困りごと、原因——を、一文に煮つめます。型はこれです。

〈誰〉は〈場面〉で〈困りごと〉に困っている。〈解決〉なら助かるはずだ。

模擬店に当てはめると、こうなります。

シフト担当の部員は、行列をさばきながらの会計で、暗算のミスに困っている。計算をアプリに任せられるなら、助かるはずだ。

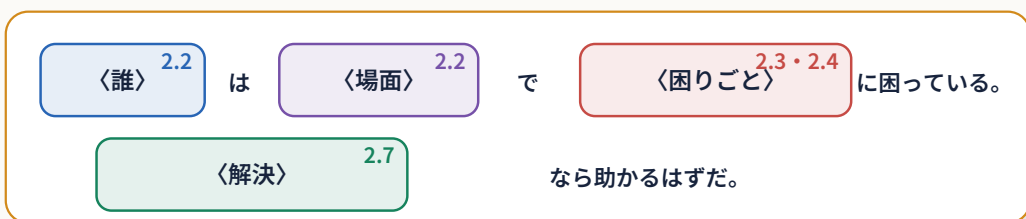
この一文を価値仮説と呼びます。日常語で言うと、「これで助かるはず」の一文です。

大事なのは「仮説」のほうです。この一文は、まだ確かめ終わっていません。文化祭まで待たずに、予行演習で部員に試してもらい、計算のやり直しや操作の迷いが減るかを見ます。当日の利用からも、さらに確かめていきます。

だから、いまは堂々と「仮に」決めてよいのです。あとで外れていたとわかったら、それは失敗ではなく、確かめられたという前進です。

図 2-4

価値仮説の型



模擬店の記入例

シフト担当は、行列中の会計で暗算ミスに困っている。
計算をアプリに任せられるなら助かるはずだ。

仮説
未検証

4つの穴を埋めても、まだ答えではなく仮説

図2-4 価値仮説の4つの穴

この型は、点検表としても働きます。どこかの穴が埋まらないなら、その材料がまだ足りません。〈誰〉が「みんな」になってしまうなら2.2へ、〈困りごと〉がぼんやりしているなら2.3と2.4へ戻ってください。

持ち歩き用に、短くたたんだ形も作っておきましょう。

シフト担当の部員が、行列をさばきながら会計でミスをしてしまう。それを防ぎたい。

この一文が、次の第3章の出発点になります。手帳の1行に収まるサイズまで削れたら、価値仮説は完成です。

2.7 「作らない」という選択肢

価値仮説ができました。すぐにでも作りたくなりますが、開発責任者として、最後にひとつだけ検討することがあります。そもそも、アプリを作るべきなのか。

型の〈解決〉の席に座れる候補は、アプリだけではありません。会計ミスを防ぐだけなら――

- ・ 紙の早見表: よく出る組み合わせの合計金額を印刷して貼る。コストはほぼゼロ
- ・ 電卓: 手元のものを使える。ただし商品ごとの金額入力と、お釣りの計算が必要
- ・ 既存のレジアプリ: 設定して使い始められる候補。料金、操作、割引への対応を実際に確かめる
- ・ 人の工夫: 会計係を固定して2人体制にする。道具は増えないが、人手が要る

これらで足りるなら、作らないのが正解です。それは負けではありません。「アプリを作るまでもなく課題が解決した」という、いちばん安上がりな勝ちです。

「紙の早見表で十分でした」も、課題解決の成果です。困りごとが減ることがゴールで、アプリはそのための手段だからです。

逆に、ここで真剣に比べたからこそ、「作る」と決めたときの理由が強くなります。模擬店の場合はどうでしょう。今年の店はメニューを5品に増やす予定です。しかも、ドリンクとフードのセットを安くする案や、最終日の午後に値引きセールをやる案が出ています。

そこで、候補を同じ条件で比べます。紙の早見表は割引の切り替えに対応できるか。電卓は暗算をどこまで減らせるか。既存アプリは、初めての部員にも使えそうか。2人体制は確保できるか。自作アプリにも、作成・検証・当日の対応という負担があります。

本書の店では、割引の組み合わせと切り替えがあることを決め手にしました。「店の割引ルールを組み込み、商品を押すだけで計算できる」という仮説を、小さな自作アプリで試します。店の条件が違えば、比較の結論も変わります。自分の題材でも、選んだ理由と、他の案で足りなかった点を一行ずつ残しましょう。

アプリを作ること自体が、目的になっていないか。作るのは楽しいからこそ、ときどき価値仮説の一文へ戻ります。迷ったら「この機能で、どの困りごとが減る?」が合言葉です。

AIと一緒に直す

道具がそろったので、章の冒頭をやり直しましょう。あのとき渡したのは機能リストでした。今度は、困りごとを渡します。

あなた → AI

文化祭の模擬店(飲み物と焼き菓子、現金のみ、屋外テント)の相談です。シフト担当の部員が、行列をさばきながらの会計で、暗算ミスとお釣りの間違いをしてしまいます。この困りごとの解決策を、アプリを作る以外の方法も含めて5つ挙げてください。それぞれの弱点も添えてください。まだ何も作らないでください。

AI → あなた

会計ミスを防ぐ解決策を5つ挙げます。

1. 価格の早見表を印刷して掲示する——品数や割引が増えたと作り直しが必要です
2. 電卓を常備する——打ち間違いは残り、お釣りの計算は別に必要です
3. 会計係を固定して2人体制にする——ピーク時の人手を確保できるかが課題です
4. 既存のレジアプリを導入する——高機能ですが、有料プランや初期設定の手間があります
5. 店専用の簡単な会計アプリを自作する——作る手間はかかりますが、店のルールに合わせられます

どれを選ぶかは、割引の予定や当日の人員体制によって変わります。決まっている条件があれば教えてください。

冒頭のログと見比べてください。同じAIです。機能リストを渡したときは、黙って全部作りました。

困りごとを渡したら、選択肢と弱点を並べて、条件を聞き返してくる相談相手に変わりました。AIを黙った作業係にするか、考える相棒にするかは、あなたが渡す言葉が決めます。

なお、5つの案の中身は、あなたの画面では違うはずですが、それでかまいません。「アプリ以外の案が混ざって出てくること」が確認できれば、この依頼は機能しています。もしAIが相談に乗らず、いきなり作りはじめてしまったら、依頼の最後の「まだ何も作らないでください」の一行を思い出してください。この「頼み方」の話は、第4章でまとめて鍛えます。

別の条件でも動くか試す

この章の流れ——聞く、分ける、一文にする、作らない案と比べる——は、題材を選びません。会社なら、会議室が取れない、日報を書くのが面倒、月末の経費の記録が合わない。学校なら、部費の管理、実験ノートの整理。同じ型がそのまま使えます。

経理担当の同僚は、月末の経費精算で、レシートの転記ミスに困っている。撮影するだけで記録が残るなら、助かるはずだ。

うまくできたかどうかの目安は、模擬店の例と似ていることではありません。あなたが書いた一文を読んだ人が「ああ、それは困るね」と言ってくれること。そして、あなた自身がその現場を見に行けることです。

というわけで、次はあなたの番です。あなた自身の題材を、演習で探します。ここで選んだ題材は、第3章以降も「もうひとつの題材」として、模擬店と並走します。

選ぶときの基準は3つ。頻度(どのくらいよく起きるか)、痛さ(起きたとき、どのくらい困るか)、そして自分が現場を確かめられるかです。最初の題材は、利用者に会うか連絡を取り、作った後も試してもらえるものにすると進めやすくなります。

図2-5では、候補を頻度と痛さで並べ、よく起きて痛いものを、現場を確かめられるかで1つに絞る流れを見てください。

図 2-5

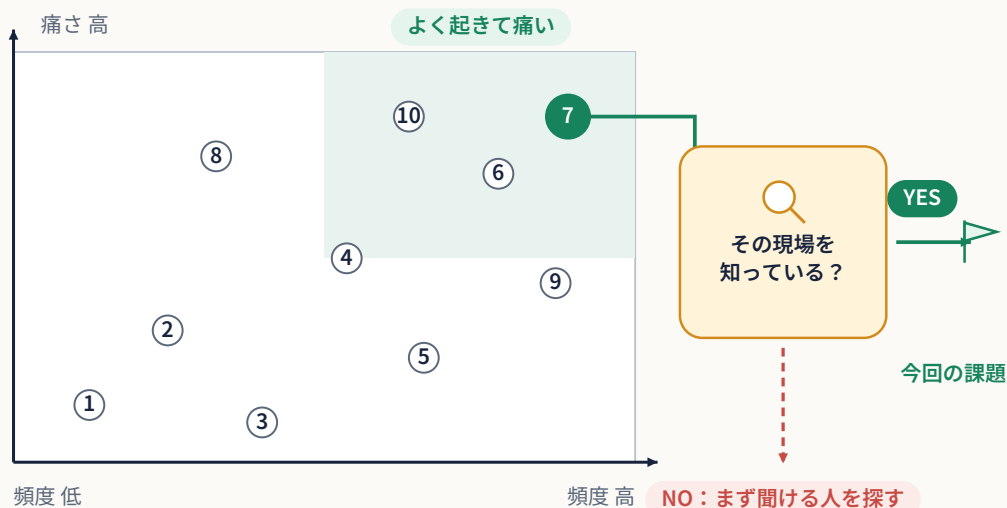


図2-5 課題候補を選ぶ3条件

演習2 宝(困りごと)の探索

- ミッション: 身の回りの不便を10個集め、アプリで解く価値のあるものを1つ選び、課題定義書を書く
- 時間の目安: 60分(探索30分+選定15分+課題定義書15分)
- クリア条件:
 1. 不便リストが10個ある(自分が直接見たか、体験したものであること)
 2. 10個それぞれに「誰が」「どの場面で」の1行が付いている
 3. 選んだ1つに、頻度・痛さ・現場を知っているか、の3つの評価が書いてある
 4. 価値仮説が型(〈誰〉は〈場面〉で〈困りごと〉に困っている。〈解決〉なら助かるはずだ)の通りに書けている
 5. 「作らない場合の代替りの手段」を1つ以上、検討した跡がある
- つまづいたら:
 - 10個出ないときは、昨日1日を朝から順に思い出す。「小さく舌打ちした瞬間」が宝の在りかです
 - 自分の不便が見つからないときは、家族や友人の困りごとでもかまいません。ただし「15分のインタビューを頼める相手」がいるものに絞ってください
 - 課題と解決策が混ざっていないか不安なら、AIに「この文は困りごとですか、解決策ですか」と判定させてみる

- ・ 検討の結果「作らない」が正解になったら、それも前進です。解決手段を1行で記録し、不便リストの次の候補で選定からやり直してください
- ・ 裏ダンジョン: 選ばなかった9個から「アプリ以外で解決できるもの」を3つ選び、解決手段を1行ずつ書く。「作らない」の練習になります
- ・ 冒険の書に記録: 不便リスト10個+課題定義書

(授業向け: ペアで不便リストを交換し、「それは課題? 解決策?」を指摘し合ってから選定に進む)

課題定義書の書式は、この章の材料がそのまま項目になっています。

図 2-6

課題定義書・記入例

誰が
初めて店番する1年生

利用場面
行列中・片手はトング

困りごと
複数商品の合計とお釣りを暗算し、ミスする

原因候補
行列中の暗算
まだ原因は未確定

確かめた方法
先輩への聞き取り
次は予行演習

選定: 頻度 高 | 痛さ 高 | 現場を知る YES

価値仮説
計算を任せれば会計の負担が減るはず
検証: 予行演習で操作と検算を比べる

比較案: 早見表・電卓

自作で割引支援を検証

空欄版

後章でも参照

なぜ選び、なぜ作るのかを一枚で追える

図2-6 課題定義書の1枚シート

この章のまとめ

- ・ 機能は解決策の部品にすぎない。機能リストを渡せばアプリはできてしまうが、できることと役に立つことは別である
- ・ 課題(困りごと)・原因・解決策は別のもの。同じ課題でも、原因が違えば効く解決策は変わる
- ・ 原因は机の上で当てず、観察と、過去の具体的な出来事を聞く短いインタビューで確かめる。聞いた事実と、考えた原因候補は分けて記録する
- ・ 価値仮説は「〈誰〉は〈場面〉で〈困りごと〉に困っている。〈解決〉なら助かるはずだ」の一文で書き、まだ確かめ終わっていない仮説として扱う

- アプリは解決策の候補の一つ。紙・電卓・既存の道具・人の工夫で足りるなら、作らないのが正解である

理解度チェック

1. 「会計でミスが起きる」「暗算でお釣りを計算している」「会計アプリを作る」は、それぞれ課題・原因・解決策のどれに当たりますか。
2. 2.6で立てた価値仮説(計算をアプリに任せられるなら、会計の暗算ミスに困らずに済むはず)を、文化祭の当日より前に確かめる方法を1つ挙げてください。
3. 紙の早見表と電卓で、会計ミスが十分に減りました。それでも「アプリを作らなかったのは失敗だ」と言えますか。理由も答えてください。

解答のめやす

1. 順に、課題・原因(の候補)・解決策です。「会計アプリを作る」は困りごとではなく、解決策の一つです(→2.3)
2. 友達に客の役を頼んで会計の予行演習をし、同じ注文で、計算のやり直しや操作の迷いが減るかを比べる(→2.4、2.6)
3. 言えません。ゴールは困りごとが減ることで、アプリはそのための手段です。ほかの手段で足りるなら、作らないのがいちばん安上がりな正解です(→2.7)

60秒でクエスト報告

行列中の会計ミスに注目し、聞き取りから暗算を原因候補に挙げました。紙や電卓とも比べ、店の割引に合う小さなアプリを試します。予行演習で、計算のやり直しや操作の迷いが減るかを確かめます。

——口頭試問の第1問「誰の何を解決しますか」に、選んだ理由まで添えた答えです。

レベルアップ!

Lv.2 称号: 困りごとハンター

- 手に入れた装備: ユーザー、課題、利用場面、価値仮説
- 冒険の書: 課題定義書(+不便リスト10個)

次の第3章では、「ミスを防ぎたい」という願いを、AIに渡して確かめられる決めごと——仕様へ変えます。

第3章 「こんな感じ」をアプリの仕様に変える

クエスト3: 願いを仕様に鍛える鍛冶屋

文化祭まで、あと25日。

この章のねらい

- ・ 要求と仕様を見分け、機能ではない大事な条件も仕様書き出せる
- ・ 受け入れ条件を、具体的な入力と期待する結果の形で書ける
- ・ やらないことで範囲を絞った1ページ仕様書の点検を、AIに頼める

今回のクエスト

第2章で選んだ題材は、文化祭の模擬店で使う会計アプリでした。「会計ミスが減らしたい」という一文を、今日は作って確かめられる決めごとへ変えます。

ミッションは、このアプリをAIに「正しく」頼めるようになること。章の終わりには、1ページ仕様書という、この先ずっと使える武器が手に入ります。

まずAIに作らせてみよう

第2章で、あなたは「誰の、どんな困りごとを解決するか」を決めました。模擬店の例でいえば、こうです。

シフト担当の部員が、行列をさばきながら会計でミスをしてしまう。それを防ぎたい。

では、さっそくAIに頼んでみましょう。多くの人が、最初是这样頼みます。

あなた → AI

文化祭の模擬店で使う会計アプリを作ってください。

AI → あなた

模擬店向けの会計アプリを作りました。コーヒー300円、紅茶300円、ケーキ500円のメニューボタンがあり、押すと合計金額が計算されます。会計ボタンで確定し、履歴に残ります。

プレビューを見ると、メニューが並び、押すと合計が出ます。会計アプリの形が見えてきました。では、このまま自分の店で使えるでしょうか。

ボス出現——「言った通りに」の正体

ところが、自分の模擬店に当てはめて使ってみると、次々に困ったことが起きます。

- ・ メニューが違う(うちの店にケーキはない。チャイがある)
- ・ セット割がない(うちはドリンク+フードのセットで50円引きにする予定なのに)
- ・ タイムセールsの仕組みがない(最終日の午後は、売り切るために全品2割引にしたいのに)

- セット割とタイムセールが重なったらどうなるのか、誰も決めていない
- 店長だけが見るはずの売上合計が、誰の画面にも見えている

欲しかったものと違う。そう感じたら、元の依頼文に戻ってみます。

そこには、セット割も、タイムセールも、店長だけができる操作も書かれていません。今回のずれには、まだ伝えていない店の決めごとが含まれています。

言われていないことを、AIは自分の想像で埋めました。序章で「やさしい雰囲気にして」がAIの解釈しだいだったのと、同じ現象です。

依頼文と期待を並べると、ずれは2種類に分けられます。書いてあるのに違うこと(AIの見落とし)と、まだ決めていないことです。この章で扱うのは、後者の「まだ決めていないこと」。それをAIに渡せる形にするのが、今回のクエストです。

このボスを倒す武器が、仕様です。

3.1 要求と仕様の違い

まず、2つの言葉を区別します。

要求は、ほしい結果のことです。「会計のミスを防ぎたい」「行列を早くさばきたい」。日常語で言うようお願いごとです。

仕様は、作るものの決めごとです。「メニューは5品で、ボタンで選ぶ」「セットは50円引き」「タイムセールは2割引」。日常語で言う注文書です。

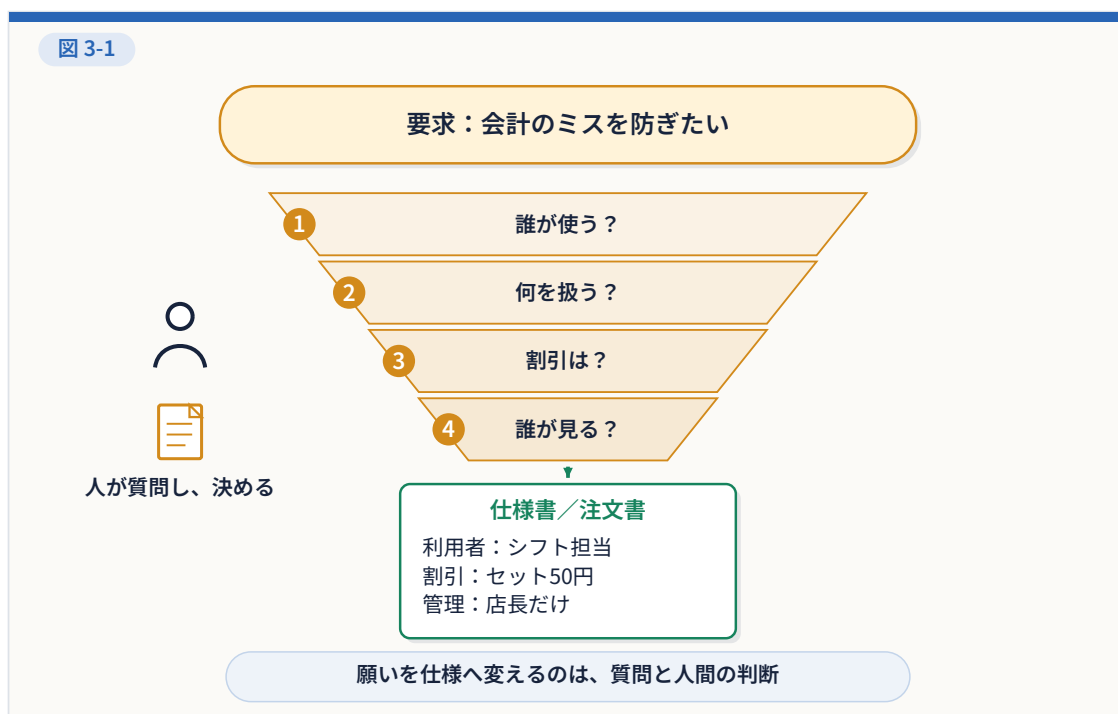


図3-1 願いごとを仕様へ変える漏斗

図3-1では、願いごとが「誰が使う?」「割引は?」といった質問を通り、仕様の決めごとに変わる流れを追ってください。

仕様は、作るためにも、できたものを確かめるためにも使います。「750円になる」と書いてあれば、画面に740円と出たとき、何が違うかを指させます。注文書は、検査の物差しにもなるのです。

※厳密に言う: ソフトウェア工学では、この変換を要求定義・要件定義と呼び、それだけで一冊の本になる分野です。本書では「1ページ仕様書が書ける」ことをゴールにします。

3.2 機能要求と「機能ではない大事な条件」

仕様に書くことは、大きく2種類あります。

1つめは何かができるか(機能)。「メニューを選べる」「合計を計算する」「履歴が残る」。

2つめが見落とされがちです。機能ではないけれど大事な条件。模擬店の例で言えば――

- 行列があるので、アプリへの入力から金額確認まで10秒以内を目標にする（現金や商品の受け渡し時間は別に測る）
- 使うのは初めてシフトに入る1年生。説明なしで使えてほしい
- 屋外のテントで使う。日光の下でも見える画面がいい
- 現金のみ。しかも1円玉のお釣りを扱いたくない(小銭を数えていると行列が伸びるから)

こうした条件は、機能の一覧には現れません。でも、これを伝えないと「正しく動くのに、現場で使えないアプリ」ができあがります。とくに最後の「1円玉」は、このあと3.6で大仕事をします。覚えておいてください。

※厳密に言う: 速さや操作のしやすさは非機能要求、現金のみ・端数の扱いは業務ルールとして整理できます。分類名よりも、測り方や現場の制約まで仕様に書くことが大切です。

3.3 ユーザーストーリーとユースケース

仕様を考える道具を2つ紹介します。どちらも書式はゆるくてかまいません。

ユーザーストーリーは、次の型の一文です。

〈誰〉として、〈何〉をしたい。なぜなら〈理由〉。

模擬店なら――

- シフト担当として、商品を選んで会計し、押し間違いを取り消したい。なぜなら行列の中でも正しく記録したいから。
- 店長として、タイムセールを自分の判断ですぐ始めたい。なぜなら「売れ残りそう」と気づいた瞬間に動きたいから。
- 店長として、時間帯ごとの売上を見たい。なぜならタイムセールが効いたのか知りたいから。

〈誰〉が2種類出てきたことに注目してください。シフト担当と店長では、やりたいことも、やってよいことも違います(セールを始めるのは店長だけ、のはずです)。この「誰」の区別は、第9章(権限)で大きな意味を持ちます。

ユースケースは、操作の物語です。うまくいく流れだけでなく、うまくいかない分岐も書きます。

客が注文する → 担当がメニューを押す → 合計が出る → 現金を受け取る → 会計ボタン
 分岐A: 客が注文を変えた → 選んだ品を取り消せる?
 分岐B: 会計の途中でタイムセールが始まった → この客はセール価格? 通常価格?

分岐を考えはじめると、書くべき仕様が自分で見えてきます。「あ、取り消し機能が要るんだ」「セールの切り替わりの瞬間を決めていなかった」という具合です。

ひとつ注意。ストーリーもユースケースも、各3〜5個で止めてください。全部を書き尽くそうとすると、仕様書づくりが文化祭より大変になります。

3.4 画面一覧と画面遷移

次に、アプリを画面の単位で考えます。模擬店の会計アプリなら、画面は3つで足ります。

1. 注文画面(メニューを選び、合計を出す)
2. 履歴画面(今日の会計の一覧。取り消しもここ)
3. 店長画面(売上の集計と、タイムセールの開始・停止。店長だけが見る)

画面から画面への移動を画面遷移と呼びます。矢印でつなぐだけの図で十分です。

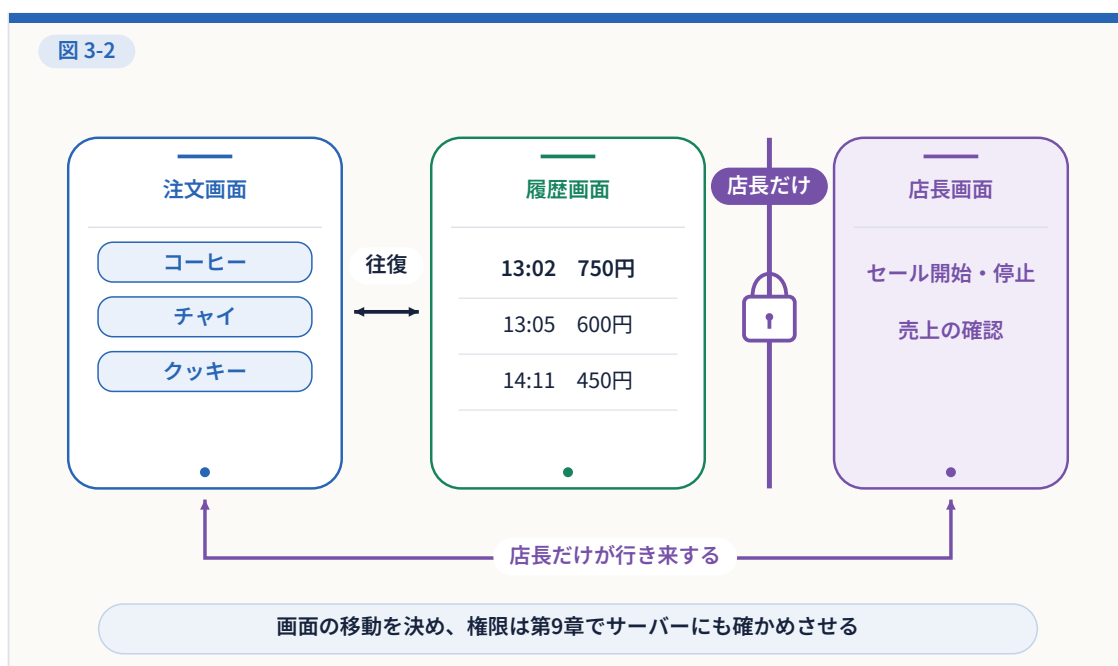


図3-2 3画面の移動と権限

図3-2では、画面どうしをつなぐ矢印と、店長の画面にだけ付いた「店長だけ」の境界を見てください。

▼コーナー「先輩エンジニアならここを見る」

画面数を数えると、作る範囲を話し合いやすくなります。ただし、画面の数だけでは仕事量は決まりません。履歴の取り消し、通信の失敗、店長の権限など、各画面に必要な動作も合わせて見ます。

3.5 入力するデータ、表示するデータ

画面が決まったら、そこを流れるデータに名前を付けます。難しく考えず、3列の表にするだけです。

入力するもの	記録されるもの	表示するもの
メニューの選択、数量	注文の内容	選択中の品と小計
会計ボタン	会計の時刻、確定した合計、適用した割引率	合計金額と割引の内訳
(店長)セール開始・停止、割引率	セールの時間帯と割引率	「セール中」の表示
(店長)集計ボタン	—	日ごと・時間帯ごとの売上

データに名前が付くと、AIとの会話が正確になります。「あれがうまく出ない」ではなく「履歴画面に、適用した割引率が出ない」と言えるからです。

2行めの「適用した割引率」に注目してください。会計を確定したときの率と金額を記録すれば、後でセール設定を変えても、当時の会計をたどれます。3.3で店長が知りたがっていた「タイムセールが効いたのか」も、この記録があれば比べられます。あとで何を知りたいかから逆算して、記録する項目を決めるのがコツです。

※厳密に言うとう: 売上の変化だけで、セールの効果を断定することはできません。記録は、効果を比べるための材料です。

3.6 業務ルールと計算ルール

ここが会計アプリの心臓部です。あなたの店のお金のルールを、日本語で1行ずつ書き出します。

- メニューは5品(チャイ400円、コーヒー300円、クッキー200円、ドーナツ250円、ラムネ150円)
- ドリンクはチャイ・コーヒー・ラムネ。フードはクッキー・ドーナツ。セット判定には、この分類を使う
- セット割: ドリンク1点+フード1点の組で50円引き
- タイムセール: 店長が開始すると全品2割引。開始・停止・割引率の変更は店長だけ
- 店長画面で、日ごと・時間帯ごとの売上を集計する
- 金額は、会計を確定するときに有効なセール状態で決め、適用した率と金額を記録する。操作の途中で金額が変わったら、確定前にもう一度確認する。確定済みの金額は、後のセール変更で書き換ええない

さて、ここで問題です。セット割とタイムセールが重なったら、どうしますか?

- 案A: 併用する——小計からセット割50円を引き、その残りを2割引
- 案B: 有利なほうだけ——セット割と、タイムセール割引額の大きいほうだけを適用

実は、案Aと案Bで合計金額が変わります。コーヒー2つとクッキー1つ(小計800円)なら、案Aは $(800-50) \times 0.8 = 600$ 円。案Bはセット割50円よりセール割引160円のほうが大きいので640円。40円違います。

AIには、案ごとの計算結果や長所・短所を出してもらえます。しかし、どちらを採るかは店の決定です。本書では案Aの併用に決め、仕様書にもそう書きます。

もうひとつ、決めごとが要ります。3.2の「1円玉を扱いたくない」を思い出してください。いまは2割引だからきれいな数字ですが、仕様には「割引率の変更は店長だけ」と書きました。

もし当日、店長が「3割5分引きだ!」と言い出したら? セット割を引いた後の金額が550円なら、 $550 \times 0.65 = 357.5$ 円になります。現金では払えない端数を、どう扱いますか。

本書では——割引後の端数は、10円未満を切り捨てる(357.5円→350円)と決めます。請求額が10円単位になるので、1円玉や5円玉のお釣りを避けられます。切り捨てる分だけ売上額も変わるため、店として決めておくルールです。

現場を知っているあなただからこそ書ける1行です。AIは、あなたの店のレジに1円玉がないことを知りません。

計算のやり方を決め、AIに実装を頼み、具体例で一致を確かめる。この3つをつなげることが、会計アプリではとくに大切です。

決めたルールは、そのままコードに対応します。見てみましょう(読めなくて大丈夫です。日本語の1行とコードの1行が対応している、という一点だけ見てください)。

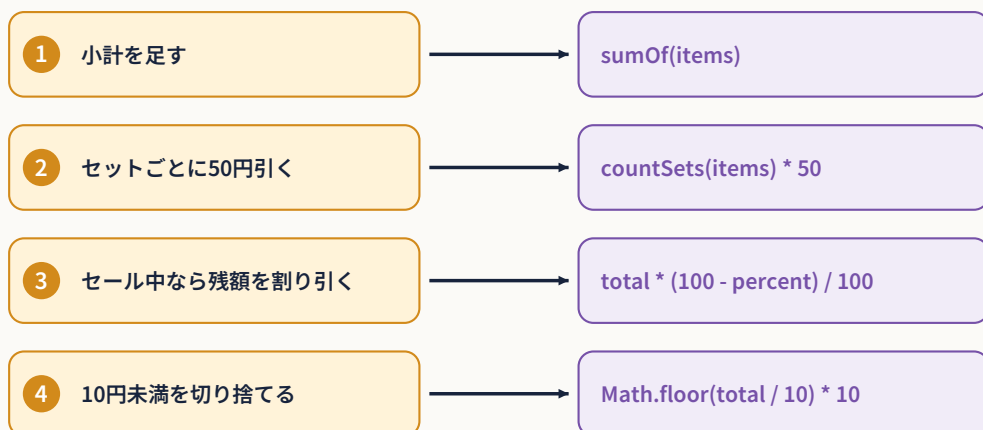
役割ラベル: 計算

```
[javascript]
// 合計 = (小計 - セット割) にタイムセール割引(案A: 併用)。10円未満は切り捨て
function calcTotal(items, saleRate) {
  const subtotal = sumOf(items);           // 小計
  const setOff = countSets(items) * 50;    // セット割: 1セットにつき50円引き
  let total = subtotal - setOff;
  const percent = Math.round(saleRate * 100); // 割引率を整数の%にする(0.2→20)
  if (percent > 0) total = total * (100 - percent) / 100; // タイムセール(率は店長が設定)
  return Math.floor(total / 10) * 10;       // 端数: 10円未満切り捨て(1円玉を扱わない)
}
```

図 3-3

人が決める日本語仕様

AIが書くコード



番号と線を追えば、仕様のどこがコードになったか分かる

図3-3 仕様の1行とコードの1行

ここでは `sumOf` が商品の小計、`countSets` が成立するセット数を返すものとしています。こうした関数の読み方は、第6章で扱います。いまは、日本語のルールとコードの行を対応させて追えれば十分です。

ただし、`percent` の行だけは、仕様書に対応する1行がありません。コンピューターの小数計算で生じる、ごく小さな誤差を避けるための工夫です。なぜ必要なのかは、第6章で読み解きます。

3.7 受け入れ条件と「やらないことリスト」

仕様書の仕上げに、2つのリストを付けます。

1つめは受け入れ条件。「この動きなら、決めた仕様を満たす」の一覧です。コツは、入力と期待する結果を具体的に書くこと。数値だけでなく、誰が操作できるか、取り消した記録をどう扱うかも条件になります。

- 通常時、コーヒー2つとクッキー1つで、合計750円になる(セット割50円引き)
- 同じ注文が、タイムセール(2割引)中なら600円になる(セット割と併用・案A)
- タイムセールの開始・停止と割引率の変更は、店長だけができる(画面を隠すだけでなく、権限の確認が必要)
- 会計を取り消すと、通常の履歴一覧と売上集計から除かれる。元の記録は取り消し済みとわかる形で残す
- 割引後に10円未満の端数が出たら、切り捨てられる

受け入れ条件は、あとの章で二度おいしい働きをします。第12章では、そのままテストの種になります。セールが始まる「ちょうどの瞬間」の注文はどっちの価格?—という意地悪な問いもそこで扱います。そして、口頭試問の「正しく動くことをどう確認しましたか」への答えにもなります。

2つめはやらないことリスト。今回は作らない、と宣言するものです。

- QRコード決済(現金のみ)
- 在庫管理
- 事前予約

作らないと決めるのは、負けではなく作戦です。やらないことを決めると、「今回作る範囲」の線が引けます。この範囲のことをスコープと呼びます。スコープの線を引かないと、AIとの開発は際限なく広がります(AIは頼めば何でも足そうとするからです)。

▼コーナー「このプロンプトの何が悪い?」

「いい感じの会計アプリを作って。決済とかも全部対応で」

悪い点は3つ。①「いい感じ」は全部AIの想像で埋まる ②「決済とか」の「とか」は範囲が無限 ③完成の条件がないので、いつまでも終わらない。この章を終えたあなたなら、もう書き直せるはずです。

3.8 MVPとして範囲を小さくする

第1章で出てきたMVPを、ここで具体化します。確かめたいのは、「部員が行列の中でも、計算に迷わず会計できるか」。その問いに必要な機能を選びます。

本書では、機能を3つのまとまりに絞ります。3つという数は普遍的な正解ではなく、今回の範囲を小さく保つ目安です。

模擬店の例なら――

1. 注文と合計計算(セット割・タイムセール込み)
2. 会計の記録と取り消し
3. 店長画面(セールの開始・停止と、日ごと・時間帯ごとの集計)

「お気に入りメニュー」「売上グラフ」「かわいいアニメーション」は、最初の検証では後回しにします。必要だとわかったら、その理由を添えて次の候補に戻せます。

以上の材料を、1枚にまとめたものが1ページ仕様書です。

図 3-4

模擬店会計／1ページ仕様書	
目的・利用者	初めて店番する部員と店長。行列中の暗算ミスを減らす。
ストーリー：3件	① 担当：商品を選んで会計し、押し間違いを取り消す ② 店長：売れ残りに気づいたら、セールを開始する ③ 店長：時間帯ごとの売上を見て、セールの結果を知る
画面：3つ	注文 ⇄ 履歴 / 注文 ⇄ 店長（店長だけ）
記録するデータ	注文の内容・会計時刻・合計・適用したセール率 取り消し済みの印。通常履歴と集計からは除く。
メニュー：5品	飲物：チャイ400円／コーヒー300円／ラムネ150円 食物：クッキー200円／ドーナツ250円
計算の順番	小計 → 1セットごと50円引き → セール中は2割引 最後に10円未満を切り捨て。確定時の率と額を記録。
受け入れ条件：5件	① コーヒー2＋クッキー1＝通常750円 ② 同じ注文＝2割引セール中600円 ③ セールの開始・停止・率変更は店長だけ ④ 取り消し済み記録を保持し、通常履歴・集計から除外 ⑤ 割引後の10円未満を切り捨て
作らない：3件	決済連携 / 在庫管理 / 事前予約

仕様を一枚で共有し、詳しい操作・権限・テストへつなぐ

図3-4 1ページ仕様書の全体像

図3-4では、この章で決めてきた項目が、1枚のどの欄に入るかを確かめてください。

AIと一緒に直す

では、この仕様書を持って、もう一度AIのところへ行きましょう。今度の依頼はこうなります。

あなた → AI

文化祭の模擬店の会計アプリを作ってください。仕様は次の通りです。

【誰が使う】シフト担当の部員(初めての1年生も)と店長

【画面】注文画面/履歴画面/店長画面(店長だけが見る)の3つ

【メニュー】チャイ400円、コーヒー300円、クッキー200円、ドーナツ250円、ラムネ150円

【計算】ドリンク1点+フード1点のセットで50円引き。タイムセール中は全品2割引で、セット割と併用(小計-セット割のあとに2割引)。割引後の10円未満は切り捨て

【セールの操作】開始・停止・割引率の変更は店長だけ

【受け入れ条件】①通常時、コーヒー2つとクッキー1つで750円 ②同じ注文が2割引セール中は600

円③セール操作は店長だけ④取り消し済みの記録を保持し、通常履歴と集計から除外⑤割引後の10円未満は切り捨て

【やらないこと】決済連携、在庫管理、予約

まず、この仕様で不明な点や、決まっていないことがあれば質問してください。

AI → あなた

仕様を確認しました。実装の前に3点、確認させてください。

1. セット割は、1回の会計で複数セット成立しますか(ドリンク2点+フード2点なら100円引き?)
2. タイムセールは、店長が停止し忘れた場合、自動で終わりますか。それとも停止されるまで続きますか
3. 確定前の注文変更と、確定後の会計取り消しは、どちらも必要ですか

見てください。AIが作る前に、質問を返してきました。しかも鋭い質問です。複数セットの扱いも、停止し忘れも、たしかにあなたはまだ決めていません(2番は文化祭当日、本当に起きそうです)。

冒頭の短い依頼では、店の決めごとが見えていませんでした。仕様書を渡し、不明点を尋ねるよう頼むと、決まった部分と未決定の部分を一緒に点検できます。

これが、この章のビフォーアフターです。同じAI、同じあなた。変わったのは、渡した紙が「願いごと」から「注文書」になったことだけです。

別の条件でも動くか試す

この1ページ仕様書という型は、模擬店以外でもそのまま使えます。サークルの部費管理、ゼミの本の貸し出し、職場の備品予約。第2章で選んだあなた自身の題材で、同じ1ページ仕様書を書いてみてください。

うまく書けたかどうかは、質問が返ってくるかだけでは決まりません。別の人が仕様を読み、同じ入力から同じ結果を予想できるかを確かめます。AIには未決定の点を探してもらい、あなたが答えた内容を仕様書へ書き戻しましょう。

演習3 会計アプリを仕様化する

- ミッション: 会計アプリ(またはあなたの題材)の1ページ仕様書を書き、AIに渡す
- 時間の目安: 60分
- クリア条件:
 1. 1ページ仕様書の全項目が埋まっている
 2. 受け入れ条件が5つ以上あり、少なくとも2つに具体的な数字が入っている
 3. 割引(または、あなたの題材の「重なったらどうする?」問題)について、自分で決めた1行がある
 4. やらないことリストが3つ以上ある
 5. AIに未決定の点を点検させ、質問への回答または「今回は決めなくてよい理由」を仕様書に書き戻した
- つまづいたら:

- 全項目を自力で埋めなくてよい。空欄のまま渡して「空欄について質問してください」とAIに頼むのも正しい使い方
- 計算ルールで迷ったら、電卓で1例だけ手計算してみる。手で計算できないルールは、まだ決まっていないルール
- (授業向け)ペアで仕様書を交換し、相手の仕様書の「AIが想像で埋めそうな穴」を3つ探し合う
- 裏ダンジョン: 割引の併用ルール(案A・案B)を両方AIに実装させ、同じ注文(コーヒー2つとクッキー1つ)で合計がいくら違うか確かめる
- 冒険の書に記録: 1ページ仕様書+MVP範囲表

この章のまとめ

- 要求は「ほしい結果」(願いごと)、仕様は「作るものの決めごと」(注文書)。仕様は、作るだけでなく、できたものを確かめる物差しにもなる
- 仕様には機能だけでなく、速さ・使う人・使う場所・お金の扱いといった「機能ではない大事な条件」も書く。書かなければ、正しく動くのに現場で使えないアプリになる
- 割引の併用や端数の扱いのように、答えが一つに決まらないルールは、AIに比べてもらえても、決めるのは店である。本書では併用(案A)と、10円未満の切り捨てに決めた
- 受け入れ条件は、入力と期待する結果を具体的に書く。「コーヒー2つとクッキー1つで、通常750円、2割引のセール中は600円」と書けば、そのまま確かめに使える
- やらないことを決めてスコープの線を引き、確かめたい問いに必要な機能だけをMVPとして残す

理解度チェック

1. 「会計のミスを防ぎたい」と「セットは50円引き」は、それぞれ要求と仕様のどちらに当たりますか。
2. タイムセール(2割引)中に、チャイ1つとクッキー1つを会計する場面を、受け入れ条件の形(入力と期待する結果)で1行に書いてください。金額は本書の計算ルールで求めます。
3. AIが「売上グラフも付けましょうか」と提案してきました。役に立ちそうなので、そのままMVPに入れてよいと言えますか。

解答のめやす

1. 「会計のミスを防ぎたい」は要求(願いごと)、「セットは50円引き」は仕様(注文書)です(→3.1)
2. 「タイムセール(2割引)中、チャイ1つとクッキー1つで、合計440円になる(セット割と併用・案A)」。小計600円からセット割50円を引いて550円、その2割引で440円です。10円未満の端数は出ません(→3.6、3.7)
3. すぐには言えません。MVPは、確かめたい問い(部員が行列の中でも計算に迷わず会計できるか)に必要な機能に絞ります。売上グラフは最初の検証では後回しにし、必要だとわかったら理由を添えて次の候補に戻します(→3.7、3.8)

60秒でクエスト報告

画面は注文・履歴・店長の3つです。合計は、小計から成立したセットの割引額を引き、セール中はさらに2割引。最後に10円未満を切り捨てます。検算はコーヒー2つとクッキー1つで、通常750円、セール中600円です。セール操作は店長だけ。取り消し済みの会計は通常履歴と集計から除き、元の記録は残します。この5つの受け入れ条件を検証し、決済連携・在庫管理・予約は今回の範囲から外します。

——気づいたでしょうか。これはそのまま、口頭試問の第1問「このアプリは、誰の何を解決しますか」への答えになっています。

レベルアップ!

Lv.3 称号: 仕様を書ける人

- 手に入れた装備: 要求、仕様、ユースストーリー、ユースケース、画面遷移、受け入れ条件、スコープ
- 冒険の書: 1ページ仕様書+MVP範囲表

次の第4章では、この仕様書をAIへの「仕事の依頼」に変えます。どこから作り、何を確かめ、次へ進むか。段取りを決める章です。その前に——AIの3つの質問が、まだあなたの答えを待っています。

第4章 AIへの指示を「プロンプト」から「仕事の依頼」に変える

クエスト4: 仲間に仕事を頼む技

文化祭まで、あと23日。

この章のねらい

- ・ 目的・利用者・制約・完成条件の4項目で、AIへの依頼を書ける
- ・ 実装の前に計画と仮定・不明点を出させ、仕様とのずれを見分けられる
- ・ 作業を1回1変化のタスクに割り、変えないルールを指示書に書ける

今回のクエスト

第3章で、あなたは1ページ仕様書を手に入れました。そしてAIは、作る前に3つの質問を返してきたところでした。この章のミッションは、その仕様書をAIへの仕事の依頼に変えることです。修正する場所と、確かめる結果を決めてから頼む。小さな段取りで、変更を追えるようにします。

章の終わりには、AI開発ブリーフ、タスクリスト、プロジェクト指示書という3点セットが手に入ります。この先の全章で、AIに何かを頼むたびに使う道具です。

まずAIに作らせてみよう

最初の仕事は、AIの3つの質問に答えることです。①セット割は1回の会計で複数成立するか ②タイムセールは自動で終わるか ③確定前の注文変更と確定後の会計取り消しをどう扱うか。決定した内容を仕様書に書き戻します。

決めて、伝えます。

あなた → AI

3つの質問に答えます。

1. セット割は、1回の会計で複数成立します。ドリンク2点+フード2点なら100円引きです。
2. タイムセールは自動では終わりません。店長が停止するまで続きます。ただし停止し忘れが心配なので、仕様を1つ足します——セール中は、店長画面に「セール中」を大きく表示してください。
3. 確定前は選択中の商品を取り消せます。確定後は会計全体を取り消せます。確定後の取り消しは、元の記録を取り消し済みとして残し、通常の履歴一覧と売上集計から除きます。

これで仕様書が3項目育ちました。チャットの返答だけで終わらせず、仕様書の同じ項目も更新します。

とくに2番に注目してください。「停止し忘れたら?」というAIの質問が、「セール中を大きく表示する」という新しい仕様を生みました。AIの質問に答えることは、仕様を育てることです。

さて、仕様は固まりました。意気揚々と「この仕様で、アプリを全部作ってください」と頼むと、AIは数分で組み上げてきます。注文画面、履歴画面、店長画面。セット割も、タイムセールも、取り消しも、集計もある。

プレビューでコーヒー2つとクッキー1つを押すと、750円。セールを始めると600円。第3章で書いた条件を、具体的な操作で確かめられるようになりました。

ボス出現——直すたびに、別の場所が壊れる

ところが、ここからが本番でした。使い込むうちに、細かい注文が出てきます。あなたはAIにこう頼みました。

あなた → AI

履歴画面の取り消しボタンが小さくて押しにくいので、直してください。

ボタンは大きくなりました。ところがその晩、集計を見ると様子が変わります。取り消したはずの会計が、売上に残っています。

頼んで直してもらおうと、今度は店長画面の「セール中」表示が消えました。それも直してもらったら、検算で750円になるはずの注文が740円になっています。原因はまだわかりません。小計、セット数、割引率を順に比べる必要があります。

一つ直したのに、別の結果まで変わった。この連鎖を止めるには、二つの点を見直します。

1つめは、変更の範囲です。ボタンを大きくするだけなら、変えるのは見た目だけで足ります。取り消しの処理は集計ともつながっているのだから、そこまで書き換えると影響が広がります。何を変更したかが追えれば、予想外の影響も調べやすくなります。

2つめは、変更後の確認です。「押しやすくなったか」に加え、750円/600円の検算や、取り消し後の集計も確かめます。変えてはいけない条件を依頼に添え、実際に守られたかを見ます。

これがこの章のボスです。倒し方は、うまい言い回しを探すことではありません。仕事の頼み方——段取りです。ここから、その道具を1つずつ手に入れます。

4.1 良いプロンプトより、良い前提情報

プロンプト(prompt)とは、AIへの依頼文のことです。日常語で言うと、そのまま「依頼文」です。

依頼の言い回しを考えると、まずAIが何を知っている状態で受け取るかを確かめます。仕様書、いまのコード、今回変えたい場所。同じ一文でも、参照できる材料が違えば、提案も変わります。

このAIに渡っている前提情報のことを、コンテキスト(context)と呼びます。日常語で言うと「AIに渡す前提情報」。自分に引きつけた問いはこうです——「いまのこのAIは、うちの店の何を知っていますか」。

新しく開いた会話で、第3章の仕様書が自動的に参照されるとは限りません。「今回参照する仕様書はこれです」と示し、計算ルールを短く説明してもらいましょう。説明が仕様書と合っていれば、共通の前提を確認できます。

図4-1は、同じ「取り消しボタンを大きくして」という依頼が、前提の有無でどう分かれるかを並べたものです。

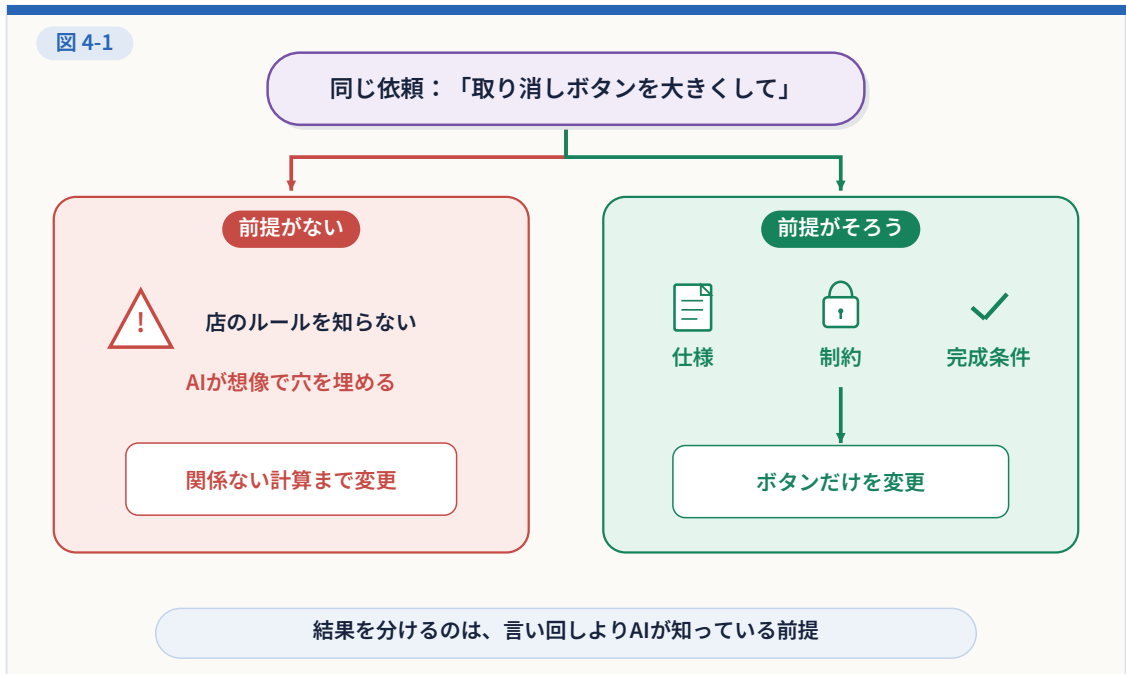


図4-1 同じ依頼でも前提で結果が変わる

だから、依頼文を磨く前に確認することが1つあります。いまのAIに、仕様書は渡っていますか。

※厳密に言う：前の会話の引き継ぎ方は、ツールや設定によって違います。また、一度に処理できる情報量には上限があり、長い会話は要約などを経て引き継がれる場合があります。過去に伝えたことが、常に同じ細かさで参照されるとは限りません。大事な前提の置き場所は、4.7で用意します。

4.2 目的、利用者、制約、完成条件を伝える

とはいえ、毎回仕様書をまるごと貼って「よろしく」でも足りません。1回の依頼に必要な前提を、型にして渡します。本書ではこれをAI開発ブリーフ——仕事の依頼票と呼びます。書くことは4つだけです。

- 目的: なぜやるのか(困りごとに戻る)
- 利用者: 誰のためか(シフト担当? 店長?)
- 制約: 守ってほしい条件
- 完成条件: どうなったら完了か

制約は、日常語で言うと「守ってほしい条件」。コツは、してほしいことより先に変えないでほしいことを書くことです。自分に引きつけた問い——「このアプリで、AIに勝手に触られたら一番困る場所はどこですか」。

会計アプリなら、計算ルールです。メニュー5品と画面3つも固定です。

完成条件は、日常語で言うと「どうなったら完了か」。第3章の受け入れ条件の兄弟です。受け入れ条件がアプリ全体の合格ラインなら、完成条件は今回の依頼1件ぶんの合格ライン。数字で書くコツも同じです。

図4-2は、冒頭の取り消しボタンの依頼を、4項目に当てはめた記入例です。

図 4-2

AI開発ブリーフ／仕事の依頼票

依頼名：取り消しボタンを押しやすくする

1	目的 会計の取り消しを、忙しい店番でも押しやすくする	なぜ
2	利用者 模擬店のシフト担当者	誰
3	制約 計算ルール・メニュー5品・画面3つは変えない	変えない
4	完成条件 対象端末で誤タップなく取り消せる 検算750円／600円が変更前と一致	できた？

目的 → 利用者 → 制約 → 完成条件の4項目

図4-2 AI開発ブリーフの型

▼コーナー「先輩エンジニアならここを見る」

プロが同僚から仕事を頼まれて、最初に聞き返す言葉は決まっています。「それ、どうなってたら完了?」です。完成条件のない依頼は、人間のプロ同士でも事故になります。AIが相手でも、まったく同じです。

4.3 一度に全部作らせない——実装前に計画を出させる

まだ全体の見通しが立たない変更では、ブリーフに次の一文を添えます。

「まず計画だけ出してください。実装はまだしないでください。」

仕様書を渡して、この一文を添えます。すると、こんな返事が来ます。

AI → あなた

計画を提案します。

1. 注文画面(メニュー5品と数量、小計の表示)

2. 割引計算(セット割→タイムセール→10円未満切り捨ての順。750円/600円で検算)
 3. 履歴画面と取り消し(取り消しは集計にも反映)
 4. 店長画面(セールの開始・停止、「セール中」の大きな表示、日ごと・時間帯ごとの集計)
- この順で進めてよいですか。

計画の何がうれしいのか。作る前なら、ずれを言葉の段階で直せることです。もし計画の4に「集計は日ごとだけ」とあれば、「時間帯ごとも必要です」と先に補えます。影響の大きい変更ほど、この確認が効きます。

返ってくる計画は毎回同じではありません。順番や分け方が本と違ってても、心配は要りません。計画が仕様と食い違ってないかをあなたが確かめられれば、先に進めます。

図4-3では、左の「いきなり実装」と右の「まず計画」で、ずれに気づく時点を比べてください。

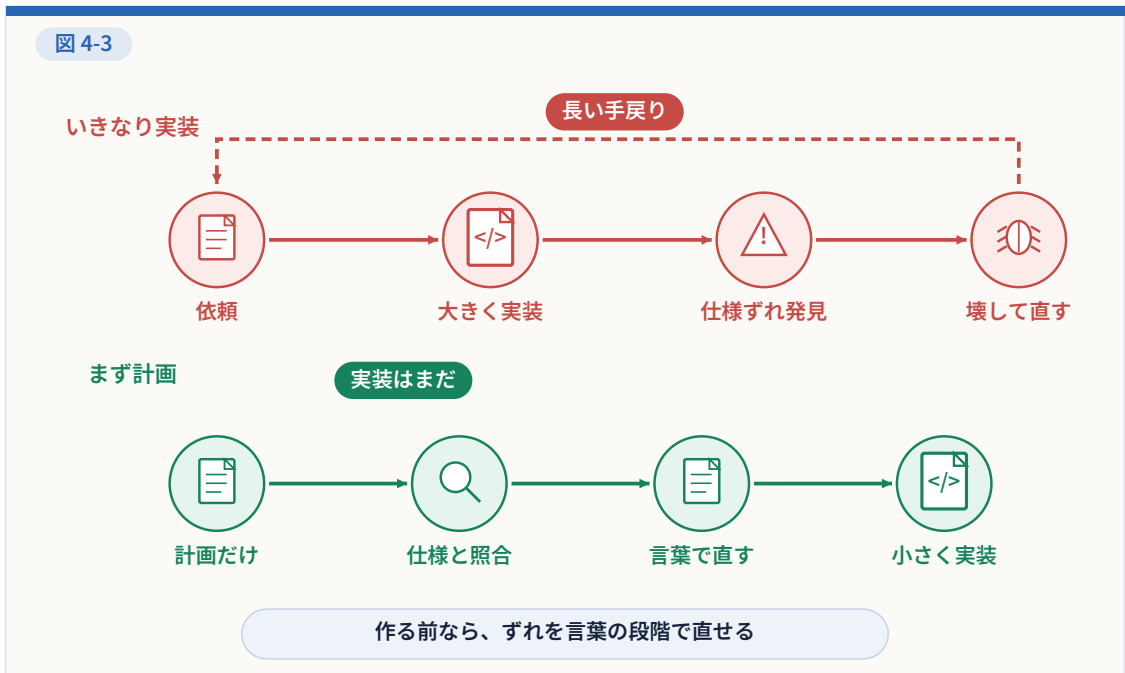


図4-3 計画が手戻りを減らす

4.4 大きな機能を小さなタスクに分ける

計画が出てても、「では全部どうぞ」に戻ってはいけません。計画を小さく割ります。これをタスク分割と呼びます。日常語で言うと「大きな仕事を小さく割る」です。

原則はひとつだけ。1回の依頼=1つの変化。

「取り消しを直して、ついでに集計も見やすくして、あと色も変えて」——これは3つの変化です。まとめて変更すると、予想外の結果が出たとき、どの変更が原因かを調べる範囲が広がります。1つずつ確かめれば、まず直前の変更から調べられます。

ちょうどよい大きさの目安も1つだけ。完成条件が1〜2行で書けること。書けないタスクは、まだ大きすぎます。

図4-4は、さっきの3つの変化をまとめた依頼を3つのタスクに割り、それぞれに完成条件を付けた例です。



図4-4 タスクと完成条件の一覧

何回にも分けて頼むのは、まどろっこしく見えるかもしれません。でも冒頭の壊れの連鎖を思い出してください。小さく割った道のほうが、結局は早く着きます。

4.5 AIに仮定と不明点を列挙させる

第3章の終わり、AIは頼んでもいないのに3つの質問を返してきた——のではありません。仕様書の最後に「不明な点があれば質問してください」と書いたから、返ってきたのです。これも型にしましょう。

「実装の前に、あなたが置く仮定と、仕様で決まっていないことを列挙してください。」

AIは、決まっていないことを黙って想像で埋めます。第3章で見た通りです。列挙させると、想像が質問に変わります。あなたが答えると、質問が仕様が変わります。

この章の冒頭がまさにそうでした。「停止し忘れたら?」が「セール中を大きく表示」という仕様になったのです。

仮定のほうは、たとえば「履歴は新しい順に並べる、と仮定します」のような形で返ってきます。それでよければ「はい」と答えるだけです。まずいものが混ざっていたときこそ、この型の出番です。

そして大事なのは、答えた内容を仕様書に書き戻すことです。会話を読んでいない人もAIも、同じ決定を参照できるようにするためです。

4.6 AIに自分でテストさせる——実装役・テスト役・レビュー役を分ける

ここまでの型で、作る流れはだいぶ安全になりました。仕上げて、確かめる側の質を上げます。

自分で作ったものを自分で確かめると、思い込みごとと確かめてしまう——人間の話ですが、AIも似た振る舞いをします。実装した会話の流れのまま「確認して」と頼むと、「問題ありません」と返ってきがちです。そこで、役割を分けます。

- 実装役: プリーフの通りに作る
- テスト役: 壊すつもりで確かめる。検算と、意地悪な入力
- レビュー役: 仕様書と見比べて、漏れと余計な変更を指摘する

分け方は2通りあります。同じ会話で「ここからはテスト役です」と観点を切り替えるか、別の会話を開くか。別の会話には仕様書に加え、対象のコードや動かし方、必要なテストデータを渡します。仕様書だけなら検査項目は作れても、実装が合っているかは確かめられません。

あなた → AI

ここからはテスト役になってください。いま実装した割引計算を、壊すつもりで確かめてください。まず検算: コーヒー2つとクッキー1つで、通常750円、タイムセール中600円。そのあと、意地悪な注文を3つ考えて試してください。

AI → あなた

テスト役として、通常時とセール中を分けて確認しました。コーヒー2つ+クッキー1つは通常750円、セール中600円です。追加の確認は通常時で実施しました。①ラムネ1つは150円 ②コーヒー3つ+クッキー1つは、セット割1組で1,050円 ③何も選ばずに会計すると、0円の会計が履歴に記録されました。0品の扱いを決める必要があります。

この作例では、0品の会計という穴が見つかりました。「0品では会計を確定できない」と決めたら、仕様書に書き戻します。

AIの「確認した」という報告には、実行した方法と結果も添えてもらい、自分でも同じ操作を試しましょう。AIが実際に動かせない環境なら、「確認済み」とは扱わず、試す手順を出してもらいます。

役割を分けたAIたちは、あなたが率いる小さなパーティです。あなたの仕事はコードを書くことではなく、誰に、何を、どの順で頼むかを決めること。本書があなたを「開発責任者」と呼ぶのは、この意味です。

図 4-5

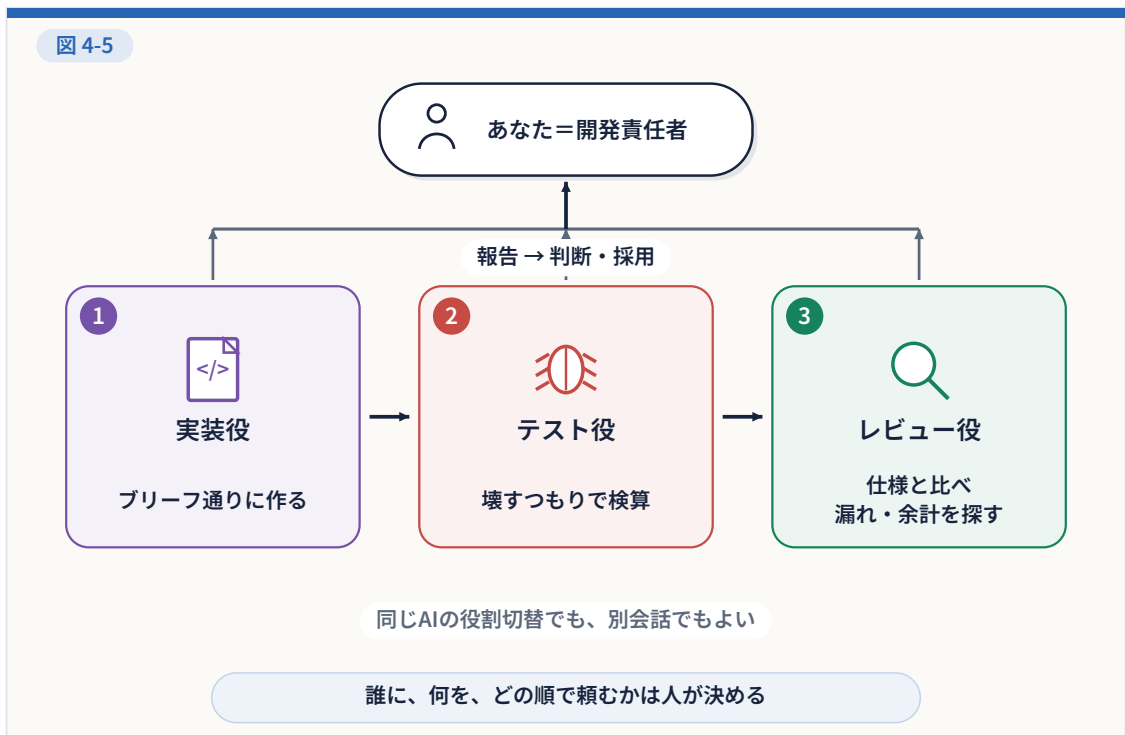


図4-5 AIチームの役割分担

※厳密に言うと: 役割を宣言したり会話を分けたりしても、検査の独立性や正しさが保証されるわけではありません。観点を増やし、仕様と実行結果を照合するための工夫です。本格的なテストの設計は第12章で扱います。

4.7 プロジェクト指示書とREADME

ここまでで、1回の依頼に添えるものが増えました。仕様書、ブリーフ、計画、役割。毎回全部を貼り直すのは大変です。そこで、毎回言わなくても守らせたいことを、置いておく場所を作ります。

それがプロジェクト指示書です。AI開発環境によっては、指定した場所の指示ファイルを読み込ませる機能があります。名前や置き場所、読み込む範囲は、使うツールの公式ヘルプで確認します。この指示ファイルに、店の共通ルールを書いておきます。次の例では、「変えないルール」と「仕事の進め方」の2つのまとまりに注目してください。

会計アプリ プロジェクト指示書

変えないルール

- メニューは5品: チャイ400円/コーヒー300円/クッキー200円/ドーナツ250円/ラムネ150円
- セット割: ドリンク1点+フード1点で50円引き(1会計で複数成立)
- タイムセール: 全品2割引。開始・停止・割引率の変更は店長画面のみ。自動終了はしない
- 割引は併用: (小計-セット割)に割引を掛け、10円未満は切り捨て
- 検算: コーヒー2+クッキー1=通常750円、セール中600円
- 取り消し済み記録は保持し、通常履歴と集計から除く。0品では会計を確定しない

仕事の進め方

- 実装の前に、計画と、仮定・不明点を出すこと
- 1回の依頼では1つの変化だけ。関係ない場所を書き換えないこと
- 変更後は検算(750円/600円)を確認して報告すること

見てください。この章で学んだ型が、そのまま指示書の中身になっています。指示書があると、依頼文は短くなります。

たとえば「指示書に従って、取り消しボタンの大きさだけを変更してください。対象端末で押し間違いなく操作でき、検算750円/600円が変わらないことを確認してください」と頼めます。報告では、実施した確認と未確認の項目を分けてもらいましょう。

もうひとつ、似た場所に **README.md** というファイルを作ります。こちらはAI向けではなく人間向け——このプロジェクトが何で、どう動かすかを書く表紙です。いまは「仕様書の要約を貼っておく場所」と覚えれば十分です。第10章と第14章で、引き継ぎの道具として育てます。

4.8 直らないときに、同じ指示を繰り返さない

最後に、それでも直らないときの話です。

直らないと、「もう一度ちゃんと直して」と送りたくなります。けれど新しい情報がなければ、AIは同じような修正を繰り返しがちです。

AIの返事は毎回すこし変わるので、同じ指示の再送は「当たりが出るまで引き直す」のと同じです。たまには当たります。けれど、なぜ当たったのかを、あなたは説明できません。

同じ症状が残るとわかったら、頼み方を切り替えます。手は3つあります。

1. 言い方を変える: 「動かない」ではなく、観察した事実を書く。「取り消すと、履歴からは消えるが、集計の数字が減らない」
2. 小さくする: タスクをさらに割る。「まず、取り消しのとき集計がどう更新されるはずか、説明だけしてください」
3. 戻す: 変更前の状態に戻して、やり直す。いつでも戻れるようにする本格的な道具は第10章で手に入ります。それまでは、うまく動いていた時点の内容を控えておくのが保険になります

原因を自分で切り分ける方法は、第11章で本格的に学びます。この章では「同じ指示を繰り返さない」——これだけ持ち帰ってください。

▼コーナー「このプロンプトの何が悪い?」

「さっきの不具合、まだ直ってないんだけど。ちゃんとして」

悪い点は3つ。①症状がない(何が、どうなる?) ②どう操作するとそうなるかがない ③「ちゃんと」はAIの想像で埋まる。第3章の「いい感じ」と同じ病気です。あなたはもう、書き直せるはずです。

AIと一緒に直す

冒頭の事故現場に戻りましょう。壊れの連鎖の始まりは、「押しにくいので直してください」の一言でした。この章の型で、同じ依頼を頼み直します。

あなた → AI

履歴画面の取り消しボタンを押しやすくしたいです。

【目的】屋外で急いでいるシフト担当が、押し間違えないため

【利用者】シフト担当(初めての1年生も)

【制約】計算ルールと、履歴・集計の中身には手を入れない。変えるのは見た目だけ

【完成条件】当日使うスマートフォンで、隣の会計を誤って押さずに取り消せる。取り消し済みの記録は保持し、通常履歴と集計から除く動作が変わらない。検算750円/600円が通ったまままず計画だけ出してください。実装はまだしないでください。

AI → あなた

計画: 履歴画面のボタンの大きさと間隔だけを変更します。計算と集計には触れません。よろしければ実装します。

(あなたの承認後)実装しました。確認結果: ボタンは拡大済み。取り消し後、履歴と集計は一致。検算750円/600円、ともに一致です。

同じ「直して」でも、今度は確認する操作と結果が決まっています。AIの報告を読み、手元のスマートフォンでも操作してみます。完成条件があれば、合っている点も、残った不具合も具体的に示せます。

失敗版と改善版の違いは、文章のうまさではありません。渡した前提と、頼んだ大きさです。

別の条件でも動くか試す

この章の型は、会計アプリ専用ではありません。ブリーフの4項目も、「まず計画だけ」の一文も、役割分けも、第2章で選んだあなた自身の題材にそのまま使えます。

AIの返事が本と同じである必要はありません。変更する範囲を理解でき、完成条件を実際に確かめられるかを目安にします。計画や質問は、そのための道具です。

章の通りに進まないときは、依頼を半分の大きさにして頼み直してください。この本で出会う困りごとの多くは、「小さくする」で乗り越えられます。

演習4 曖昧な依頼と、仕事の依頼を比べる

- ミッション: 会計アプリに同じ機能(お釣り計算——預かり金額を入れるとお釣りを表示)を、2通りの頼み方で追加し、結果を比べる
- 時間の目安: 60分
- やり方:
 - 同じ出発点のコピーを二つ用意し、AとBを別々に試す。Aで変更したアプリへ続けてBを頼むと、比較する条件が変わってしまう
 - A: 「お釣りも計算できるようにして」とだけ頼む
 - B: ブリーフで頼む。制約(計算ルールと画面構成を変えない)、完成条件(セール中600円の注文で1,000円を預かると、お釣り400円と表示される)を付け、「まず計画だけ」を添える
- クリア条件:

1. AとBを同じ3観点（操作・計算結果・変更範囲）で比べ、一致した点と違った点を記録した
 2. Bの依頼に制約と完成条件が入っていて、完成条件に数字がある
 3. Bでは、実装の前に計画または質問が返ってきた
 4. タスクリストとプロジェクト指示書ができていて、指示書に計算ルールが書いてある
- つまづいたら:
 - Aがうまくいったら、そのまま記録する。一度の比較で頼み方の優劣を証明する演習ではない。Bなら、何を変え、何を確認したか説明できるかも見る
 - 計画が返ってこないときは、依頼の最後に「実装はまだしないでください」をもう一度
 - (授業向け)ペアでBの依頼文を交換し、相手の環境で試す。依頼文だけで伝わるかの試金石になります
 - 裏ダンジョン: レビュー役に「このプロジェクト指示書に足りないことを3つ」挙げさせ、採用するかどうかを自分で判断する
 - 冒険の書に記録: AI開発ブリーフ+タスクリスト+プロジェクト指示書

この章のまとめ

- AIの提案は、言い回しより、渡した前提情報(コンテキスト)で変わる。依頼文を磨く前に、仕様書が渡っているかを確かめる
- 依頼には、目的・利用者・制約・完成条件の4項目を付ける。制約には変えないでほしいことを、完成条件には今回1件ぶんの合格ラインを数字で書く
- 影響の大きい変更は、実装の前に計画と、仮定・不明点を出させる。ずれは作る前に言葉で直し、答えた内容は仕様書に書き戻す
- 1回の依頼では1つの変化だけを頼む。完成条件が1〜2行で書けない作業は、まだ大きすぎる
- 毎回守らせたい計算ルールや仕事の進め方は、プロジェクト指示書に書いて置いておく

理解度チェック

1. AI開発ブリーフに書く4項目を挙げてください。また、「制約」には何を先に書くとよいですか。
2. 「取り消しを直して、集計も見やすくして、ボタンの色も変えて」と1回で頼もうとしています。どう頼み直しますか。最初の依頼に付ける完成条件の例も1つ書いてください。
3. 割引計算を実装した会話のまま「確認して」と頼むと、「問題ありません」と返ってきました。これで確認済みと言えますか。理由も教えてください。

解答のめやす

1. 目的・利用者・制約・完成条件。制約には、してほしいことより先に、変えないでほしいこと(会計アプリなら計算ルール)を書きます(→4.2)
2. 3つの変化なので、3回の依頼に分けます。完成条件の例は「履歴の各行で取り消しボタンを押しやすく、検算750円/600円が変わらない」など、1〜2行で確かめられるもの(→4.4、4.2)

3. 言えません。同じ流れのままだと、思い込みごと確かめがちです。テスト役に切り替えるか別の会話を開き、実行した方法と結果を出してもらい、自分でも750円/600円などを試します(→4.6)

60秒でクエスト報告

仕様をまとめて作らせた後、変更の影響を追いにくくなりました。そこで、いまは1回の依頼を1つの変化に分けています。目的・利用者・制約・完成条件を依頼に付け、決めたルールは仕様書とプロジェクト指示書へ戻します。変更後は750円/600円の検算を行い、実行結果と未確認の点を記録します。次に直す場所を、結果から決められるようにするためです。

——前半は口頭試問の第1問「誰の何を解決しますか」の続きです。そして後半は、この先の問い——どう変更したか、正しさをどう確かめたか——への足場になっています。

レベルアップ!

Lv.4 称号: パーティリーダー

- 手に入れた装備: プロンプト、コンテキスト、制約、完成条件、タスク分割
- 冒険の書: AI開発ブリーフ+タスクリスト+プロジェクト指示書

次の第5章では、視点を「頼み方」から「アプリの中身」へ移します。ブラウザの向こう側を見に行く、構造の地図の始まりです。ところで——あなたの依頼がAIに届いたあと、アプリはいったいどこで動いているのでしょうか。

第5章 アプリはどこで動いているのか

クエスト5: 画面の向こう側の地図

文化祭まで、あと21日。

この章のねらい

- ・ ブラウザ・サーバー・データベース・外部APIの4役で、自分のアプリの構成図を描ける
- ・ 開発者ツールのネットワークタブで、リクエストとレスポンスの往復を観察できる
- ・ 「通信していない」と「保存していない」を区別し、記録が消える理由を説明できる

今回のクエスト

第4章までで、会計アプリの試作が動き出しました。メニューを押せば合計が出て、会計すれば履歴に並びます。けれど、その記録はどこにあるのでしょうか。第Ⅰ部で決めた機能を、第Ⅱ部では中の仕組みから見直します。

この章から第Ⅱ部、構造の地図に入ります。今回のミッションは、この本ではめずらしいものです。

何も作りません。何も直しません。そのかわり、いま動いているアプリの「中」を、初めて覗きます。

章の終わりに手に入るのは、**アプリ構成図**。画面の向こう側の地図です。この地図があると、AIの返事が読め、壊れたときに見る場所の見当が付くようになります。第15章まで、そしてこの本を閉じたあとも、ずっと使い続ける1枚です。

まずAIに聞いてみよう

この章では、開発用サーバーからファイルを配り、会計の記録はブラウザのメモリにだけ置く構成を例にします。実際のAIの生成結果は違うことがあるので、観察して自分の構成図に直してください。物語は、店長役の友達にアプリを見せた翌日からです。

「昨日の夜、家からあのアプリを開こうとしたら、開けなかったよ。当日はテントで使うんだよね。大丈夫?」

答えられますか。なぜ開けなかったのか。そもそもあのアプリは、いま、どこに「ある」のか。

あなたのパソコンの中でしょうか。インターネットのどこかでしょうか。わからないので、AIに聞いてみます。

あなた → AI

このアプリは、いまどこで動いているのですか。友達が夜に家から開けなかったのは、なぜですか。

AI → あなた

フロントエンドはブラウザ上で動作しています。ファイルの配信元は、開発環境内のサーバーです。クライアントからのリクエストにレスポンスを返せるのは、そのサーバーに接続できるあいだです。

会計を保存するバックエンドは、まだありません。友達が開けなかった原因を特定するには、サーバーの稼働状況とURLの共有範囲を確認する必要があります。

……読めましたか。

ボス出現——中が見えない

フロントエンド、サーバー、リクエスト。見覚えのある単語があっても、まだ一つの景色にはつながらないかもしれません。

そして質問し返そうにも、どの単語がわからないのかも、わからない。

これが、序章の表にあった第2の壁「中が見えない」です。たちが悪いことに、この壁は、ぶつかっても痛くありません。中が見えないままでも、AIは動くものを作り続けてくれるからです。開発は「進んでしまいます」。

仕組みの見当が付けば、不具合のときにも「ブラウザの中か、通信か、保存先か」と調べる場所を絞れます。口頭試問の第2問——「ボタンを押すと、データはどこを通りますか」への答えも、そのまま構成図に描けるようになります。

このボスは、力では倒せません。倒すのに必要なのは、地図です。

まずは「誰が、誰に、何を頼んでいるか」を追います。専門用語は、その場面に一つずつ名前を付けるために使いましょう。章末で、さっきの返事をもう一度読みます。

5.1 ブラウザとアプリの違い

「アプリ」と聞くと、スマートフォンにインストールするものを思い浮かべるかもしれません。では、あなたの会計アプリはどうでしょう。インストールしていません。ブラウザで開いています。

ブラウザは、ページを表示するためのソフトです。大事なのは、その働き方です。ブラウザは、URLを入れると、その住所へ「このページの材料をください」と取りに行きます。そして届いた材料——画面の設計図や、動きの指示書のファイル一式——を、その場で組み立てて動かします。

いまの会計アプリは、ブラウザが読み込んだファイルを動かすWebアプリです。同じブラウザでも、届くファイルが違えば、動画サイトにも地図にもなります。なお、以前に取得したファイルを再利用する「キャッシュ」が働くこともあり、毎回すべてを受け取り直すとは限りません。

図 5-1

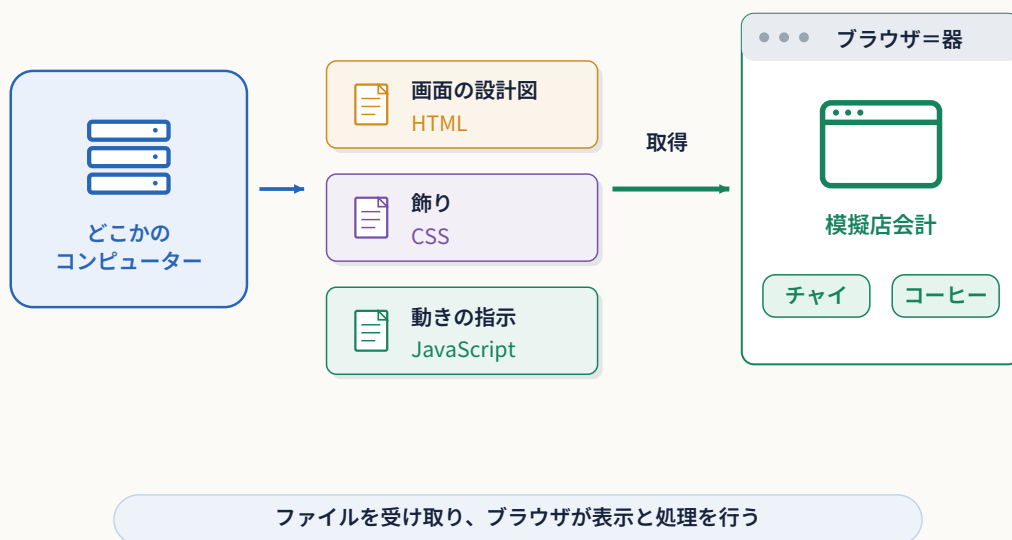


図5-1 ブラウザへアプリが届く

届いてくるものの先頭を、すこしだけ見てみましょう。読めなくて大丈夫です。「画面の設計図が、文字のファイルとして届く」ことだけ確認してください。

役割ラベル: 表示

```
[html]
<!-- 開いた瞬間に届く「画面の設計図」の冒頭部分 -->
<title>模擬店会計</title>
<button>チャイ 400円</button>
<button>コーヒー 300円</button>
```

では、この一式を届けてくれているのは、誰なのでしょう。

5.2 クライアントとは何か、サーバーとは何か

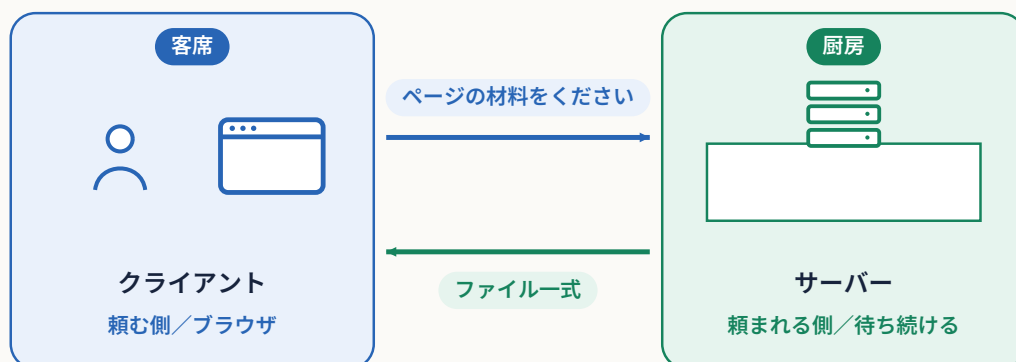
言葉を2つ、同時に手に入れます。クライアント(client)は、頼む側。日常語で言うと注文する客、つまりあなたの手元です。

サーバー(server)は、頼まれる側。日常語で言うと、注文を受けて働く厨房です。

レストランを思い浮かべてください。客席のあなたは、厨房の中を知りません。注文すると、料理が返ってくる。知っているのはそれだけです。

アプリの世界も、まったく同じ形をしています。あなたのブラウザが客席から「ページの材料をください」と注文し、どこかのコンピュータが厨房としてそれに応える。この厨房役が、サーバーです。

図 5-2



サーバーは機械の種類ではなく、頼まれる側という役割

図5-2 客席と厨房の往復

厨房役が応答できなければ、新しい注文に料理は返りません。サーバーも同じで、利用者が使う時間帯に依頼へ応答できる必要があります。実際には、常時動かす方式も、依頼に応じて処理を起動する方式もあります。

※厳密に言うと: サーバーは機械の種類ではなく、役割の名前です。あなたのパソコンも、頼まれる側に回ればサーバーになれる。同じ人が、店では店員、家に帰れば客になるのと同じです。

ここまでで、店長の質問を調べる手がかりができました。作例の配信元は開発環境のサーバーです。これが停止していたら、ページのファイルを新しく届けることはできません。

ただし、「開けない」という症状だけで、停止が原因だとは決められません。URLの有効期限、アクセスできる人の制限、ネットワークの問題も候補です。「厨房が動いているか」「そこまで注文が届くか」を分けて確認します。

そして、この章でいちばん大事な問いを渡します。「このアプリでは、どれがサーバーですか。止まると、何が起きますか。」この問いは、あなたが今後どんなアプリに関わるときも、最初に立てるべき問いになります。

5.3 フロントエンドとバックエンド

次の2語は、仕事の場所の名前です。フロントエンド(frontend)は、画面側。日常語で言うと、手元で起きていることです。

バックエンド(backend)は、向こう側の処理。厨房の中で起きていることです。

フロントエンドは、客席で起きることの全部です。メニューが見える、ボタンが押せる、押した瞬間に色が変わる。バックエンドは、厨房の中の仕事の全部です。注文を受け付ける、記録を残す、集計する。

作例の会計アプリで仕分けると、こうなります。

出来事	どちら側か
メニューのボタンが並んで見える	フロントエンド(手元)
押すと合計金額が変わって見える	フロントエンド(手元)
会計の記録を一時的に持つ	この作例ではフロントエンド
日ごと・時間帯ごとの売上を計算する	この作例ではフロントエンド

保存や集計という仕事の名前だけで、処理の場所は決まりません。いまは手元で済ませっていますが、複数人で同じ記録を使い、権限を守るには、サーバー側の処理や保存先が必要になります。自分のアプリでどこが担当しているかを、次の節で観察します。

予告をひとつだけ。第3章で「タイムセールの操作は店長だけ」と決めました。この約束を本当に守れる場所は、手元ではなく向こう側です。手元の画面は、見る人の側にあるぶん、ごまかしが効いてしまうからです。

くわしくは第9章で扱います。いまは「守りは厨房でやるものらしい」とだけ、地図の隅に書いておいてください。

5.4 リクエストとレスポンス

客席と厨房のあいだを往復しているものに、名前を付けます。行きがリクエスト(request)、日常語で言うと注文。帰りがレスポンス(response)、日常語で言うと返事です。

本書ではまず、1回の注文に1回の返事が返る、基本的な往復を見ます。失敗すれば返事を受け取れないこともあります。

リクエストには、宛先(URL)と用件が書いてあります。レスポンスには、頼まれたもの——さっき見た設計図のファイルなど——が載っています。この会話の書き方には世界共通の作法があるのですが、その名前は第7章のお楽しみにします。

図 5-3

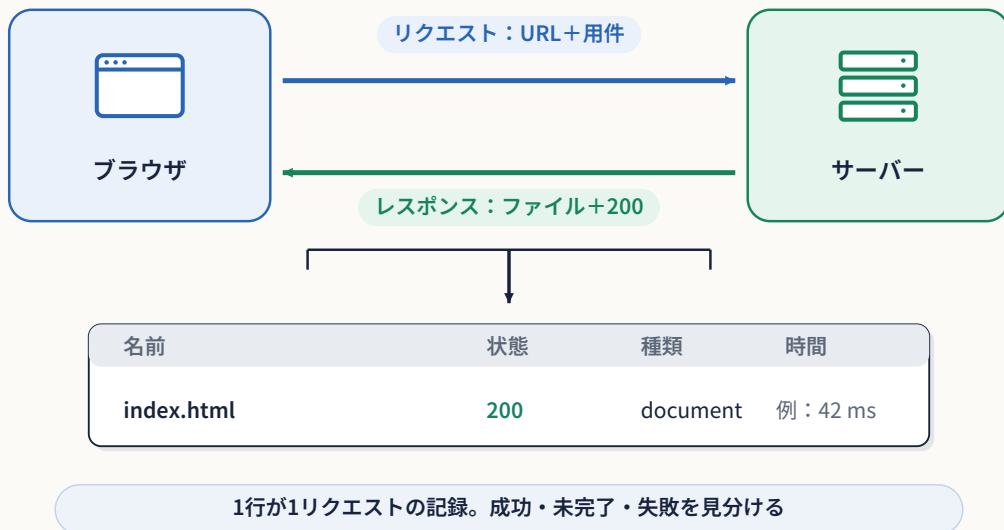


図5-3 通信をネットワークの1行で追う

さて、この往復はふだん目に見えませんが、見るための道具が、あなたのブラウザに最初から付いています。

開発者ツールです。日常語で言うと、ブラウザに隠れている観察窓。誰でも、いますぐ、無料で開けます(開き方はブラウザごとにすこし違うので、付録Aにまとめました)。

開発者ツールの中のネットワークというタブ——さっきの「往復」を一覧にしてくれる窓です——を開いたまま、会計アプリを再読み込みしてください。——行が、ずらっと並びます。

一覧の1行は、一つのリクエストを追う記録です。成功した行にはレスポンスの情報が入り、未完了や失敗の行もあります。ページを開くと、設計図、飾り付け、動きの指示書などを、別々に取り寄せている様子が見えます。

行の中に **200** という番号も見えるはずです。返事に付く番号で、200は「成功」の意味。番号の一覧表は、第7章で手に入ります。

では、次の実験です。ネットワークタブを開いたまま、会計ボタンを押してください。

作例では、会計を保存するための行は増えません。

メニューを押しても、会計しても、履歴を開いても、この作例では会計のデータがサーバーへ送られていません。

※厳密に言うと: 会計と関係のない行が増えることもあります(開発環境の更新確認など)。また、すでにつながっている回線の中を流れる通信は、新しい行として表示されない場合があります。行が増えたら宛先と中身を確認、行数だけで結論を出さないようにします。

ここでわかったのは、通信の有無です。通信していないことと、保存していないことは同じではありません。ブラウザ内にも、再読み込みをまたいで記録できる仕組みがあります。保存先は、再読み込み後の動作とコードを合わせて確認します。5.7で、その違いを図にします。

出典: ブラウザ内の保存については、MDN [Web Storage API](#)を参照。

▼コーナー「先輩エンジニアならここを見る」

プロがアプリの不調を調べるとき、真っ先に開く場所のひとつが、このネットワークタブです。「そもそも注文は出たのか。返事は返ってきたのか」。画面をいくら見つめてもわからないことが、ここでは1行で見えます。観察窓の場所を知っている——それだけで、あなたの調べる力は段違いになります。

5.5 ローカルで動くことと、インターネットで動くこと——クラウドとは何か

では、配信元のサーバーはどこにあるのでしょうか。ローカル(local)は、ここでは自分が作業するコンピュータの中、という意味で使います。一方、ブラウザで使うAI開発環境では、作業用サーバー自体がクラウド上にあることもあります。「開発用だからローカル」とは限りません。

ローカルのサーバーは、自宅の台所に似ています。人に出す前に、まず自分で試せます。ただし、手元で動かしていても、外部のサービスや本物のデータにつないでいれば、影響は外にも及びます。練習には練習用のデータを使いましょう。

ローカルだけで動くURLは、通常、そのまま友達の端末からは開けません。クラウド上の開発プレビューでも、利用者が制限されていたり、環境の停止で使えなくなったりします。置き場所と、誰がいつアクセスできるかは、別々に確かめる項目です。

文化祭当日に必要なのは、使う時間帯に、許可した部員の端末から安定して接続できる場所です。開発中のパソコンの起動状態に依存しないようにするなら、配信や処理を外部のサービスに任せる方法があります。

その代表がクラウド(cloud)です。コンピュータの処理能力や保存場所などを、ネットワーク経由のサービスとして使います。厨房の設備を自分で一式そろえる代わりに、必要な働きを借りるイメージです。

ファイルを配るサービス、処理を実行するサービス、データベースのサービスなど、借りる単位はさまざまです。どれを使っても、利用条件、料金、停止時の対応は確かめます。

※厳密に言うと: クラウド(雲)という名前ですが、実体は空にはありません。データセンターと呼ばれる地上の建物の中の、実在するコンピューターです。あなたのアプリのデータにも、必ず物理的な置き場所があります。

利用者に届ける公開のやり方は、第14章で扱います。いまは地図に「配信元はどこか」「止めると何が使えなくなるか」「誰が開けるURLか」を書きます。友達への答えも、調べた内容に基づいて説明できるようになります。

5.6 画面の四状態——表示中、読み込み中、データなし、エラー

観察窓から、もうひとつ大事なものが見えます。ネットワークタブの各行には、往復にかかった時間が出ています。往復には時間がかかるのです。手元の厨房なら一瞬でも、遠くの厨房なら、待たされる瞬間が生まれます。

外からデータを取得する画面では、四つの状態を考えると漏れを減らせます。履歴画面を例にします。

状態	レストランで言うと	履歴画面で言うと
表示中	料理が目にある	履歴が並んでいる
読み込み中	注文が厨房に行っていて、待っている	「読み込んでいます…」
データなし	「本日はまだ注文がありません」	会計0件。「まだ会計はありません」
エラー	厨房と連絡が取れない	「履歴を読み込めませんでした」

図 5-4



図5-4 画面の四状態

履歴がきれいに並んだ状態だけでなく、待っているとき、まだ記録がないとき、取得できないときにも、使う人へ状況を伝えます。

とくに、下の2行に注目してください。「データなし」と「エラー」は、別の出来事です。「今日はまだ1件も売れていない店」と、「記録を取りに行けない店」。後者では売上を確認できていないので、0円と断定してはいけません。

もし連絡が取れない日に店長画面が「売上0円」と出したら、店長は「今日はまったく売れなかったのか」と誤解します。データが無いことと、データを取りに行けないことを、画面は区別して見せる責任があります。

いまの会計アプリは往復がないので、読み込み中やエラーには、まだほとんど出会いません。でも第7章で外部のデータを借り始めた瞬間、この四状態は毎日の現実になります。図に描く習慣を、今日から始めておきましょう。

5.7 アプリ構成図の描き方

材料がそろいました。地図を描きます。

本書の構成図の登場人物は、4人だけです。

1. あなた(ブラウザ)——注文する側。画面はここに映る
2. サーバー——注文を受けて働く厨房
3. データベース——記録を消えないように残す、ノート係(まだ出会っていない仲間です。第8章で登場します)
4. 外部API——よそのアプリに仕事を頼むときの、その相手(こちらも第7章で登場します)

構成図では、この4役を基本に配置をそろえます。後の章では、ログインの仕組みや秘密の設定値の置き場所なども描き足します。そのときも、まず4役のどこに関係するかを考えると、位置を見失いにくくなります。

描き方のルールは3つです。

- 配置: 左から「あなた(ブラウザ)」→「サーバー」→「データベース」の順に置く。「外部API」は右上に別枠で置く
- 矢印: 必ず往復1組で描く。行き(リクエスト)に用件を、帰り(レスポンス)に返ってくる中身を書く
- 点線: まだ登場していない(使っていない)人は、点線で描く

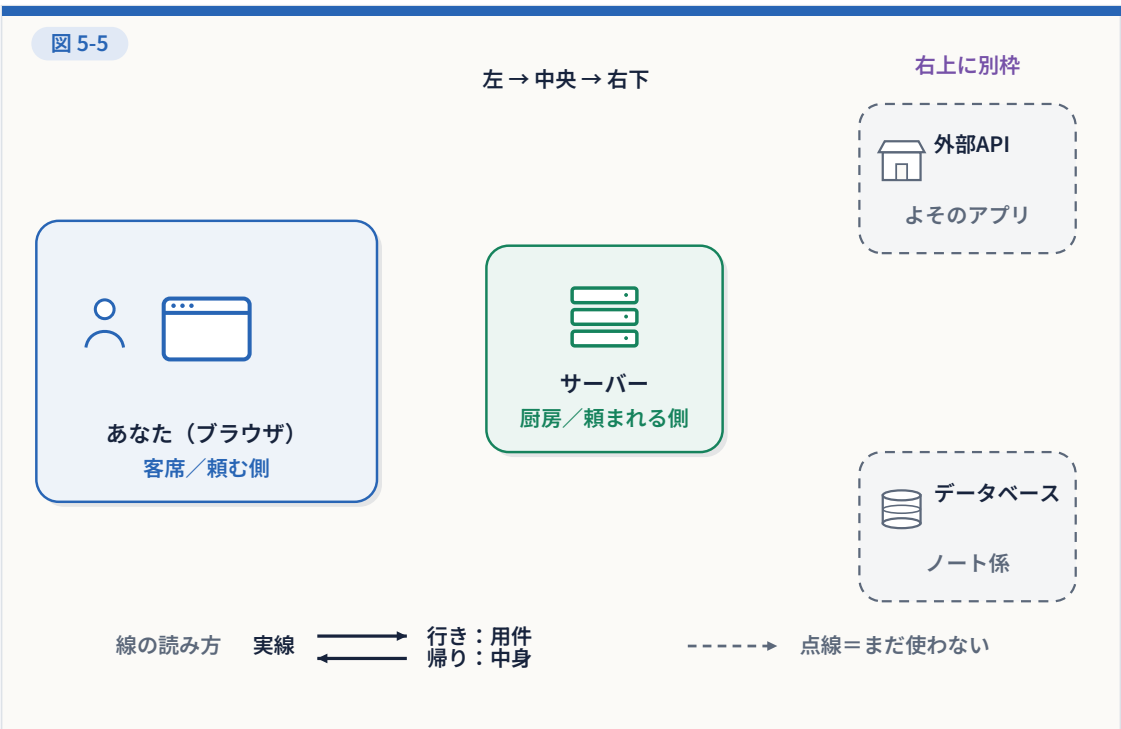


図5-5 4登場人物の共通配置

では、いまの会計アプリを、正直に描きます。

図 5-6

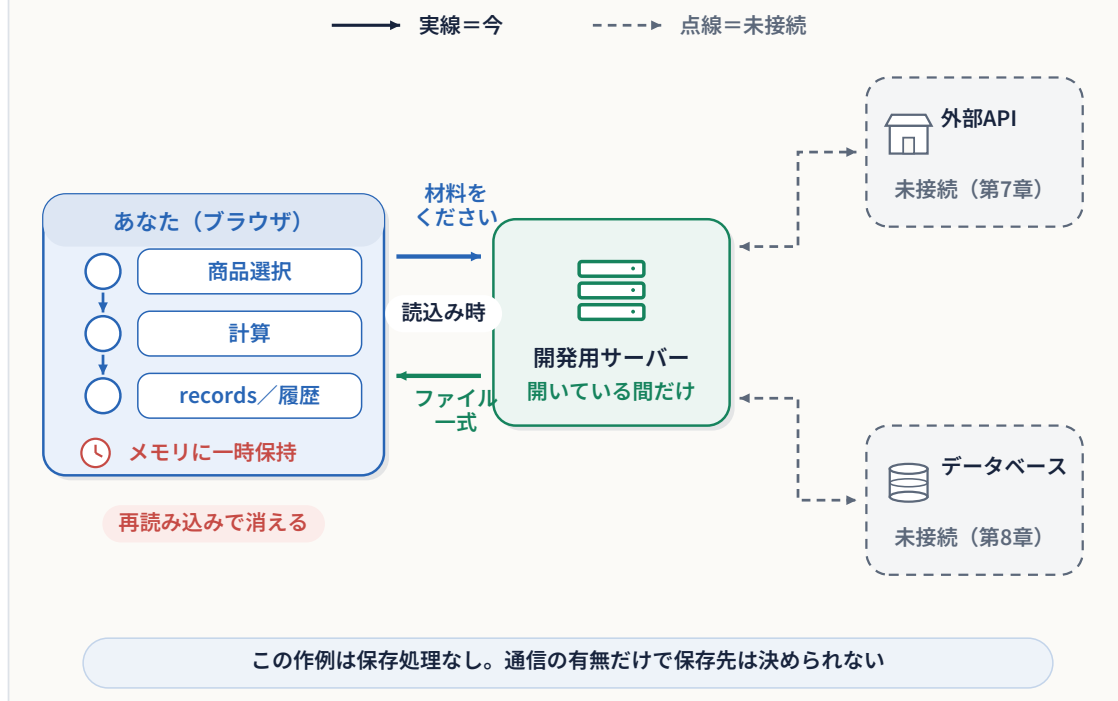


図5-6 現在の会計アプリの構成

ファイルを読み込む通信は、図では1組にまとめました。実際には複数のリクエストがあります。その後の会計操作は左の枠の中で進み、データベースと外部APIはまだ使っていません。

コードでも確かめておきましょう。次は、処理の流れを短く示した学習用の例です。**records**は、ページを開くたびに空になる配列で、別の保存先へ書き込む処理はないものとします。

役割ラベル: 計算・表示

```
[javascript]
// 会計ボタンが押されたとき。どの行も、あなたのブラウザの中で実行される
function onCheckout() {
  const total = calcTotal(order, saleRate); // 合計を計算する(手元)
  records.push({ time: now(), total }); // 履歴のメモに書き足す(手元)
  showHistory(records); // 履歴画面を描き直す(手元)
}
// records はメモリにだけ置いており、別の保存先には書き込んでいない
```

この作例で記録が消えるのは、メモリ上の配列にだけ追加し、再読み込み後に復元する仕組みがないからです。自分のアプリで記録が残るなら、**localStorage** など別の保存先を使っていないか、AIとコードを確認しましょう。通信・保存先・再読み込み後の結果がそろると、理由を説明できます。

言葉で聞き、図で描き、コードで確かめる——同じ事実を3つの層で見ました。この見方は、この先のどの章でも繰り返します。

最後に、構成図のいちばんの効用を言います。描きにくい場所が、次に確かめる場所です。

矢印の行き先で手が止まったら、そこがあなたの地図の空白です。空白は恥ではありません。序章の「分からなかったことリスト」と同じで、次に埋める場所の印です。

AIと一緒に直す——もう一度、あの返事を読む

章の冒頭の、読めなかった返事に戻りましょう。

フロントエンドはブラウザ上で動作しています。ファイルの配信元は、開発環境内のサーバーです。クライアントからのリクエストにレスポンスを返せるのは、そのサーバーに接続できるあいだです。会計を保存するバックエンドは、まだありません。

「画面はブラウザの中。ファイルは開発用サーバーから届く。会計を保存する処理は、まだ向こう側がない」。単語が、地図の場所とつながってきました。友達が開けなかった件では、配信元の稼働と、URLの共有範囲を調べればよさそうです。

さらに一歩進めます。冒頭のあなたの質問は「どこで動いているの?」という漠然としたものでした。だからAIも、一般的な用語で答えるしかありませんでした。地図を手に入れたいまは、聞き方そのものを変えられます。

あなた → AI

このアプリのコードと設定を確認し、「あなた(ブラウザ)」「サーバー」「データベース」「外部API」の4役に分けて説明してください。使っていないものは「未使用」、確認できないものは「不明」としてください。会計ボタンからたどる処理と保存先を、根拠になるコードの場所も添えて教えてください。

AI → あなた

1. あなた(ブラウザ): メニューの表示、合計の計算、履歴の保持と表示を、すべてブラウザの中で行っています。
2. サーバー: 開発環境のサーバーが、最初のアクセス時に画面のファイル一式を届けています。それ以外の仕事はしていません。
3. データベース: まだ使っていません。会計履歴はブラウザの中にだけあり、再読み込みで消えます。
4. 外部API: まだ使っていません。
会計ボタンを押したとき、データはブラウザの外には出ません。

今度は、地図と照らせる答えになりました。ただし、読みやすい説明が正しいとは限りません。AIが示したコードと、自分が観察した通信や保存の結果が一致するかを見ます。説明の形と、説明の根拠。この両方を頼むのがコツです。

別の条件でも動くか試す

この観察は、ふだん使うニュースサイトなどでも試せます。ネットワークタブを開いたまま再読み込みし、ページや画像を取りに行くリクエストを探してください。大きなサービスは4役だけでは細部を表せませんが、ブラウザから何が見えるかを整理する入口にはなります。

開発者ツールの見た目や開き方は、ブラウザによってすこしずつ違います。この本の説明と画面が違って、「再読み込みすると行が並ぶ」ことさえ観察できていれば、先へ進んで大丈夫です。AIに構

成を聞いたときの答えも毎回変わりますが、4人への振り分けが読み取れれば、あなたの聞き方は機能しています。

演習5 会計ボタンの旅を描く

- ミッション: 会計アプリの構成図と、「会計ボタンを押してから、履歴画面に出るまで」のデータフロー図を矢印で描く
- 時間の目安: 30分
- クリア条件:
 1. 構成図に4人の登場人物がすべて描かれ、まだ使っていないものが点線になっている
 2. ネットワークタブを開き、再読み込みの瞬間に行が並ぶことを自分の目で観察した
 3. データフロー図のすべての矢印に、「手元(ブラウザの中)」か「向こう側(サーバー)」かが書いてある
 4. 再読み込み後に履歴が残るかを試し、保存先と結び付けて理由を一文で説明できる(本書の作例と異なってよい)
- つまづいたら:
 - 開発者ツールの開き方は付録Aへ。ネットワークタブが空っぽのときは、タブを開いたまま再読み込みする
 - 行が多すぎて怖くなったら、読まなくてかまいません。「並んだ」ことだけ確認すれば十分です
 - 矢印の行き先で手が止まったら、それが空白の印。図5-6と見比べて、どこが違うかをAIに質問する
 - (授業向け)ペアで図を交換し、相手の図の矢印を1本ずつ「これは手元? 向こう側?」と質問し合う
- 裏ダンジョン: ふだん使うサービスで、「押すと行が増えるボタン」と「押しても増えないボタン」を1つずつ見つけ、その違いを構成図の言葉で説明してみる
- 冒険の書に記録: アプリ構成図+データフロー図

この章のまとめ

- ブラウザは、サーバーから届いたファイルを組み立てて動かす。作例の会計アプリでは、画面と計算はブラウザの中(フロントエンド)で動いている
- 頼む側がクライアント、応える側がサーバー。行きがリクエスト、帰りがレスポンス。往復は、開発者ツールのネットワークタブで1行ずつ見える
- 通信の有無だけでは、保存の有無は決まらない。保存先は、コードと再読み込み後の動きもあわせて確かめる
- 「開けない」と言われたら、配信元が動いているかと、誰がいつそのURLに接続できるかを分けて調べる

- ・ 外からデータを取る画面では、表示中・読み込み中・データなし・エラーの四状態を考える。「0件」と「取得できない」は別の出来事

理解度チェック

1. 作例の会計アプリで、フロントエンドが担当している仕事を2つ挙げてください。
2. 友達から「夜、家からアプリを開けなかった」と言われました。まず確かめる点を2つ挙げてください。
3. 会計ボタンを押しても、ネットワークタブに行が増えませんでした。これだけで「記録はどこにも保存されていない」と言えますか。理由も答えてください。

解答のめやす

1. メニューの表示、合計の計算、履歴の保持と表示のうち2つ(→5.3)
2. 配信元のサーバーが動いているか。そのURLに友達が接続できる設定か(有効期限・共有範囲)(→5.2、5.5)
3. 言えません。ブラウザの中にも、再読み込みをまたいで残せる保存先があるからです。コードと、再読み込み後に記録が残るかもあわせて確かめます(→5.4、5.7)

60秒でクエスト報告

作例では、開発用サーバーからファイルを受け取り、ブラウザの中で注文・計算・履歴表示を行います。会計時に保存の通信はなく、記録はメモリ上の配列にだけ置いているため、再読み込みすると消えます。通信がないことだけでは保存の有無は決められないので、コードと再読み込み後の結果も見ました。人に渡す前には、配信元がいつ動き、誰が接続できるかも確かめます。

——口頭試問の第2問は「ボタンを押すと、データはどこを通りますか」でした。作例の答えは「ブラウザの中で、注文から計算、配列への追加、履歴表示へ進みます」。自分の図でも、確認した経路を指さして説明できれば、この章のクリアです。

レベルアップ!

Lv.5 称号: 構成図の描き手

- ・ 手に入れた装備: クライアント、サーバー、フロントエンド、バックエンド、リクエスト、レスポンス、ローカル、クラウド、開発者ツール
- ・ 冒険の書: アプリ構成図+データフロー図

次の第6章では、いま描いた地図のいちばん左の枠——ブラウザの中に入ります。そこで動いている「動きの指示書」、つまりコードです。安心してください。書けるようになる必要はありません。仕様書と見比べながら「読める」ようになるための章です。

第6章 コードを全部書かなくても、読めるようになる

クエスト6: 呪文の解読

文化祭まで、あと19日。

この章のねらい

- ・ コードの行を、計算・表示・データ・通信の4つの役割に仕分けられる
- ・ 変数・配列・条件分岐・関数などを、仕様書の日本語と対応させて説明できる
- ・ AIの「直しました」を、変更された行と期待する合計の両方で確かめられる

今回のクエスト

画面には750円と出ています。では、セット割の50円は、どこで引かれているのでしょうか。この章では、動いている画面から一步奥へ入り、AIが書いたコードを開きます。

ゴールは、仕様とコードを結び付けて読めるようになることです。日本語の仕様と、図と、コードを見比べて、「セット割はこの行」「画面を変えるのはこの部品」と指させるようになることを目指します。コードを書く作業はAIに任せながら進められます。

動作を試すことに加え、変更された行も確かめられる。その小さな手がかりが、AIの「直しました」を検品する助けになります。章の終わりには、セット割を変更し、変わった場所と変わらなかった場所を説明してみましょう。

まず、AIのコードを開いてみよう

第4章で、あなたは1ページ仕様書をAIに渡し、会計アプリを作らせました。アプリは動いています。注文画面でコーヒー2つとクッキー1つを押せば、ちゃんと750円と出ます。

ところで——その計算は、どこにあるのでしょうか。

画面のどこにも計算式は見えません。でも、どこかに書いてあるはずです。書いてある場所が、AI開発環境のファイルの中です。序章の図で「今日は使わない」と薄く描かれていた、あのファイル置き場を、今日初めて開きます。

開くと、名前の付いたファイルがいくつも並んでいます。どれかひとつを選ぶと——出ました。英単語と記号がぎっしり詰まった、あの画面です。図6-1では、その中に見覚えのある業務の数字(50や10)がまじっていることにだけ目を留めてください。図は、タイムセールを行を省いた通常時の抜粋です。セールの行まで含めた完全な形は、6.5で読みます。

図 6-1



図6-1 コードを開いた瞬間

ボス出現——AIのコードが、呪文にしか見えない

初めてこの画面を開くと、手が止まるかもしれません。「呪文にしか見えない」。英単語だけでなく、カッコや記号まで意味を持っていそうです。

ここで1行目から順に、全部理解しようとする、知らない言葉のたびに立ち止まることになります。最初の目的は、英単語を訳し尽くすことではなく、750円を生む計算を探すことでした。

先に結論を言います。全部読む必要は、最初からありません。

既存のコードを調べるときは、知りたい動きにあたりを付け、関係する場所から読む方法が役立ちます。あなたが身につけるのも、その「あたりの付け方」です。

そして、そのための最初の道具は、実はもうあなたの手元にあります。

6.1 ソースコードとは何か

ソースコード(略してコード)とは、アプリの動きを1行ずつ書き下した指示書です。日常語で言うと、アプリの楽譜です。演奏(実行)するのはコンピューターですが、楽譜は人間にも読める形で書かれています。

AIが作ったのも、この楽譜です。画面もボタンも計算も、すべてこの文章から生まれています。

ここで、種明かしをひとつ。第3章で **calcTotal** のコードを見せたとき、上に小さく「役割ラベル: 計算」と付いていたのを覚えていますか。あれは、この章のための伏線でした。

本書では、コードを読むための目印として、仕事を次の4種類に分けます。プログラム全体の厳密な分類ではなく、会計アプリの見取り図です。

- 計算——値段や合計を求める
- 表示——画面に出す、書き換える
- データ——覚えておく、並べておく
- 通信——アプリの外に頼みに行く

これが本書の色分けです。コード例には、役割のラベルを添えます。「計算はここに固まっているな」「ここから外へ頼んでいるな」と、まとまりを探してください。

図6-2は、図6-1と同じコードを行ごとに塗り分けたものです。計算の行が固まっていること、通信の行がまだ0行であることを見てください。

図 6-2

通常時の抜粋を、役割で塗り分ける

```

デ 1  const menu = products;
デ 2  let currentItems = [];
計 3  function calcTotal(items) {
計 4  const subtotal = sumOf(items);
計 5  const setOff = countSets(items) * 50;
計 6  return Math.floor((subtotal - setOff) / 10) * 10;
計 7  }
表 8  showTotal(calcTotal(currentItems));
表 9  renderHistory();

```

4つの役割

- 計算
- 表示
- データ
- 通信
0行
まだ外と話していない

役割の色で、必要な処理のまとまりを探す

図6-2 コードの4色地図

1行が複数の役割を持つこともあります。どちらの色かで迷ったら、主な仕事を書き添えれば十分です。ここから6.2～6.6で、読むための基本語を見ていきます。意味を暗記するより、「自分のアプリならどれに当たるか」をひとつずつ見つけてください。

6.2 変数は「名前を付けた箱」——文字、数字、真偽値

変数は、値をしまっておく名前を付けた箱です。コードのいたるところに出てきます。実物を見ましよう。見るのは、3つの箱の中身の違いです。

役割ラベル: データ

```

[javascript]
const price = 300;    // 数字の箱: コーヒーの値段
const name = "コーヒー"; // 文字の箱: 品名
let saleActive = false; // 真偽値の箱: いまタイムセール中か

```

値の種類を型と呼びます。**300**は数字、**"コーヒー"**は文字(正式には文字列)です。見た目が数字でも、**"300"**のように引用符で囲まれていれば文字として扱われます。

まずは3つだけ覚えれば足ります。文字、数字、そして真偽値——「はい(**true**)」か「いいえ(**false**)」だけが入る箱です。

3行目に注目してください。仕様書の「タイムセール」という日本語は、コードの中では **saleActive** という箱の中身ひとつに姿を変えています。セール中なら **true**、そうでなければ **false**。日本語とコードの対応の、いちばん小さな単位がこれです。

図6-3は、3つの箱を、名前・中身・種類の組で並べ直したものです。

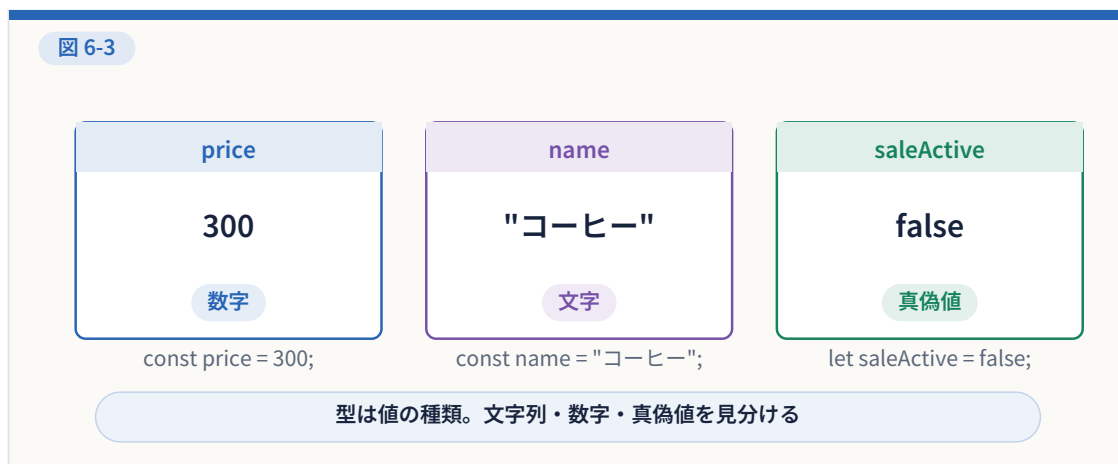


図6-3 変数と型の3つの箱

※読み方の補足: **const**は値を入れ直さない名前、**let**は入れ直せる名前の宣言です。JavaScriptにはほかの型もありますが、ここでは数字・文字・真偽値の違いを押さえれば十分です。

6.3 配列とオブジェクト

箱がひとつずつでは、メニュー5品を扱えません。箱をまとめる方法が2つあります。

配列は、並んだ箱です。順番があり、何番目、と取り出せます。オブジェクトは、名前付きの箱のセットです。関係する箱をひとつとめて、名前を取り出せます。

本書のメニュー5品を、コードで表すとこうなります。見るのは、**[]**と**{}**、2種類のかっこです。

役割ラベル: データ

```
[javascript]
const menu = [
  { name: "チャイ", price: 400, kind: "ドリンク" },
  { name: "コーヒー", price: 300, kind: "ドリンク" },
  { name: "クッキー", price: 200, kind: "フード" },
  { name: "ドーナツ", price: 250, kind: "フード" },
  { name: "ラムネ", price: 150, kind: "ドリンク" }
];
```

読み方は2つだけです。`[]` は並び(配列)、`{}` はひとまとまり(オブジェクト)。つまりこれは「5個のオブジェクトが並んだ配列」で、日本語に戻すと「メニュー表」です。セット割の判定に使う「ドリンクかフードか」も、`kind` という箱でちゃんと覚えられています。

もうひとつ、注文1件を表す例です。説明のために、保存項目の一部を抜き出しています。見てほしいのは、中身の顔ぶれです。

役割ラベル: データ

```
[javascript]
const order = {
  time: "13:05",           // 会計の時刻
  items: ["コーヒー", "コーヒー", "クッキー"], // 品目(配列)
  total: 750,              // 合計
  saleApplied: false       // セール適用の有無
};
```

第3章の3.5で作ったデータの表を思い出してください。「会計の時刻、合計、セール適用の有無」に品目を加えた4項目が、オブジェクトの中身に対応しています。仕様書で決めた項目に、コードでは名前が付く。第8章では、会計番号やセット割額も含め、長く残すための形に整えます。

図 6-4



4つの項目をorderオブジェクトにまとめる

図6-4 仕様表と注文オブジェクト

6.4 条件分岐と繰り返し

データが読めたら、次は動きです。上から順に進む流れに加えて、ここでは「道を分ける」「同じ仕事を繰り返す」の2つを見ます。

1つめ、条件分岐。日常語で言うともし～ならです。見るのは、先頭の `if` の1語です。

役割ラベル: 表示

```
[javascript]
```

```
if (saleActive) {
  showBanner("タイムセール中"); // もしセール中なら、画面に帯を出す
}
```

ifを見たら「もし」と読んでください。この例では、仕様書の「もしタイムセール中なら」が、**if (saleActive)**に対応しています。別の書き方をする場合もありますが、探す条件は同じです。

2つめ、繰り返し。「1件ずつ順番に」です。店長画面の「日ごとの売上」を考えます。ここでは**orders**に、その日の集計対象の注文だけが入っているものとします。見るのは、先頭の**for**です。

役割ラベル: 計算

```
[javascript]
let daySum = 0;
for (const order of orders) { // 注文を1件ずつ取り出して
  daySum = daySum + order.total; // 合計に足していく
}
```

forを見たら「1件ずつ、何かしているな」と読みます。何をしているかは、中の1~2行を見れば十分です。

ここで大事な読み方の作法をひとつ。**of**や**+**の細かい文法は、わからなくても飛ばしてよいのです。探すのは文法ではなく、仕様書の日本語の気配です。

図6-5では、左の「もし」が道を2つに分け、右の「1件ずつ」が同じ足し算を繰り返す形を見比べてください。

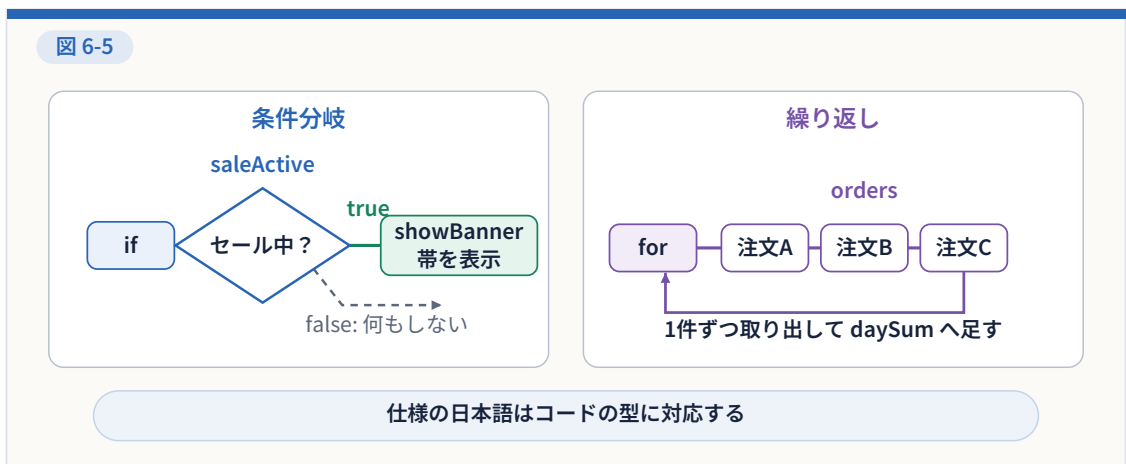


図6-5 もしと一件ずつ

※読み方の補足: JavaScriptでは**forEach**や**map**など、配列の各要素を扱う書き方にも出会います。見慣れない形なら、「何を1件ずつ取り出し、何をしているか」とAIに聞いてみましょう。

6.5 関数とイベント

関数は、名前を付けた手順です。レシピだと思ってください。材料を渡すと、書かれた手順どおりに作業して、結果を返してくれます。一度名前を付ければ、何度でも呼び出せます。

そして、ここがこの章のクライマックスです。第3章の3.6で「読めなくて大丈夫です」と言って見せた、あのコードを再掲します。今度は、読みます。探すのは、セット割の「50」と端数の「10」——仕様書で決めた数字です。

役割ラベル: 計算

```
[javascript]
// 合計 = (小計 - セット割) にタイムセール割引(案A: 併用)。10円未満は切り捨て
function calcTotal(items, saleRate) {
  const subtotal = sumOf(items);           // 小計
  const setOff = countSets(items) * 50;    // セット割: 1セットにつき50円引き
  let total = subtotal - setOff;
  const percent = Math.round(saleRate * 100); // 割引率を整数の%にする(0.2→20)
  if (percent > 0) total = total * (100 - percent) / 100; // タイムセール(率は店長が設定)
  return Math.floor(total / 10) * 10;       // 端数: 10円未満切り捨て(1円玉を扱わない)
}
```

ここでは**items**に選択した商品、**saleRate**に割引率を渡します。通常時は**0**、2割引なら**0.2**です。**saleActive**が「セール中か」を表す札なのに対し、**saleRate**は計算に使う数字です。上から1行ずつ、仕様書の計算ルールと突き合わせます。

コードの行	仕様書の日本語
function calcTotal(items, saleRate)	「注文の品々」と「割引率」を材料に、合計を求める手順
const subtotal = sumOf(items)	小計を出す
const setOff = countSets(items) * 50	セット割: 1セットにつき50円引き
let total = subtotal - setOff	小計からセット割を引く
const percent = Math.round(saleRate * 100)	(仕様書には無い)割引率を20のような整数にそろえる
if (percent > 0) total = ...	もしタイムセール中なら、その残りを割り引く(併用・案A)
return Math.floor(total / 10) * 10	10円未満の端数は切り捨てる

どうでしょう。仕様書の計算ルールが、コードの行とほぼ1対1で並んでいます。あなたが第3章で日本語で決めた「併用・案A」も「10円未満切り捨て」も、ちゃんとここにあります。

知らない単語はまだあるはずですが(**Math.floor**は「切り捨ての道具」、**Math.round**は「四捨五入の道具」くらいで十分)。それでも「セット割の50円はこの行だ」と、あなたはもう指させます。

表の中で、仕様書の日本語が無い行が1つだけあります。**percent**の行です。コンピューターは、0.3のような小数をぴったりには覚えられません。700円にそのまま0.7を掛けると489.99…円になり、10円未満を切り捨てると480円。正しい490円より、10円少なくなってしまう。そこで、割引率をいったん30のような整数に直してから計算しています。

仕様書に無い行は、「仕様を正しく実現するための工夫」か、「頼んでいない余計な変更」のどちらかです。見つけたら、どちらなのかをAIに聞いて確かめましょう。この行は前者です。

途中で呼ばれている **sumOf** や **countSets** も、このアプリ用に用意する関数です。JavaScriptに最初から付いている名前ではありません。レシピから「別ページの手順」を参照する感覚です。まず名前から仕事を推測し、計算が合わないときにその中身まで追います。

関数の名前が読めると、コードは目次になります。

図6-6は、さっきの表を、仕様の手順とコードの行を線で結ぶ形に描き直したものです。

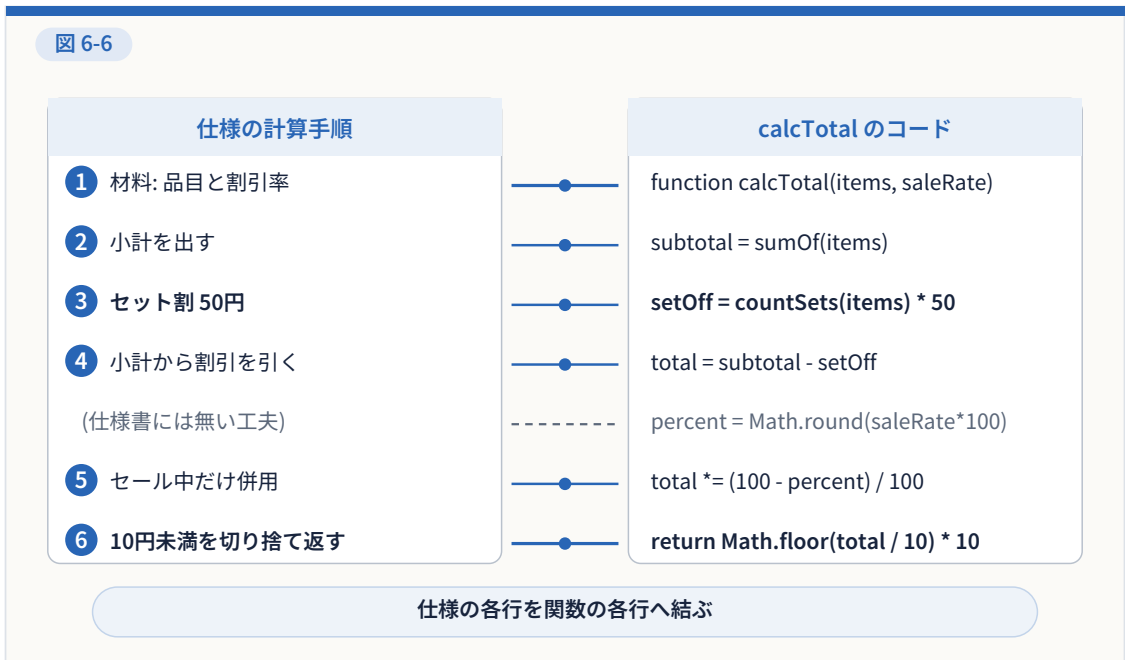


図6-6 計算仕様と関数の対応

では、この手順はいつ動くのでしょうか。書いてあるだけでは、コードは動きません。きっかけがイベント——「押された」などの出来事です。コードはふだん待機していて、出来事が起きた瞬間、結び付けられた関数が走ります。

見るのは **click** の1語と、その中で呼ばれている **calcTotal** です。

役割ラベル: 表示・計算

```
[javascript]
// 「会計」ボタンが押されたら、合計を計算して画面に出す
checkoutButton.addEventListener("click", () => {
  const total = calcTotal(currentItems, saleRate); // 計算
  showTotal(total); // 表示
});
```

addEventListener("click", ...) は、「押されたら、これをやって」という登録です。第5章で追った「ボタンを押すと」の、コード側の入口です。この抜粋は計算と表示まで。サーバーへの送信が必要なら、その処理が続きます。

6.6 状態(ステート)とコンポーネント

残る道具は2つ。どちらも画面のための言葉です。

ステートは、いまの状態です。いま選ばれている品。いまの合計。いまセール中かどうか。

画面は、ステートをそのまま映して描かれます。ステートが変わると、画面が描き替わります。さっきの **saleActive** も、ステートの一員でした。

序章で、再読み込みしたら気分の記録が消えたのを覚えていますか。あの記録は、画面のステートとしてメモリ上にだけ置かれていました。メモリ上だけの状態は、再読み込みすると最初からやり直しになります。

※厳密に言うと: 消えない場所へ保存し、起動時に読み戻す仕組みを加えれば、再読み込み後も画面の状態を復元できます。その保存先——序章で「ノート」と呼んだもの——は第8章で扱います。

コンポーネントは、画面の部品です。注文画面は1枚の絵ではなく、部品の組み合わせでできています。メニューボタンの部品が5個、合計表示の部品が1個、履歴の1行も部品です。

AIは画面を部品単位で組み立てています。だから、直してほしいときも「合計表示の部品だけ直して」と部品単位で頼むと、よく通ります。

図6-7では、左のステートの値(選んだ品・合計・セール中か)が、右の画面の部品にそのまま映っていることを確かめてください。

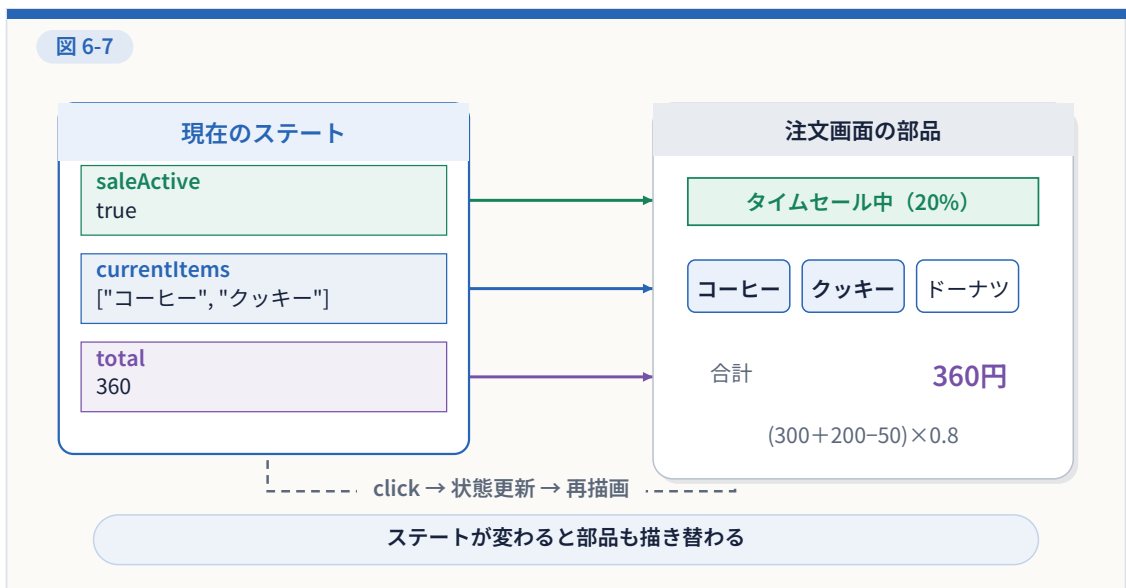


図6-7 ステートが画面を描く

読むための道具がそろいました。ここまでの「日本語⇄コード」を1枚にまとめたものが、この章の成果物、日本語・図・コードの対応表です。図6-8の記入例は、「セット割は50円引き」という1つの仕様を、画面の合計表示とコードの行まで追ったものです。

図 6-8

仕様書の日本語	画面・図の場所	コードの場所	役割色
記入例 セット割は50円引き	合計表示	calcTotal setOff の行	計算
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

一つの仕様を画面とコードまで追跡する

図6-8 日本語・図・コード対応表

AIと一緒に直す——読めると、変更が怖くなくなる

道具がそろったところで、実戦です。文化祭の作戦会議で、セット割を50円から80円に変えることになったとしましょう。コードは書きません。仕様の言葉で頼みます。

あなた → AI

仕様を1つ変更します。セット割を「1セットにつき50円引き」から「1セットにつき80円引き」に変えてください。変更したら、コードのどこを変えたのか、変更点を教えてください。

AI → あなた

変更しました。変更点は1か所です。calcTotal 関数の中で、セット割の金額を求めている行を countSets(items) * 50 から countSets(items) * 80 に変えました。ほかの行は変更していません。

ここからが、この章の腕の見せどころです。この報告を、あなたは自分の目で検品できます。

ファイルを開き、calcTotal を探す(検索機能を使ってかまいません)。6.5の表の3行目にあたる行が * 80 になっている。ほかの行は元のまま。報告どおりです。

動作でも確かめられます。コーヒー2つとクッキー1つなら、800-80で720円になれば正しい。タイムセール中なら(800-80)×0.8=576、10円未満切り捨てで570円。受け入れ条件の数字も、もう自分で計算し直せます。

変更箇所を見て、期待する数字で動作を確かめる。コードと画面の両方から、同じ変更を検品できました。なお、80円はこの章の変更練習用です。演習のあとに50円へ戻し、第7章以降は通常時750円・2割引中600円の共通条件で進めます。

6.7 コメントを信用しすぎない

さて、変更後に次のような行が残っていたとします。見るのは、行の中ほどの数字と、`//` から後ろの説明文です。

役割ラベル: 計算

```
[javascript]
const setOff = countSets(items) * 80; // セット割: 1セットにつき50円引き
```

数字は80に変わったのに、後ろの説明が「50円引き」のまま——。

コードの中の `//` から始まる部分をコメントと呼びます。人間(とAI)のためのメモで、アプリの動きには一切影響しません。そして、影響しないからこそ、直し忘れが起きます。動いているのはあくまでコードで、コメントはコードに貼られた付箋です。

読み分けの基準はこうです。動作を決めるのはコード、正しい動作を決めるのは仕様。コメントとコードが食い違ったら、まず仕様を見返し、どちらを直すべきか判断します。

今回は仕様も80円に変更したので、コメントの直し忘れです。AIに「コメントも80円引きに合わせて直して」と頼んでください。半年後のあなたと、次の店長が、同じ行で迷わずに済みます。

6.8 AIにコードを説明させる質問——「この変更は仕様のどこに対応するか」

最後に、読む力を何倍にも増やす道具を配ります。AIへの質問です。

その前に、うまくいかない聞き方から。呪文の画面を前にした人は、たいていこう聞きます。

あなた → AI

このコードを全部説明してください。

AI → あなた

このアプリはコンポーネントベースで構成され、状態管理には……(以下、専門用語の説明が延々と続く)

返ってきたのは、翻訳という名の第二の呪文でした。「全部説明して」では、読む側が知りたい場所をAIも絞れません。いま確かめたい計算ルールに、質問の範囲を合わせましょう。

コツは、仕様との対応で聞くことです。この5つを、そのまま使ってください。

- 「この関数は、仕様書のどの行に対応しますか」
- 「セット割の80円は、どのファイルの何行目にありますか」
- 「この変数(`saleRate`)が0だと、何が起きますか」
- 「この変更で、変えていない場所はどこですか」

- 「この画面は、どんな部品(コンポーネント)でできていますか」

あなた → AI

calcTotal 関数を説明してください。ただし、仕様書の計算ルール(セット割、タイムセール、端数)のどの行に対応するかを、コード1行ずつ教えてください。専門用語は使わないでください。

AI → あなた

対応は次のとおりです。**sumOf(items)** が「小計を出す」、**countSets(items) * 80** が「セット割: 1 セットにつき80円引き」、**if (saleRate > 0) ...** が「タイムセール中なら割引く」、最後の行が「10円未満の切り捨て」に対応します。

この対話例で変えたのは、質問の範囲です。

第3章で、願いごとを注文書に変えました。この章では、説明の依頼に「何との対応を知りたいか」を加えます。返ってきた説明はコードと照合し、納得できない行だけ聞き直してください。

▼コーナー「先輩エンジニアならここを見る」

3つめの質問「この変数が0だと何が起きますか」は、プロがコードレビューで使う型そのものです。ふつうの値ではなく、端っこの値(0、空っぽ、マイナス)を差し込んで揺さぶる。**calcTotal** に0品の注文が来たら? この揺さぶりは第12章で「境界値」という正式な名前をもらいます。

別の条件でも動くか試す

この読み方は、会計アプリ以外でも同じように使えます。

そして忘れないでください。AIの書くコードは毎回違います。変数名も、行の順番も、この本と同じにはなりません。それでも、次の3つが確認できれば、あなたは読めています。

1. コードのどこかに、メニュー5品(あなたの題材のデータ)が見つかる
2. 計算ルールの数字(80、割引率、10円)が、コードの中に見つかる
3. AIの説明を見たあと、自分でも仕様とコードを1組以上対応させて説明できる

見つからないときは、探すこと自体をAIに頼んでください。「セット割はどのファイルにありますか」で十分です。それでも対応する処理が見つからなければ、仕様がまだ実装されていない可能性もあります。画面で試す確認に戻りましょう。

演習6 呪文の解読

演習6-1 コードを色分けする

- ミッション: 会計アプリのコードから20行ほどをAIに抜き出させ、1行ずつ「計算・表示・データ・通信」の4色に仕分ける
- 時間の目安: 30分
- クリア条件:
 1. 15行以上を自分で仕分けた

2. AIの仕分けと比べ、食い違った行について双方の理由を言えた
 3. 見つけた役割を説明できた。現れない色があれば、その理由を説明した
- つまづいたら:
 - 箱(変数)を作っている行は、まず「データ」を疑う
 - **show** や画面の言葉があれば「表示」、数式があれば「計算」
 - 迷った行は保留にして飛ばす。全部塗ることが目的ではない
 - 裏ダンジョン: 「通信」の行が見つからなかったら、なぜ無いのかをAIに聞く(いまのアプリは、まだ外の誰とも話していないのかもしれない——次章の予告です)
 - 冒険の書に記録: 色分け済みのコード1画面

演習6-2 仕様を変えて、変更点を説明する

- ミッション: コードを書き直さず、セット割を50円→80円にAIに変更させ、変更点がどこかを自分の言葉で説明する
- 時間の目安: 30分
- クリア条件:
 1. 変更後、コーヒー2つとクッキー1つの合計が720円になる
 2. 同じ注文が、タイムセール中は570円になる
 3. 変更されたファイルと行を、自分で指させる
 4. 「変えていない場所」を1つ言える(例: 端数の切り捨ては元のまま)
- つまづいたら:
 - 合計が合わないときは、まず **80** をファイル内で検索する
 - 見つからなければ「80円はどの行にありますか」とAIに聞く
 - コメントが「50円」のまま残っていないかも見る(6.7)
 - (授業向け)ペアで互いのAIの変更点を交換し、相手の720円を検算し合う
- 裏ダンジョン: 端数ルールを「1円未満切り捨て」に変えたら570円がいくらになるか、予想してから変更させる
- 冒険の書に記録: 日本語・図・コードの対応表(記入済みのもの)

練習が済んだら、セット割を50円へ戻し、コメントと仕様書もそろえます。通常時750円、2割引中600円になれば、次章へ進む準備は完了です。

この章のまとめ

- コードは1行目から全部読まなくてよい。知りたい動きにあたりを付け、計算・表示・データ・通信の役割でまとまりを探す

- 変数は名前を付けた箱、配列は並んだ箱、オブジェクトは名前付きの箱のセット、関数は名前を付けた手順。仕様書で決めた項目や計算ルールは、コードの中で名前を持つ
- 関数は、「押された」などのイベントをきっかけに動く。画面は、いまの状態(ステート)を映して描き替わる
- 動作を決めるのはコード、正しい動作を決めるのは仕様。コメントは動きに影響しないので、直し忘れが残りやすい
- AIへの質問は「全部説明して」ではなく、「仕様書のどこに対応するか」で範囲を絞る。「直しました」は、変更された行と、期待する数字での動作の両方で確かめる

理解度チェック

1. 次のものを、この章の用語で答えてください。①値に名前を付けてしまっておく箱 ②「もし〜なら」で道を分ける書き方 ③名前を付けた手順
2. 画面に「タイムセール中」の帯が出ないと相談されました。コードのどこを探せばよいか、手がかりになる語を挙げて説明してください。
3. セット割を80円に変えるよう頼むと、AIは「変更点は1か所です」と報告し、コーヒー2つとクッキー1つの合計は720円と出ました。これだけで検品は済んだと言えますか。

解答のめやす

1. ①変数 ②条件分岐 ③関数(→6.2、6.4、6.5)
2. 「もし」と読む if で始まる行と、そこで使われている「セール中か」を表す真偽値の変数(作例では saleActive)を探します。その中に帯を出す表示の処理があるかも見ます(→6.4、6.2)
3. 言えません。セール中の570円も試します。コードでは、セット割の行が80に変わり、ほかの行(端数の切り捨てなど)が元のままかを自分で指さして確かめます。コメントが「50円引き」のまま残っていないかも見ます(→6.5、6.7)

60秒でクエスト報告

会計ボタンを押すと、「押された」というイベントが起きて、**calcTotal** という関数が動きます。関数は、注文の配列から小計を出し、セット割を引き、セール中ならさらに割引いて、10円未満を切り捨てます。結果はステートに入り、合計表示の部品が描き替わります。私はコードを書いていませんが、仕様書のどの行がコードのどの行に対応するかを指させます。だからAIの修正が仕様どおりかどうかを、自分で確かめられます。

—第5章では、Q2「ボタンを押すと、データはどこを通りますか」に構成図で答えました。今回は同じ問いに、コードの中まで潜って答えています。深掘り版のできあがりです。

レベルアップ!

Lv.6 称号: コードの読み手

- 手に入れた装備: 変数、型、配列、オブジェクト、条件分岐、繰り返し、関数、イベント、ステート、コンポーネント(+役割の色分け)
- 冒険の書: 日本語・図・コードの対応表

次の第7章では、この章の計算例にはなかった役割——通信が主役になります。自分のコードに通信の行があった人も、なかった人も、今度はその先を追ってみましょう。文化祭本部のイベント情報を受け取る窓口、APIをたずねます。

第7章 API——アプリ同士がお願いし合う仕組み

クエスト7: 伝説の窓口をたずねて

文化祭まで、あと17日。

この章のねらい

- APIへの頼み方を、エンドポイントとHTTPメソッドを使って説明できる
- JSONの1件と一覧の配列を見分け、どの項目が画面に出るかを説明できる
- 404・429・壊れたJSONをモックで再現し、案内表示と他画面の無事確かめられる

今回のクエスト

会計アプリの隣に、もうひとつ画面がほしくなりました。文化祭のイベント一覧です。ステージ発表の時間、他の店の出し物——お客さんに「次は何時に何があるの?」と聞かれるたび、紙のパンフレットを探すのは大変です。

ただし問題がひとつ。イベント情報はあなたの店のものではなく、文化祭の本部が管理しています。あなたのアプリの外にあるのです。

外にあるものを、借りてくる。それがこの章の技術、APIです。

ミッションは2つ。イベント一覧を表示すること。そして、借り先が閉まっても、あなたのアプリが壊れないようにすること。

まずAIに作らせてみよう

まず、外の窓口をまねた練習用の「イベント情報API」を、自分の開発環境に用意します。7.3にあるイベント1件ぶんのJSON例を、次の依頼と一緒にAIへ貼り付けてください。付録Aにも、準備と確認の手順があります。

あなた → AI

練習用に、固定のイベント一覧をJSONで返すAPIを作ってください。窓口は `/mock/events` とし、一緒に貼るイベント1件の例をもとに、IDが重ならない2件の架空データを用意します。応答の一番外側は配列 `[...]` とし、1件だけでも配列で返してください。会計アプリには「イベント」画面を追加し、そのAPIから取得した名前・場所・開始時刻を、開始時刻順に表示してください。

AI → あなた

イベント画面を追加しました。指定のAPIからイベント一覧を取得し、開始時刻の順に表示しています。

プレビューにイベント一覧が並びます。「12:00 軽音ライブ 体育館」「13:30 ダンス部 中庭」——。

ここで、データの出どころに注目してください。一覧画面のコードにイベントを書き込んだのではなく、APIの返事を使っています。

第5章で描いた構成図を思い出してください。画面はサーバーの窓口に「イベント一覧をください」と頼み、返事を受け取って並べます。練習では自分のサーバーに窓口を作りますが、本番で文化祭本部のAPIを使うときも、頼み方と返事を確かめる手順は同じです。

その「頼む先」の正体を見ていきましょう。

7.1 APIを「機能の窓口」として理解する

API(Application Programming Interface)は、ソフトウェアの機能を、決められた方法で使うための窓口です。この章では、ネットワーク越しにサーバーへ頼むWeb APIを扱います。

日常語で言うと、電話注文の窓口です。人間のお客さん用の店構え(画面)はありませんが、決まった番号にかけて、決まった言い方で頼めば、決まった形で品物が返ってきます。相手の厨房の中がどうなっているかは、知らなくていい。知らなくていいところが窓口のありがたさです。

図7-1では、アプリが決められた窓口とだけ往復している点を見てください。

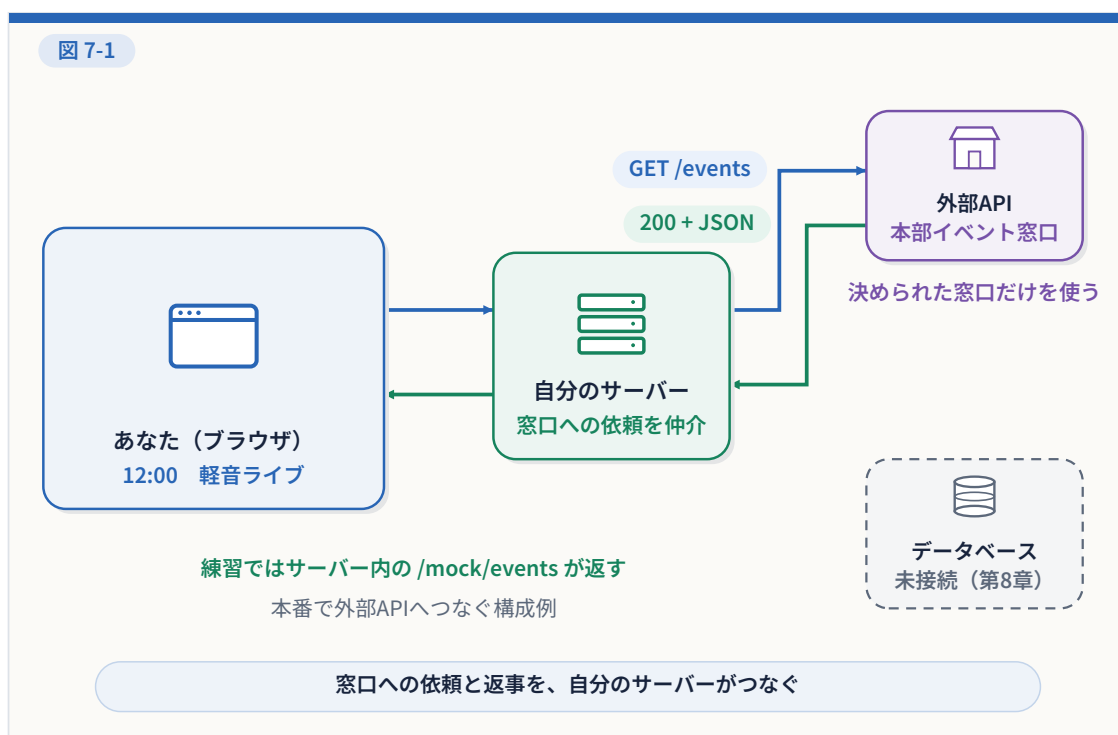


図7-1 APIは外部機能の窓口

地図、天気、決済、そしてAIにも窓口があります(AIのAPIは第15章の発展課題で扱います)。必要な機能をAPIで借りられれば、地図を一から作らずに、文化祭の会場案内に力を注げます。自分のアプリの仕事と、外へ頼む仕事を分けられるのが、APIの利点です。

7.2 エンドポイントとHTTPメソッド

窓口には住所があります。それがエンドポイントです。見た目はURLです。

- `…/events` — イベント全部の一覧
- `…/events/3` — 3番のイベント1件だけ

練習用の `/mock/events` も住所のひとつで、ネットワークタブのイベント取得の行に見えるはずですよ。

そして、窓口への用件には種類があります。第5章で学んだリクエストに、用件の種別(メソッド)が付いているのです。代表は4つ。

用件	意味	例
GET	ください	イベント一覧をください
POST	登録してください	新しいイベントを登録して
PUT	全体を置き換えてください	3番のイベント情報を、新しい内容一式で置き換えて
DELETE	消してください	3番のイベントを取り消して

「作る・読む・変える・消す」——次の第8章で出会うCRUDに結びつく並びです。

※厳密に言うと: これはよくある設計例で、メソッドとデータベース操作が必ず1対1に対応するとは限りません。

この「住所+用件+返事」のやりとり全体の作法に付いた名前がHTTPです。第5章でブラウザとサーバーの会話を観察しましたが、あの会話の作法の正式名称が、実はHTTPでした。

ネットワークタブでイベント取得の行の詳細を開くと、用件(Method)の欄に `GET` と出ているはずですよ。

※読み方の補足: 一部だけを変更する窓口には `PATCH` も使われます。「開始時刻だけ変更して」なら、こちらが合うことがあります。実際には、使うAPIの仕様書でメソッドと渡す項目を確かめます。

7.3 JSONの読み方

窓口からの返事は、多くの場合JSONという形式で届きます。実物を見ましょう。イベント1件ぶんです。

役割ラベル: データ

```
[json]
{
  "id": 3,
  "name": "軽音ライブ",
  "place": "体育館",
  "start": "12:00",
  "reserved": false
}
```

初めて見ると記号だらけですが、読み方のルールは3つだけです。

1. `{ }` は、ひとかたまり(イベント1件)
2. `"名前": 値` のペアで、1つの情報(`"place": "体育館"`)
3. `[ ]` は、並び(イベントが6件なら `[{…},{…},…]`)

JSONは、データを受け渡すための共通の形式です。第6章のオブジェクトとよく似ていますが、項目名は二重引用符で囲み、コメントは書けません。まずは「項目名と値のペア」が読めれば、返事と画面を見比べられます。

困ったときは、秘密情報や個人情報を除いた返事の例をAIに渡し、「このJSONのどの値が、画面のどこに表示されますか」と聞いてみてください。説明の材料が具体的になります。

図7-2では、JSONの項目が画面のどこに表示されるかを、線でたどってください。



図7-2 JSONと画面の対応

APIが返す「一覧」は、上の1件を配列 `[]` に並べた形です。今回の練習用APIは、1件でも `[{…}]`、0件なら `[]` を返します。画面への取り込みは、AIが書いたコードがやっています。次の抜粋では、`for` の行で配列から1件ずつ取り出します(読む練習です。第6章の色分けを思い出してください)。

役割ラベル: 通信・表示

```

[javascript]
// イベントAPIから一覧を取得して表示する
async function loadEvents() {
  const res = await fetch("/mock/events"); // 窓口にGETで頼む(通信)
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  const events = await res.json(); // 返事をデータとして読む
  for (const ev of events) {
    addCard(ev.start, ev.name, ev.place); // 1件ずつ画面に並べる(表示)
  }
}

```

`fetch`が「窓口に頼みに行く」の合図、`await`が「その結果を待つ」の合図です。`res.ok`は、返事の番号が成功を示しているかを確認しています。404(見つからない)の返事が返っても、`fetch`だけでは失敗として止まりません。

そのため、返事の番号を確かめます。壊れたJSONを読めない場合も含め、失敗したときは案内の表示へつなぎます。その受け止め方を7.7で整えます。

7.4 ステータスコード——返事には番号が付いている

HTTPの返事には、3桁のステータスコードが付いています。「返事は届いたが断られたのか」「サーバーで処理に失敗したのか」を調べる手がかりです。通信がつながらず、HTTPの返事自体が届かない場合は、番号が得られないこともあります。

番号	意味	日常語で言うと
200	成功	「はい、どうぞ」
400	頼み方が変	「ご注文の意味がわかりません」
401	有効な認証情報がない	「会員証を確かめられません」
403	依頼を許可できない	「その注文はお受けできません」
404	対象が見つからない	「その窓口や品物はありません」
429	頼みすぎ	「少し間をあけてください」
500	向こうの故障	「厨房でトラブルが起きています」

401の「認証」は、頼む側が誰かを確かめることです(第9章で扱います)。

ここで開発者ツールのネットワークタブ(第5章)を開いて再読み込みすると、イベント取得の行に「200」が付いているのが見えます。

覚えなくて大丈夫です。表を引ければいい(付録Bにも載せてあります)。ただ、ひとつだけ法則を持ち帰ってください。

▼コーナー「先輩エンジニアならここを見る」

まず番号の先頭の数字を見ます。4で始まる返事なら、頼み方・対象・会員証・回数制限などを確認。5で始まる返事なら、返事をしたサーバー側の処理を調べます。ただし番号は出発点です。429なら待ち時間、404なら正しいURLと対象の有無など、APIの説明と返事の内容を合わせて判断します。

7.5 API仕様書の読み方、SDKとの違い

窓口の使い方をまとめたものが、API仕様書です。最初は次の4点から読みます。

1. 住所(エンドポイントの一覧)
2. 頼み方(メソッド、渡す項目、7.6の会員証が要るか)
3. 返ってくる形(JSONの見本)
4. 料金と回数制限(無料枠はどこまでか)

もうひとつ、AIのコードを読んでいるとSDKという言葉に出会います。SDKは、窓口とのやりとりを楽にする道具箱です。電話のかけ方を覚えなくても、ボタンひとつで注文してくれる専用電話機、と思ってください。

▼コーナー「AIの言うことを信じてよい?」

AIは、APIの仕様を記憶で書くことがあります。そして記憶は古いことがあります。存在しないエンドポイント、変わってしまった頼み方——もっともらしいコードが、実は思い出で書かれている。対策はひとつです。仕様書のURLか抜粋を、依頼と一緒に貼って渡す。第4章の「良いプロンプトより、良い前提情報」は、APIの場面でこそ効きます。

7.6 APIキーとレートリミット

本格的な窓口の多くは、APIキーを求めてきます。APIキーは窓口の会員証です。会員証があると、窓口は「誰が、何回頼んだか」を数えられます。

練習用の `/mock/events` は、会員証なしで返事をくれる窓口です。

窓口が回数を数える目的のひとつが、レートリミット(回数制限)です。1分間に60回まで、1日に1000回まで——さっきの表の429は、これを超えたときの返事でした。無料枠と料金の話は第14章で扱います。

ここでは掟をひとつだけ。会員証は、他人に見せてはいけません。コードの中に直接書いてもいけません(正しい置き場所は、この後の第9章で「金庫」と一緒に学びます。破るとどうなるかは第13章で見ます)。

ボス出現——窓口が閉まっていた日

さて、イベント一覧は動いています。ここで想像してください。

文化祭の当日、本部のサーバーが落ちたら?

想像で終わらず、練習用APIで体験しましょう。AIに「練習環境だけで、正常・404・429・壊れたJSONを選んで返せる切り替えを追加して」と頼みます。3通りの返事を、1つずつ受け取ってみます。

1. 対象が見つからない(404): 404を返す練習モードを選ぶ
2. 頼みすぎと判定される(429): 429を返す練習モードを選ぶ
3. 返事が壊れている: 途中で切れたJSONが返ってくる

失敗を扱う処理がなければ、一覧が空のまま止まったり、読み込み表示が消えなかったり、画面が真っ白になったりします。どんな見え方になるか、自分のアプリで観察してください。429を出すために本物のサービスへ大量のリクエストを送る必要はありません。

考えてみれば理不尽です。壊れたのは本部のサーバーなのに、お客さんの目の前で「壊れたアプリ」に見えるのは、あなたのアプリです。お客さんは本部のサーバーなんて知りません。

目指すのは、外部の失敗を、必要以上に広げないことです。イベント情報が取れなくても、会計の計算と記録は続けられる設計にします。

7.7 読み込み表示とエラー表示

第5章で、画面には四状態(表示中・読み込み中・データなし・エラー)がある、と学びました。いまのイベント画面は「表示中」しか作っていません。残りをAIに頼みましょう。

あなた → AI

イベント画面に、次の3つの状態を追加してください。

1. 読み込み中: 「読み込んでいます…」の表示
2. APIが失敗したとき(404や429、JSONが壊れているときも): 「イベント情報を取得できませんでした。時間をおいて、もう一度お試しください」と表示。アプリの他の画面はそのまま使えること
3. イベントが0件のとき: 「本日のイベントはありません」と表示

AI → あなた

3つの状態を追加しました。API取得の失敗はすべて捕捉し、日本語の案内を表示します。会計画面など他の機能には影響しません。

もう一度、3通りの返事を試してください。案内が出ることに加え、会計画面で通常時750円の注文を確定できるかも見ます。AIの完了報告だけでは、他の機能が守られた証拠にはなりません。

この設計を、表の形で残しておきます。これが成果物のエラー処理表です。

起きること	ユーザーに見せる文言	アプリの動き
読み込みに時間がかかる	読み込んでいます…	他の操作を妨げず待つ
404/429/500/壊れたJSON	イベント情報を取得できませんでした。時間をおいてお試しください	イベント画面のみ案内表示。他画面は通常どおり
イベントが0件	本日のイベントはありません	正常表示(エラーではない)

「0件」がエラーではなく正常、という行に注目してください。データがないことと、取りに行けないことは、別の出来事です。この区別ができる人のアプリは、現場で信頼されます。

7.8 モックAPIを使った安全な練習

最後に、種明かしです。この章で使った「練習用のイベントAPI」のようなものを、モックと呼びます。本物そっくりの、練習相手です。ネットワークタブに見える `/mock/events` の `mock` が、この言葉です。

なぜ本物のAPIで練習しないのか。理由は3つあります。

- 本物には、回数制限や料金がある
- 本物の仕様は予告なく変わり、教材が壊れる
- そして何より——本物は、都合よく壊れてくれない

404も429も壊れたJSONも、モックだからこそ安全に、何度でも再現できました。

第12章では、この「同じ条件をもう一度作れる」性質を、テストにも使います。

別の条件でも動くか試す

次は、扱いたい題材の公開APIをひとつ選び、公式の仕様書を読んでつないでみましょう。天気や郵便番号など、返事と画面を対応させやすい題材が向いています。料金・APIキーの要否・回数制限を先に確認します。ブラウザからの直接呼び出しを許可しないAPIや、秘密のキーが必要なAPIは、9.7のサーバー経由の形で扱います。

確認するのは、正しい返事を表示できることと、失敗したときに案内できることです。失敗の確認は、先ほどの練習用APIで繰り返せます。APIを替えるときも、最初に整えたエラー処理表を持っていきましょう。

演習7 イベント一覧+失敗演習

- ミッション: モックAPIからイベント一覧を表示し、3つの壊れ方すべてで「真っ白にならない」アプリにする
- 時間の目安: 60分
- クリア条件:
 1. イベント一覧が表示される
 2. 404・429・壊れたJSONの3パターンすべてで、画面が真っ白にならず、日本語の案内が表示される
 3. その間も、会計画面(アプリの他の機能)は通常どおり使える
 4. エラー処理表が3行以上埋まっている
- つまづいたら:
 - まず、開発者ツールのネットワークタブ(第5章)で番号と返事を見る。番号がなければ、接続できたかやブラウザのエラーも確認する
 - 番号があれば7.4の表を引き、頼み方、APIキー、待ち時間、サーバーの状態のどれを調べるか決める
 - 直らないときは、まとめて頼まず「読み込み中の表示だけ追加して」と機能単位で小さく頼み直す(第4章)
- 裏ダンジョン: 練習用APIで返事を遅らせ、「3秒待っても返事がないとき」に案内を出す(タイムアウト)。再試行できることと、待つのをやめても会計が続けられることを確かめる
- 冒険の書に記録: API接続仕様(住所・頼み方・返ってくる形・制限)+エラー処理表

この章のまとめ

- APIは、ソフトウェアの機能を決められた方法で使うための窓口。窓口の住所がエンドポイント、用件の種類がHTTPメソッドで、GETは「ください」にあたる

- JSONは、「"項目名": 値」のペアでデータを表す。波かっこ {} は1件のひとつかたまり、角かっこ [] は並びで、一覧は1件を配列に並べた形で届く
- 返事にはステータスコードが付く。200は成功、4で始まれば頼む側、5で始まれば相手側を調べる出発点。404が届いても fetch だけでは止まらないので、番号を確かめ、失敗時は案内を出して他の機能を使えるままにする
- APIキーは窓口の会員証で、他人に見せず、コードに直接書かない。回数制限(レートリミット)を超えると、429の返事が返る
- 失敗の練習は、本物の窓口ではなくモックで行う。モックなら、試したい失敗を安全に、何度でも再現できる

理解度チェック

1. APIから、波かっこ {} で囲まれたひとつかたまりと、角かっこ [] の中に {} が並んだものの、2種類の返事が届きました。それぞれ何を表していますか。
2. 文化祭当日、イベント画面を開いたときに、ネットワークタブのイベント取得の行に429が出ていました。何が起きていると考えられますか。また、そのとき画面とアプリはどうなっているべきですか。
3. イベントを取得するコードで、fetch がエラーで止まりませんでした。これだけで「イベントの取得は成功した」と言えますか。理由も答えてください。

解答のめやす

1. 前者はイベント1件ぶんのひとつかたまり、後者はイベントの並び(一覧)です(→7.3)
2. 頼みすぎと判定された(回数制限を超えた)と考えられます。少し間をあけて頼み直します。画面は真っ白にせず「イベント情報を取得できませんでした」の案内を出し、会計画面は通常どおり使えるべきです(→7.4、7.6、7.7)
3. 言えません。404などの失敗の返事が届いても、fetch だけでは止まらないからです。返事の番号(res.ok やステータスコード)で、成功かどうかを確かめます(→7.3、7.4)

60秒でクエスト報告

イベントのボタンを押すと、私のアプリが、本部のイベントAPIという窓口でGETで「一覧をください」と頼みます。返事はJSONという形式で届き、アプリがそれを読んで画面に並べます。窓口が404や429を返したときや返事が壊れていたときは、画面を真っ白にせず、日本語の案内を出して、他の画面はそのまま使えるようにしてあります。

——これは口頭試問の第2問「ボタンを押すと、データはどこを通りますか」への、そのまま答えの型です。

レベルアップ!

Lv.7 称号: 窓口と話せる人

- 手に入れた装備: API、エンドポイント、HTTP、JSON、ステータスコード、APIキー、レートリミット、モック
- 冒険の書: API接続仕様+エラー処理表

借り物のデータは、表示できるようになりました。では、あなた自身のデータ——会計の記録は、いまだどこにあるのでしょうか。序章のあの事件を思い出してください。再読み込みで、消えていませんか。次の第8章で、いよいよ「ノート」を手に入れます。

第8章 データはどこに保存されるのか

クエスト8: ついに、ノートを手に入れる

文化祭まで、あと14日。

この章のねらい

- 記録の保存先を、規模・共有・安全の3つで選び、理由を説明できる
- 会計のテーブルを、主キー・列・データ型を決めて設計図に描ける
- 取消を論理削除で残し、集計から除かれたことを金額で確かめられる

今回のクエスト

会計アプリは、ずいぶん頼もしくなりました。注文画面で合計が出て、履歴が残り、店長画面には集計とタイムセールのスイッチがあります。第7章ではイベント一覧まで付きました。

そこで今夜は、部員を集めて通し練習です。本番さながらに、注文を次々にさばいてみます。合計が合っているかだけでなく、記録が残るかも確かめる夜です。

ミッションは、記録の置き場所を選び、再読み込みや端末の交代を越えて使えるようにすることです。さらに、取り消した記録をどう残すか、写しから何を戻せるかまで決めます。序章で予告した「ノート」に、今日は罫線を引いていきます。

ボス出現——会計履歴も、再読み込みで消えた

通し練習は順調でした。30件あまりの会計を打ち込み、履歴がたまり、店長画面の集計も動いています。ところが、タブレットを次のシフト役に手渡した拍子に、指が画面を引っばりました。再読み込みです。

履歴画面が、まっさらになりました。32件の会計が、全部消えました。集計もゼロです。

この光景に、見覚えがありませんか。序章の気分記録アプリと、まったく同じ事件です。あのときは「日記として致命的」で済みました。今回は笑えません。

これが文化祭当日の昼に起きたら、レジの現金と売上を突き合わせる手がかりを失います。タイムセールの前後を比べたくても、比較する記録がありません。

なぜ消えたのか。答えは序章と同じです。会計の記録は、画面のステート(第6章)として持たれていただけでした。

このアプリのステートは黒板です。書くのも消すのも速い反面、保存先がなければ、再読み込みで元の空の状態に戻ります。

残したければ、ノートに書き写しておく必要がある——序章でそう予告しました。そのノートの正体を明かし、実際に手に入れるのが、この章です。

8.1 メモリと永続化——消える記憶、残る記憶

まず、アプリの記憶には2種類あることを押さえます。

1つめは、アプリが動いているあいだだけの記憶です。コンピューターが作業中の値を置いておく場所で、メモリと呼ばれます。日常語で言うと、作業机の上です。

画面の作業中のステートは、ここに置かれます。再読み込みや電源オフで失われるため、それだけでは長く残す記録には向きません。

2つめは、電源を切っても残る記憶です。作業机の上のものを、残る場所へ書き写すことを永続化(persistence)と呼びます。日常語で言うと「消えないように残すこと」です。

図8-1では、再読み込みのあと、黒板(メモリ)は空になり、ノートへ写した会計だけが残るところを見てください。

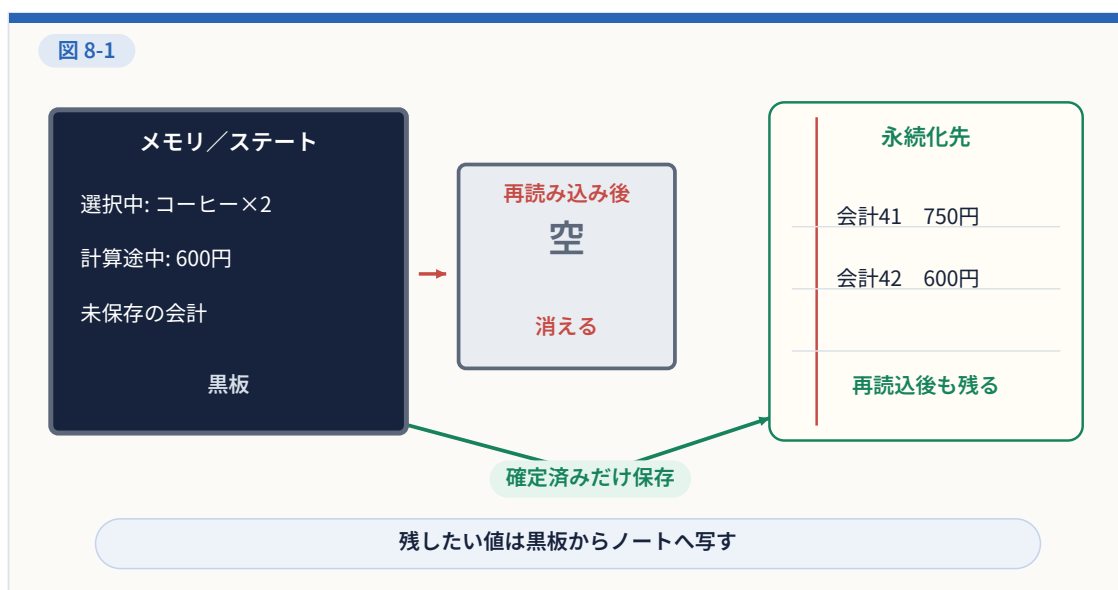


図8-1 消える黒板、残るノート

ここで、自分のアプリに引きつけた問いをひとつ。このアプリで、消えては困るデータはどれですか。会計の記録は、困ります。では、選択中の注文はどうでしょう。会計前なら、消えても打ち直せば済むかもしれません。何もかも永続化する必要はありません。消えては困るものを選ぶところから、保存の設計は始まります。

8.2 ローカルストレージ、ファイル、スプレッドシート、データベース

「ノート」といっても、1種類ではありません。図8-2で、保存方式ごとの特徴を比べます。上位・下位という順位ではなく、どこで共有し、どう守るかを選ぶための表です。

- **ローカルストレージ**: 同じ端末・ブラウザの、そのサイト用の小さなメモ帳です。再読み込みでは消えませんが、別の端末とは自動で共有されません。ブラウザのデータ削除などでは消えることがあります

- **ファイル**: 文章や表の形で書き残す方式です。手元にもサーバーにも置けます。公開先によっては一時的な領域しか使えないため、保存先が残る場所かを確認します
- **スプレッドシート**: 表計算サービスのシートに書き込む方式です。人がそのまま開いて読めるのが強みです
- **データベース**: データを記録し、探し、守ることだけを仕事にする、専用の仕組みです

序章で予告したノートとして、この会計アプリではサーバー側のデータベースを選びます。記録をまとめて保存し、条件を指定して探し、複数の端末から使うための仕組みです。保存や検索を助けてくれますが、正しい設定やバックアップは別に必要です。

図 8-2

会計アプリの条件から選ぶ

複数端末 売上データ 消えると現金不一致

↓

保存方式	規模	共有	安全・復元
ローカルストレージ	小	一台	端末依存
ファイル	小～中	サーバー経由	別途備え
表計算シート	小～中	人と共有	権限管理
データベース	中～大	複数端末	権限・復元

規模・共有・安全で保存先を選ぶ

図8-2 保存場所の選び方

どの方式も、ノートの一種です。どれを選ぶかの物差しは3つあります。**規模**(何件たまるか)、**共有**(何台の端末・何人で見えるか)、**安全**(消えたり壊れたりしたら、どれだけ困るか)。

1台で試す短いメモなら、ローカルストレージが手軽です。複数の端末で同じ記録を見たいなら、サーバー側の共有できる保存先を選びます。スプレッドシートなども候補ですが、会計の保存・検索・取り消しを扱う本書では、データベースを使います。

会計アプリはどうでしょう。レジのタブレットと店長のスマートフォン、最低2台で使います。消えたら現金と合わなくなります。共有と復元を備えた保存先が必要です。

※厳密に言うと: データベースという言葉は本来「データの集まり」を指し、それを管理するソフトはDBMSと呼ばれます。本書では区別せず、まとめて「データベース」と呼びます。

AIと一緒に直す――ノートを手に入れる

では、直しに行きましょう。事件の直後、多くの人はこう頼みます。

あなた → AI

会計の履歴が、再読み込みすると全部消えます。消えないようにしてください。

AI → あなた

会計履歴をローカルストレージに保存するようにしました。再読み込みしても、履歴と集計が残ります。

練習用の会計32件を打ち直して試すと、今度は再読み込みしても、32件がそこにあります。消えた記録が戻ったのではなく、これからの記録が残るようになったのです。

ところが翌日、店長役の部員が自分のスマートフォンで店長画面を開くと、売上はゼロです。レジのタブレットには32件ある。どちらも壊れていません。

ローカルストレージは「その端末だけのメモ」だからです。タブレットのメモは、店長のスマートフォンからは見えません。

最初の依頼が悪かったわけではありません。ただ、「何台で共有するか」を言っていませんでした。言われていない部分は、AIが手軽な保存方式で埋めた——第3章のボスと同じ現象が、保存場所でも起きるのです。今度は、物差しごと伝えます。

あなた → AI

保存先を変えたいです。会計の記録は、レジのタブレットと店長のスマートフォンの、どちらから読み直しても同じになるようにしてください。サーバー側のデータベースを使います。記録する項目は、会計日時、品目と数量、販売時単価、合計金額、セット割額、セール適用の有無と適用した割引率です。サーバーが会計確定時のセール状態で計算し、保存に成功してから「会計完了」と表示してください。すでにある記録は、消さずに移行方法を説明してください。

AI → あなた

データベースを用意し、会計を記録するテーブルを作りました。会計を確定するとサーバーに送られて1件ずつ保存され、履歴画面と店長画面はデータベースから読み込んで表示します。どの端末から開いても、同じ履歴が見えます。

確かめましょう。タブレットで会計する。保存完了を確認し、店長のスマートフォンで履歴を読み直すと、同じ1件が現れる。両方で再読み込みしても消えない。

片方で会計したとき、もう一方の画面が自動で書き換わる機能は、別の設計です。ここではまず「同じ保存先を読み直せること」を確認します。

最後に通信を切り、保存できない会計が「完了」と表示されないことも試します。「画面に出た」と「保存できた」を分けられれば、記録の置き場所を変えた効果が確かめられます。

図 8-3

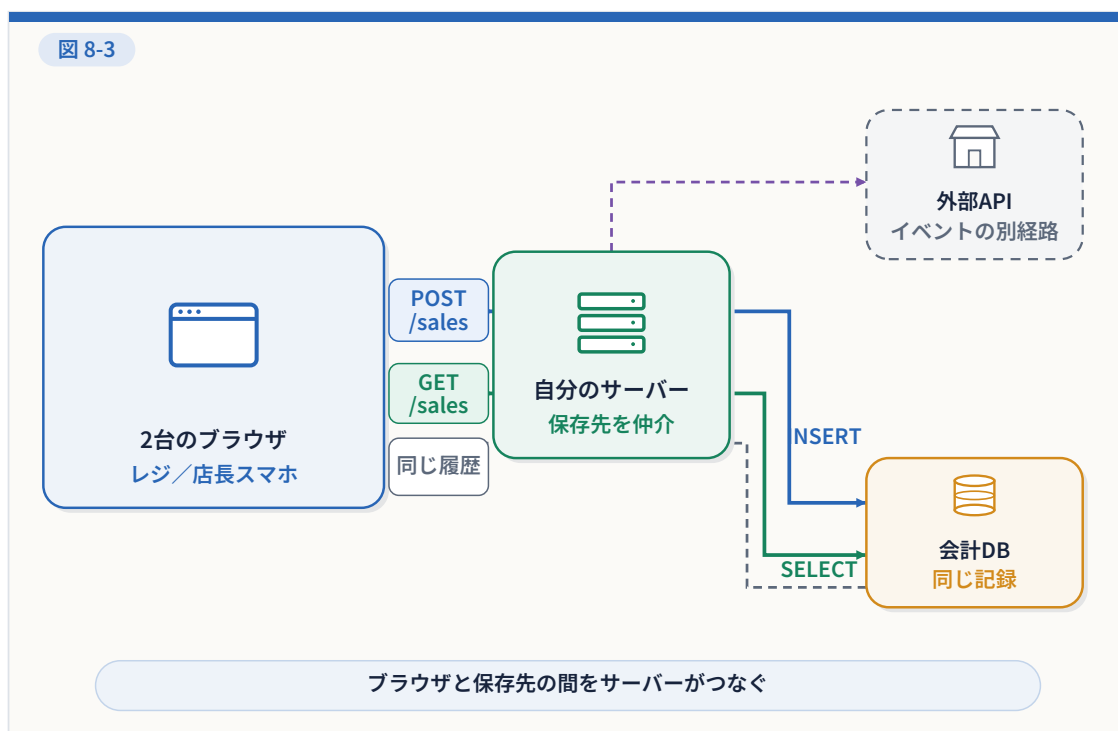


図8-3 データベースを加えた構成

図8-3では、2台の端末が、どちらもサーバーを通して同じ保存先を読み書きしている点を見てください。

第5章の構成図を覚えていますか。あのときデータベースは、まだ点線でした。今日、4人目のデータベースが実線で加わり、この本の構成図はようやく全員そろいました。

8.3 テーブル、行、列——IDと主キー、スキーマ、データ型

手に入れたノートの中を、のぞいてみましょう。AIに「会計はどんな形で保存されていますか。表で見せてください」と頼むと、図8-4のような表が出てきます。

テーブルは、ノートの1ページです。日常語で言うと、1種類の記録を集めた表。横の行が記録1件(会計1回)、縦の列が項目です。

図 8-4

主キー=行の背番号

列=項目の種類

id INTEGER	time 日時	items TEXT	total INTEGER	set_discount INTEGER	sale_applied BOOLEAN
41	13:02	コーヒー×2 クッキー×1	750	50	false
42	14:11	コーヒー×2 クッキー×1	600	50	true
43	14:13	ラムネ×1	120	0	true

1行=会計1件

日時は同日の時刻のみ表示／列の一部を抜粋

同じ品目でも、会計番号・時刻・適用した割引は別

図8-4 会計テーブルの読み方

41番と42番を見てください。品目はまったく同じです。人間なら「同姓同名」で混乱する場面です。取り消したいのはどちらの会計なのか。「コーヒー2つとクッキー1つのやつ」では特定できません。そこで、行を1件ずつ見分けるための主キーを決めます。ここでは会計番号が、行の背番号です。主キーは、同じテーブルの中で重複できず、空にもできません。本書では、一度付けた会計番号を変えない運用にします。

背番号があるから、「42番を取り消して」と取り違えずに言えます。同じ注文が1日に何十回あっても、です。

次に、スキーマ。列の名前と、そこに何を書くかを先に決めたものです。日常語で言うと、ノートの罫線。

じつは、あなたはもう下書きを持っています。第3章3.5で作った「入力・記録・表示」の表——あれがスキーマの下書きでした。「セール適用の有無を記録する」と決めたのも、あの表です。

罫線には、もうひとつ決めごとが含まれます。データ型、つまり列ごとに入るものの種類です。第6章で出会った「型」との再会です。

合計は数値、時刻は日時、セール適用は真偽値。型を決めておくと、「750」と「七百五十円」が同じ列に混ざる事故を防げます。

スキーマの正体を、コードでも見ておきましょう(読めなくて大丈夫です。日本語の表と1対1で対応している、という一点だけ見てください)。

役割ラベル: データ

```
[sql]
CREATE TABLE sales (      -- 会計の表を作る
  id      INTEGER PRIMARY KEY, -- 会計番号
  time    TIMESTAMP NOT NULL, -- 日付を含む会計日時
  total   INTEGER NOT NULL,  -- 合計金額(円)
  set_discount INTEGER NOT NULL, -- セット割で引いた額(円)
  sale_applied BOOLEAN NOT NULL, -- セール適用の有無
  sale_rate NUMERIC NOT NULL, -- 適用率(通常0、2割引0.2)
  canceled BOOLEAN NOT NULL DEFAULT false -- 取消済みか(8.7)
); -- ※品目の列の持ち方は8.6で見直します
```

time の行のコメントに注目してください。第6章の**time: "13:05"**は画面向けの短い例でしたが、保存する会計日時には日付も含めます。時刻だけでは、別の日の同じ時間帯の会計と見分けられないからです(8.5)。

※厳密に言うと: これは列の対応を読むための例です。日時の型や会計番号の自動発行方法は、使うデータベースに合わせます。また、コードの**setDiscount**・**saleApplied**と、表の**set_discount**・**sale_applied**は、同じ項目を別の命名規則で表しています。

▼コーナー「先輩エンジニアならここを見る」

テーブルの一覧は、アプリを理解する手がかりになります。記録の形がわかれば、どんな集計ができるか、何をまだ記録していないかが見えるからです。成果物の「データベース設計図」は、次の担当者への案内図にもなります。

8.4 CRUD——作る、読む、変える、消す

記録に対する基本操作を、4つに整理します。作る(Create)、読む(Read)、変える(Update)、消す(Delete)。頭文字をとってCRUDと呼びます。日常語で言うと、記録の基本4動作です。

第7章で、窓口への用件の表は「CRUDに結び付く並び」だと予告したのを覚えていますか。ここで回収します。

会計アプリでしたいこと	CRUD	窓口への用件(第7章)
会計を記録する	Create(作る)	POST(登録してください)
履歴・集計を表示する	Read(読む)	GET(ください)
記録を書き換える	Update(変える)	PUT(全体を置換)/PATCH(一部を変更)
記録を消す	Delete(消す)	DELETE(消してください)

これは窓口の設計例です。会計ボタンを押す→POSTで記録を頼む→テーブルに1行増える。履歴画面を開く→GETで一覧を頼む→テーブルから行を読む。窓口への用件と、保存先で起こることをつなげて読めるようになります。

アプリの機能は、ほとんどがCRUDの組み合わせで分解できます。「お気に入り登録」は、お気に入りを作る・読む・消す。「プロフィール編集」は、読む・変える。

自分のアプリに引きつけた問いはこうです。このアプリの各機能は、CRUDの4語でどう言い直せますか。言い直せた機能は、AIに頼める形になっています。

8.5 検索とクエリ

店長には、知りたいことがあります。「今日の14時台の売上は、セール適用の分だけでいくらだったのか」。数件なら目で足せますが、文化祭3日分、800件を毎回調べるのは大変です。

データベースの得意技が、まさにこれです。データベースは、質問に答えるのが仕事のノートです。その質問文をクエリと呼びます。日常語で言うと、データへの質問文です。

店長の質問をクエリにすると、次のようになります。各行の右にある日本語のコメントと見比べてください。

役割ラベル: 計算

```
[sql]
SELECT COALESCE(SUM(total), 0) FROM sales
WHERE time >= '2026-11-07 14:00:00' -- 例: 11月7日14時以降
      AND time < '2026-11-07 15:00:00' -- 同日15時より前
      AND sale_applied = true      -- セール適用あり
      AND canceled = false;       -- 取消済みは除く(8.7)
```

読み方は3語だけです。**SELECT**=何を出すか。**FROM**=どのページ(テーブル)から。**WHERE**=どんな条件の行だけ。

WHEREの条件には、日付まで書いてあります。同じ14時台でも、別の日の会計を混ぜないためです。日付は練習用の例で、時刻は保存と集計で同じ基準(この例では文化祭会場の時刻)にそろえます。

SELECTのすぐ後の**COALESCE**は、対象が0件のときに合計を0円として返すための道具です。

JSONのときと同じ掟を使います。書けなくていい、読めればいい。あなたが書くのは日本語の質問で、クエリへの翻訳はAIの仕事です。

※厳密に言うと: この質問の言葉の代表がSQLという言語です。データベース製品ごとに方言がありますが、上の3語はほぼ共通です。

ただし、ひとつ急所があります。日本語の質問がクエリに翻訳できるかどうかは、スキーマ次第ということです。もし「セール適用」の列がなかったら、この質問には誰も——AIもデータベースも——答えられません。

記録するときに書いておかなかったことは、あとから聞けないのです。第3章3.5の「あとで何を知りたいかから逆算して、記録するものを決める」は、この日のための備えでした。

8.6 データ同士の関係と重複データ

図8-4の品目の列には、「コーヒー2、クッキー1」と品名を書き込んでいました。ここに罣が潜んでいます。

文化祭の朝、チャイを400円から450円に値上げしたとしましょう。注文画面、メニュー管理、集計用の一覧がそれぞれ別の価格を持っていると、直すたびに食い違いが生まれます。品名も「チャイ」と「ちゃい」でゆれると、同じ商品として集計しにくくなります。

原因は同じです。同じ事実を、何か所にも重複して書いているからです。直すときに1か所でも忘れると、ノートの中で事実が食い違います。

対策は、ページを分けて、背番号でつなぐことです。メニュー表には商品番号・品名・現在価格、会計表には会計番号・日時・合計を置きます。さらに、1件の会計に含まれる商品ごとに注文明細を作り、会計番号・商品番号・数量でつなぎます。「会計41番の明細に、商品1番を2つ」と書く形です。

図 8-5



現在の価格と、会計時点の単価を分けて残す

図8-5 番号でつなぐデータ

図8-5では、**menu**(メニュー表)・**sale_items**(注文明細)・**sales**(会計表)の3つが、番号でつながっているところをたどってください。

ただし、会計した時点の金額は、履歴として残します。注文明細にはそのときの単価を、会計表には割引後の合計、セット割額、適用したセール率を保存します。金額はサーバーが会計確定時の状態で計算し、あとでセールを切り替えても、確定済みの金額は変えません。

値上げ後の「現在価格450円」と、昨日売った「販売時単価400円」は、同じ事実の重複ではありません。時点の違う2つの事実です。

メニューの価格を1か所で見ると、新しい注文には新価格が使われ、過去の明細は販売時の価格を保ちます。「現在の情報」と「その時点の記録」を分けておけば、値上げのあとも、過去の売上を説明できる帳簿になります。

8.7 入力値の検証、削除と論理削除

ここからは、ノートを守る話です。守る相手は2つ。うっかりと、取り消しです。

まず、うっかり。注文画面の数量に、マイナスが入ったらどうなるでしょう。「ドーナツ -3個」の会計が保存できてしまうと、合計から750円が引かれた記録が残ります。打ち間違いでも起きますし、もしわざとやる人がいたら、それは売上の改ざんです。

対策は、ノートに書き込む前の見張りです。注文に含める数量は1以上の整数、合計は0円以上。条件を外れた入力は、保存せずに案内します。画面での入力制限に加えて、サーバーでも数量を検証し、商品価格と割引のルールから合計を計算します。ブラウザから届いた合計を、そのまま帳簿に写さないためです。

「入力を信用しない」という開発の大原則の、これが入口です。本格的な話は第13章でやります。悪意のある入力は、マイナスどころではないからです。

次に、取り消し。第3章の受け入れ条件4を覚えていますか。

会計を取り消すと、通常の履歴一覧と売上集計から除外される。取消済みの記録は残す

素直に実装すると「取り消し=行を消す」です。ところが、行を本当に消すと困ることが起きます。押し間違いで取り消したら、二度と戻せません。閉店後に現金が合わないとき、「何がいつ取り消されたか」を調べる手がかりも、一緒に消えています。

そこで論理削除という書き方を使います。行を消す代わりに、「取消済み」の列に印を付け、履歴の表示と集計からは、印の付いた行を除外するのです。紙の帳簿で、間違えた行を修正液で消さず、線を引いて「取消」と書き添える——あの作法と同じです。

SQLで書くと、次の2行になります。1行目が印を付ける操作、2行目が印のない行だけを足す集計です。

役割ラベル: データ

```
[sql]
UPDATE sales SET canceled = true WHERE id = 42; -- 42番に「取消」の印を付ける
(変える)
SELECT SUM(total) FROM sales WHERE canceled = false; -- 集計は、印のない行だけ(読む)
```

第3章で紙に書いた受け入れ条件が、印を付ける1行と、除外して集計する1行になりました。通常の履歴と集計からは外れ、記録には残っています。8.5のクエリにも、同じ **canceled = false** が入っています。

取り消した日時や担当者も後で確認したいなら、それらを別の項目として保存します。印だけでは「いつ、誰が」はわかりません。

図8-6では、取消の前後で行は残ったまま、**canceled** の値だけが変わる場所を見てください。

図 8-6



図8-6 記録を残す論理削除

なお、本当に消すべきデータもあります。消すことが義務になる個人情報などは、第13章であらためて扱います。

8.8 CSV入出力とスプレッドシート連携

文化祭が終わると、部長には仕事が残っています。顧問の先生への会計報告です。先生が使う報告書にも数字を移す必要があります。紙と表計算の世界へ、データを持ち出す手段が要ります。

その共通語がCSVです。表を「コンマ区切りの文章」に仕上げる形式で、ほぼすべての表計算ソフトが読み書きできます。日常語で言うと、表の共通語です。

会計履歴をCSVにすると、次のようになります。1行目が列の名前、2行目からが会計1件ずつです。

役割ラベル: データ

```
会計番号,会計日時,合計,セット割,セール適用,取消済み
41,2026-11-07 13:02:00,750,50,なし,
42,2026-11-07 14:11:00,600,50,あり,取消
43,2026-11-07 14:13:00,120,0,あり,
```

AIには、こう頼みます。「店長画面に、会計履歴をCSVで書き出すボタンを付けてください。取消済みの行も、印付きで含めてください」。書き出したCSVをスプレッドシートに読み込めば、グラフも報告書もそちらで作れます。

逆方向もできます。メニュー表をスプレッドシートで管理して、アプリに読み込む。部員みんなが表計算に慣れているなら、それも立派な設計です。

ひとつだけ注意があります。書き出した瞬間から、そのCSVはアプリの管理の外にあります。

渡した先で数字が書き換えられても、アプリは気づけません。アプリが管理する原本はデータベース、CSVは書き出した時点の写し。この区別を忘れないでください。

そして、この「写し」という考え方が、最後の道具につながります。

8.9 バックアップの考え方

データベースの記録は、画面の再読み込みでは消えません。でも、次のような事故への備えは必要です。

- 操作のうっかり(「履歴を全部消して」を試しに実行してしまう)
- 保存先のサービス側の故障や停止
- AIに「データベースを作り直して」と頼んだら、中身ごと初期化された

特に、3つめは開発中にも起こり得ます。「表の構造を変えたい」と「記録を消してよい」は別の依頼です。変更前に、既存の記録をどう保つかを確認しましょう。

備えの名前がバックアップです。ノートの写しを、別の場所にとっておくこと。日常語で言うと、ノートの控えです。考えることは3つだけです。

1. いつ写すか——閉店後に1回なら、失い得るのは最大でその日の記録。1日分を失えないなら、もっと短い間隔にする
2. どこへ写すか——元と別の場所へ。同じデータベースの隣のページに写しても、データベースごと消えたら一緒に消えます
3. 戻せるか——本番とは別の練習場所で復元し、件数・合計・取り消しの扱いまで確認する

8.8のCSVは、金額を照合できる緊急の控えになります。ただし、あのCSVには注文明細やテーブル間の関係がなく、権限や設定も入りません。そのままではアプリ全体を復元できません。

復元には、使うデータベースのバックアップ機能で、必要なデータと表の構造を保存します。データベースの外で管理している設定やログインの仕組み(第9章で扱います)も含めて、何をどこから戻すかを書き留めます。「前日まで戻せる」と言えるのは、その保存範囲と復元手順を確かめたあとです。

CSVは照合に、バックアップは復元に。この2つを分けて備え、第14章で運用手順に仕上げます。

別の条件でも動くか試す

保存場所は、題材と失いたくない度合いで選びます。読書メモを1台で試すならローカルストレージが手軽ですが、大切な読書記録なら書き出しもほしい。ゼミの貸し出し記録を係の自宅からも見るなら、共有できる保存先が必要です。どの保存先を選び、なぜその選択かを言えることが、この章のクリアの証です。

AIが選ぶ保存先やテーブルの形は、この本と同じにはなりません。それでも、①再読み込みしても記録が残る、②必要な端末すべてで同じ記録が見える、の2つが確認できていれば先へ進めます。保存まわりでAIが行き詰まったときは、まとめて頼まないでください。「まず会計1件を保存して、読み出して表示する」だけの、最小の依頼に戻すのが近道です(第4章)。

演習8-1 あの事件を、自分の手で解決する

- ミッション: 序章の気分記録アプリを、保存できるアプリに変更する
- 時間の目安: 30分
- クリア条件:
 1. 再読み込みしても、記録が残っている
 2. 記録がどこに保存されているかを、一言で言える
 3. 序章の「分からなかったことリスト」にあった「保存ってどこにされるの?」の行に、自分の言葉で答えを書き込めた
- つまづいたら:
 - 使い方を伝え、保存方式を指定して頼む(「ローカルストレージに保存して」)
 - 1台で試すなら、ローカルストレージと書き出しから始めてもよい
 - (授業向け)ペアで相手のアプリを操作し、保存先と、その選択理由を説明し合う
- 裏ダンジョン: 同じアプリを2つの端末で開き、記録が共有されるか確かめる。共有されないなら、その理由を図8-2の比較表で説明する
- 冒険の書に記録: 答えが書き込まれた「分からなかったことリスト」

クリアしたら、あの再読み込みを、わざともう一度やってみてください。画面が戻ってきて——新しく保存した記録は、そこにあります。今度は、残った理由も説明できます。

序章のあの夜、あなたは消えた記録の前で、なぜ消えたのかを説明できませんでした。いまのあなたは、原因を言えて、直せて、確かめられます。リストの1行が消える——序章で約束した、レベルアップの瞬間です。

演習8-2 タイムセールは効いたのか

第2章で先輩は、閉店後のレジ金の照合を「一番最悪だった」と振り返っていました。記録がノートに残ったいまなら、あの照合も、この集計と突き合わせて調べられます。

- ミッション: 会計履歴から、日ごと・時間帯ごとの売上を集計し、セール適用の有無で比べる
- 時間の目安: 60分
- クリア条件:
 1. 店長画面に、日ごと・時間帯ごとの売上が表で出る
 2. セール適用の会計と通常の会計が、分けて集計されている
 3. 会計を1件取り消すと、その分だけ集計の数字が減る(履歴の見た目だけでなく、合計の金額で確認する)
 4. データ項目一覧と、テーブルを線でつないだ設計図が1枚に描けている
- つまづいたら:

- ・ 集計が合わないときは、まず取消済みの行が混ざっていないかを疑う
- ・ 8.8のCSV例の42番600円と43番120円なら、取消前は720円、42番の取消後は120円。1マスだけ手計算して、画面の数字と見比べる
- ・ 14:00ちょうどの会計がどちらの時間帯に入るかは、あなたの決めごと。決めていなかったなら、いま決めて仕様書に足す(第12章で、まさにここを意地悪に確かめます)
- ・ 裏ダンジョン: セール中の1時間と直前の1時間で、売上と会計の件数を比べる。時間帯や客数も違うので、差をそのままセールの効果とは断定できない。「記録から言えること」と「まだわからないこと」を分け、店長に報告する
- ・ 冒険の書に記録: データ項目一覧+簡単なデータベース設計図

この章のまとめ

- ・ 画面のステートはメモリに置かれ、再読み込みで消える。消えては困る記録を選び、残る場所へ書き写す(永続化)
- ・ 保存先は、規模・共有・安全の3つで選ぶ。ローカルストレージはその端末だけのメモで、複数の端末で同じ記録を使うには、サーバー側のデータベースのように共有できる保存先が要る
- ・ テーブルでは、行が記録1件、列が項目。主キーで行を取り違えずに指し、スキーマで列の名前と型を決める。記録していない項目は、あとから質問できない
- ・ 同じ事実は1か所へ書き、番号でつなぐ。ただし会計確定時の単価や割引は、その時点の記録として残し、現在の価格で書き換えない。会計日時は日付まで含めて残す
- ・ 取消は行を消さずに印を付け(論理削除)、通常の一覧と売上から除く。CSVは照合用の写しで、アプリを戻すにはバックアップと、確かめた復元手順が要る

理解度チェック

1. 主キーとは何ですか。会計のテーブルに主キーがないと、どんな困りごとが起きますか。
2. チャイを400円から450円に値上げした翌日、店長画面で前日の売上まで増えていました。記録の持ち方のどこに問題があると考えられますか。
3. 取消ボタンで会計の行を削除するようになったところ、履歴からも集計からも消えました。第3章の受け入れ条件どおりと言えますか。

解答のめやす

1. 行を1件ずつ見分けるための背番号です。ないと、品目がまったく同じ会計41番と42番を区別できず、「42番を取り消して」と言えません(→8.3)
2. 過去の会計を、メニュー表の現在価格で計算し直していると考えられます。注文明細に販売時単価を、会計表に確定時の合計や割引を保存し、集計にはそれを使います(→8.6)
3. 言えません。受け入れ条件は「会計を取り消すと、通常の履歴一覧と売上集計から除かれる。元の記録は取り消し済みとわかる形で残す」です。行は消さずに取消の印を付け、集計は印のない行だけで行います(→8.7)

60秒でクエスト報告

会計はサーバー側のデータベースに保存し、保存成功を確認してから完了にします。会計表には会計番号、日時、合計、割引額などを、注文明細には商品番号、数量、販売時単価を残します。取り消しは行を消さず、取消の印を付けて通常の履歴と集計から除外します。CSVは金額を照合する控えです。復元には別のバックアップと手順を用意し、どの時点まで戻せるかを練習場所で確かめます。

——これは口頭試問の第3問「データはどこに保存されていますか」への、そのまま答えの型です。「データベースです」の一言で終わらせず、どこに・どんな形で・消えたらどうするか、まで言えれば合格です。

レベルアップ!

Lv.8 称号: 記録者

- 手に入れた装備: 永続化、データベース、テーブル、主キー、スキーマ、CRUD、クエリ、バックアップ
- 冒険の書: データ項目一覧+簡単なデータベース設計図

ノートは手に入りました。ただし、いまのノートには誰でも書き込めて、誰でも読めます。店長画面のURLさえ知っていれば、シフト担当の1年生でもタイムセールを開始できてしまう——第3章の仕様書には「操作は店長だけ」と書いたはずです。次の第9章では、アプリに「あなたは誰か」を確かめさせます。

第9章 利用者、ログイン、権限を理解する

クエスト9: 扉と鍵と通行証

文化祭まで、あと12日。

この章のねらい

- ・ 認証と認可の違いを、部員証とスタッフパスのたとえで説明できる
- ・ 権限マトリクスを書き、×の操作がサーバー側で断られることを確かめられる
- ・ 秘密の保管場所(Secrets)と渡し方(環境変数)を分けて、秘密情報一覧を書ける

今回のクエスト

第8章で、会計の記録はデータベースに残るようになりました。レジと店長の端末から同じ記録を読めます。では、そのつながりは誰に開かれているのでしょうか。シフト担当の端末から、店長の操作までできてしまいますか。

第3章の仕様書に、あなたはこう書きました。「タイムセールを開始・停止と割引率の変更は、店長画面からしかできない」。この1行、実はまだ守られていません。この章のミッションは、誰であるかを確かめ、誰に何を許すかを決めて、それをアプリに本当に守らせることです。

まずAIに作らせてみよう

素直に頼んでみましょう。

あなた → AI

会計アプリにログイン機能を追加してください。利用者はシフト担当と店長です。店長画面(売上集計とタイムセールの操作)は、店長だけが見られるようにしてください。

AI → あなた

ログイン機能を追加しました。ログインすると、店長のアカウントには店長画面へのボタンが表示されます。シフト担当のアカウントにはボタンが表示されません。

試してみると、たしかにその通りに動きます。店長でログインすればボタンがあり、シフト担当にはない。仕様通りに見えます。この章が3ページで終わりそうな気配すらします。

ボス出現——鍵のかかっていない扉

シフト担当役の部員が、練習中に声を上げました。「店長のボタンはないんだけど、売上の集計が見えた。タイムセールも開始できた。」

部員は、自分たちの練習用アプリで、第5章のネットワークタブを開いていました。店長の操作がどこへリクエストを送るかを観察すると、`/api/admin/summary`(集計)や`/api/sale/start`(セール開始)という窓口が見えました。

そして、シフト担当のままその窓口で直接頼んでみた。すると——通ってしまった。

第7章を思い出してください。画面のボタンは、押されるとAPIという窓口のリクエストを送っているだけです。つまりボタンを消しても、窓口は開いたままです。店の正面の扉を閉めても、裏口に鍵がかかっていなければ、誰でも入れます。

「店長だけが見られる」という要求に、ボタンを隠すだけの実装では足りませんでした。画面に見えなくても、窓口へは頼めてしまう。この抜けを、次は受け入れ条件に含めます。失敗の原因は、専門用語を知らなかったことではありません。実装と検証で埋めるべき穴が、見つかったのです。

この扉に、鍵と門番を付けていきましょう。まず言葉の整理からです。

9.1 認証と認可の違い——「あなたは誰か」と「何をしてよいか」

この章の背骨になる区別です。似た字面ですが、別の仕事をします。

認証(authentication)は、あなたは誰かを確かめることです。日常語で言うと、部員証の確認。ログインとは、部員証を見せる行為です。

認可(authorization)は、確認できたその人が何をしてよいかを確かめることです。日常語で言うと、スタッフパスの確認。部員証で「山田さんだ」とわかってても、バックヤードに入れるのはパスを持つ人だけです。

図 9-1

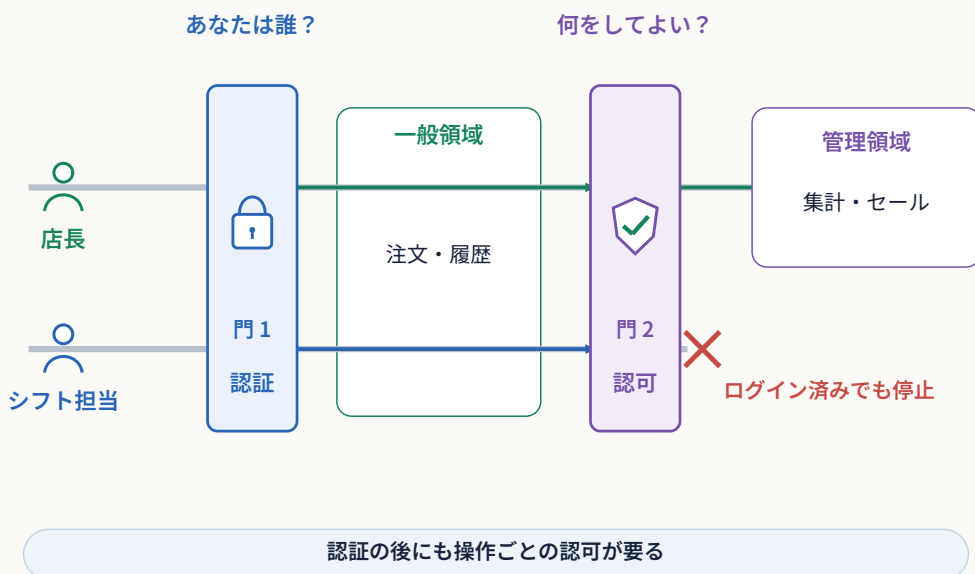


図9-1 認証と認可の2つの門

図9-1では、1つめの門を通ったシフト担当が、2つめの門で止められる点を見てください。

冒頭の事件を、この言葉で言い直せます。あのアプリには認証はありました(ログインはできた)。なかったのは認可です。部員証を確認したあと、「この人はバックヤードに入ってよいか」を誰も確かめていませんでした。

ログイン機能を付けることと、権限を守ることは、別の仕事なのです。

9.2 ユーザーと管理者、ロール

では、「何をしてよいか」をどう管理するか。部員は入れ替わるので、1人ずつ同じ許可を書き足すより、役割ごとにまとめると見通しがよくなります。

そこでロールを使います。日常語で言うと、役割の札です。人ではなく札に権限を決めておき、人には札を渡します。この会計アプリの札は3枚です。

- ・ シフト担当: 注文を打ち、履歴を見て、押し間違いを取り消す
- ・ 店長: シフト担当のすべてに加えて、集計を見て、タイムセールを操作する
- ・ 文化祭本部: 各店の売上集計を見る(それ以外は何もできない)

これを表にしたものが、この章の成果物権限マトリクスです。機能を行に、ロールを列に並べ、○×を埋めるだけです。

機能	シフト担当	店長	文化祭本部
注文の入力・取り消し	○	○	×
履歴の閲覧	○	○	×
売上集計の閲覧	×	○	○
タイムセールの開始・停止・割引率変更	×	○	×

この表の力は、×を書けることにあります。本部の仕事は各店の売上集計を見ることなので、注文もセール操作も×です。

○にも範囲があります。シフト担当と店長の○は「自分の店のデータ」に限ります。複数の店を扱うなら、ロールに加え、どの店の記録へ触れてよいかもサーバーで確認します。

▼コーナー「先輩エンジニアならここを見る」

プロがこの表を受け取ってまず試すのは、○ではなく×のマスです。「シフト担当でセール開始を試したら、ちゃんと断られるか」。できることの確認は誰でもします。できないはずのことが本当にできないかの確認こそ、権限の確認です。

9.3 セッションとトークン

小さな疑問をひとつ片付けます。ログインは最初に1回だけなのに、サーバーはなぜ、そのあとのリクエストでも「この人はさっきログインした店長だ」とわかるのでしょうか。第5章で学んだ通り、リクエストは1回ずつ独立した注文のはずです。

ここで使うのがセッションです。複数のリクエストを、同じ利用者との一続きのやりとりとして扱う仕組みです。たとえるなら、入場時の本人確認を、その後の入場スタンプにつなぐ仕組みです。

以後のリクエストにはスタンプが添えられ、サーバーは毎回それを確認めます。省けるのは、毎回パスワードを入力する手間だけです。有効期限、失効、権限の確認は、リクエストごとに続けます。ログインは1回、確認は毎回——その橋渡しがセッションです。



トークンは、確認のために受け渡す文字列などのデータです。セッションを識別する値も、その一例です。「セッションとトークンは別々のログイン方式」と覚えるより、セッションは続きの関係、トークンはその確認に使う通行証、と分けると理解しやすくなります。

ログアウトでは、ブラウザの表示を変えるだけでなく、ログイン中の状態(セッション)を使えなくする処理が必要です。具体的な仕組みは認証サービス(9.4で説明します)に任せ、共用タブレットではシフト交代時にログアウトします。次の利用者が、前の店長の権限を引き継がないためです。

9.4 ログインを自作しない、パスワードをそのまま保存しない

ここで、多くの人が次にやることを先回りします。「ログイン画面と、ユーザーとパスワードの表を作って」とAIに頼むのです。すると、パスワードをそのままデータベースに保存する実装が出てくることがあります。

動きます。そして、危険です。

パスワードをそのまま保存するのは、部員名簿にみんなの暗証番号を鉛筆で書き込むようなものです。名簿が1回漏れたら、全員分が漏れます。

さらに悪いことに、人はパスワードを使い回します。あなたのアプリから漏れた1つが、その人の他のサービスまで危険にさらすのです。文化祭のアプリが背負うには、重すぎる責任です。

この本では、本人確認の仕組みは認証サービスに任せます。自作すると、パスワードの保存に加え、ログイン試行の制限、パスワードを忘れたときの回復、セッションの失効まで作らなければならないためです。あなたは、どのロールに何を許すか(認可)に集中できます。

あなたが覚えるのは、AIへの頼み方の型ひとつです。

あなた → AI

ログインは自作せず、認証サービスを使ってください。パスワードがこのアプリのデータベースに保存されない構成にしてください。

※読み方の補足: パスワードの保存には、専用の方式によるハッシュ化などが使われます。単に文字を暗号化したり、一般的なハッシュ関数を1回使ったりすれば十分というわけではありません。本書では、その管理を認証サービスに委ねます。

▼コーナー「このプロンプトの何が悪い?」

「ログイン機能を作って。ユーザー名とパスワードはデータベースに保存する方式で」

悪い点は2つ。①保存の形式を指定していないので、そのまま保存されかねない②自作を指示してしまっている。AIは頼まれれば自作します。頼む側が「自作しない」と知っていることが、ここでの安全装置です。

9.5 画面を隠すだけでは権限制御にならない——管理者用API

さて、この章でいちばん大事な節に来ました。冒頭の事件の核心です。

最初のAIが書いた守りは、こういうコードでした。ifの中で何をしているかを見てください。

役割ラベル: 表示

```
[javascript]
// 店長のときだけ、店長画面へのボタンを表示する
if (user.role === "店長") {
  showManagerButton(); // ボタンを見せる(表示)
}
```

これは表示のコードです。つまり、ブラウザの中の話です。

第5章の構成図を思い出してください。ブラウザは利用者の手元にあり、そこで動くコードや送るリクエストは、利用者の側で書き換えられます。ボタンがなくても窓口へ直接頼めるので、表示の条件だけを権限の根拠にはできません。

守りが立つべき場所は、サーバーの入り口です。

図 9-3

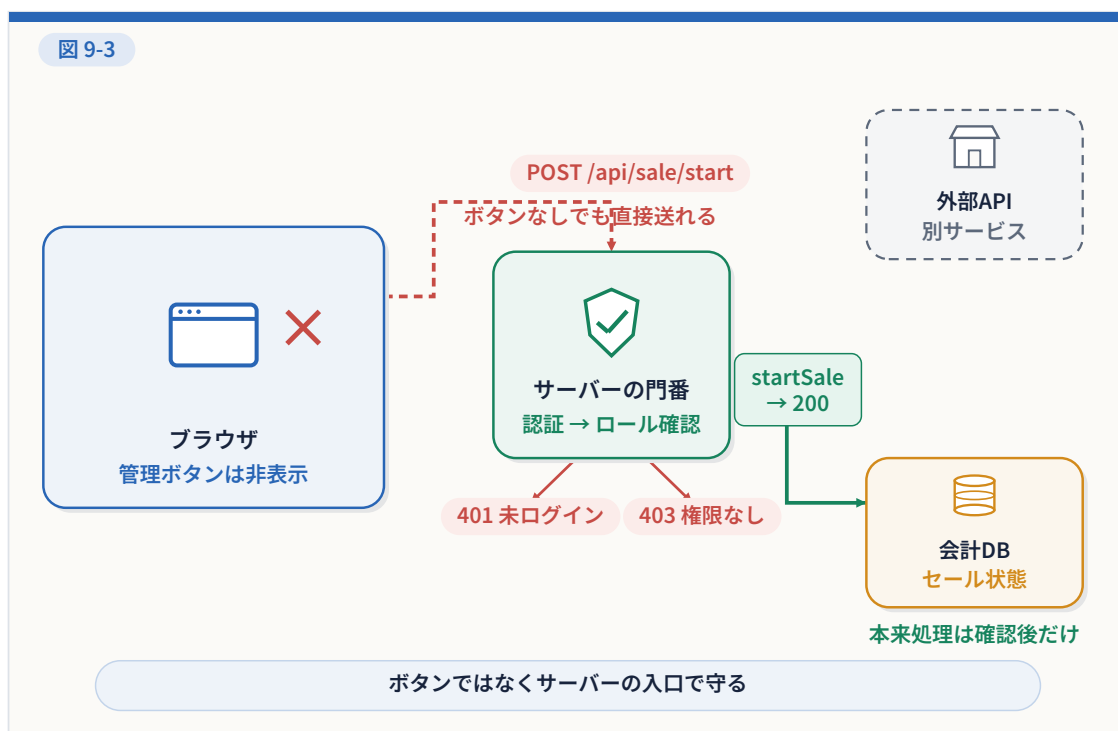


図9-3 権限確認はサーバーで

店長画面が使う窓口——集計、セールの開始・停止、割引率の変更——を、ここでは管理者用APIと呼びます。次は認証と認可に注目した抜粋です。`getLoginUser`は、認証サービスが検証した情報から利用者を特定する処理を表し、ブラウザが自己申告したロールは使いません。

役割ラベル: 通信

```
[javascript]
// セール開始の窓口。仕事の前に、必ず二段の確認をする
app.post("/api/sale/start", (req, res) => {
  const user = getLoginUser(req);    // スタンプから利用者を特定(認証の確認)
  if (!user) {
    return res.status(401).send("ログインしてください");
  }
  if (user.role !== "店長") {        // ロールを確認(認可の確認)
    return res.status(403).send("この操作は店長だけができます");
  }
  startSale(req.body.rate);          // ここでようやく本来の仕事
  res.status(200).send("タイムセールを開始しました");
});
```

この例では、有効なログインを確認できなければ401、ログイン済みでも許可のないロールなら403です。番号を見れば、図9-1の2つの門(認証と認可)のどちらで止めたかがわかります。

※厳密に言うと: 上の抜粋は、認証と認可の確認だけを取り出したものです。実際の管理者用APIでは、割引率の範囲、対象の店、意図しないサイトからの操作要求なども検証します。

ではブラウザ側の「ボタンを隠す」は無意味かということ、そうではありません。あれは親切です。使えない機能を見せないのは良い画面設計です。

ただし親切は防御ではない。見た目の制御はブラウザで、本当の確認はサーバーで。両方やって、初めて仕様の1行が守られます。

AIと一緒に直す

材料はそろいました。冒頭の失敗した依頼と見比べながら、頼み直します。

あなた → AI

店長画面の守り方を作り直します。ボタンを隠すだけでなく、サーバー側で確認してください。

1. ログインは自作せず、認証サービスを使う。パスワードはこのアプリのデータベースに保存しない
2. 利用者にはロールを持たせる: シフト担当/店長/文化祭本部
3. 権限は次の表の通り(権限マトリクスを貼る)。管理者用APIはサーバー側で毎回ロールを確認し、権限がなければ403を返す
4. 受け入れ条件: シフト担当でログインしたまま `/api/sale/start` に直接頼むと、403が返ること

AI → あなた

認証サービスを組み込み、すべての管理者用APIにロールの確認を追加しました。シフト担当がセール開始の窓口で直接頼むと403を返し、画面には「この操作は店長だけができます」と表示されず。

最初の依頼(「店長だけが見られるように」)との違いは3つです。確認する場所(サーバー側で)を指定した。権限の表を渡した。そして受け入れ条件を、抜け道側で書いた——「ボタンが見えないこと」ではなく「直接頼んでも403が返ること」。

第3章で学んだ受け入れ条件が、ここでは防御の検査になっています。

仕上げるに、自分の練習用アプリで確かめます。シフト担当でログインしたまま、セール開始の窓口へ直接頼む手順を、AIに示してもらいましょう。403が返ることと、実際にセールが開始されていないことの両方を見てください。拒否の番号だけ返し、処理は進んでしまう実装も見逃さないためです。

9.6 最小権限と、個人情報を集めすぎないこと

権限マトリクスの×の話に、名前を付けます。最小権限——必要な人に、必要なだけ許す、という原則です。日常語で言うと、合鍵をむやみに配らない、です。

店長が席を外すとき、「念のため全員に店長権限を」としたくなるかもしれません。その場では便利でも、誰が操作できるのかがわからなくなります。代行する人を決め、必要な間だけ権限を渡すほうが、交代の手順も明確になります。

迷ったら×にして、必要になったそのときに○へ変える。権限は後から広げるほうが、後から狭めるより簡単です。

そして、最小権限には双子の原則があります。データも、必要な分しか持たない。

この会計アプリは、お客さんの名前も連絡先も記録していません。第3章のデータ表を見返してください——注文の内容、時刻、合計。それだけです。会計に「誰が買ったか」は要りません。

顧客名や連絡先を持たなければ、それらが会計データから漏れることもありません。持たないことで、守る対象を減らせます。ただし、部員のログイン情報や操作記録など、アプリが別に扱うデータの確認は必要です。

「あとで何かに使えるかも」と項目を増やす前に、何のために使い、いつまで残すかを考えます。この判断を、第13章のデータ点検につなげていきます。

9.7 APIキーと環境変数——秘密の置き場所は金庫

最後に、第7章からの宿題を回収します。APIキーは窓口の会員証、他人に見せない、コードに直接書かない——ではどこに置くのか。「金庫」の正体を明かします。

APIキーは、公開先の秘密管理機能(Secrets)に保管し、必要なときだけサーバーの処理へ渡します。これが「金庫」に当たる場所です。

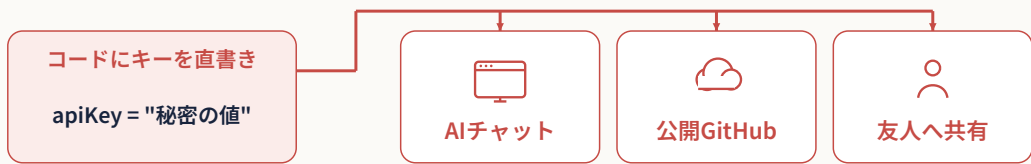
渡し方のひとつが環境変数です。環境変数そのものに、暗号化やアクセス制限が自動で付くわけではありません。保管は金庫、受け渡しは環境変数と、役割を分けて考えます。次のコードでは、`EVENT_API_KEY`という名前だけが書かれ、キーの実物がないことを確かめてください。

役割ラベル: 通信

```
[javascript]
// サーバー側のコード例(Node.js)。キーの実物は書かない
const apiKey = process.env.EVENT_API_KEY; // 環境変数を読む
const res = await fetch(url, {
  headers: { "X-API-Key": apiKey } // 会員証を提示して頼む(通信)
});
```

図 9-4

× 秘密をコードに書くと、コピー先へも渡る



○ 秘密を使うのはサーバー。ブラウザには結果だけ



図9-4 秘密はコードではなく金庫へ

図9-4は上が悪い例、下がよい例です。下の例で、キーがブラウザを通らないことを見てください。

通り道も大切です。ブラウザ→自分のサーバー→外部APIの順に頼み、キーはサーバーから外部APIへ送るときだけ添えます。ブラウザへ返すのは必要な結果だけです。環境変数に入れていても、ブラウザ向けのコードや返事に埋め込めば利用者へ届いてしまいます。

なぜ、秘密をコードから分けるのか。コードは、思った以上にあちこちへ写るからです。

AIとの会話に貼る。友だちに見せる。コードの置き場(GitHubなどの共有サービス。第10章でくわしく学びます)に上げる。そのすべてに、書き込まれた秘密も一緒に旅をします。

公開されたキーは、コピー先から回収できません。漏れた可能性があれば、コードから消すだけでなく、発行元で古いキーを無効にして作り直します(第13章で扱います)。

だから掟はこうです。秘密はコードに書かない。AIとの会話にも貼らない。金庫に入れ、名札で呼ぶ。

そして、何が秘密なのかを一覧にしておきます。これが2つめの成果物、秘密情報一覧です。

秘密情報	置き場所	貼ってはいけない場所
秘密として扱うイベントAPIキー	サーバー用Secrets。実行時に環境変数などで渡す	コード、AIとの会話、公開設定、ブラウザへの返事

秘密情報	置き場所	貼ってはいけない場所
パスワードを含むDB接続情報	サーバー用Secrets	同上
利用者のパスワード	認証サービスに任せる(自分では持たない)	あらゆる場所

1行目の「置き場所」は、保管する場所(Secrets)と渡し方(環境変数など)を分けて書いています。

※読み方の補足: 環境変数には公開してよい設定も置けます。また、APIには公開用と秘密用のキーを区別するものもあります。名前だけで判断せず、発行元の説明を確認します。第10章では手元の設定ファイル、第14章では公開先の設定として再登場します。

別の条件でも動くか試す

この章の考え方は、題材が変わってもそのまま使えます。ゼミの本の貸し出しなら、部員と管理者。職場の備品予約なら、社員と総務。まず権限マトリクスを書き、×のマスサーバー側で守らせ、×が本当に断られるかを直接確かめる——手順は同じです。

AIの作り方は毎回違います。認証サービスの選択も、画面の見た目も、紙面と違ってかまいません。×の操作がサーバーで拒否され、データも変わらず、○の操作はこれまで通り動くことを確認します。この章では、未ログインは401、権限違いは403という設計で確かめます。

うまくいかないときは、一度に全部を頼まないことです。「セール開始の窓口に、店長かどうかの確認だけ追加して」と、窓口1つぶんまで小さくして頼み直してください(第4章の型です)。

演習9-1 権限の漏れたアプリを直す

- ミッション: 自分の練習用アプリで、シフト担当が集計閲覧やセール開始をできないことを確認し、漏れがあれば修正する。すでに守られていれば、その証拠を残す
- 時間の目安: 60分
- クリア条件:
 1. シフト担当でログインすると、店長画面が開けない(URLを直接開いても案内が出る)
 2. シフト担当のまま、セール開始の窓口に直接頼むと403が返り、セールも開始されない
 3. 店長でログインすると、集計もセール操作もこれまで通り使える
 4. 上の1～3が、再読み込みしても変わらない
- つまづいたら:
 - まず、どの窓口が守られていないかを特定する。ネットワークタブで、店長の操作がどの住所に飛ぶかを観察する(第5章・第7章)
 - 「画面で隠せました」とAIが報告してきたら、「サーバー側の確認はどこにありますか」と聞き返す
 - (授業向け)ペアで攻守に分かれ、相手のアプリの×のマスを破れるか試し合う

- ・ 裏ダンジョン: ログインしていない人(401)とロール違いの人(403)で、画面の案内文を出し分ける
- ・ 冒険の書に記録: 修正前後の「破れた手順→断られた証拠(403)」のメモ

演習9-2 権限マトリクスを完成させる

- ・ ミッション: シフト担当・店長・文化祭本部の3ロールで、機能×役割の○×表を作る
- ・ 時間の目安: 30分
- ・ クリア条件:
 1. アプリの全機能が行に並び、全マスに○か×が入っている(「未定」を残さない)
 2. ×のマスのうち2つ以上を、実際に操作して「断られること」を確認した
 3. 秘密情報一覧が2行以上埋まっている
- ・ つまづいたら:
 - ・ 機能の洗い出しに漏れがないか、AIに「この表にない操作がアプリにあれば挙げてください」と確認させる
 - ・ 迷ったマスは×にする(最小権限。あとで○に変えるほうが簡単)
- ・ 裏ダンジョン: 「会計の取り消し」は誰まで○か、店の仲間と議論して決める。取り消しは売上を消せる操作——集計の信頼に関わる、権限設計のいい練習台です
- ・ 冒険の書に記録: 権限マトリクス+秘密情報一覧

この章のまとめ

- ・ 認証は「あなたは誰か」、認可は「何をしてよいか」の確認で、別の仕事である。ログインできても、操作ごとに認可を確かめなければ権限は守れない
- ・ 権限はロールごとに決め、機能×ロールの権限マトリクスに○×で書く。迷ったマスは×にし(最小権限)、×の操作が本当に断られるかを確認する
- ・ ボタンを隠すのは画面の親切であって、防御ではない。管理者用APIは、サーバー側でリクエストごとに認証と認可を確認する
- ・ ログインは自作せず、認証サービスに任せる。利用者のパスワードを、このアプリのデータベースに保存しない
- ・ APIキーなどの秘密は、コードやAIとの会話に書かない。Secretsに保管し、環境変数などでサーバーの処理だけに渡し、ブラウザには渡さない

理解度チェック

1. 認証と認可は、それぞれ何を確かめる仕事ですか。
2. 会計アプリで、イベントAPIキーを使って外部APIから情報を取ります。キーはどこに保管し、どの通り道で使いますか。

3. シフト担当の画面から、店長画面へのボタンを消しました。これで「タイムセールの操作は店長だけ」が守られたと言えますか。理由も教えてください。

解答のめやす

1. 認証は「あなたは誰か」(部員証の確認)、認可は「その人が何をしてよいか」(スタッフパスの確認)を確かめます(→9.1)
2. サーバー用Secretsに保管し、環境変数などでサーバーの処理へ渡します。ブラウザ→自分のサーバー→外部APIの順に頼み、キーはサーバーから外部APIへ送るときだけ添えます。ブラウザへは結果だけを返します(→9.7)
3. 言えません。ボタンを隠すのは表示の制御で、窓口(管理者用API)へは直接リクエストを送れるからです。サーバー側で毎回ログインとロールを確認し、シフト担当なら403で断る必要があります(→9.1、9.5)

60秒でクエスト報告

このアプリには、シフト担当、店長、文化祭本部の3つのロールがあります。誰であるかは認証サービスで確かめ、許可された操作かはサーバーが毎回確認します。シフト担当からセール開始を直接頼んでも403で断られ、セールは開始されません。秘密のAPIキーはサーバー用Secretsで管理し、外部APIへ頼むときだけ使います。ブラウザには渡しません。会計に必要な顧客名や連絡先も集めていません。

——卒業制作の発表で「このアプリでは、誰が、何をできますか」と聞かれたら、これが答えの型です。権限マトリクスを指させれば、証拠つきで答えられます。

レベルアップ!

Lv.9 称号: 門番

- 手に入れた装備: 認証、認可、ロール、セッション、トークン、最小権限、環境変数
- 冒険の書: 権限マトリクス+秘密情報一覧

アプリはもう、扉と鍵を持つ一人前の姿になりました。ところで——ここまで育ったこのアプリ、中身はいったい何個のファイルでできているのでしょうか。そして、AIに大きな変更を頼んで壊れたら、昨日の状態に戻せますか。次の第10章では、プロジェクトの中身と変更の歴史を手に入れます。金庫に入れた秘密が「コードの置き場」で漏れない作法も、そこで仕上げます。

第10章 プロジェクトの中身と変更履歴を管理する

クエスト10: セーブなしで、ボスに挑むな

文化祭まで、あと10日。

この章のねらい

- ・ プロジェクトのファイルを、ソース・設定・借りた部品に見分けられる
- ・ 変更をコミットし、diffで「どの行が変わったか」を確かめられる
- ・ 動いていたコミットを指定して戻すよう頼み、750円・600円を確かめられる

今回のクエスト

会計アプリは、ずいぶん育ちました。仕様書があり、構成図があり、データは保存され、店長だけがセールを操作できます。ここで一度、手を止めます。この章では新しい機能をひとつも作りません。かわりに、足元を見ます。

あなたのアプリは、実際には何でできているのか。AIはこれまで、どこの何を書き換えてきたのか。そして——壊れたとき、どうやって昨日に戻るのか。

ミッションは2つです。プロジェクトの中身を案内できること。そして、動いていたコードを記録し、必要なときに戻せることです。第6章で開いたファイルに加えて、「ターミナル」「バージョン管理」を使い、変更を安心して試す足場を作ります。

まずAIに作らせてみよう——本番前の魔改造

文化祭が3日後に迫った夜を想像してください。アプリは動いています。動いているのに、欲が出ます。「見た目が地味かも」。

あなた → AI

アプリ全体のデザインをモダンにしてください。配色を一新して、ボタンにアニメーションも付けてください。

AI → あなた

デザインを刷新しました。配色をダークトーンに変更し、メニューをカード型に、ボタンには押したときのアニメーションを追加しました。あわせて画面のコードを整理しました。

見違えました。勢いに乗って、あなたは続けます。「履歴画面もカード型に」「店長画面にグラフっぽい見た目を」「フォントも変えて」。夜中の1時間で、AIは十数回の変更をこなしました。満足して眠ります。

ボス出現——昨日は動いていたのに

翌朝、シフト担当役の友達から連絡が来ます。「コーヒー2つとクッキー1つで、800円って出るよ」。

血の気が引きます。第3章で決めた受け入れ条件は「通常時750円(セット割50円引き)」でした。セット割が、消えています。昨夜、割引の話なんて一度もしていないのに。

あわてて頼みます。

あなた → AI

昨日の夜、デザインを変える前の状態に戻してください。

AI → あなた

変更前の状態に戻しました。配色とボタンのデザインを以前の形に復元しています。

一見、戻ったように見えます。でも、よく見るとボタンの並び順が違います。計算を確かめると750円には戻ったものの、今度はタイムセール中の表示が出なくなりました。「戻して」を繰り返すほど、アプリは誰も見たことのない状態へ変わっていきます。

ここで、恐ろしい事実に気づきます。「昨日の夜の状態」を正確に知っている者が、どこにもいないのです。あなたは覚えていない。AIも覚えていません。

チャットの履歴は残っていますが、あれは会話の記録であって、アプリの状態の記録ではありません。AIの「戻しました」は、記憶をたよりにそれらしく作り直ただけです。

「もう最初から作り直そうか」。そんな考えが浮かんでも、9章ぶんの積み上げはできれば残したいものです。必要なのは、AIの記憶を何度も問い直すのではなく、比較できる変更の記録です。

変更の記録がないと、戻る場所も原因も探しにくくなります。記録があれば、動いていた地点を選び、どこが変わったかを絞れます。まずは、その記録に何が入るのか、アプリの「中身」から見ていきましょう。

10.1 プロジェクトとは何か——ファイルとフォルダーを開けてみる

AI開発環境の「ファイル」を開いてください。第6章では計算を探すために開きました。今回は、アプリ全体の荷物を見渡します。

図 10-1

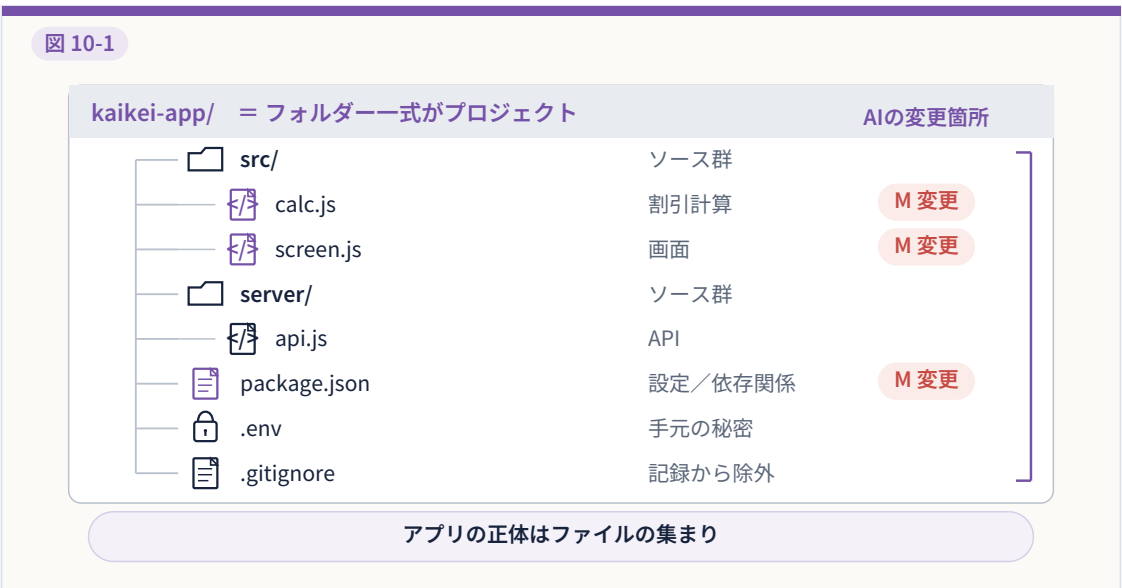


図10-1 プロジェクトの中身

図10-1では、1つのフォルダーに、計算・画面・設定のファイルが並ぶ点を見てください。

このフォルダー一式まるごとを、プロジェクトと呼びます。あなたのアプリの正体は、画面でもチャット履歴でもなく、このファイルの集まりです。AIに「変更して」と頼むたび、AIはこの中のどれかのファイルを書き換えていました。

ファイルは大きく2種類あります。ソースファイルは、アプリの本体です。第6章で読んだコードは、ここに入っています。

設定ファイルは、アプリの周りの決めごとです。どんな部品を借りるか、どこに公開するか。本体ではないのに、壊れるとアプリごと動かなくなる、意外な急所です。

全部読める必要はありません。案内は、AIに頼めます。

あなた → AI

このプロジェクトのファイル構成を、初心者向けに案内してください。割引計算はどのファイルにありますか。

ファイル名が本書と違っても、AIの案内が「割引計算はここ」「公開先の設定はここ」と示していれば十分です。あなたも、第6章で探した1行が、どんなフォルダーの中に置かれているかまで案内できるようになります。

10.2 あなたのアプリは借り物の上に立っている

ファイルの一覧で、**node_modules**という大きなフォルダーに気づくかもしれません。「こんなにたくさん書いてもらった覚えはないのに」。JavaScriptのプロジェクトでは、ここに大量のファイルが入ることがあります。

書いてもらっていません。それは借り物です。

ライブラリは、できあいの部品集です。日付の計算、画面の部品、グラフの描画——世界の誰かが作って公開した部品を、あなたのアプリは組み込んで使っています。

パッケージは、その部品を配るときの箱の単位です。ひとつ取り寄せると、その部品が必要とする別の部品も芋づる式に届きます。だから数万ファイルになるのです。

この「別の部品を必要とする関係」を**依存関係(dependency)**と呼びます。**package.json**には、アプリ名や起動方法に加え、必要な部品の一覧を記録できます。次の見本で見るのは、**dependencies**の中に並んだ部品の名前です(JSONにはコメントを書けないため、説明はコードの外に置きます)。

役割ラベル: データ(設定)

```
[json]
{
  "name": "kaikei-app",
  "dependencies": {
    "react": "^18.3.0",
    "date-fns": "^3.6.0",
    "recharts": "^2.12.0"
  }
}
```

この例では、**react**は画面、**date-fns**は日付処理、**recharts**はグラフを作るライブラリです。右側の文字列は、使えるバージョンの指定です。

さらに、アプリを組み立てる骨組みを提供するフレームワークと、コードを動かす実行環境であるランタイムがあります。たとえば、画面を動かすブラウザや、サーバーでJavaScriptを動かすNode.jsが、実行環境に当たります。図10-2では、自分のコードが、借り物の層のいちばん上に薄く載っているのを見てください。すべてのアプリが、同じ部品を同じ順に積むわけではありません。



図10-2 借り物の上に立つアプリ

自分の会計ルールを書くコードは、その周囲のたくさんの部品に支えられています。部品を借りることで、文化祭ならではの注文や割引に、作業を集中できます。

使っている部品がわかれば、不具合や更新のお知らせが出たときに、自分のアプリへの影響を確認できます。その点検を第13章と第14章で扱います。

※厳密に言うと: ライブラリとパッケージの区別は現場でも曖昧で、ほぼ同義に使われます。「部品(ライブラリ)が、箱(パッケージ)で届き、リスト(依存関係)で管理される」と覚えれば実用上は十分です。また、例のバージョンは読み方の見本で、新しい開発に勧めるものではありません。実際に使った部品のバージョンをそろえるため、ロックファイル(例: `package-lock.json`)も一緒に管理します。

10.3 ターミナルとコマンド—AIの提案を、人間が承認する場

次の道具はターミナルです。日常語で言うと、コンピューターと文字で話す窓口。ボタンやメニューの代わりに、命令文(コマンド)を1行打つと、コンピューターがそれを実行します。

本書では、コマンドを自分で覚えて打つ必要はありません。部品の取り寄せやアプリの起動も、AIに頼めます。AIは実行の前に、あなたの許可を求めることがあります(確認画面が出るかは、環境と設定によります)。ターミナルは命令の実行と結果を見る場所、承認はその実行を通すかを決める仕組み、と区別しておきましょう。

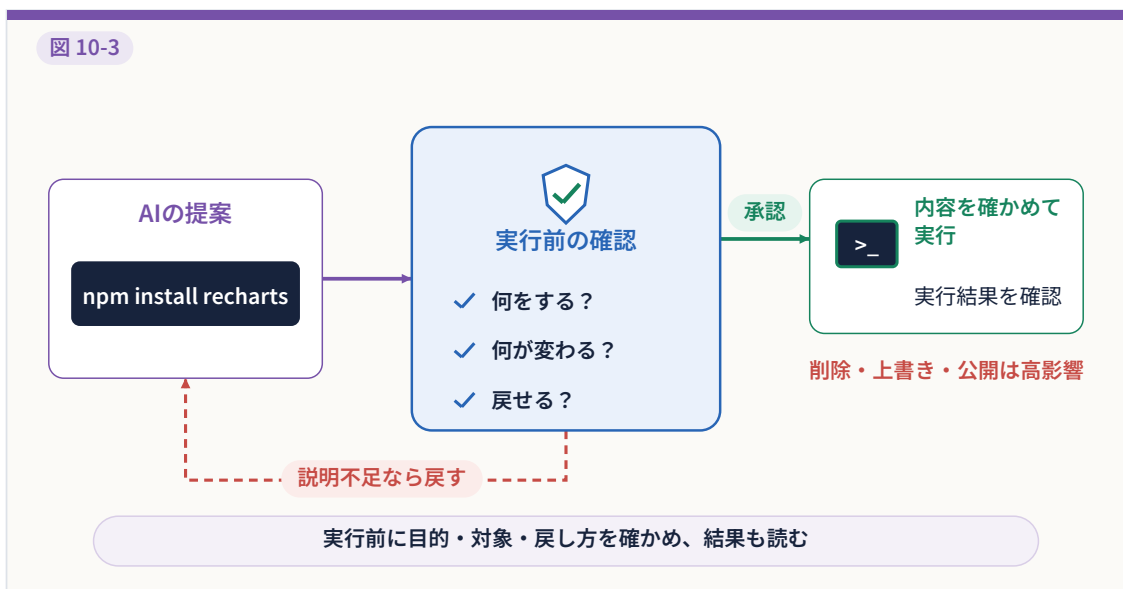


図10-3では、AIの提案したコマンドが、承認をってから実行される点を見てください。

承認の作法は、第9章の権限の考え方と同じです。説明できないものを、通さない。

わからないコマンドが来たら、「何のために、どのフォルダーで、何を变えますか」と聞きます。たとえば図10-3の `npm install recharts` なら、「グラフの部品を取り寄せ、`package.json`などの部品の一覧と `node_modules`が変わる」という答えになるはずです。

削除・上書き・公開を伴うときは、対象と戻し方まで確認してください。説明と実行結果を対応させる習慣が、見知らぬ1行を扱う手がかりになります。

10.4 開発用設定と公開用設定——金庫の置き場所

もうひとつ、設定ファイルの話です。同じアプリでも、手元で作っているときと公開して使うときでは、設定が違います。

- 手元(開発用): 練習用のモックAPI(第7章)につなぐ。データベースも練習用
- 公開先(公開用): 本部の本物のAPIにつなぐ。データベースも本番用

この「場所ごとに違う値」をコードへ渡す方法のひとつが、第9章の**環境変数**です。接続先や秘密のキーを、開発用と公開用で別々に設定します。手元では`.env`というファイルから読み込む構成も使われますが、`.env`は通常のテキストファイルです。名前を付けただけで暗号化されるわけではありません。

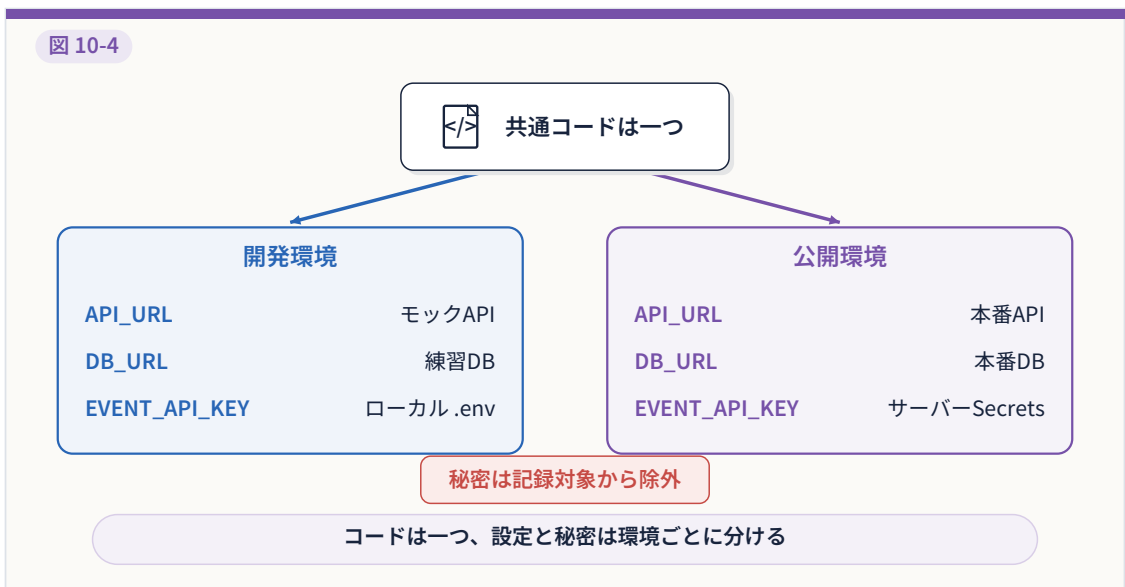


図10-4 環境ごとの設定差し替え

図10-4では、コードは一つのまま、設定と秘密だけが環境ごとに変わる点を見てください。

公開先では、秘密の値はサーバー用Secretsへ登録します。これが第9章の「金庫」です。

公開してよいURLや表示名と、秘密のキーは区別して扱います。また、開発用の設定から本番のデータへ誤って接続しないようにします。このあとの変更履歴にも、秘密の実物は含めません。

10.5 GitとGitHub——リポジトリ、コミット、diff

いよいよ、この章の主役です。

Git(ギット)は、プロジェクトの変更履歴を記録する道具です。世界中の開発者が使っている、事実上の標準です。あなたのAI開発環境の「バージョン管理」の中身も、多くの場合Gitです。

Gitでファイルと変更履歴を管理する場所をリポジトリと呼びます。いまのファイルに加え、記録した時点の姿をたどれます。自動ですべての編集が残るわけではありません。

残す変更を選び、ひと区切りとして記録する操作がコミットです。コミットすると、記録対象のファイルのその時点の状態が、メモ(コミットメッセージ)や時刻とともに残ります。記録対象に入っていないファイル、サーバーのデータベースの中身、Secretsに登録した値はコミットに入らず、別に管理します。

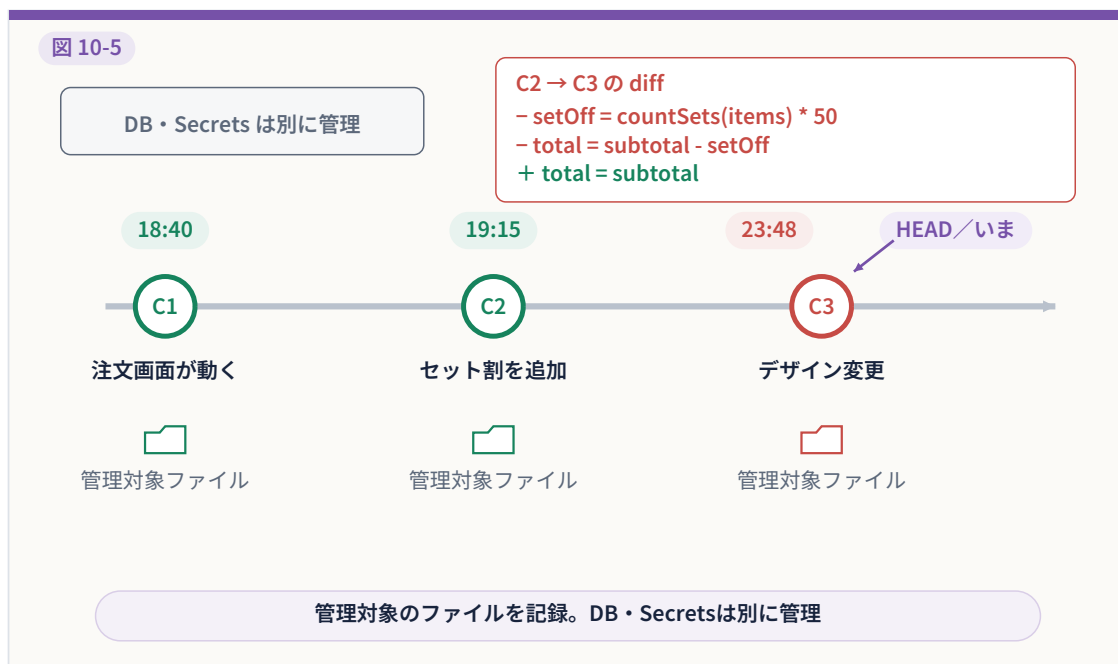


図10-5 コミットという記録

コミットは、コードを育てるときのセーブです。「セット割が動く」と確かめた地点に、目印を残せます。

進んだら記録する。大きく変える前にも記録する。ただし、会計データの控えは第8章のバックアップが担当します。コードのセーブとデータのバックアップがそろって、戻すための備えになります。

次に見たいのは、「前回の記録と、いま、どこが違う?」。この差分をdiff(ディフ)と呼びます。コミット同士でも、最後のコミットと未記録の編集でも比べられます。

第6章でAIに報告させていた「変更点」——あれの正体が、このdiffです。もし昨夜のあなたがコミットしていたら、朝の事故はdiffでこう見えたはずです。

役割ラベル: 計算(diffの読み方: 「-」が消えた行、「+」が増えた行)

```
[diff]
function calcTotal(items, saleRate) {
  const subtotal = sumOf(items);
  - const setOff = countSets(items) * 50; // セット割: 1セット50円引き
  - let total = subtotal - setOff;
  + let total = subtotal;
```

```
const percent = Math.round(saleRate * 100);
if (percent > 0) total = total * (100 - percent) / 100;
return Math.floor(total / 10) * 10;
}
```

セット割を求める行と、それを引く行が消えています。この例では、見た目の変更と一緒に入ったコード整理が、800円になった原因でした。

変更前と並べて見られるので、調べる場所が絞れます。第3章の仕様と、第6章で読んだ計算の対応が、ここでも役立ちます。「見た目の依頼なのに、計算の行が変わっている」と気づけるのです。

GitHub(ギットハブ)は、リポジトリをオンラインで保管・共有するサービスです。引き継ぎの置き場所にも使えます(第14章と第15章)。

手元のコミットをGitHubへ送る操作をpush(プッシュ)と呼びます。送信済みの記録は、手元のパソコンとは別の場所に残ります。コミットしただけでは、まだ手元にしかない点に注意してください。

最後に、約束をひとつ。秘密を書いた.envは、コミットに含めない。リポジトリは人に渡るものだからです。

Gitには、まだ記録していないファイルを記録対象から外す.gitignoreという設定があります。秘密を含む.envをここに指定し、AIに「記録対象と過去の履歴に秘密がないか、値を表示せず確認して」と頼みます。すでに記録したファイルは、あとから.gitignoreへ書くだけでは外れません。秘密を記録してしまった場合は、まず発行元でキーを無効化し、履歴への対応も進めます。

▼コーナー「先輩エンジニアならここを見る」

コミット履歴は、そのアプリの成長日記になります。「セット割を追加」「10円未満の切り捨てを修正」なら、後から探せます。「修正」だけでは何を直したかわかりません。メッセージは未来の自分への、短い道案内です。

10.6 ブランチとマージ——本体に触らずに試す

コミットの列は、一本道とは限りません。途中から枝分かれさせることができます。この枝がブランチです。

図 10-6

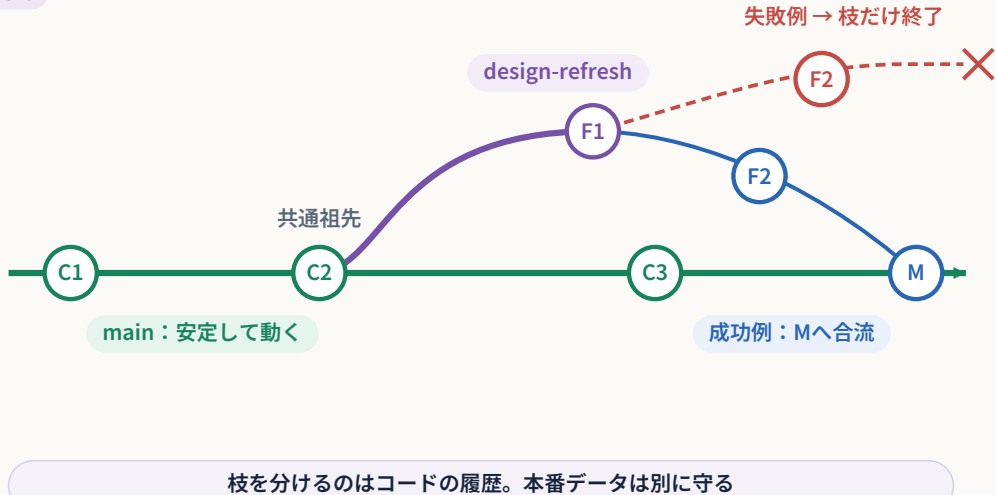


図10-6 安全に試すブランチ

ブランチで記録した変更は、合流させるまで本体のブランチには入りません。「デザイン全面刷新」を別の枝で試し、受け入れ条件を満たしたら本体へ取り込みます。これがマージです。採用しないなら本体へ戻り、試した変更を残すか整理するか決められます。

ただし、枝が分かれるのはコードの履歴です。同じ本番データベースへ接続して書き換えたり、その枝を公開したりすれば、本番に影響します。試作の枝には、練習用のデータと設定を組み合わせます。

ひとり開発のいまは、「大工事の前は、AIに『ブランチを切って作業して』と頼めるらしい」と知っていれば十分です。ブランチが本領を発揮するのは、複数人で同じアプリを育てるときです。本書では深入りせず、第15章の卒業制作で必要になったら思い出してください。

10.7 ロールバック—AIに変更させる前のチェックポイント

道具はそろいました。仕上げは「戻る」です。

過去のコミットの状態にプロジェクトを戻すことをロールバックと呼びます。全滅したら、直前のセーブから再開する——ゲームの鉄則そのままです。ただしゲームと違って、開発のロールバックは負けではありません。「動いていた状態」という安全地帯に一度戻り、そこから小さく頼み直すのは、プロが日常的に使う前進のための一手です。

図 10-7

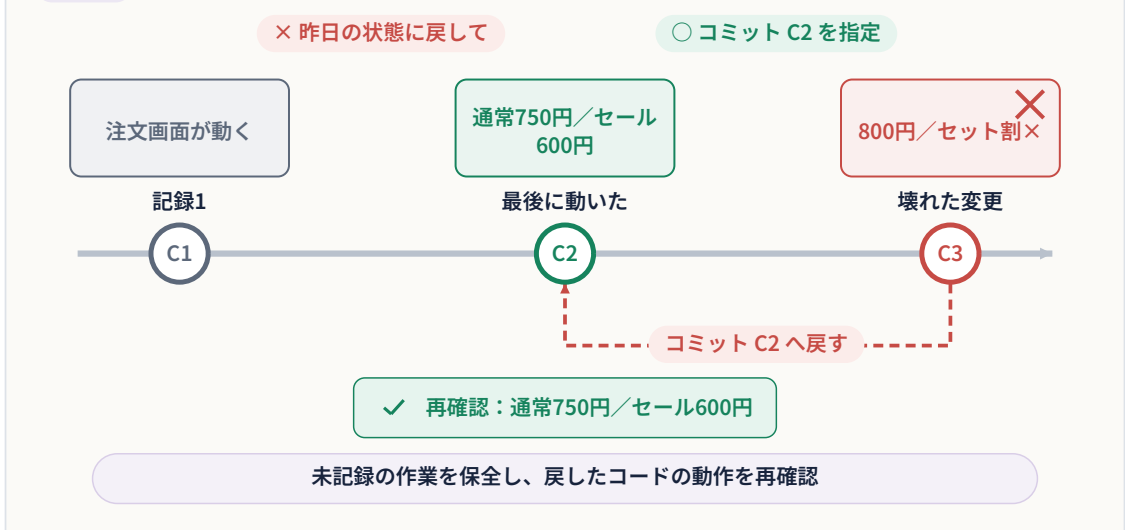


図10-7 動いていた地点へ戻る

図10-7では、記憶ではなく、コミットC2を指定して戻す点を見てください。

さて、ここまでの道具を使うのに、コマンドの暗記はまったく要りません。Gitには何百というコマンドがありますが、あなたがAIに頼む操作は、当面2つだけです。

1. 記録する：「変更差分と記録対象を確認して、秘密を除いてコミットしてください。メッセージは『セット割が動く状態』で」
2. 戻る：「履歴と未コミットの変更を見せてください。残す作業を保全し、動いていたコミットのコードへ戻す方法を説明してから実行してください」

朝の事故を思い出してください。あのとき「昨日の状態に戻して」は失敗しました。今度の頼み方と、何が違うのでしょうか。

あなた → AI

昨日の夜の状態に戻してください。(×: 朝の頼み方)

あなた → AI

コミットの一覧を見せてください。「セール操作を店長画面に」のコミットの状態に戻してください。(○: 記録がある場合の頼み方)

上は、AIの記憶に頼んでいます。AIは会話の印象から「それらしい状態」を作り直すしかなく、結果は毎回揺れます。

下は、記録を指しています。メッセージに加え、日時とコミットIDも確かめれば、戻したいコードを一意に選べます。戻したあとに、通常時750円・セール中600円と、変えていない機能を確認します。公開中のアプリへの反映やデータベースの復元は、コードを戻す操作とは別です。

※読み方の補足: Gitの「戻る」には、過去の状態を取り出す方法や、変更を打ち消すコミットを追加する方法などがあります。共有済みの履歴を勝手に書き換えたり、未コミットの作業を失ったりしないよう、残すものを先に指定します。手順の候補はAIに説明させ、実行後に履歴と動作を確認します。

そして、この章でいちばん大切な作法を、ここに置きます。

AIに大きな変更を頼む前に、必ずコミットする。

「大きな」の目安は、影響範囲が言えない変更です。「デザインを全部」「構成を整理」「～を作り直して」のような依頼が典型です。AIがどのファイルを何個変えるか、頼んだ本人にも予測できません。

予測できない変更の前には、記録を打つ。第4章で「一度に全部作らせない」を学びましたが、それでも大きく頼みたい場面はあります。この作法があれば、大きく頼む勇気と、失敗しても戻れる保険が、両立します。

別の条件でも動くか試す

AI開発環境によって、この章の道具の見え方はさまざまです。

- 「バージョン管理」ボタンの奥に、コミットの一覧が見えるもの
- チャットの横に「チェックポイントに戻る」ボタンが出るもの
- 自動で細かく記録を取ってくれるもの(名前が「コミット」でないこともあります)

見た目がどうであれ、確認することは2つだけです。記録の一覧が見られること。選んだ記録の状態に戻れること。この2つが確認できれば、あなたの環境の流儀がこの本と違っても、この章はクリアです。見つからなければ、AIに「このプロジェクトをGitで管理して。今の状態を最初のコミットにして」と頼んでください。

演習10 タイムマシン

練習用の環境で、変更して戻る演習です。本番の会計記録には接続せず、まず小さな変更で一往復してみます。

- ミッション: 会計アプリを「動く状態を記録→試しに変更→記録した状態へ戻る」の一往復させる
- 時間の目安: 30分
- 手順: ①通常時750円・セール中600円を確認してコミット ②練習用の枝で「セット割を一時的に0円にして」と頼み、その変更もコミット ③通常時800円・セール中640円を確認 ④残す変更を確かめて元のコードへ戻し、750円・600円を再確認 ⑤README(第4章)に起動と確認の方法を書き、コミット
- クリア条件:
 1. コミット履歴に、自分の言葉のメッセージが付いたコミットが2つ以上並んでいる
 2. 一時変更により、合計が800円・640円に変わったことを確認した
 3. ロールバック後、受け入れ条件(通常時750円/セール中600円)がふたたび成立している

4. リポジトリに**README.md**があり、アプリ名・何をするアプリか・起動の仕方が書いてある

・ つまづいたら:

- ・ 記録の道具が見つからないときは、AIに「このプロジェクトをGitで管理して」から
- ・ 戻れなくなったら、履歴と未コミットの変更を見る。戻す記録をコミットID・メッセージ・日時で特定する
- ・ 方法がわからないときは「750円になっていたコードへ戻したい。いまの作業は失わず、共有済みの履歴も書き換えない方法を説明して」と頼む
- ・ 裏ダンジョン: 別のブランチでデザイン変更を試し、750円・600円などの受け入れ条件を満たすことを確認してからマージする
- ・ 冒険の書に記録: リポジトリ(GitHubに預けたもの)+コミット履歴の画面+README

(授業向け)ペアでリポジトリを交換し、相手のコミット履歴だけを読んで「このアプリがたどった歴史」を1分で説明し合おうと、メッセージの良し悪しが体感できます。

この章のまとめ

- ・ アプリの正体は、ソースファイルと設定ファイルが入ったフォルダー一式、つまりプロジェクトである。自分のコードは、依存関係として記録された借り物の部品に支えられている
- ・ ターミナルのコマンドは暗記しなくてよい。AIが出したコマンドは、何をするか・何が変わるか・戻せるかを説明させてから承認する
- ・ コミットは、記録対象のファイルのその時点の状態を、メッセージと時刻付きで残す。diffを見れば、どの行が変わったかがわかる
- ・ AIに大きな変更を頼む前にコミットする。戻すときは「昨日の状態」ではなくコミットを指定し、戻したあとに受け入れ条件を確かめる
- ・ Gitで戻せるのは、コードなど記録対象のファイルである。データベースの中身はバックアップ、秘密はSecretsで別に守り、秘密を記録してしまったら発行元でキーを無効にする

理解度チェック

1. コミットとdiffは、それぞれ何ですか。
2. デザイン変更のあと、会計が800円と出るようになりました。「昨日の状態に戻して」の代わりに、AIにどう頼みますか。戻したあとに確かめることも教えてください。
3. AIに頼んだ変更のあと、データベースの会計記録が一部消えていました。動いていたコミットへコードを戻せば、消えた記録も元に戻ると言えますか。理由も教えてください。

解答のめやす

1. コミットは、残す変更を選び、その時点のファイルの状態をメッセージや時刻とともに記録する操作です。diffは、前回の記録といまなど、2つの状態の差分です(→10.5)
2. コミットの一覧を見せてもらい、動いていたコミットをメッセージ・日時・コミットIDで特定して、その状態に戻すよう頼みます。戻したあと、通常時750円・セール中600円と、変えていない機能を確認します(→10.7)

3. 言えません。コミットに入るのは記録対象のファイルで、サーバーのデータベースの中身は入らないからです。コードを戻す操作とデータの復元は別で、記録の復元には第8章のバックアップを使います(→10.5、10.7)

60秒でクエスト報告

動いていたコードをGitのコミットとして残しています。変更後に壊れたら、履歴から戻す地点を選び、diffで変わった行を見ます。いまの未コミット作業を保全してから、必要に応じてコードを戻し、750円・600円などの条件を再確認します。Gitに入れないSecretsや本番データは別に管理し、データの復元には第8章のバックアップを使います。

——これは口頭試問の第5問「壊れた場合、どこを確認し、どう戻しますか」への、そのまま答えの型です。9章まで積み上げてきたあなたのアプリは、今日から「壊しても戻せるアプリ」になりました。

レベルアップ!

Lv.10 称号: セーブの達人

- 手に入れた装備: ライブラリ、パッケージ、フレームワーク、依存関係、ターミナル、Git、リポジトリ、コミット、diff、ブランチ、ロールバック
- 冒険の書: リポジトリ+コミット履歴+README
- 身についた作法: 動いたらコミット。大きく頼む前にコミット

次の第11章は、故障さがしです。ロールバックは強力ですが、あれは「避難」であって「原因の解明」ではありません。戻っても、また同じ壊れ方をすることがあります。バグの正体を突き止め、AIに丸投げせず切り分ける技術——バグモンスター図鑑を持って、原因の森へ入ります。

第11章 エラーをAIに丸投げせず、原因を切り分ける

クエスト11: モンスターには弱点がある

文化祭まで、あと8日。

この章のねらい

- ・ 故障を、何をしたら・何が起きたか・どうなってほしいかの3点で書ける
- ・ 入力・処理・保存・表示に分けて、原因のある区間を絞り込める
- ・ 修正後に、バグ本人・近所・端っこの3つを確かめられる

今回のクエスト

あなたの会計アプリは、ずいぶん立派になりました。履歴はデータベースに残り(第8章)、セールの操作は店長だけができ(第9章)、変更の歴史はコミットで守られています(第10章)。

ボタンからデータベースまで道がつながると、うまく動かないときの候補も増えます。「保存されない」の原因は、ボタン、通信、保存処理のどこにあるのでしょうか。

この章では、故障を手がかりに変えます。必要なのは、全コードを読める力よりも、「ここまでは正しい」と確認できる場所を一つずつ増やす力です。

この章のミッションは、原因の候補を自分で絞り込み、AIと確かめながら直すことです。章の終わりには、症状、再現手順、修正と確認をつないだバグ報告書が手に入ります。

ボス出現——「直して」を5回繰り返した夜

文化祭まで、あと8日。この節では少しだけ時計を進めて、前夜に起きる事件を先に見ておきましょう。

事件は、店長役の部員が割引率をいろいろ変えて練習していたときに起きました。

「ねえ、35%引きにすると、チャイとラムネで357円って出るんだけど」

検算してみましょう。チャイ400円+ラムネ150円で550円。どちらもドリンクなのでセット割はなく、35%引きで357.5円。仕様では割引後の10円未満は切り捨てなので、350円になるはずです。

画面には357円。7円の差。

エラーの文字はどこにも出ていません。アプリは平然と動いています。ただ、答えだけが違う。

あなたはAIにこう頼みます。

あなた → AI

計算がおかしいので直してください。

AI → あなた

割引計算を修正しました。端数処理の丸め方を見直しています。ご確認ください。

確認すると、まだ357円です。「直っていません。直してください」。

AIは今度は別の場所を直します。すると、通常時のコーヒー2つとクッキー1つが、750円ではなく740円になりました。壊れていなかった場所が、壊れたのです。「そこは戻してください。それで、さっきのを直して」――。

5回目の「直して」を送ったころには、どこが正しくてどこが壊れているのか、見えにくくなっています。本書では、このように確認の材料が増えないまま修正を繰り返す状態をAIループと呼びます。変更だけが積み重なると、最初の不具合と修正の副作用も区別できなくなります。

このループを抜けるには、AIの答えを待つ前に、渡す材料を増やすことです。どの入力で、何円になるはずが、何円になったか。その差を伝える道具を、これから順番に装備していきます。

11.1 バグとエラーの違い

まず、原因につける名前と、表に現れる現象を分けておきましょう。

バグ(bug)は、プログラムや設計にある不具合です。計算の丸め方を間違える、保存処理を呼び忘れる、といった原因を指します。

エラーは、処理に失敗した状態や、その通知です。バグが原因でエラーになることもあれば、通信切断など、プログラムの外の事情でエラーになることもあります。

調査の入口として役立つのは、エラーの通知があるか、通知なしに結果が違うかという見方です。前者ならメッセージやログを読み、後者なら仕様と実際の値を比べます。さっきの357円は、通知を出さずに計算結果を間違えるバグです。

この章では、この2つの現れ方を「悲鳴が出る」「静かに間違う」と呼びます。バグとエラーを、互いに重ならない故障の種類として覚える必要はありません。図11-1では、通知があるかないかで、調べ方の入口が変わる点を見てください。

図 11-1



図11-1 不具合の2つの現れ方

エラーの通知があれば、発生した場所や時刻を調べる足がかりになります。ただし、表示された場所が根本原因とは限りません。

一方、357円のバグは、2割引の検算では姿を見せませんでした。その注文では端数が出なかったからです。エラーが出ないことだけでは、正しく計算できたとは言えない。この点を、最初の道しるべにしましょう。

11.2 再現手順の書き方

序章で「エラー文を省略しない」という作法を覚えました。メッセージを途中で切ると、原因を探す手がかりも落としてしまうからです。共有前には、キーや個人情報などの秘密だけを伏せます(第13章)。

通知なしに結果が違うときは、メッセージだけに頼れません。そこで、故障を報告するときは次の3点セットで書きます。これはエラーの通知がある場合にも役立ちます。

1. 何をしたら(操作)
2. 何が起きたか(実際の動作)
3. 本当はどうなってほしいか(期待した動作)

これが再現手順です。日常語で言うと「同じ故障をもう一度起こすための手順書」。357円のバグなら、こう書けます。

- ① チャイ1つとラムネ1つを注文し、割引率35%のタイムセール中に会計を押すと
- ② 合計が357円と表示される
- ③ 仕様は「割引後の10円未満は切り捨て」なので、350円になってほしい

③があることに注目してください。正解の根拠は、いまのコードではなく、仕様書(第3章)です。AIにも仕様書を渡し、「実装は357円、仕様は350円」と比較できる状態にします。仕様が曖昧なら、先に店の人とルールを決め直します。

もうひとつ、大事な性質があります。再現できる故障は、半分直ったようなものです。逆に「たまに変になる」としか言えない故障は、プロでも苦戦します。

まず①を固めて、何度でも同じ壊れ方をさせられるようにする。遠回りに見えて、これが最短路です。

図11-2では、エラーの通知があるときも、3点を同じようにそろえることを確かめてください。

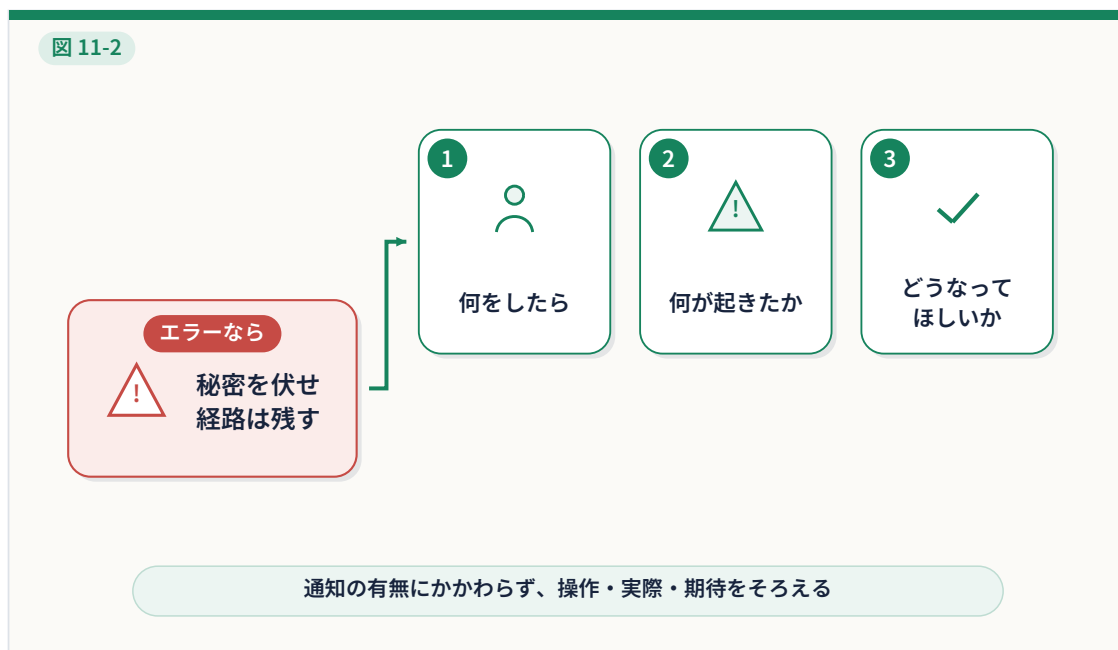


図11-2 再現手順の3点セット

11.3 エラーメッセージとスタックトレース

エラーの通知には、発生箇所を探す手がかりが付いてきます。次は、0個の注文を正しく扱えない計算処理を想定した作例です。見るのは、1行目の英文と、2行目の`calc.js:12`です。

```
TypeError: Cannot read properties of undefined (reading 'price')
    at calcTotal (calc.js:12)
    at onCheckout (order.js:34)
    at HTMLButtonElement.<anonymous> (main.js:8)
```

1行目がエラーメッセージ、悲鳴の内容です。英語ですが、全部読めなくてかまいません。2行目から下、`at`で始まる並びがスタックトレースです。日常語で言うと、故障現場の住所と、そこへ至る経路です。

- いちばん上の`calc.js:12`が現場の住所。「`calc.js`というファイルの12行目」
- その下は、現場までの経路。「ボタンが押され→`onCheckout`が呼ばれ→`calcTotal`の中で事故」

まず、いちばん上の住所を見ます。ライブラリの内部なら、その下にある自分のコードの住所までたどりましょう。AIに渡すときも、秘密を伏せたうえで経路を残します。「表示された行で何が足りず、その値はどこから来たか」と聞けば、現場から原因へさかのぼれます。

「エラーが出ました」とだけ伝えるのは、住所を破り捨てて「どこかで事故がありました」と通報するようなものです。

図11-3で、「悲鳴の内容」「現場の住所」「呼び出しの経路」が、それぞれどの行かを確かめてください。



図11-3 スタックトレースの読み方

11.4 ログとブラウザの開発者ツール

悲鳴も出ない、画面にも手がかりがない。そんなときは、アプリの中の出来事をのぞく窓が2つあります。どちらも、第5章で出会った開発者ツールの中にあります。あのとき「観察窓」だった道具を、この章で武器に昇格させましょう。

1つめは**コンソール**タブ。エラーの悲鳴が流れてくる場所ですが、もうひとつの働きがあります。ログの表示です。

ログは、アプリが自分で書き残す動作の記録。日常語で言うと航海日誌です。「会計ボタンが押された」「小計550、割引率0.35、合計357.5」のように、所要所で日誌を書かせることができます。

検証用の環境で「割引計算の前後に、小計・割引率・計算結果をログへ出して」とAIに頼みます。出力された値を自分の検算と比べれば、数字がどこまで正しかったかがわかります。注文者の情報や認証のトークンは記録しません。

2つめはネットワークタブ。アプリとサーバーのあいだの通信が、1件ずつ全部見えます。第7章で使った、返事に付く番号(ステータスコード)もここに出ます。保存されない、一覧が出ない——「向こう側」がからむ故障のときは、まずこの窓です。

図11-4では、Consoleに処理の中身(ログ)が、Networkに通信の成否が並ぶことを見比べてください。Networkの **201** は「作成できた」という成功の番号です。200と同じく2で始まる番号は成功、と覚えておけば十分です。

図 11-4

Console

アプリの独り言

14:02:11

total = 350

14:02:12

TypeError: items

14:02:12

calc.js:12

Network

外との依頼と返事

名前

状態

時間

POST /sales

201

82ms

GET /events

500

1.2s

GET /menu

200

46ms

処理の中身は Console、通信の成否は Network

図11-4 開発者ツールの2つの窓

11.5 入力、処理、保存、表示に分けて切り分ける

道具はそろいました。次は、探し方の作戦です。

「アプリのどこかが壊れている」と考えると、途方に暮れます。そこで、第5章で描いたアプリ構成図を、故障さがしの地図として使い直します。ボタンを押してから結果が出るまで、データは決まった道を通るのでした。その道を、4つの区間に分けます。

入力 → 処理 → 保存 → 表示

図 11-5

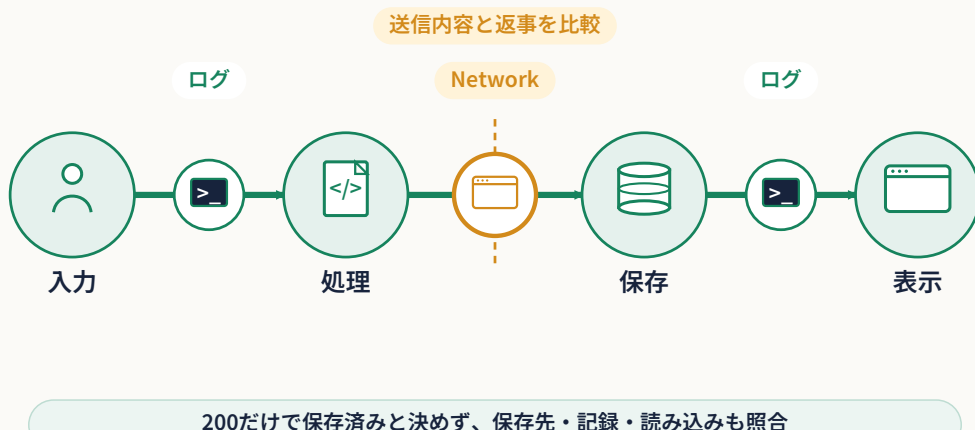


図11-5 故障さがしの一本道

図11-5の区間の境目ごとに、何が入って、何が出たかを比べましょう。たとえば「履歴が再読み込みで消える」なら、ネットワークタブで保存のリクエストを見ます。

- ・ リクエストが飛んでいない → ボタンから送信までの経路を調べる
- ・ リクエストが飛んだ → 送信した内容、返事の番号、返事の本文を確認する
- ・ 返事が200でも履歴がない → 保存先・保存された行・読み込み条件を順に照合する

200はリクエストへの成功応答であり、期待したデータが正しい保存先に入った証明ではありません。間違った内容を送った場合も、別のデータベースに保存した場合も、200が返る実装はありえます。

このように観察を増やし、原因候補を狭めるのが切り分けです。「どこかが怪しい」を、「送信内容までは正しい。次は保存先を見る」に変える捜査です。

※厳密に言うと: 図11-5は、観察のために簡略化した地図です。実際には、保存に失敗して画面へ戻る道や、外部APIへの寄り道もあります。複数の原因が重なることもあるので、「確認済み」と「未確認」を分けて記録することを優先してください。

▼コーナー「先輩エンジニアならここを見る」

数字が違うなら計算の入力と出力、再読み込みで消えるなら保存と読み込み、真っ白なら表示と通信。症状から最初の観察場所を選びます。見当は仮説なので、記録が合わなければ別の区間へ移ります。

11.6 仮説を一つずつ試す——変更範囲を小さくする

区間が絞れたら、仮説を立てます。357円のバグなら「切り捨てが『10円未満』ではなく『1円未満』になっているのでは」。ここで、守ってほしいルールが2つあります。

仮説は一度にひとつ。2つ同時に直して直った場合、どちらが効いたのかわかりません。それでは、次に同じ故障が出たときのための経験値が残りません。

変更範囲は小さく。AIに頼むときに「計算の関数だけを直して。ほかの場所を変えないで」と範囲を区切ります。あの夜のループの後半で通常価格まで壊れたのは、「直して」のたびにAIが広い範囲を書き換えたからです。範囲を区切れれば、壊れうる場所も区切られます。

そしてもうひとつ。試す前にセーブ——第10章の言葉で言えば、コミットです。直す前の状態にいつでも戻れることを確かめてから、変更する。これで、「悪化し続けるループ」にはまっても、いつでも抜け出せます。

AIと一緒に直す

材料がそろいました。357円のバグを、今度は正しい型で報告します。あの夜の「計算がおかしいので直してください」と見比べてください。

あなた → AI

会計アプリの割引計算にバグがあります。

【再現手順】①チャイ1つとラムネ1つを注文し、割引率35%のタイムセール中に会計を押すと②合計が357円と表示されます③仕様(割引後の10円未満は切り捨て)では350円になるはず

【確認済み】割引なしの合計550円は正しい。2割引のとき(コーヒー2つ+クッキー1つ=600円)も正しい

【仮説】切り捨てが「10円未満」ではなく「1円未満」になっていませんか

【依頼】計算の関数だけを確認して直してください。ほかの場所は変更しないでください

AI → あなた

ご指摘のとおりでした。`calcTotal`の最後の切り捨てが`Math.floor(total)`となっており、1円未満の切り捨てになっていました。仕様の「10円未満切り捨て」に合わせて修正しました。変更はこの1行だけです。

原因は、`calcTotal`の最後にある`return`の行でした(第3章で見た計算ルールの、実装違いです)。修正前と修正後の行を見比べてください。

役割ラベル: 計算

```
[javascript]
// 修正前(バグ): 1円未満の切り捨てになっていた
return Math.floor(total); // 357.5 → 357
// 修正後: 仕様どおり、10円未満を切り捨てる
return Math.floor(total / 10) * 10; // 357.5 → 350
```

この例では、違いは`/ 10) * 10`があるかないかでした。修正の大きさは不具合によって違いますが、「どの入力でどうずれるか」がわかれば、変更の理由を確かめられます。次は、350円になったことと、ほかの計算を壊していないことを実際に試します。

第3章で「願いごと」を「注文書」に変えたら、AIの働きが変わりました。あれとまったく同じ構図です。

AIの返事が本と同じでなくてもかまいません。原因の場所(ファイルと行)と、何をどう変えたかの説明が返ってくることが確認できれば、先へ進めます。逆に、説明なしの「修正しました」だけが返ってきたら、「どの行をどう変えたのか教えてください」と聞いてください。説明できない修正を受け取らないのは、開発責任者の仕事です。

11.7 「直ったように見える」を疑う

350円が出ました。直りました——と言い切る前に、最後の関門があります。あの夜、357円を直そうとして、750円が壊れたのを思い出してください。

修正は、別の場所を壊すことがあります。「直ったように見える」と「直った」のあいだには、まだ距離があるのです。

だから、直したあとには必ず3つを確かめます。

1. バグ本人: チャイ+ラムネ、35%引きで350円になるか
2. バグの近所: 通常時750円、2割引600円は、まだ正しいか(第3章の受け入れ条件の検算値)
3. 端っこ: 0個の注文、割引率0%——極端な入力でも壊れないか

2番に使ったのは、仕様書に数字で書いておいた受け入れ条件です。あの数字が、今回の変更で既存の計算を壊していないか確かめる物差しになります。

とはいえ、直すたびに3つを手で確かめるのは、正直つらい。この確認を機械に任せて、「動いた」を「正しい」に格上げする方法が、次の第12章のテストです。ここでは「直った、の後ももう3つ」とだけ覚えてください。

11.8 同じ修正を繰り返すAIループから抜ける

最後に、あの夜のループに、正面から決着をつけます。危険信号はこれです。同じ指示に、同じような修正が2回返ってきた。

そうになったら、3回目の「直して」を送ってはいけません。手を止めて、3つの脱出口のどれかへ向かいます。

出口1: 最小構成で再現する。アプリ全体の中で直そうとせず、故障だけを取り出した最小の再現を作ります。「新しい画面に、**calcTotal**と同じ計算をする関数だけを置いて、550円の35%引きの結果を表示してください」。余計なものが視界から消えると、AIも人間も、急に原因が見えるようになります。

出口2: 言い方を変えて、別の仮説を促す。「直して」の繰り返しは、AIを同じ考えの道に閉じ込めます。修正させるのをやめて、考えさせる質問に切り替えます。「直さなくていいので、この計算が357になる原因の候補を3つ挙げてください」「切り捨て以外に原因があるとしたら、どこですか」。候補が並べば、あとは切り分けの出番です。

出口3: ロールバックして作り直す。変更が積み重なって、何が正しいのかわからなくなったら、第10章の装備の出番です。最後に検算が通っていたコミットまで戻り、バグ報告書だけを持って、あらためて頼み直します。迷走した5回分の変更を捨てるのは、負けではありません。迷路で入口に戻るの、立派な作戦です。

図11-6では、同じ修正が2回返ってきた地点から、3つの出口へ分かれることを見てください。

図 11-6

同じ修正が2回＝危険信号

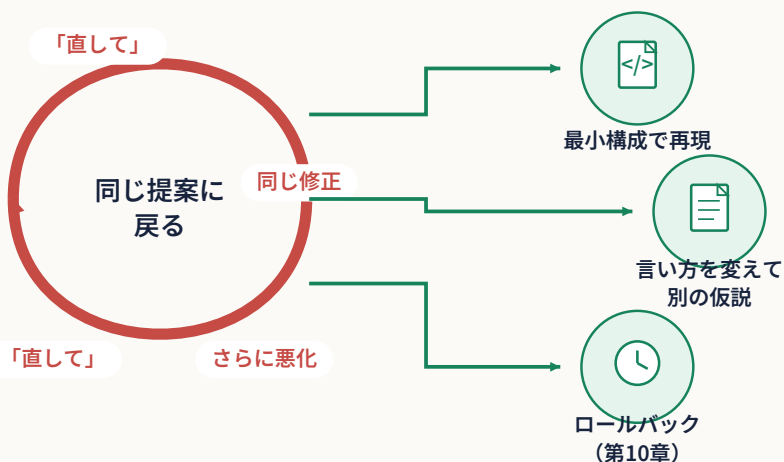


図11-6 AI修正ループからの出口

バグモンスター図鑑——診療所の常連5体

この章の総まとめとして、会計アプリで練習したい故障を5体のモンスターにしておきます。出現場所は、最初に調べる区間です。図鑑を入口に、実際の記録を見て仮説を確かめてください。

▼バグモンスター図鑑

No.1 ハスウクズシ

- ・ 出現場所: 処理の区間(計算)
- ・ 症状: エラーは出ないのに、合計の数字が仕様と違う。350円のはずが357円
- ・ 生態: 割引率がきれいな数字のあいだは端数が出ず、姿を見せない。静かに間違えるバグの典型
- ・ 弱点: 受け入れ条件の検算値。手計算と1円でも違えば発見できる。疑う場所は役割ラベル「計算」の関数だけ

No.2 キオクナクシ

- ・ 出現場所: 保存の区間
- ・ 症状: 会計はできるのに、再読み込みすると履歴が消えている
- ・ 生態: 序章の事件の再来に見えるが、データベースがあるのに消えるなら別種。「書けていない」のか「読めていない」のかで住みかが違う
- ・ 弱点: ネットワークタブ。送信内容と返事を見て、保存先の行、読み込み条件の順に調べる

No.3 マドグチトザシ

- 出現場所: 処理の区間の外れ(外部APIとの境)
- 症状: イベント一覧が真っ白。自分は何も変えていないのに、ある日突然現れる
- 生態: 正体は他人のサーバーの故障や回数制限。あなたのアプリ側に備え(第7章の案内表示)がないと、画面ごと巻きぞえにする
- 弱点: 返事の番号と本文(第7章)。4xxなら入力・認証・回数制限、5xxなら相手側の処理を調べる足がかりになる。故障の原因を確かめつつ、自分のアプリの案内表示も確認する

No.4 カラウチ

- 出現場所: 入力の区間
- 症状: ふだんは元気に動くのに、特定の入力——0個の注文、空欄、巨大な数——のときだけ悲鳴を上げて止まる
- 生態: 悲鳴(エラー)が出るぶん正直者。スタックトレースに現場の住所を残していく
- 弱点: 再現手順の①「何をしたら」。入り口で変な値をはじく仕組み(第8章)を頼めば退治できる

No.5 カクレトビラ

- 出現場所: 表示の区間(の手前に立つ門番)
- 症状: 店長画面だけが表示されない。ほかの画面は全員ぶん正常
- 生態: 故障ではなく「わざと隠す仕組み」(第9章の権限)の設定違いであることが多い。誰でログインしているかによって症状が出たり消えたりする
- 弱点: 「誰として操作したか」を再現手順の①に必ず書く。ロールの判定を疑う

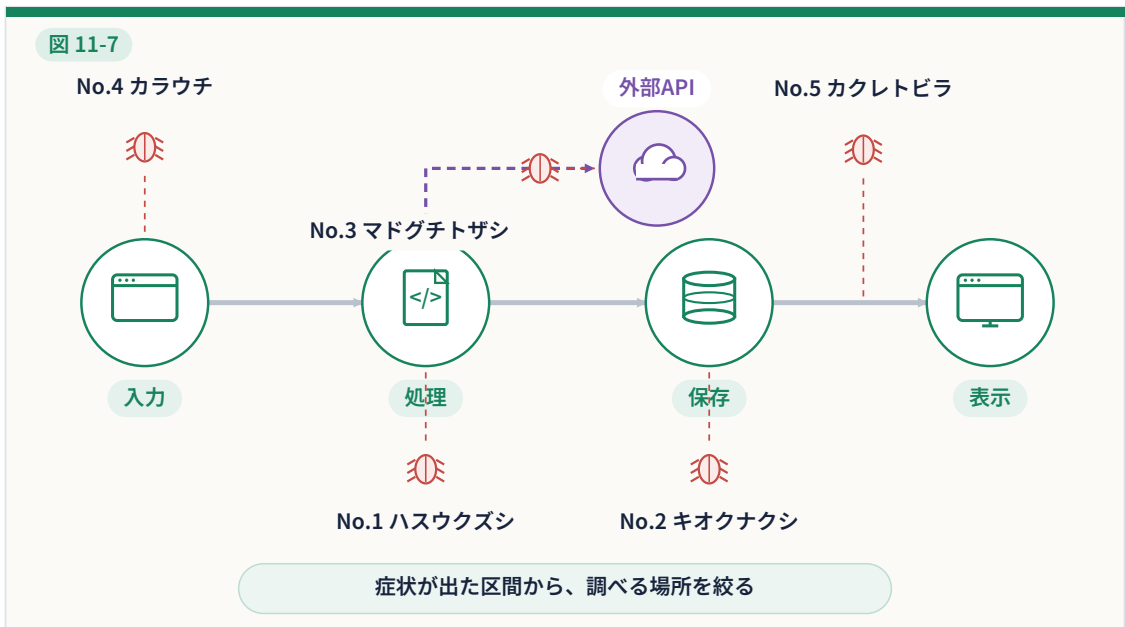


図11-7 バグモンスター出現地図

名前を覚える必要はありません。大事なのは、故障に出会ったとき「これは4区間のどの住人か」を言えることです。この図鑑は、付録B(つまり別シート)の症状索引と対応しています。

別の条件でも動くか試す

この章の型は、会計アプリ専用ではありません。再現手順の3点セットと四分割の切り分けは、あなた自身の題材でも、仕事のシステムの不具合報告でも、そのまま通用します。

うまく使えているかの目安は、観察した事実と、修正後の確認がつながっていることです。AIの説明が詳しくても、実際のログや再現結果と合わなければ、仮説として見直します。

もし本の通りに進まなくても、行き先は決まっています。同じ修正が2回返ってきたら、11.8の3つの出口のどれかです。

演習11 バグ診療所

- ミッション: 自分の会計アプリの検証用コピーで、図鑑の5症状を一つずつ再現し、診断・修正の報告書を書く。AIには「検証用コピーだけに、今回調べる症状を1つ作って。答えは別に控えて」と頼む。外部APIの失敗はモックで作り、本番データは使わない
- 時間の目安: 60分
- クリア条件:
 1. 5症状それぞれに、再現手順の3点セット(何をしたら・何が起きたか・どうなってほしいか)が書けている
 2. 各症状について、原因が四分割のどの区間だったかを記録した
 3. 修正後に次の5つが確認できる——チャイ1つ+ラムネ1つの35%引きが350円/再読み込み後も履歴が残る/イベント一覧の失敗時に案内が表示される/0個のまま会計を押しても止まらない/店長でログインすると店長画面が開く
 4. どの修正のあとでも、通常750円・2割引600円の検算が崩れていない
- つまづいたら:
 - 悲鳴が出ていれば、秘密を伏せ、スタックトレースを省略せず渡す
 - 悲鳴が出ていなければ、ネットワークタブで送信内容と返事を照合し、未確認の区間を調べる
 - 1体直すごとにコミットする(第10章)。同じ修正が2回返ってきたら、3つの出口へ
- 裏ダンジョン: タイムアタック。ペアで相手のアプリに新しいバグを1体仕込み、交換して、バグ報告書の完成までの時間を競う(出題者は「どの区間のバグか」を答えられること)
- 冒険の書に記録: バグ報告書5枚(症状・再現手順・原因の区間・修正内容・確認記録)

(授業向け)グループの場合は、5症状を分担して1人1体を担当し、報告書を回し読みして「再現手順どおりに再現できるか」を相互チェックすると、報告の型が早く身につきます。

この章のまとめ

- 不具合には、エラーの通知が出るものと、通知なしに結果が違うものがある。エラーが出ないことだけでは、正しく動いたとは言えない
- 故障は、何をしたら・何が起きたか・どうなってほしいかの3点で報告する。正解の根拠は、いまのコードではなく仕様書
- 入力・処理・保存・表示の区間の境目で、何が入って何が出たかを比べ、原因の候補を狭める。返事が200でも、正しく保存された証明にはならない
- 仮説は一度にひとつ、変更範囲は小さくして試す。直したあとは、バグ本人・近所・端っこの3つを確かめる
- 同じ修正が2回返ってきたら、「直して」を重ねない。最小構成で再現する、言い方を変えて別の仮説を促す、ロールバックする、のどれかに切り替える

理解度チェック

1. スタックトレースを受け取ったとき、最初に見るのはどこですか。AIに渡すときに伏せるものと、残すものも答えてください。
2. 会計アプリで「会計したのに、再読み込みすると履歴が消える」と報告がありました。ネットワークタブを見ると、保存のリクエストは送られていて、返事は200でした。次に何を確かめますか。
3. 357円のバグを直したら、チャイ+ラムネの35%引きが350円になりました。これで「直った」と言えますか。理由も答えてください。

解答のめやす

1. いちばん上の`at`の行にある住所(ファイル名と行番号)。ライブラリの内部なら、その下にある自分のコードの住所までたどります。キーなどの秘密は伏せ、経路は残して渡します(→11.2、11.3)
2. 送信した内容と返事の本文を確かめ、保存先・保存された行・読み込み条件を順に照合します。200は、期待したデータが正しい保存先に入った証明ではないからです(→11.5)
3. まだ言えません。修正は別の場所を壊すことがあるからです。バグ本人に加えて、近所(通常750円・2割引600円)と端っこ(0個の注文・割引率0%)も確かめます(→11.7)

60秒でクエスト報告

会計の合計が仕様と7円ずれるバグが出ました。エラーの出ない、静かな故障です。まず再現手順を固めました。チャイとラムネを35%引きで会計すると357円、仕様では350円です。入力・処理・保存・表示の四分割で考え、割引なしなら正しいことから、処理の計算部分に絞りました。切り捨てが1円単位になっているという仮説をAIに渡したら、修正は1行でした。直したあと、通常750円とセール600円の検算が崩れていないことも確認しました。

——口頭試問の第5問「壊れた場合、どこを確認しますか」には、この順序で答えられます。自分の不具合でも、観察した事実と修正後の結果を添えて説明してみましょう。

レベルアップ!

Lv.11 称号: バグハンター

- 手に入れた装備: バグ、エラー、再現手順、スタックトレース、ログ、切り分け
- 冒険の書: バグ報告書(原因・修正・確認記録)

357円は直りました。でも、考えてみてください。あのバグは、2割引のあいだずっと隠れていました。ほかにも、まだ静かに隠れている間違いがないと、なぜ言えるのでしょうか。「壊れてから探す」のこの章に対して、次の第12章は「壊れる前を見つける」です。直すたびに手でやった3つの検算を機械に任せ、「動いた」を「正しい」に変えます。セールが始まるちょうどの瞬間(13:59:59と14:00:00)の注文がどちらの価格か、という第3章からの宿題にも、そこで決着をつけます。

第12章 「動いた」から「確かめた」へ——テストで根拠を残す

クエスト12: 本番前のリハーサル

文化祭まで、あと6日。

この章のねらい

- 受け入れ条件を、前提・操作・期待する結果のテストケースに書き直せる
- 単体・結合・E2Eという範囲と、回帰という目的を見分けられる
- AIが書いたテストに、境界値や異常系の漏れがないかを確かめられる

今回のクエスト

357円のバグは直りました。けれど、来週のレジでは、練習と違う順番でボタンが押されるかもしれません。通信が遅いとき、セールが始まる瞬間、注文が0件のとき——アプリはどう振る舞うでしょう。

ここで、演劇部のことを考えてみてください。台本が完成し、稽古も済んだ。それでも、通し稽古をせずに本番を迎える演劇部はありません。この章のミッションは、あなたのアプリの通し稽古——テストです。章を終えたとき、あなたは「正しく動くことを、どのように確認しましたか」という問いに、記録を添えて答えられるようになります。

まずAIに確かめさせてみよう

テストもAIに頼めるはずです。多くの人が、最初是这样頼みます。

あなた → AI

文化祭が来週です。会計アプリが正しく動くか、テストしておいてください。

AI → あなた

主要な機能を確認しました。注文の登録、合計の計算、会計の記録と取り消し、履歴の表示、店長画面の集計が、いずれも正常に動作しています。問題は見つかりませんでした。

「問題は見つかりませんでした」。この返事で確かめたいのは、何を、どんな条件で試したかです。その記録がないまま、本番へ進む場面を見てみましょう。

ボス出現——前日リハーサルの悲劇

文化祭の前日、部員を集めて通しの練習をしたとしましょう。行列役の部員が「早く早く」と急かします。シフト担当役の1年生が、あわてて会計ボタンを2回押しました。

——履歴に、同じ会計が2件。売上の集計が水増しされました。

さらに練習の途中で店長役がタイムセールを開始すると、会計の途中だったお客さん役が言いました。「え、私はセール価格? 通常価格?」。誰も答えられません。

AIは「問題は見つかりませんでした」と言ったのに。——種明かしをすると、AIがやったのは、各機能を1回ずつ動かして見たことでした。「動いた」は、たった1回の観察です。あなたに必要なのは「正しい」——決めた条件を、全部満たしているという確認です。

動いた。でも、文化祭当日も「正しく」動くと言えるか? これがこの章のボスです。

図 12-1

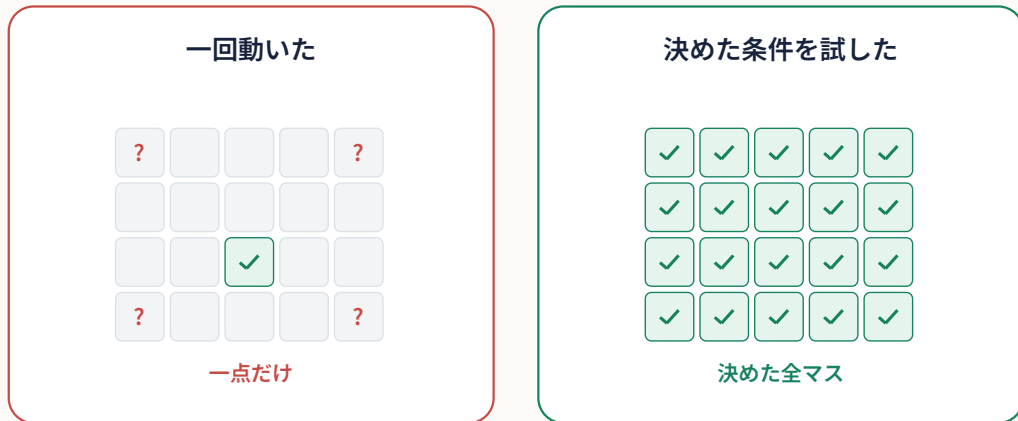


図12-1 動いたと正しいの差

12.1 テストとは何か——受け入れテスト

では、「決めた条件」のリストはどこにあるのでしょうか。朗報です。あなたはもう持っています。第3章の1ページ仕様書に書いた、受け入れ条件の5つです。

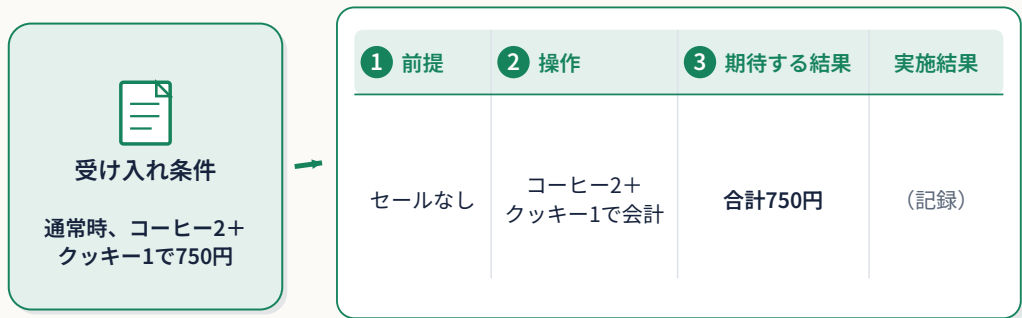
1. 通常時、コーヒー2つとクッキー1つで、合計750円になる(セット割50円引き)
2. 同じ注文が、タイムセール(2割引)中なら600円になる(セット割と併用)
3. タイムセールの開始・停止と割引率の変更は、店長だけができ、サーバー側でも権限を確認する
4. 会計を取り消すと、通常の履歴一覧と売上集計から除外され、取消済みの記録は残る
5. 割引後に10円未満の端数が出たら、切り捨てられる

第3章で「受け入れ条件は第12章で二度おいしい」と予告しました。ここがその回収です。作る前に書いた「これが確認できたら完成」の一覧は、作った後には確かめる手順書になります。

同じ1枚の紙が、設計図としても検査表としても働く。仕様書を書いた人だけが味わえる、設計の快感です。

受け入れ条件の1行ずつを、確かめられる形に書き直したものがテストケースです。日常語で言うと、確かめごと1件ぶんのカード。書式は3点セットです。「前提(どういう状態で)」「操作(何をして)」「期待する結果(どうなるはず)」。

図 12-2



同じ紙が二度働く：仕様の一行→テストの一行

図12-2 受け入れ条件からテストへ

番号	前提	操作	期待する結果	実施結果
T1	セールなし	コーヒー2+クッキー1で会計	合計750円	
T2	セール中(2割引)	同じ注文で会計	合計600円	
T3	シフト担当でログイン	店長画面とセール変更APIを試す	操作が拒否され、割引率は変わらない	
T4	会計が1件ある	その会計を取り消す	通常一覧と集計から除外され、取消済み記録は残る	
T5	セール中・割引率3割5分	ドーナツ1つで会計	160円(162.5円の端数を切り捨て)	

受け入れ条件をそのまま確かめるテストを、受け入れテストと呼びます。数字で書いた条件は、そのままテストになる——第3章で「数字で書くこと」にこだわった理由が、ここで効いてきます。

T5の「3割5分」は、端数の切り捨てを確かめるための設定です。本書のメニュー価格とセット割では、2割引にしても10円未満の端数が出ません。切り捨てを確認するなら、端数が出る注文と率を選ぶ。これがテストを設計する感覚です。

12.2 正常系と異常系

テストケースには2つの系統があります。正常系は、想定どおりの使い方であまりよくことの確認。日常語で言うと晴れの日の確認です。T1もT2も正常系です。

異常系は、想定外のことが起きても壊れないことの確認。日常語で言うと雨の日の確認です。変な入力、困った状況、意地の悪い操作。T3(権限のない人が開こうとする)は異常系です。

最初のテスト表は、注文して会計する、といった素直な操作で埋まりやすいものです。そこへ「通信が遅かったら」「権限がなかったら」と条件を一つ足してみます。

二重押しも、会計途中のセール開始も、特別な利用者だけの操作ではありません。正常な使い方の組み合わせでも、不具合は起きます。晴れの日と雨の日の両方を、具体的な場面で用意しましょう。図12-3では、道の外側に並ぶ罫を見てください。

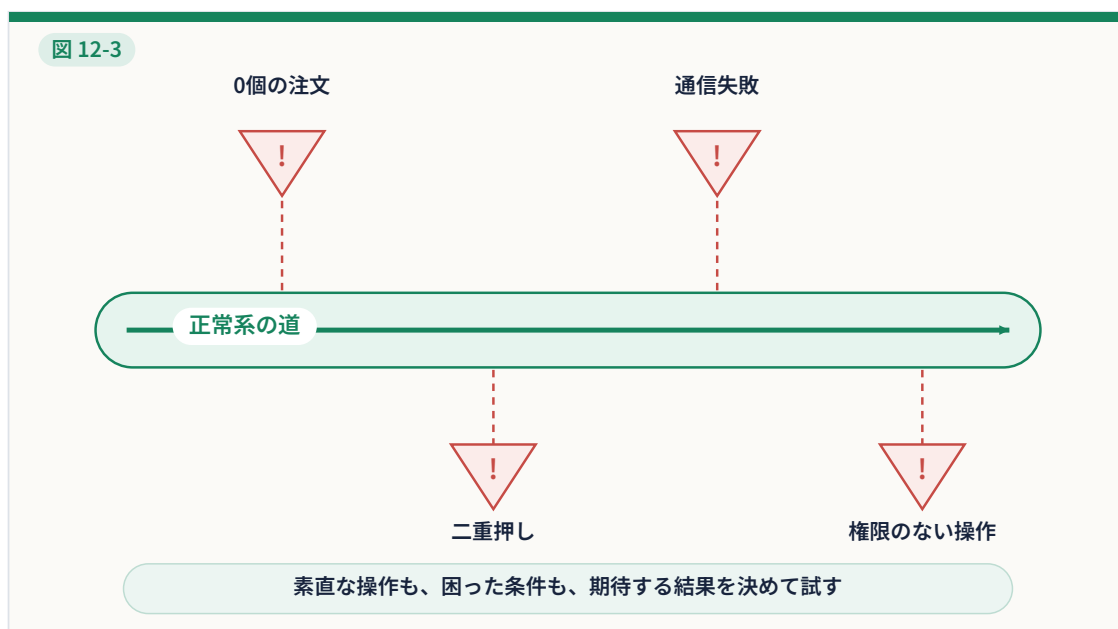


図12-3 正常系の道と異常系の罫

12.3 境界値——バグは「ちょうどの所」に棲む

次は、正常系にも異常系にもまたがる観点です。ルールが切り替わる「ちょうどの所」にある値を、境界値と呼びます。数量の上限が99なら99と100、開始時刻が14時ならその直前と直後。条件式の小さな違いが現れやすい場所です。

この章の主役は、タイムセールの境界時刻です。あなたの店の運用予定はこうでした——最終日の14時、売れ残りそうなら店長がセールを開始する。では、質問です。

- 13:59:59に会計ボタンを押した注文は、750円? 600円?
- 14:00:00ちょうど(店長が開始ボタンを押した直後)の注文は?

- 品物を選んでいる途中でセールが始まったお客さんは、どちらの価格?

第3章の「会計の途中でタイムセールが始まったら?」という分岐を、ここでテストできる形にします。本書では、サーバーが会計を確定するときのセール状態で価格を決め、適用した割引率と金額を記録することにします。端末ごとの時計や表示の遅れがあっても、判断する場所を一つにするためです。

仕様書に1行足してください。決めていないことは、テストできません。

決めたら、状態が切り替わる両側をテストします。ここでは検証用に「13:59:59に会計確定→750円」「14:00:00にセール開始を確定→その後の会計は600円」という順番を固定します。同じ時刻表示でも、会計と開始のどちらを先に処理したかで結果が変わるため、その順番も記録します。

「14時ごろの注文で試した」では足りません。バグは崖のふちに棲んでいて、「ごろ」のテストはふちを踏まないからです。

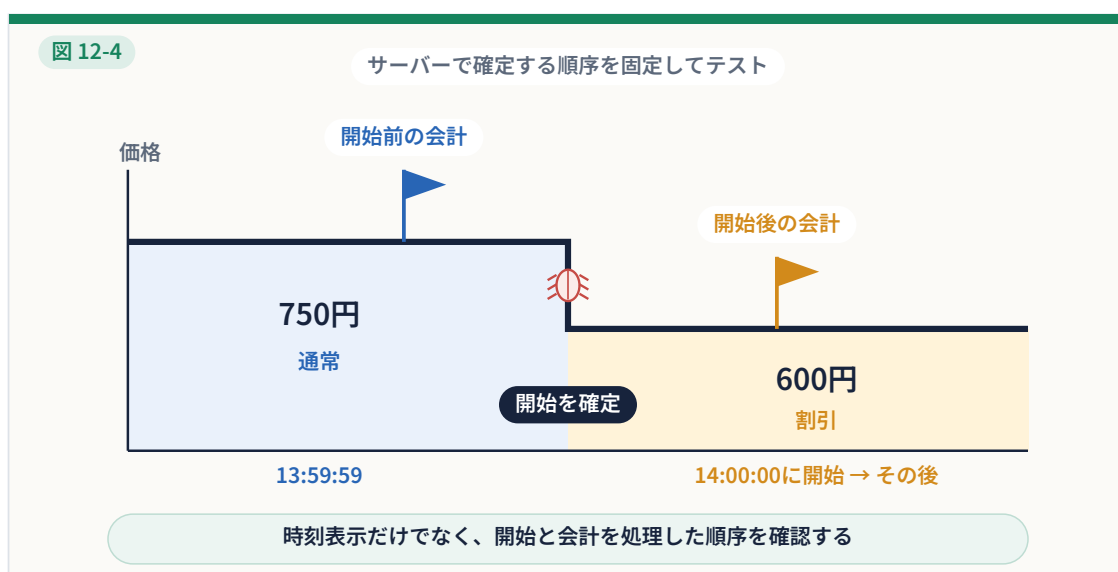


図12-4 時刻境界の両側

時刻の境界には続きがあります。リハーサルで開始したセールを止め忘れ、翌日になったら? 仕様は「自動終了なし・停止されるまで続く」でした(第4章)。期待する結果は「翌日もセール価格」で、実装がそのとおりならテストは合格します。

ところが、文化祭当日の開店時に通常価格のつもりで営業すると、意図せず2割引で売り続けることになります。

ここに、テストの大事な性質があります。テストが確かめるのは「仕様どおりか」であって、「仕様が現場に合っているか」ではありません。

この違和感に気づいたら、仕様や運用を見直します。たとえば「セール中の表示を全画面で確認できる」「開店前に店長が状態を確認する」と決めます。色だけでなく「セール中・2割引」の文字で伝える工夫は、第13章へつながります。

時刻のほかにも、境界はあります。あなたのアプリの「端っこ」を一巡りしましょう。

- 空欄・0個の注文: 何も選ばずに会計ボタンを押したら? これは第4章でテスト役のAIが見つけた穴で、あのとき「0品では会計ボタンを押せない」と決めたのでした。決めた仕様には、確かめるテストを付けます
- 非常に大きな数量: クッキー9999個、合計1,999,800円。文化祭では起きえない注文です。つまり、ほぼ確実に打ち間違い。受け付けてよい? 数量に上限を決める?
- ボタンの二重押し: リハーサル of 悲劇の正体です。素早く2回押して、記録が1件だけになるか
- 通信失敗: 会計ボタンを押した瞬間に通信が切れたら、その会計は記録された? されていない? 第7章のモックには、わざと壊すスイッチがありました。あの壊し方が、ここでそのまま再利用できます

図12-5は、時刻と数量を「値の境界」、二重押しと通信失敗を「異常系」に分けています。



図12-5 境界値の見張り塔

12.4 テストの範囲と目的——単体・結合・E2E・回帰

テストの名前は、どこまでを確かめるかという範囲と、何のために確かめるかという目的を分けると整理できます。まず、範囲を表す3つです。

- 単体テスト: 部品を1つだけ取り出して確かめる。日常語で言うと部品の検査。割引計算の関数 `calcTotal` だけを机に載せ、注文を入れて答えを見る
- 結合テスト: 部品のつながり目を確かめる。日常語で言うとなが目の検査。計算した合計が正しく保存され、保存された記録が正しく集計されるか
- E2Eテスト(End to End): 利用者の操作を端から端まで確かめる。日常語で言う通し稽古。検証用の環境で実際のブラウザを使い、メニューボタンを押して会計し、履歴画面に出るまでを流す。本番の売上を使うという意味ではない

回帰テストは、範囲ではなく目的の名前です。変更後に、すでに動いていた機能が壊れていないかを確かめ直します。前に直したバグの再発確認も含みます。単体テストを再実行する場合も、E2Eテストを再実行する場合も、この目的なら回帰テストです。会計アプリでは、変更の影響範囲に合わせてT1～T5や過去の不具合のテストを流します。

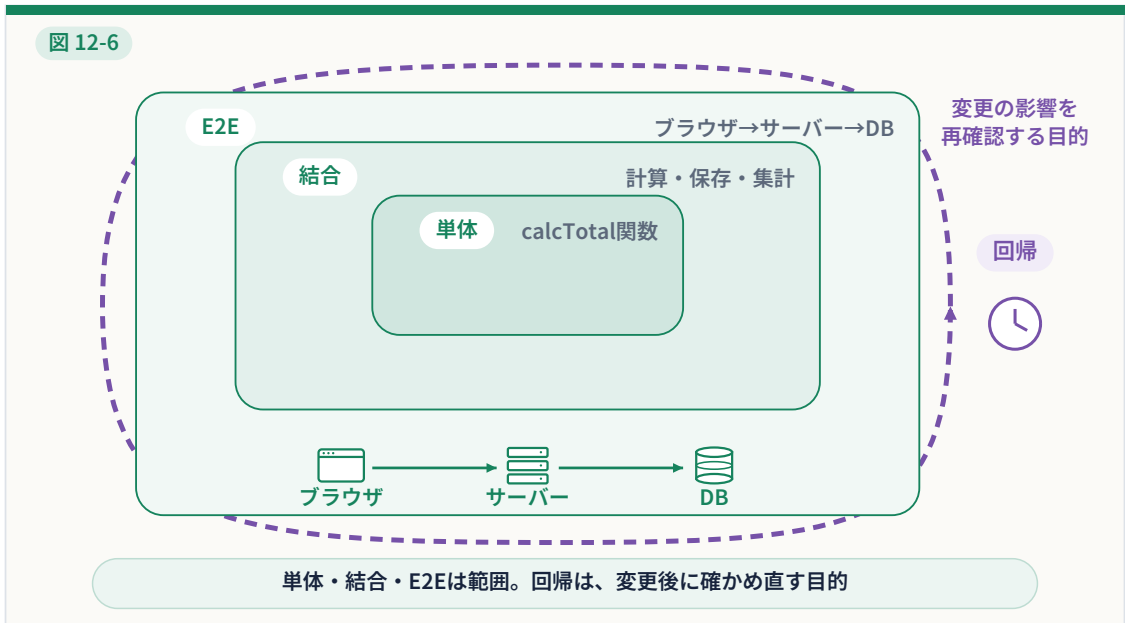


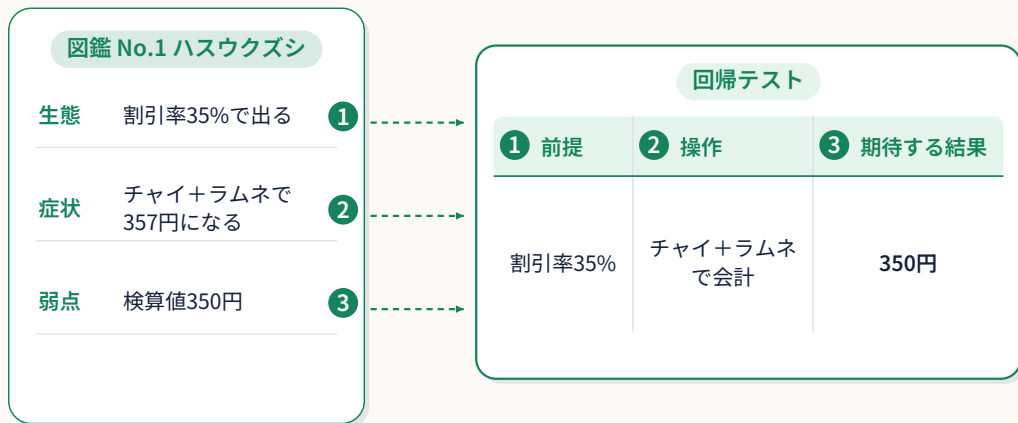
図12-6 テスト範囲の入れ子

図12-6では、回帰だけが範囲でなく目的として描かれています。

回帰テストには、心強い相棒がいます。第11章で作ったバグモンスター図鑑です。あなたが退治した5体——計算がおかしい、保存されない、APIが失敗する、特定の入力で止まる、店長画面だけ表示されない。図鑑の1ページが、そのまま回帰テストの1件になります。

「あのバグの再現手順を実行して、再発していないことを確かめる」。バグは、直しの副作用として同じ場所にぶり返すことが多いのです。図鑑は思い出の記録ではなく、現役の検査表です。図12-7で、図鑑の1ページがテストの3点セットへ移る様子を見てください。生態(出る条件)が前提に、症状を起こす注文が操作に、弱点の検算値が期待する結果になります。

図 12-7



図鑑がそのまま検査表になる

図12-7 バグ図鑑を回帰テストへ

※厳密に言うとは、「回帰」は英語のregression(後戻り)の訳です。直したはずの状態から後戻りしていないか、という意味だと知っていれば十分です。

12.5 テストデータの作り方

テストには、注文のデータが要ります。作り方のコツは4つです。

1. 手で検算できる小ささにする。コーヒー2つとクッキー1つなら、電卓で750円を出せます。品数20の注文では、期待する結果を自分が間違えます
2. 境界を狙って作る。0個、9999個、セットが成立するぎりぎりの組み合わせ、13:59:59の会計
3. 日付をまたぐデータを混ぜる。「昨日の会計」と「今日の会計」を用意しないと、日ごと集計のテストができません
4. 正解を先に書いてから、アプリに入れる。これがいちばん大事です。アプリの答えを見てから「これが正解」と表に書くと、アプリの間違いごと合格にしてしまいます。検算が先、アプリが後

ちなみに、本物のお客さんの会計履歴をテストに使い回すのは、本番が始まったあともおすすめしません。理由は第13章で扱います。

12.6 AIにテストを書かせる——意地悪はあなたの仕事

テストケース表ができれば、テストの実装はAIの得意分野です。冒頭の「テストしておいて」と何が違うか、見比べてください。

あなた → AI

会計アプリの割引計算をテストします。受け入れ条件は次の5つです(テストケース表T1～T5を貼る)。

1. T1・T2・T5の計算を、**calcTotal**関数の単体テストにしてください。T3の権限とT4の取り消しは、APIや保存を含む結合テスト、画面のE2Eテストに分けてください
2. テストは自動で繰り返し実行できる形にしてください
3. 実行して、何件中何件が合格か報告してください

AI → あなた

計算の単体テスト8件、権限と取り消しのテスト2件を作成し、検証環境で実行しました。結果は10件とも合格です。

計算8件の内訳は、通常価格2件、セット割2件、セール併用2件、端数切り捨て1件、0個の注文1件です。実行したコマンドと結果も添えます。

次は、テストの考え方を示す抜粋です。**コーヒー**などはテスト用の商品データを表し、関数の引数や商品データの形は実際のコードに合わせます。ここでは、テストケース表の1行がコードの1かたまりに対応する点に注目してください。

役割ラベル: 計算(のテストコード)

```
[javascript]
// calcTotalの単体テスト(抜粋)。「前提・操作・期待する結果」をコードにしたもの
test("T1 通常時: コーヒー2+クッキー1は750円(セット割)", () => {
  expect(calcTotal([コーヒー, コーヒー, クッキー], 0)).toBe(750);
});
test("T2 セール中(2割引): 同じ注文は600円(セット割と併用)", () => {
  expect(calcTotal([コーヒー, コーヒー, クッキー], 0.2)).toBe(600);
});
test("T5 割引率3割5分: ドーナツ1つは160円(162.5円を切り捨て)", () => {
  expect(calcTotal([ドーナツ], 0.35)).toBe(160);
});
test("T6 割引率3割: チャイ+コーヒー(700円)は490円", () => {
  expect(calcTotal([チャイ, コーヒー], 0.3)).toBe(490);
});
test("0個の注文: 合計は0円", () => {
  expect(calcTotal([], 0)).toBe(0);
});
```

expect(操作).toBe(期待する結果)——テストケース表の日本語が、そのままコードの形になっただけです。第3章で書いた「750円」という数字が、仕様書、テストケース表、テストコードと、3枚目の紙まで旅をしてきました。

T6は、第6章で見た小数の誤差を確かめる1件です(第11章の357円のバグとは別物です)。修正前の、0.7をそのまま掛けるコードなら、 700×0.7 が489.99…円になり、このテストは490円ではなく480円で失敗します。2割引や3割5分引きの例では、この誤差は合計に表れません。AIの書いたテストが2割引ばかりなら見逃すので、割引率を変えて試すのは、人が足すべき意地悪の1つです。

さて、この作例では10件すべて合格しました。表をよく見ると、まだ試していない条件があります。境界時刻のテストがありません。二重押しもありません。割引率を変更した直後の会計もありません。

AIも仕様がない条件を提案できます。それでも、テストの件数と受け入れ条件の対応は、自分で照合します。AIのテストは重要な条件を網羅するとは限らず、仕様に書いた条件を見落とすこともあるからです。

「店長が停止し忘れたら」「シフト担当が焦って2回押したら」。現場を知る人の想像とAIの提案を持ち寄り、優先して確かめる条件を選びましょう。

あなた → AI

3件追加してください。

1. サーバー上で13:59:59に会計確定→750円、14:00:00にセール開始を確定→その後の会計が600円になるか。同じ時刻に届いた場合も、確定した順序で決まるか
2. 会計ボタンを素早く2回押しても、履歴の記録が1件だけになるか
3. 割引率を2割から3割5分に変えた直後の会計で、新しい率と端数の切り捨てが正しく効くか

AI → あなた

追加して実行しました。2番が不合格で、同じ会計が2件記録されました。画面では送信中にボタンを無効にし、サーバー側では同じ会計の送信を重複登録しない対策が必要です。同じ会計IDで再送した場合にも、記録が1件になるテストを追加します。

ボタンの無効化は、連打を減らすための画面の工夫です。通信が切れて送り直す場合などには、それだけでは足りません。1回の会計に同じIDを付けて再送し、サーバーとデータベースでそのIDを一度だけ受け付ける仕組みを用意します。

※厳密に言うとは: 同じIDの送信が同時に届く場合も考え、データベースの一意制約(同じ値を2回登録させない決まり)などで重複を防ぎます。また、会計を新しく始めるときは、新しいIDにします。

テストが落ちたおかげで、本番のレジに並ぶ前に、売上が二重になる条件をつかめました。

合格も不合格も、判断の材料です。不合格なら原因を調べ、合格なら「何を確認できたか」を記録します。テストが間違いを検出できるか不安なら、検証用コピーで計算結果をわざと変え、そのテストが落ちることを確かめます。

あとの流れは、もう知っている道です。AIに修正させ、テストを全件実行し直す(回帰テスト——第11章の図鑑ぶんも含めて)。全件合格したら、コミットして記録を残す(第10章)。

繰り返す検算は自動化すると続けやすくなります。人は、条件の選択と結果の判断に時間を使えます。実行時間はテストの範囲で変わるので、短い計算テストは頻繁に、画面を通すテストは変更の節目にも実行します。

12.7 テストに通ったから安全とは限らない

最後に、この章の装備を過信しないための注意です。図12-1を思い出してください。テストの網は、あなたが張った場所にしか張られていません。

クッキー9999個は、打ち間違いを想定した意地悪でした。では、わざとおかしなデータを送りつけてくる人がいたら? 店長画面のURLを推測して直接開こうとする人がいたら? 履歴の取り消しを他人の端末から連打されたら?

権限のない人の操作は、すでにT3で扱いました。次はその範囲を広げ、画面の制限を通らずに直接リクエストを送る場合なども考えます。

テスト結果が示すのは、実行した条件では期待どおりだったという根拠です。あらゆる条件で不具合がない証明にはなりません。次の第13章では、守るデータと操作を棚卸しし、利用者の手元をどこまで信用できるかという視点で、点検を広げます。

別の条件でも動くか試す

AIが書くテストの件数・書き方・使う道具は、毎回変わります。紙面の作例と件数や見た目が違ってても、確認するのは次の3点です。

1. 受け入れ条件5つに対応するテストが存在する
2. 境界時刻・二重押し・割引率変更後のテストが存在する
3. 全件を一度に実行でき、合格・不合格の結果が表示される

この3つが確認できれば、紙面と違っていても先へ進めます。テストがうまく動かないときは、欲張らないこと。「T1の750円のテスト1件だけを動くようにして」と小さく頼み直し、1件動いたら残りを足していくのが近道です。

演習12 割引計算をリハーサルする

- ミッション: 会計アプリ(またはあなたの題材)の割引計算のテストケース表を作り、AIにテストを実装させ、実施記録を残す
- 時間の目安: 60分
- クリア条件:
 1. テストケース表に、受け入れ条件5つに対応する行がある(前提・操作・期待する結果が埋まっている)
 2. 境界値と異常系のテストが4種類以上ある(セール開始の直前・直後、セット割との併用、端数の切り捨て、0個の注文、数量上限、二重押し・再送、割引率変更、通信失敗から選ぶ)
 3. 全件を実行し、結果(合格・不合格、実行した日時)がテスト実施記録に残っている
 4. 不合格が1件でも出たら、修正→再実行→全件合格までが記録されている。1件も出なかったら、わざと計算ルールを1か所壊して、テストがちゃんと落ちることを確認する(落ちないテストは網に穴がある)
- つまづいたら:
 - テストの実行方法がわからない→AIに「テストの実行コマンドと、結果の読み方を教えて」と頼む
 - 全部作るのが大変→T1(750円)の1件だけから始める。1件動けば、残りは同じ型の繰り返し
 - AIのテストが本と違う形になった→「前提・操作・期待する結果」が読み取れて、3つの確認点(前節)が満たせていれば先へ進んでよい

- ・ 裏ダンジョン: セール前の750円の会計を、セール開始後に取り消す。確定済みの750円が集計から除かれ、600円へ再計算されないことをテストする。取り消しを再送しても二度引かれられないことも確かめる
- ・ 冒険の書に記録: テストケース表+テスト実施記録

(授業向け)ペアで互いのアプリに「意地悪テスト」を3件ずつ出し合い、相手のテストケース表に追加する。

この章のまとめ

- ・ 「動いた」は1回の観察で、「正しい」は決めた条件をすべて満たしたという確認。決めていないことは、テストできない
- ・ 受け入れ条件は、前提・操作・期待する結果の3点に書き直すと、そのままテストケースになる。期待する結果は、アプリに入れる前に検算して書く
- ・ 不具合は、ルールが切り替わる境界に現れやすい。境界の両側(13:59:59と14:00:00など)と、二重押しや通信失敗などの異常系を試す
- ・ 単体・結合・E2Eは、どこまでを確かめるかという範囲の名前。回帰は、変更後にすでに動いていた機能を確かめ直すという目的の名前
- ・ テストが示すのは、実行した条件では期待どおりだったという根拠にとどまる。割引率を変えた場合など、AIのテストが試していない条件は、人が選んで足す

理解度チェック

1. 単体テスト・結合テスト・E2Eテスト・回帰テストのうち、範囲ではなく目的を表す名前はどれですか。その目的も答えてください。
2. タイムセールは14:00:00に始まります。コーヒー2つとクッキー1つの注文で、セール開始の境界をテストするには、どの会計を試し、それぞれ何円を期待しますか。
3. AIが書いた割引計算のテストは、2割引の例ばかりで、全件合格しました。これで「どの割引率でも正しく計算できる」と言えますか。理由も答えてください。

解答のめやす

1. 回帰テストです。変更後に、すでに動いていた機能が壊れていないか(前に直したバグの再発も含めて)を確かめ直します(→12.4)
2. 13:59:59に確定した会計は750円、14:00:00にセール開始を確定したあとの会計は600円です。境界の両側を試し、開始と会計を処理した順序も記録します(→12.3)
3. 言えません。テストが示すのは、実行した条件で期待どおりだったことだけです。たとえば0.7をそのまま掛けるコードでは、チャイ+コーヒー(700円)の3割引が490円ではなく480円になりますが、2割引の例では誤差が表れません。割引率を変えたテストを足します(→12.6、12.7)

60秒でクエスト報告

正しく動くことは、第3章で決めた受け入れ条件5つを、テストケース表にして確認しました。通常時750円、セール中600円という計算を確認しました。さらに、セール開始の境界時刻の前後、0個の注文、ボタンの二重押し、割引率を変更した直後の端数処理も試しています。二重押しで会計が2件記録されるバグがテストで見つかり、修正して全件合格させました。テストは自動化しており、今後なにかを直すたびに全件を再実行します。実施記録はこれです。

——お気づきでしょうか。これはそのまま、口頭試問の第4問「正しく動くことを、どのように確認しましたか」への答えです。しかも最後の一言は、記録という証拠つきです。

レベルアップ!

Lv.12 称号: 検証者

- 手に入れた装備: テストケース、正常系、異常系、境界値、単体テスト、結合テスト、E2Eテスト、回帰テスト
- 冒険の書: テストケース表+テスト実施記録

次の第13章は、セキュリティです。「誰が、どの入口から、何を変えられるか」。画面を通さない送信や、キーの置き場所まで点検を広げ、公開するための守りを確かめます。

第13章 セキュリティ、プライバシー、権利を後回しにしない

クエスト13: 悪意という名のモンスター

文化祭まで、あと4日。

この章のねらい

- ・ 守るものを数え、悪用を「誰が・何をすると・誰が何を失うか」で書ける
- ・ 冒頭の4つの攻撃で、入力検証と権限の確認がサーバー側で効くかを確かめられる
- ・ キー・個人情報・依存部品などの点検結果を、根拠つきでチェックリストに書ける

今回のクエスト

会計は正しい金額で記録され、テストも通りました。次に確かめたいのは、その記録や割引率を、誰が、どこから変更できるかです。画面のボタンを隠すだけでは、裏側の操作まで守れませんでした(第9章)。

残り4日。この章では、設計時から考えてきた権限やデータの持ち方を、公開前に点検します。この点検は、公開後も機能を増やすたびに見直すための型になります。

問いは、「この操作を、許可していない人が試したら?」。章の終わりには、守るもの、確認方法、残った課題を記録したセキュリティ・プライバシーチェックリストが手に入ります。

まず、自分のアプリを攻撃してみよう

いつもと違う使い方を試します。自分のアプリの検証用コピーを用意し、架空の注文だけで次の操作をしてください。営業中のアプリや、許可を得ていない人のアプリでは試しません。

1. 注文の数量に「-5」を入れてみる
2. 会計の取り消しボタンを、連打してみる
3. 店長画面のURLを、シフト担当の端末で直接開いてみる
4. 割引率の入力欄に「200」と入れてみる

第9章と第12章の対策が効いていれば、拒否や重複防止の結果が出るはずです。結果は「防げた」「防げなかった」「未確認」に分け、画面だけでなく保存された内容も見ます。

この章では、1番のマイナス数量が通ってしまう実装を例に考えます。合計金額が静かに減るのに、画面にはエラーが出ない、という不具合です。

ボス出現——想像していなかった相手

なぜ通ってしまうのでしょうか。たとえば、入力欄ではマイナスを拒否していても、サーバーが同じ検査をしていなければ、画面を通らない送信で抜けられます。テストの条件を増やすとき、どこを通って届く操作かも見ておく必要があります。

第12章では、急いでいるシフト担当や、操作の途中で切れる通信を考えました。今度は、権限を持たない人がURLや入力を意図的に変える場面を加えます。善意か悪意かを問わず、許可されていない操作は、サーバーで拒否できる必要があります。

「小さな模擬店なら、狙われないのでは」と思うかもしれませんが。けれど、利用者の意図を確かめてから守り始めるわけにはいきません。

「マイナスを入れたら、どうなるか見たかった」。動機はそれで十分です。しかも第14章でアプリを公開すれば、相手は文化祭の数百人ではなく、インターネット全体になります。

この章のボスは、画面を使ってもらえるはず、という前提に隠れた抜け道です。

13.1 セキュリティは最後に追加する機能ではない

セキュリティと聞くと、「完成したアプリに、あとから鍵を掛ける作業」を想像しがちです。この順番の思い込みが、事故のもとになります。

家にたとえましょう。鍵は最後に付けられます。でも「金庫を置く部屋を作るか」「窓をどこに開けるか」は、設計図の段階の話です。建ててから「窓が多すぎた」と気づいても、壁は簡単には動かさません。

図 13-1

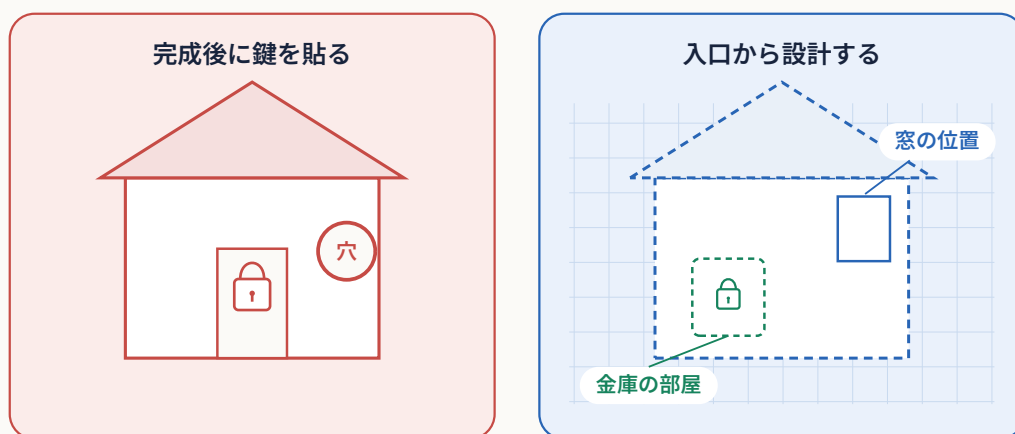


図13-1 後付けの鍵と設計の守り

アプリも同じです。「お客の名前を記録するか」「セールの操作を誰に許すか」は設計の決めごとで、あとから貼れるシールではありません。

実は、あなたはもう体験済みです。第3章で「セールの操作は店長だけ」と仕様に書いたこと。第9章で「お客の個人情報を持たない」と決めたこと。あれが、設計のときからセキュリティを考えるということでした。

だからこの章は、新しい作業を積み増す章ではありません。ここまでの決めごとを、「守り」の目でもう一度見回る章です。

ここで、この章の中心の言葉を入れます。守りに空いた穴のことを脆弱性(vulnerability)と呼びます。日常語で言うと、攻める側から見た「破れ目」です。冒頭の実験で通ってしまったマイナス数量は、あなたのアプリの脆弱性を1つ見つけた、ということです。

13.2 守るべきデータと機能——悪意のある利用者を想像する

守りを固める最初の作業は、壁を高くすることではなく、守るものを数えることです。会計アプリで棚卸しをすると、こうなります。

守るもの	悪用されると起きること
売上データ	書き換えられると、現金と記録が合わなくなる
セールの操作(割引率)	誰かが全品10割引にすると、店の売上が消える
会計履歴	全部消されると、集計も当日の精算もできない
イベントAPIの会員証(APIキーを使う構成の場合)	盗まれると、許可された範囲で外部の窓口を他人に使われる

コツは、ひとつずつ日本語の疑問文にすることです。「誰かが割引率を10割にしたら?」「売上を書き換えたら?」「履歴を全部消したら?」。疑問文になった瞬間、対策は第9章の言葉で答えられるようになります。

割引率の変更は店長のロールだけ。履歴の削除も認可で守る。守るものの一覧に、番人の名前を書き込んでいく作業です。

図13-2では、入力欄・店長操作の権限・APIキー・売上データの4か所の印を見てください。

図 13-2

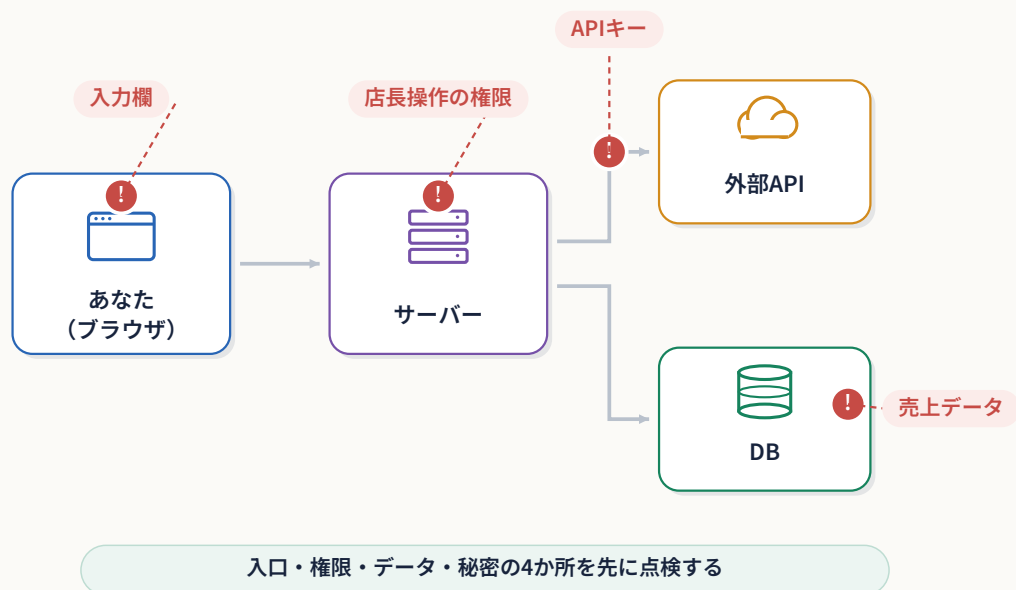


図13-2 アプリの狙われる場所

※厳密に言うとは第7章の標準演習で作る `/mock/events` には、外部サービスのキーはありません。表の4行目と図13-2の外部APIは、キーが必要な外部サービスへ切り替えた場合の点検対象です。使っていない秘密を用意する必要はありません。

▼コーナー「先輩エンジニアならここを見る」

プロは新機能の相談を受けると、まず「これで悪いことをするなら?」と考えます。性格が悪いのではなく、技術です。悪用のシナリオが書けない機能は、守り方も決められないからです。この章の演習2は、その思考法の練習です。

13.3 入力値を信用しない

次は、入り口を固めます。第8章で、空欄やおかしな値を弾く「入力値の検証」をやりました。あのときは、押し間違いからデータを守るためでした。同じ仕組みを、この章では悪意から店を守る壁として使い直します。

正式な名前は入力検証(input validation)。日常語で言うと、入り口の門番です。

冒頭の実験でマイナスが通ったアプリの中身は、こうなっています。数量の `qty` が、検査されないまま合計の計算に使われている点を見てください。

役割ラベル: 計算(わざと危険にしたコード)

```
[javascript]
// 画面から来た数量を、そのまま信じて足している
function addItem(order, menuId, qty) {
  order.items.push({ menuId, qty }); // qty が -5 でも素通り
```

```
order.total = calcTotal(order.items); // 合計が減る
}
```

クッキー2個の注文に「クッキー -1個」を混ぜると、合計は400円から200円へ、静かに半分になります。シフト担当は画面の合計を信じて現金を受け取るので、店は200円損をします。

数字の欄だけではありません。自由に文字を書ける欄に、文章のかわりにコードの断片を入れてくる人もいます。それが画面にそのまま表示されたり、データベースへの命令に混ざったりすることがあります。すると、記録の盗み見や書き換えまでできてしまうのです。防ぐには、入力検証に加えて、受け取った値を命令と混ぜない仕組みが必要です。

※厳密に言うと: データベースの命令に混ぜる手口をSQLインジェクション、ブラウザで意図しないコードを動かす手口をXSS(クロスサイトスクリプティング)と呼びます。入力検証だけでは防げません。SQLには値を命令と分けて渡すパラメーター化クエリ、画面表示には文脈に合った出力エンコードなどが必要です。AIには「該当するコードと、その対策を示して」と頼み、使用するライブラリの公式資料と照合します。[OWASP入力検証ガイド](#)

もうひとつ、大事な掟があります。検査は、画面側だけでは足りません。第5章の地図を思い出してください。

画面はお客の手元(クライアント)で動いています。手元にあるものは、いじれるのです。画面の検査は善意の人の入力ミスには効きますが、悪意の人は画面を通らず、窓口(API)に直接頼むことができます。だから門番は、サーバー側にも立てます。

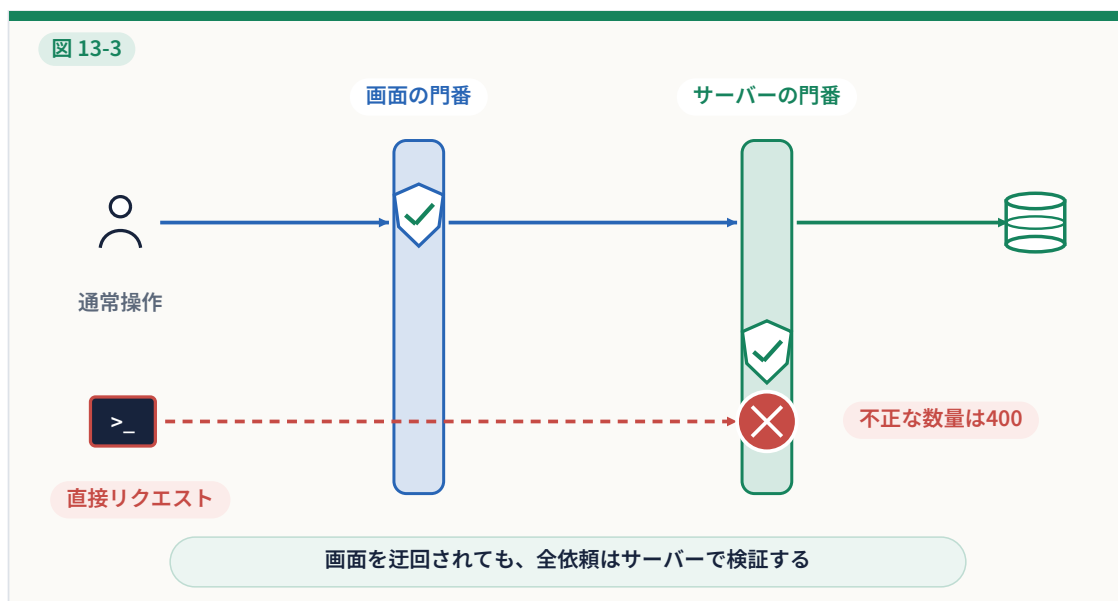


図13-3 2人の門番

AIへの直し方は、範囲を数字で示するのがコツです。

あなた → AI

会計アプリの入力検証を強化してください。

1. 数量: 1~99の整数のみ。それ以外はサーバー側で400を返し、画面に「数量が正しくありません」と表示

2. 割引率: 0～90の整数(%)のみ。店長画面からの変更でも同じ検証をする
3. 検証は画面側とサーバー側の両方に入れ、なぜ両方必要かをコメントに書いてください

AI → あなた

3点とも実装しました。サーバー側では、受け取った瞬間にすべての入力を検査します。画面側の検証は利用者への素早い案内のため、サーバー側の検証は画面を経由しないリクエストを止めるためです。

直したコードでは、`if` の条件で「整数か」「1～99の範囲か」を確かめ、外れたら先へ進ませない点を見てください。

役割ラベル: 計算(直したコードの門番部分)

```
[javascript]
// 外から来た値は、使う前に検査する(サーバー側)
if (!Number.isInteger(qty) || qty < 1 || qty > 99) {
  return respond(400, "数量が正しくありません");
}
```

400という番号、覚えていますか。第7章の「ご注文の意味がわかりません」です。あのときは受け取る側でした。今度は、あなたの窓口が言う番です。

13.4 認証・認可の不備、APIキーの漏えい

第9章の事件を覚えているでしょうか。店長画面をメニューから消しただけで、URLを知っていれば誰でも開けた——あの「隠しただけの店長画面」です。あのときは部員の操作ミスの問題として直しました。

この章の目で見ると、深刻さが一段上がります。来場者の誰か1人がURLを試すだけで、割引率を自由に変えられたのです。画面を隠すことは、権限制御にならない(第9章)。守りの言葉で言い直すと、認可の抜けは、最もよく狙われる脆弱性のひとつです。

今日できる点検は、冒頭の実験3番です。シフト担当の端末で店長画面のURLを直接開き、割引率の変更がサーバーで拒否されるかを確認めます。

そしてもうひとつ、第7章から持ち越した宿題をここで回収します。会員証(APIキー)の掟を破ると、どうなるか。次のコードでは、`API_KEY` にキーの実物の文字列がそのまま書かれている点を見てください。

役割ラベル: 通信(わざと危険にしたコード)

```
[javascript]
// 会員証をコードに直書きしている
const API_KEY = "sk_live_a1b2c3d4...";
fetch("https://.../events", { headers: { Authorization: API_KEY } });
```

何が事故なのか。あなたのコードは、第10章からリポジトリで管理されています。そのリポジトリは、参考のため、あるいは提出のために、公開設定にされることがよくあります。その瞬間、直書きされた会員証は世界中から見えるようになります。

読み取られたキーは、そのキーに許された範囲で不正利用されるおそれがあります。データへのアクセスだけでなく、従量課金の利用が増える可能性もあります。GitHubにも秘密情報を検出する仕組みがありますが、検出を待たず、キーを公開しない設計にします。[GitHubの秘密情報検出](#)

図 13-4

公開したキーは、削除だけでは秘密に戻せない



漏れたら、削除より先にキーを無効化・再発行

図13-4 APIキー漏えいの連鎖

対処は、第9章の金庫を使うことです。キーはサーバー側のSecretsに保管し、コードには値を直接書かず、環境変数などの設定から読み出します。ただし、環境変数という名前なら秘密になるわけではありません。ブラウザ向けのコードへ埋め込まれる設定には、秘密のキーを入れません。

直したコードでは、キーの実物ではなく `EVENT_API_KEY` という名前だけが書かれている点を見てください。

役割ラベル: 通信(サーバー側のコード例)

```
[javascript]
const API_KEY = process.env.EVENT_API_KEY; // サーバーの設定から読む
```

▼コーナー「先輩エンジニアならここを見る」

一度コミットした会員証は、コードから消しても履歴(第10章)に残ります。プロは「漏れたかもしれない」と思ったら、消す作業より先に、そのキーを無効化して新しい会員証に切り替えます。合鍵が出回った家は、ドアを拭くより先に錠前を替えるのです。

13.5 個人情報と機密情報——ログに残してはいけない情報

守るデータの中でも、特別扱いが必要なものがあります。個人情報——名前、連絡先、顔写真など、それだけで、あるいは組み合わせで「誰なのか」が特定できる情報です。日常語で言うと、その人の名札になる情報。

売上データが漏れれば店が困りますが、個人情報が増えると、アプリと関係のない場所まで被害が広がります。だから扱いの重さが違うのです。

※日本の個人情報保護法では、生存する個人を識別できる情報に加え、個人識別符号を含む情報も個人情報に当たります。ほかの情報と容易に照合して個人を識別できる場合も含まれます。ニックネームやIDに替えるだけで、個人情報でなくなるとは限りません。[個人情報保護委員会・通則ガイドライン](#)

ここで、第9章のあの決定が効いてきます。会計アプリは、お客の名前も連絡先も記録しない設計にしました。あのときは「集めすぎない」という慎みの話でした。

集める情報が減れば、守る対象も減ります。必要のないデータは、最初から持たない。そのうえで、売上記録や店員のアカウントなど、残る情報を守ります。入力欄がなくても、ログや外部サービスに情報が残らないかも確認します。

見落としやすい抜け道が、ログ(第11章)です。調査に便利だからと「とりあえず全部記録」すると、エラーの記録に入力内容が丸ごと写り込みます。将来、予約機能などで名前を扱う日が来たら、原則はこうです——個人は写さず、操作は写す。

名前や連絡先、パスワード、APIキー、認証トークンをそのままログに出しません。「いつ、どの担当者IDが、何を変更し、成功したか」のように、調査に必要な操作を記録します。IDを含むログにも閲覧権限と保存期間を決めます。ログの書き方だけでなく、誰が読めるかも点検項目です。

[OWASPログガイド](#)

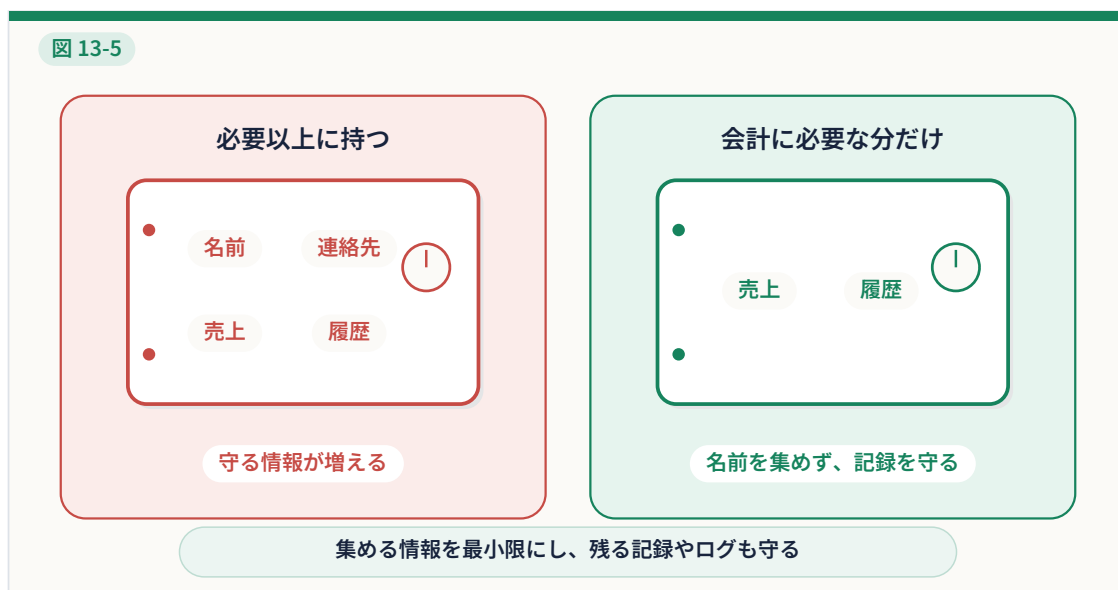


図13-5 持たないデータは漏れない

図13-5では、会計に必要な分だけを持つ側に、名前や連絡先が最初から入っていない点を見比べてください。

13.6 AIに社内データを入力してよいか

守りの話を、あなたの手元まで引き寄せます。開発のあいだ、あなたはAIにいろいろなものを貼り付けてきました。

エラー、コード、仕様書。では——部員の名簿は? 会社で使うなら、顧客リストや売上の実データは?

クラウド型AIに入力した内容は、サービスのサーバーへ送られます。保存期間、学習への利用、管理者の設定、契約条件はサービスごとに違います。手元だけで動かすAIでも、外部接続や記録の設定を確認する必要があります。

だから原則はひとつ。本物の個人情報・機密データを、AIに貼らない。動作確認に人のデータがほしければ、第12章で作った架空のテストデータを使えば足ります。

学校や会社のデータなら、利用できるサービスと入力できる情報を、所属組織のルールで確認します。判断できない場合は担当者に相談し、練習には架空データを使います。エラーログを共有するときも、この確認を一呼吸挟みましょう。

13.7 外部ライブラリと依存関係のリスク——古いパッケージ、オープンソースライセンス

第10章で、あなたのアプリが大量の借り物の部品——ライブラリやパッケージ——の上に立っていることを見ました。借り物は、守りの観点でも点検が要ります。部品にも脆弱性が見つかるからです。

部品の脆弱性は、その部品を使うアプリにも影響することがあります。影響するバージョンと、その機能をどう使っているかを調べる必要があります。図13-6では、自分で書いていない奥の部品の穴が、アプリへ届く道筋を追ってください。

図 13-6

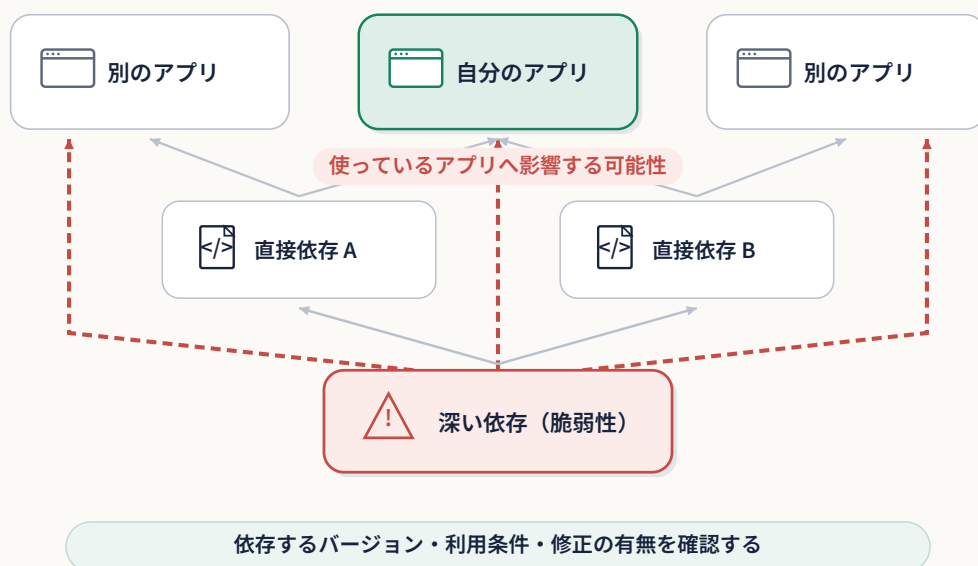


図13-6 依存部品から広がる穴

対策は、使用中の部品とバージョンを把握し、脆弱性の情報と更新内容を確認することです。「古いから危険」「新しいから安全」とだけ判定せず、影響と修正の有無を見ます。更新の実務は第14章で扱います。

公開前には、AIに「依存関係の一覧と固定バージョンを読み、既知の脆弱性の有無、情報源、影響する条件、修正版を示してください」と頼めます。自動の検査結果も読み、何を確認できて何が未確認かを記録します。

借り物には、もうひとつ別の種類の約束事が付いています。ライセンス——部品の利用条件書です。日常語で言うと、「貸すけれど、この約束は守ってね」の紙。

オープンソースは、商用を含む利用を認める考え方ですが、無条件という意味ではありません。著作権表示やライセンス文の保持、配布方法によってはソースコードの提供など、条件はライセンスごとに違います。ソースが見えても「商用利用禁止」などの条件が付くものは、オープンソースとは区別します。

文化祭で配るものも確認対象です。AIに部品とライセンスの一覧を出させ、各部品のLICENSEファイルと利用・配布の方法を照合しましょう。[Open Source Initiativeの定義](#)

13.8 AI生成コードと著作権

「AIが書いたコードは、誰のものなんですか」。第1章(1.6)から持ち越した問いです。ここでは、2つに分けて考えます。

1つめは、生成物に著作権が生じるか。日本では、人がAIを道具として創作したといえるか、創作的な寄与などを個別に見て判断します。AIが出したという事実だけで、一律に決まりません。

2つめは、他人の著作権を侵害していないか。生成物に自分の権利があるかという話とは別です。既存の著作物との類似性や依拠性などが問題になり、AIを使ったことだけで確認が不要にはなりません。文化庁はこの2つの論点を整理しています。[文化庁「AIと著作権について」](#)

本書としての行動指針は3つです。

- 利用条件を確認する。AIサービスの規約、使用した部品や素材のライセンス、所属組織のルールを読む
- 気になる類似や出典を調べる。既存コードに似た長い断片や著作権表示が出たら、そのまま配布せず、出典と利用条件を確認する。判断が難しい公開物は担当者や専門家へ相談する
- AI利用と自分の判断を記録する。第15章の記録に、何を生成させ、何を確認したかを残す。記録自体が権利処理の代わりになるわけではない

13.9 アクセシビリティ——色だけに依存しない表示

この章の見出しには、セキュリティ、プライバシーと並んで、もうひとつ「権利」という言葉があります。気をつけるべきことは、悪意ではありません。あなたのアプリが、意図せず誰かを閉め出していないか。

会計アプリの注文画面は、タイムセール中に「セール中」の赤い表示を出します。もしこれが赤い丸だけだったら、どうでしょう。

色の見え方は人によって違います。第3章で想定した、屋外の日光で画面が見つらい場面もあります。赤い丸だけでは、色を区別できないときにセール中かどうか伝わりません。

色だけに意味を持たせず、文字や形も使って伝えます。「セール中・全品2割引」と書けば、色の違いが見えなくても情報が残ります。これはWCAGの「色の使用」の基準にも沿った考え方です。

[W3C・色の使用](#)

図 13-7



図13-7 色だけに頼らない表示

こうした「誰にとっても使えるか」の設計をアクセシビリティ (accessibility) と呼びます。日常語で言うと、入り口の段差をなくすことです。

文字の大きさ、ボタンの押しやすさ、読み上げソフトへの対応まで裾野の広い分野ですが、まずは色の一歩から。AIへは、こう頼みます。「この画面を、色の区別に頼らなくても伝わるように改善してください」。

13.10 セキュリティレビューを別のAIにも依頼する——最終判断は人間が行う

最後は、点検のやり方です。実装時と同じ前提で見続けると、見落としが残りやすくなります。そこで、レビューの役割と確認項目を改めて指定します。

具体的には、作ったAIとは別のAI(別のサービスでも、同じサービスの新しい会話でもかまいません)に、レビューだけを頼みます。第4章の「実装役とレビュー役を分ける」の、守り版です。

頼み方にコツがあります。まず、うまくいかない例から。

あなた → AI

このアプリは安全ですか？

AI → あなた

全体としてよく構成されています。一般的なセキュリティのベストプラクティスに従うことをおすすめします。

ほめられただけで、何も点検されていません。「安全ですか」と聞かれたAIは、安心させる返事を作りがちです。役割と視点と数を指定して、頼み直します。

あなた → AI

あなたはセキュリティレビューの担当者です。次の仕様とコードを確認し、根拠のある危険を深刻な順に挙げてください。件数を無理にそろえず、「該当コード」「成立する条件」「確認方法」「直し方」を添え、未確認の推測は分けてください。

仕様: 文化祭の会計アプリ。セールの操作は店長のみ。お客の個人情報は持たない。(コードを添付)

AI → あなた

1. 割引率を変更するAPIが、サーバー側でロールを検査していません。シフト担当のトークンでも割引率を変更できます。直し方: 店長ロールの検査を追加…
2. 履歴の一括削除に確認がなく…(以下略)

具体的な場所と直し方が返ってきました。指摘の中身は毎回同じではありませんし、的外れが混ざることもあります。それでいいのです。

指摘を採用するかどうかを決めるのは、レビュー役のAIではなく、あなたです。ここだけは外注できません(第1章)。採用した指摘は直し、第12章のテストに追加して「直ったまま」を守ります。

▼コーナー「AIの言うことを信じてよい?」

「指摘はゼロでした」は、「安全」の証明ではありません。OWASP Top 10は代表的なリスクを知る入口です。具体的な実装の確認には、本章で紹介したOWASPの各ガイドなどを使います。AIを替えても見落としが共通することもあるので、コード・テスト・設定の事実で確かめます。

別の条件でも動くか試す

この章の点検は、どんな題材でもそのまま回ります。守るものの棚卸し→入力の門番→権限と会員証→持たないデータ→別のAIでレビュー。あなたの題材で一巡させてみてください。

AIレビューの指摘は毎回変わりますが、クリアの目安は揺れません。冒頭の4つの攻撃(マイナス数量・連打・URLの直開き・範囲外の割引率)がすべて防がれていることが確認できれば、先へ進めます。

図13-8は、チェックリストの記入例です。結果だけでなく、確認日と方法まで書く点を見てください。

図 13-8

セキュリティ・プライバシー点検				
分類	確認すること	結果	確認日	方法
入力	負の値・文字入力を拒否	合格	例 9/10	異常値テスト
権限	店長操作をサーバーで確認	_____	_____	_____
秘密情報	鍵をコード・ログに残さない	_____	_____	_____
個人情報	必要最小限だけを持つ	_____	_____	_____
借り物	依存部品の更新を確認	_____	_____	_____
権利	素材・ライセンスを確認	_____	_____	_____
表示	色・形・文字を併用	_____	_____	_____
点検	コードとテストで根拠を確認	_____	_____	_____

図13-8 セキュリティ・プライバシー点検

演習13-1 セキュリティ間違い探し

- ミッション: 本章13.3・13.4の危険なコード例と、自分のアプリの検証用コピーを使い、入力・権限・キー・ログ・依存部品の5観点で点検する。実際の秘密や本番データは使わず、キーの例は無効なダミー文字列にする
- 時間の目安: 30分
- クリア条件:
 1. APIキーの直書き・入力検証・サーバー側の認可を含む5観点について、「問題あり/問題なし/未確認」と根拠を書けた
 2. 見つけた危険それぞれに、「どう悪用できるか」を1行で書けた
 3. それぞれの直し方を、この章の節番号(13.3、13.4など)で指させる
- つまづいたら:
 - 13.2の疑問文の型を使う。「誰かが〇〇したら?」を、入力欄とURLと割引率に当てはめる
 - 画面だけでなくコードの中の文字列を見る。`"sk_live..."`のような長い英数字は要注意
 - 判断できない点は、別のAIに該当コードと確認方法を示させる。問題の件数をそろえるために指摘を作らない

- ・ 裏ダンジョン: 見つけた危険をひとつ選んで実際に直し、第12章のテストケース表に「悪意の異常系」として追加する
- ・ 冒険の書に記録: 発見リスト(危険・悪用シナリオ・直し方)

演習13-2 「このアプリが悪用されるとしたら」を5通り考える

- ・ ミッション: 自分のアプリ(会計アプリ、または自分の題材)の悪用シナリオを5通り書き、いまの守りで防げるかを判定する
- ・ 時間の目安: 30分
- ・ クリア条件:
 1. 「誰が・何をすると・誰が何を失うか」の型で、シナリオが5通り書けた
 2. 5通りすべてに「守れている/守れていない」の判定と、その根拠が書けた
 3. 「守れていない」ものには、AIへ出す修正依頼文まで書けた
 4. 結果をセキュリティ・プライバシーチェックリスト(図13-8)にまとめた
- ・ つまづいたら:
 - ・ 登場人物を入れ替える。いたずらな来場者、不満のある部員、隣の店、来年の引き継ぎ相手
 - ・ 「盗む」以外の動詞で考える。書き換える、消す、止める、なりすます、覗く
 - ・ (授業向け)ペアで交換し、相手のアプリの悪用シナリオを1つずつ追加し合う
- ・ 裏ダンジョン: 同じ問いを別のAIにも出し、自分が思いつかなかったシナリオを1つ採用してチェックリストに足す
- ・ 冒険の書に記録: セキュリティ・プライバシーチェックリスト(この章の総まとめ)

この章のまとめ

- ・ セキュリティは、完成後に貼る鍵ではない。何を記録し、誰に何を許すかという設計の決めごとから始まる。守りに空いた穴を脆弱性と呼ぶ
- ・ 外から来た値は信用せず、範囲を決めてサーバー側でも検査する。画面の検査は利用者への案内、サーバーの検査は画面を通らない依頼を止めるためにある
- ・ 画面を隠すことは、権限制御にならない。APIキーはコードに書かずサーバー側の金庫に置き、漏れたかもしれないときは、消すより先に無効化して切り替える
- ・ 必要のないデータは、最初から持たない。名前・連絡先・パスワード・キーは、ログにも出さず、AIにも貼らない。練習には架空データを使う
- ・ 借り物の部品は、脆弱性とライセンスを確かめて使う。表示は色だけに頼らず文字や形でも伝え、AIレビューの指摘を採用するかは人間が根拠で決める

理解度チェック

1. 会計アプリのログに、そのまま書いてはいけない情報を3つ挙げてください。かわりに何を記録しますか。
2. 公開設定のリポジトリに、APIキーを直書きしたコードをコミットしていたと気づきました。まず何をしますか。そのあと、コードをどう直しますか。
3. 「注文画面で数量を1〜99に制限しているので、サーバー側の検証は要らない」と言えますか。理由も答えてください。

解答のめやす

1. 名前、連絡先、パスワード、APIキー、認証トークンのうち3つ。かわりに「いつ、どの担当者IDが、何を変更し、成功したか」という操作を記録します(→13.5)
2. まず、そのキーを無効化して新しいキーに切り替えます。コードから消しても、コミットの履歴に残るからです。そのあと、キーはサーバー側のSecretsに置き、コードは環境変数などの設定から読み出す形にします(→13.4)
3. 言えません。画面は利用者の手元で動くのでいじれますし、画面を通らずに窓口(API)へ直接頼むこともできます。サーバー側でも同じ範囲を検査し、外れたら400で拒否します(→13.3)

60秒でクエスト報告

売上データ、セールの操作、会計履歴、APIキーを守ります。数量と割引率はサーバー側でも検証し、セールの変更には店長の権限を確認します。キーはサーバー側の設定に置き、お客の名前や連絡先は集めません。レビューの指摘は、コードとテストで確かめました。問題が起きたら操作記録で影響を調べ、必要なら停止します。コードは公開版を戻し、データはバックアップからの復元を判断します。漏れたキーは無効化します。

——最後の一文が、口頭試問の第5問「壊れた場合、どこを確認し、どう戻しますか」への答えの型です。この章を終えたあなたは、「壊れた」だけでなく「壊された」まで含めて、この問いに答えられます。

レベルアップ!

Lv.13 称号: 守り手

- 手に入れた装備: 入力検証、脆弱性、個人情報、ライセンス、アクセシビリティ
- 冒険の書: セキュリティ・プライバシーチェックリスト

次はいよいよ第14章、公開です。守りを固めたアプリを、インターネットの側へ出します。待っているのは最後の壁——「ローカルでは動くのに、公開先では動かない」です。

第14章 アプリを公開し、 使い続けられる状態にする

クエスト14: 開店の日

文化祭、前日。そして当日。

この章のねらい

- アプリを公開し、自分以外の端末で会計が保存されることを確かめられる
- 公開先で動かない原因を3類型で切り分け、本番の環境変数を確かめられる
- 監視・バックアップ・連絡先・終了後のデータの扱いを、運用計画に書ける

今回のクエスト

序章の最後に、ひとつ約束をしました。「いま見ているURLは仮のもの。ちゃんと公開する方法は第14章で」——今日が、その日です。

レジ横に貼るQRコードを、別の部員が読み取る。あなたのパソコンを開かなくても、注文が記録される。準備してきた店を、いよいよ通りに向かって開けます。

ミッションは2つです。アプリを公開し、自分以外の人のスマートフォンで動かすこと。そして、公開したあとも使い続けられる状態を作ること。公開は開発の終わりではありません。営業の始まりです。

まずAIに公開させてみよう

頼み方は、いつも通りでかまいません。

あなた → AI

このアプリを、インターネットに公開してください。クラスの友達が、自分のスマートフォンから使えるようにしたいです。

AI → あなた

公開の準備をします。アプリをビルドし、ホスティングサービスへデプロイしました。次のURLで公開されています: <https://...>(あなたのアプリのURL)

必要なアカウントや設定がそろえば、こうした流れでURLを発行できます。開くと、たしかに会計アプリです。けれど、確かめたいのは、URLが出たその先のことです。

先に言っておくと、この章のボスはもう近くにいます。URLが手に入ることと、そこでアプリが正しく動くことは、別の話なのです。

手順の細部はサービスごとに違います。操作は付録Aの確認項目を入口に、使用するサービスの公式ガイドと照合してください。本文では、いま何が公開先へ運ばれ、何を別に設定するかを見ていきます。

14.1 ビルドとデプロイ

いまAIがやった2つの作業に、名前を付けます。

ビルド(build)は、アプリを「届けられる形」にまとめる作業です。開発中のプロジェクトには、下書きやメモ、開発専用の道具が散らばっています。そこから、お客さんに出すものだけを選んで梱包する。日常語で言うと開店準備の梱包です。

デプロイ(deploy)は、梱包した荷物を店舗に運び込み、開店できる状態に並べることです。日常語で言うと店舗への引っ越し。あなたの作業場(ローカル。第5章)から、お客さんの来られる場所へ、アプリが引っ越したのです。

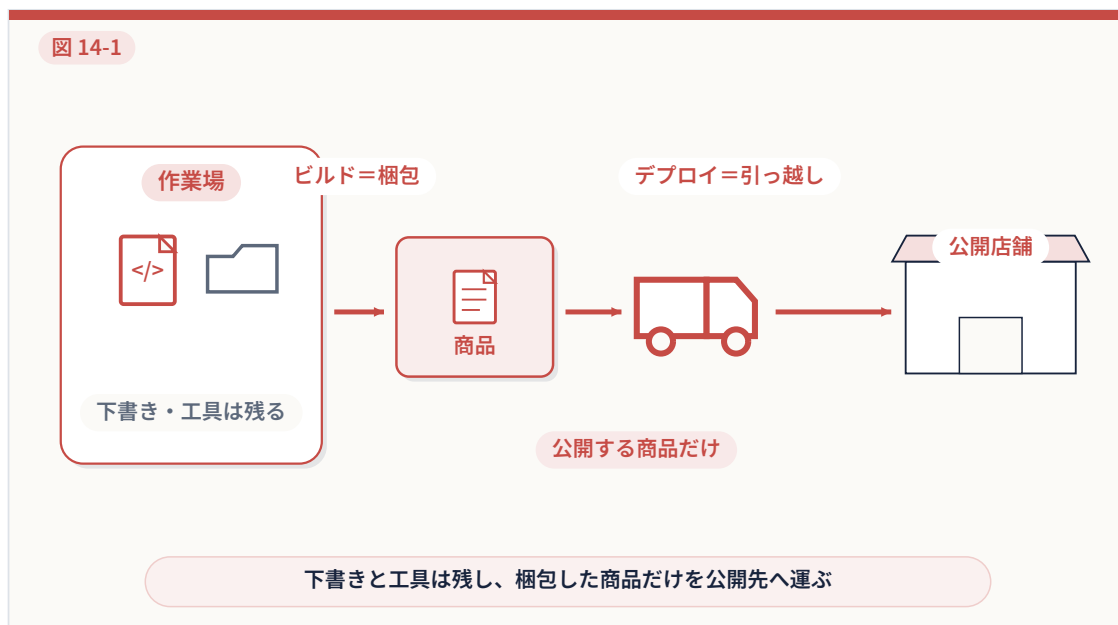


図14-1 ビルドからデプロイへ

※厳密に言うと: ビルドがほとんど要らない構成のアプリもあります。それでも「手元の形」と「届ける形」は別物、という考え方は共通です。

14.2 ホスティング、URL、ドメイン

では、引っ越し先の「店舗」はどこにあるのでしょうか。あなたのパソコンの中ではありません。パソコンを閉じたら店も閉まるのでは、店になりません。

店舗の土地と建物を貸してくれるサービスがホスティング(hosting)です。日常語で言うと、24時間営業できる場所を貸してくれる大家。実体は、第5章で学んだクラウド——借り物のコンピューター群——の一区画を、あなたの店用に借りることです。

店には住所が要ります。それがURLです。たとえば<https://your-app.example/checkout>なら、[your-app.example](https://your-app.example/checkout)がドメイン名、[/checkout](https://your-app.example/checkout)がその中の行き先です。ドメインは看板に書く店名、URLはその店の入口までを含む住所、と考えると区別できます。

ホスティングサービスが用意するドメインの一部を借りて公開することもできます。独自のドメインを使う場合は、登録・更新の費用や設定を確認します。

序章のプレビューURLは、開発中の状態を確認するための入口でした(有効期限や共有範囲はサービスによって違います)。公開版は、どの版を利用者に届けるかを決めて運用するものです。

なお、URLを知っていることは、管理者である証明になりません。公開版でも認証と認可は必要です。

図14-2では、住所(URL)と看板(ドメイン)、それらが指す建物(ホスティング)の対応を確かめてください。

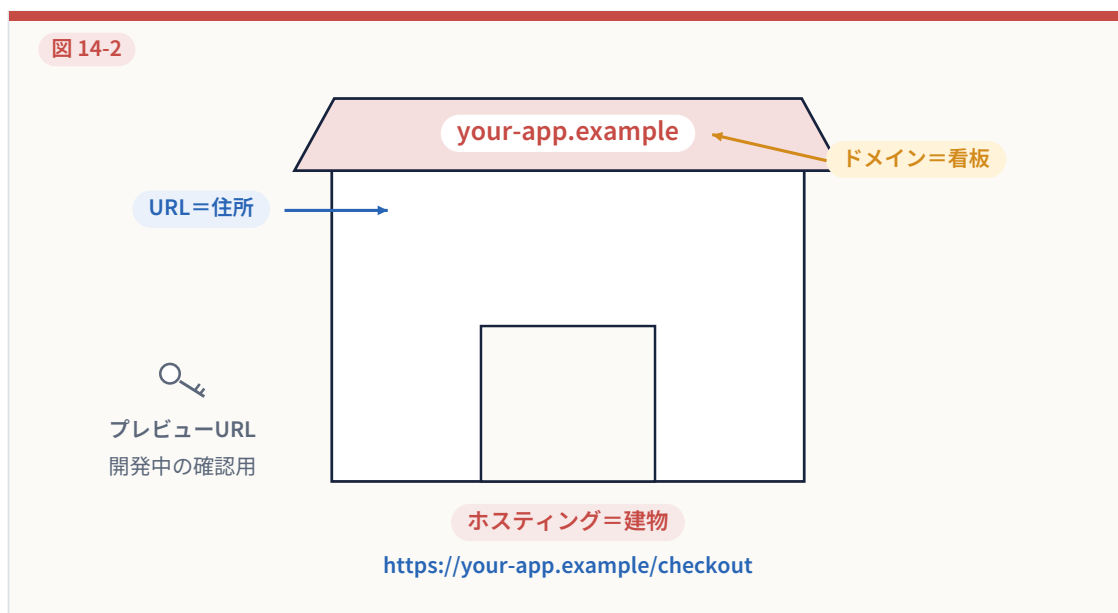


図14-2 公開店舗の住所と看板

※厳密に言うと: ドメインは世界で早い者勝ちの貸し出し制で、持つなら年単位の料金がかかります。仕組みのくわしい話は本書では扱いません。

ボス出現——ローカルでは動くのに、公開先では動かない

さっそく、自分のスマートフォンで開いてみましょう。学校のWi-Fiは切って、モバイル回線で。これが「外から見た店」です。

注文画面が出ます。チャイのボタンを押すと、合計400円。順調です。

ところが、会計ボタンを押した瞬間——「保存に失敗しました」。履歴画面は0件のまま。店長画面は、ログインすらできません。

パソコンに戻って、ローカルで同じ操作をすると、全部動きます。テストも通ります。「ローカルでは動くのに、公開先では動かない」。

画面は開くのに、記録は残らない。第11章の故障さがしの地図で言えば、表示までは届いています。調べるのは、保存と認証の経路です。

14.3 開発環境、検証環境、本番環境——公開用の環境変数

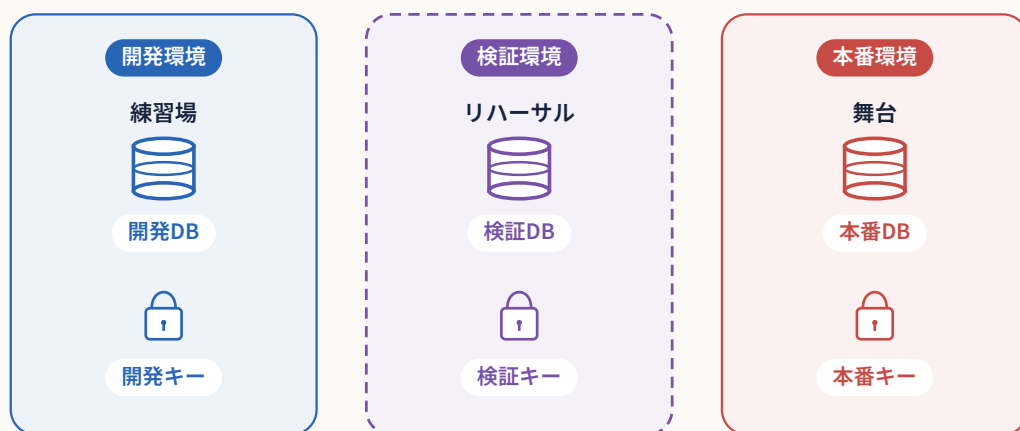
謎を解く鍵は、「環境」という言葉です。アプリが動く場所ひとそろい——コンピューター、設定、データの置き場——をまとめて環境と呼びます。あなたはいま、2つの環境を持っています。

- **開発環境**: 作るための場所。日常語で言うと練習場。架空データを使い、共有範囲を開発者に絞ります
- **本番環境(production)**: お客さんが実際に使う場所。日常語で言うと本番の舞台です

あいだに**検証環境**を置くと、本番に近い構成で、架空データを使って最終確認できます。いわばリハーサル室です。最初の公開で環境を増やせない場合も、開店前の試験データと営業データを分けます。営業開始後の破壊的なテストは、本番から切り離れた検証環境で行います。

大事なのはここからです。環境が分かれると、設定も環境ごとに分かります。第10章で「開発用設定と公開用設定」を区別しました。その代表が、第9章の金庫(Secrets)と、そこから値を受け取る環境変数です。金庫は、環境ごとに別々にあります。練習場の金庫と、舞台の金庫は、別の金庫なのです。

図 14-3



同じアプリでも、接続先と秘密は混ぜない

図14-3 3つの環境を分ける

コードの側は、どの環境でも同じです。だからこそ環境を移せます。次のコードでは、`process.env` から読む1行と、値が空のときに知らせる `if` を見てください。

役割ラベル: データ

```
[javascript]
// 保存先の住所と鍵は、コードに書かず、金庫(Secrets)から環境変数で受け取る(第9章)
const dbUrl = process.env.DATABASE_URL; // 中身は環境ごとに違う
if (!dbUrl) {
  throw new Error("DATABASE_URL が設定されていません(金庫が空です)");
}
```

コードは「金庫から読む」としか言っていません。どの金庫かは、いまいる環境が決めます。

14.4 ローカルでは動くのに公開先では動かない理由

もう、種明かしができます。ボスの正体の代表格は——本番の金庫が空だったこと、です。

ローカルの秘密を入れた `.env` は、第9章と第10章でGitの管理対象から外しました。Gitから公開する構成では、そのファイルは自動で引き継がれません。公開先のSecretsに本番用の設定を登録する必要があります。

もうひとつ注意があります。Gitに入れない設定だけでは、ビルド時に値がブラウザ向けコードへ埋め込まれることまでは防げません。秘密はサーバーだけが読む構成になっているかを確認します。図14-4では、ローカルの秘密が公開する商品に載らず、本番のSecretsは別に用意する点を見てください。

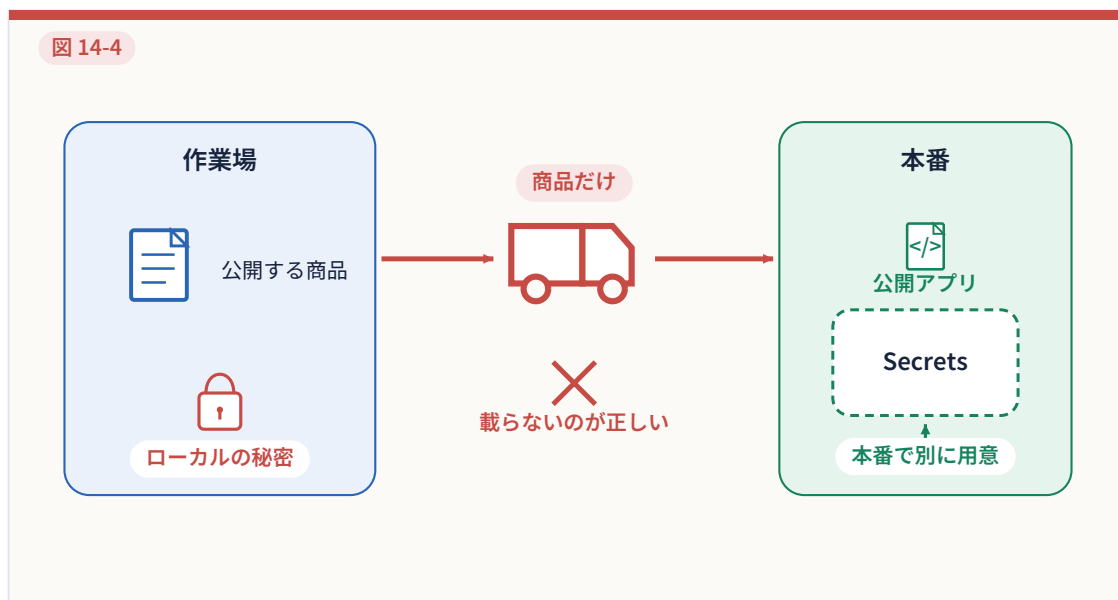


図14-4 秘密はデプロイ物に載せない

この作例では、保存先の接続設定と、認証に必要な設定が本番にありませんでした。秘密をコミットしない判断は保ちながら、公開先へ設定する工程を足せば解決できます。実際のアプリでも同じ原因かは、公開先のログで確かめます。

「公開先では動かない」には、ほかにも定番の原因が2つあります。あわせて3類型で覚えてください。

1. 金庫の用意忘れ(いま見た通り。まっ先に疑う)
2. 依存部品の入れ忘れ: 使っているライブラリが依存関係の一覧(第10章)に書かれていない。ローカルでは偶然そろっていて動くが、引っ越し先では「そんな部品は知りません」となる
3. URLの違い: `localhost`は、そのコードが動くコンピューター自身を指す住所です。利用者のブラウザから呼べば利用者の端末を指すため、開発者のサーバーには届きません。接続先のURLや、ログイン後に戻ってくる先のURLも本番用か確認します

図14-5は、この3類型を症状から引く表です。どの操作が失敗したかを手がかりに、最初に見る場所を選んでください。

図 14-5

症状	最初に見る場所
保存・ログインだけ失敗	Secrets・接続設定
起動そのものが失敗	依存部品・ビルドログ
特定の通信だけ失敗	URL直書き（localhost等）・接続先

症状が違えば、確認する場所も変わる

図14-5 公開先の症状切り分け表

AIと一緒に直す

まず、やりがちな頼み方からです。

あなた → AI

公開したら動きません。直してください。

AI → あなた

保存まわりの処理を書き換えてみました。再度デプロイして試してください。

これを何往復かすると、ローカルでは動いていたコードが、どんどん書き換わっていきます。第11章で学んだ通り、情報のない丸投げは、AIを当てずっぽうにします。壊れていない場所を直し始めたら赤信号です。深追いせず、ロールバック(第10章)しましょう。

情報を持って、頼み直します。公開先のログ(第11章)を見ると、こう出ていました。**Error: DATABASE_URL が設定されていません(金庫が空です)。**

以下は、モックから、キーを使う外部のイベントAPIへ切り替えた構成例です。標準演習の `/mock/events` を使い続ける場合、外部API用の接続先とキーは不要です。

あなた → AI

公開先で会計ボタンを押すと「保存に失敗しました」になります。ローカルでは成功します。公開先のログに上のエラーが出ています。本番環境に設定が必要な環境変数を、プロジェクトの中から探して一覧にしてください。値そのものは表示しないでください。

AI → あなた

この構成例で参照している環境変数は4つです。

1. `DATABASE_URL` — 保存先データベースの接続先
 2. `SESSION_SECRET` — ログインの仕組みが内部で使う合言葉の元
 3. `EVENTS_API_URL` — イベント情報API(第7章)の接続先
 4. `EVENT_API_KEY` — イベント情報APIを利用するための秘密のキー
- ホスティングサービスの設定画面から、本番用の値を登録してください。

「値そのものは表示しないで」の一言に注目してください。秘密はチャット欄にも出さない——第13章の点検を通ったあなたの、新しい癖です。

実際に必要な変数名は、採用した認証やデータベースの構成に合わせます。本番用の値を登録し、サービスの手順に沿って再起動や再デプロイを行います。スマートフォンで会計を記録し、再読み込み後にも残ることを確認できたら、開店の点検へ進めます。

14.5 公開直後の確認——スマートフォンでの動作確認と簡易テスト

「開店した気がする」で終わらせないために、開店日の点検をします。やることは3つです。

1. 実機で開く。自分のスマートフォンで、モバイル回線から。プレビューの確認(序章)とは違い、完全に「外」から店を見ます
2. QRコードを読み取り、配る。公開URLをQRコードにし、実際に読んで正しい入口へ届くか確認します。レジ横に貼れば、シフト交代のたびにURLを打ち込まずに済みます
3. 本番で主要な受け入れ条件を確認する。開店前に試験用の会計で750円の計算、保存、取消、権限を確認します。試験の記録を営業の集計へ混ぜない手順も決めます。大量送信や意図的な障害などの試験は、検証環境で済ませます

ローカルで全部通ったのに、また?——はい、またです。環境が変われば、結果は変わりうる。それがこの章のボスの教訓でした。舞台の上で一度も通していない芝居を、開演とは呼べません。

点検が済んだら、そばにいる誰かにQRコードを読み取ってもらいましょう。自分のパソコンの中にしかなかったアプリが、いま、人の手の中で動いている——この景色を見るために、30日歩いてきたのです。

14.6 ログと監視——利用状況、エラー通知

ここからは営業中の話です。最初の問い。文化祭の当日、アプリのエラーにいちばん最初に気づくのは、誰でしょう。

何も用意しなければ、答えは「お客さんの前で固まっているシフト担当」です。あなたに伝わるのは、行列が伸びきってから。それでは遅すぎます。

アプリの様子を続けて見張ることを監視(monitoring)と呼びます。日常語で言うと店の見回り当番です。会計アプリでは、まず次の2つから始めます。

- エラー通知: 公開先でエラーが起きたら、あなたに知らせが届くようにする。多くのホスティングに通知の仕組みがあります。「エラーが起きたら通知を受け取りたい」とAIに相談してみてください
- 利用状況: 何件使われたか、どの時間帯が多いか。異変に気づく物差しになり、タイムセール判断材料(第8章の集計)にもなります

第11章のログは「あとから調べるための記録」でした。監視は「異常に早く気づくための仕組み」です。図14-6で、監視がある場合とない場合に、誰が最初に異常を知るかを比べてください。

通知を設定したら、試験通知が担当者へ届くことも確認します。通知には検知条件や遅れがあるからです。エラーの出ない計算の静かな間違いは、現金と売上の照合でも見つけます。

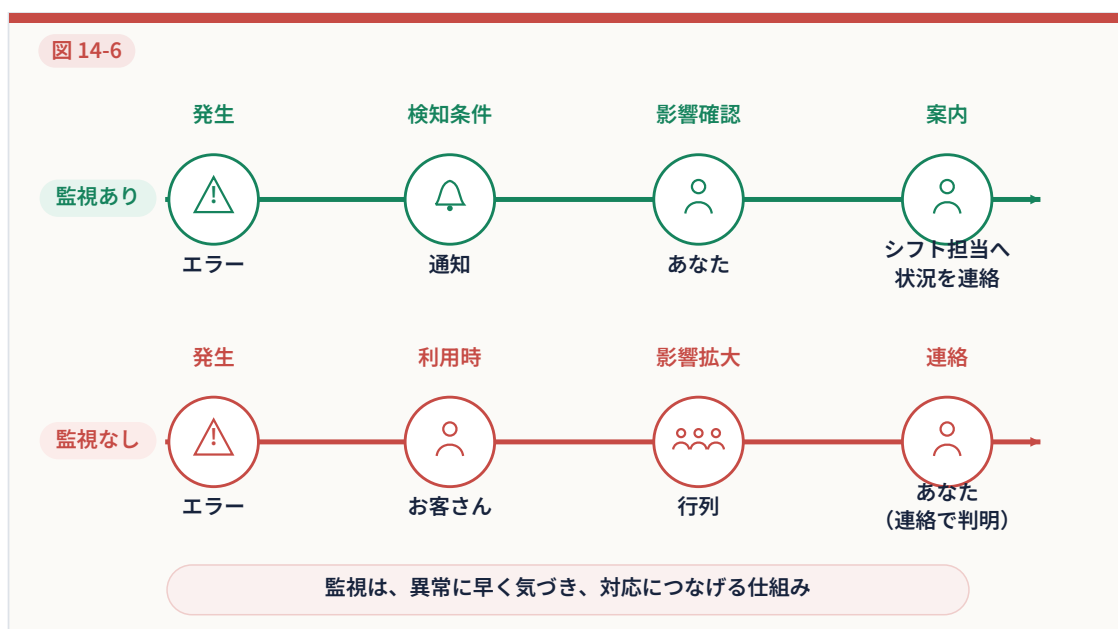


図14-6 障害に気づく2つの経路

14.7 バックアップと、データを戻す方法

第8章で「バックアップの考え方」を学んだとき、本番の話は先送りにしました。回収します。

想像してください。文化祭2日目の朝、売上データが全部消えていたら。会計報告が出せません。

模擬店の売上は部のお金であり、報告できないお金は、なかったお金と同じ扱いになりかねません。データが飛ぶことは、店の信用が飛ぶことです。

やることは3つだけです。

1. いつ取るかを時刻で決める。文化祭なら「開店前・昼・閉店後」の3回など。第8章のCSV書き出しがそのまま使えます
2. 本体と一緒に失わない場所へ保管する。別の保存先に置き、閲覧・削除の権限も決めます。CSVに必要なID、明細、取消状態が含まれるか確認します
3. 検証用の保存先へ戻す練習をする。件数だけでなく、履歴と集計が一致するか照合します。CSVだけでは認証設定やすべての関連データが戻らない場合もあります

3つめが肝心です。戻したことのないバックアップは、お守りにすぎません。「取ってあるか」ではなく「戻せるか」。問いを変えてください。

14.8 API利用料金——利用上限とクォータ

第7章でレートリミットを学んだとき、「無料枠と料金の話は第14章で」と約束しました。ここで回収します。

ホスティング、データベース、外部APIには、利用量と料金の条件があります。ここでは、似ている3つの言葉を分けます。

- 無料枠: 料金が発生しない利用量
- クォータ: 容量や呼び出し回数などに設けられた、利用量の上限
- 予算・利用通知: 費用を管理するための設定

注意したいのは、クォータがあることや予算を入力できることだけでは、請求がその金額で止まるとは限らない点です。

大事なのは、枠を超えたときに何が起きるかがサービスごとに違うことです。大きく2タイプあります。

- 止まる型: 枠を超えると、サービスが止まる(文化祭の昼にアプリが止まる)
- 課金型: 自動で有料になり、あとから請求が来る(知らないうちにお金が減る)

どちらも困りますが、いちばん困るのはどちらなのか知らないことです。使っているサービスごとに、①無料枠はどれだけか、②超えたら止まるのか課金か、③上限や通知を設定できるか、を調べます。この3点を1枚の表にしておきましょう。

調べる観点はAIに出させてかまいません。ただし料金は変わりやすいので、金額は必ず公式の料金ページで確かめてください。

文化祭でも、通信回数、保存量、再試行、公開範囲によって利用量は変わります。想定する会計件数から概算し、通知だけなのか、自動停止する設定なのかを確認します。確認した日付と公式ページも表に残すと、来年の担当者が見直せます。

14.9 障害が起きたときの連絡先、サービスを停止する方法

備えていても、止まるときは止まります。サービスが正常に使えなくなる出来事を障害(incident)と呼びます。日常語で言うと店を開けられないトラブルです。

障害対応の目的は、かっこよく直すことではありません。困る人を減らすことです。前もって決めておくのは3つです。

1. 連絡先: 自分のアプリの不具合は開発担当、イベントAPIの異常は文化祭本部へ連絡する。ホスティング全体の不調なら公式の障害情報を確認し、復旧待ちの間もシフト担当へ状況と代替手段を伝える
2. 一時停止の方法: 中途半端に壊れたまま動かすより、止めて案内を出すほうがよい場面があります。たとえば計算が狂ったまま会計を続けると、直せない被害が増えていきます。止め方は、平常時にAIと確認しておきます
3. お客さんへの案内文: 障害の最中に文章を考えてはいけません。先に書いて、貼るだけにしておきます

ただいま会計アプリが一時的に使えません。お会計は紙の伝票で行います。ご迷惑をおかけします。

紙の伝票とペンをレジの下に入れておくと、代替手段になります。伝票には番号を付け、復旧後にどの会計を登録したかを照合します。通信が途切れた直前の会計は、すでに保存されている可能性もあるため、再登録の前に履歴を確かめます。

復旧も、対象を分けます。コードの不具合は、確認済みの公開版へ戻す。失ったデータは、バックアップからの復元を検討する。Gitのコミットを戻しても、本番のデータベースは元に戻りません。データ構造を変更した版では、古いコードが今のデータを読めるかも確認します。

14.10 ライブラリの更新

開店して終わり、にならない理由がもうひとつあります。あなたのアプリは借り物の部品——ライブラリ(第10章)——でできていて、部品には新しい版が出続けます。多くは不具合や、第13章で見た「古いパッケージの弱点」の修理です。

放っておくと、どうなるか。半年後に動かそうとした後輩が、「古すぎて動かない」「危険だと警告が出る」にぶつかります。手入れをやめた店は、誰も壊さなくても静かに傷んでいくのです。

こうした公開後の手入れ全般を保守(maintenance)と呼びます。日常語で言うと店の手入れです。部品の更新は、3拍子でやります。

1. 更新の前にコミットする(第10章。戻れる場所を作る)
2. AIに更新を頼む(まずは「重要な修正だけ」で十分です)
3. 受け入れテストを一周する(第12章。更新で壊れていないかを確認める)

公開中は、重要な脆弱性や利用サービスからの通知を確認します。通常の更新は月に一度など頻度を決め、緊急の修正が必要な場合はその都度判断します。祭りのあとに停止するアプリと、来年まで公開を続けるアプリでは、必要な手入れも違います。

14.11 引き継ぎ文書——READMEと運用手順書

ところで、この店、来年は誰が開けるのでしょうか。あなたは引退するか、卒業するかもしれません。何も残さなければ、後輩はゼロから作り直します。データも、あなたの工夫も消えます。

アプリを次の人に渡せる状態にすることを引き継ぎ(handover)と呼びます。日常語で言うと店を次の代に渡すことです。渡す文書は2つ。

1つめはREADME(第10章)。これはアプリの説明書でした。何のアプリで、どう動かすか。2つめが、この章で新しく書く運用手順書です。READMEがアプリの説明なら、運用手順書は店番のマニュアルです。図14-7のように、動かし方・止め方・毎日の作業・困ったとき・お金の5項目を1枚にまとめます。

図 14-7



図14-7 1枚の運用手順書

注意はひとつだけ。金庫の中身(環境変数の値)は、手順書に書きません。書くのは「何が必要か」と「どこにあるか」までです(第9章の掟は、紙の上でも生きています)。

書き上げたら、最強のテストがあります。手順書だけを渡して、友達に店を開けてもらう(または止めてもらう)。あなたが一言も口を出さずに成功したら、その手順書は本物です。

14.12 アプリを捨てる時のデータ削除

最後の節は、少し意外な話かもしれません。店じまいの仕方です。

文化祭は終わります。アプリの役目も終わります。そのとき、中に残った売上データをどうしますか。3つに分けて決めます。

- 残すもの: 会計報告に必要な集計結果。CSVで書き出して(第8章)、部の記録として保管します

- 消すもの: 利用目的や保管期限を終えたデータ。会計報告や組織の保存ルールを確認し、不要になった明細とコピーを削除する。個人データには不要時の消去に努める規定があり、一律に「文化祭終了と同時に全部削除」とは決まらない
- 閉じるもの: ホスティングの契約、データベース、APIキー。使わなくなった会員証は無効にします(第9章)

いちばんよくないのは、動かなくなったアプリの放置です。誰も見ていないのに公開され続け、誰も直さないのに動き続ける。それは、弱点を抱えたまま無人になった店です。

閉店も運用のうちです。データベースだけでなく、CSV、バックアップ、ログなどの写しにも保存期限を決めます。削除した日と担当者を手順書へ記録して、店じまいまで完了させましょう。[個人情報保護委員会・データの管理](#)

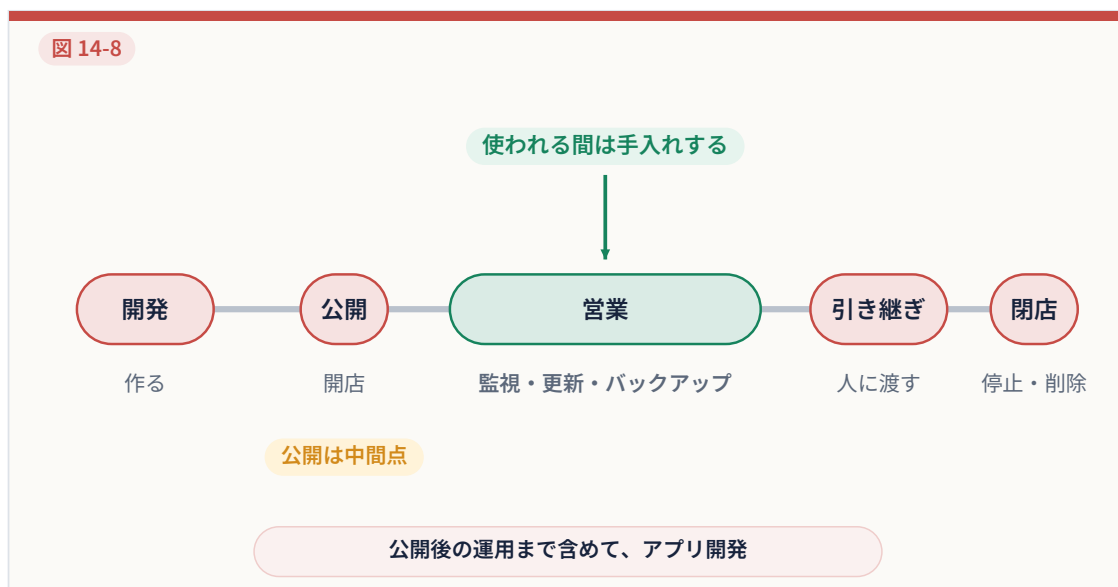


図14-8 アプリの一生

図14-8では、公開がアプリの一生の中間点で、そのあとに営業・引き継ぎ・閉店が続く点を見てください。

別の条件でも動くか試す

公開の画面や手順は、サービスによって、そして時期によっても違います。この章の通りに進まなくても、確認することは同じです。

- 自分のパソコンの外(別の端末・別の回線)から、URLでアプリが開けるか
- 会計の記録が、再読み込みしても、開発環境を閉じても残っているか
- 本番の金庫に、必要な環境変数がそろっているか

この3つがそろえば、外から使える公開版の基本を確認できています。操作で詰まったら、付録Aの項目と公式ガイドを照合します。変更で悪化した場合は、確認済みの公開版へ戻し、必要な設定やデータ構造も合っているか確認してください。

演習14-1 公開の儀

- ミッション: アプリを公開し、自分以外の人に、その人のスマートフォンから使ってもらう
- 時間の目安: 60分
- クリア条件:
 1. 公開URLを、自分以外の端末で開けた
 2. その端末で行った会計が履歴に残り、再読み込みしても消えない
 3. 開発環境をすべて閉じた状態でも、URLからアプリが開ける
 4. 受け入れテスト(第12章)のうち5件を本番でやり直し、結果を記録した
- つまづいたら:
 - 「ローカルでは動くのに」が出たら、14.4の3類型を上から順に疑う(まず本番の金庫)
 - 公開先のログを読み、秘密を伏せてエラー文と経路をAIに渡す(第11章)
 - 公開の手順は付録Aの確認項目と、使用サービスの公式ガイドへ
- 裏ダンジョン: QRコードを配って5人以上に使ってもらい、同時に使われたときに履歴や集計が壊れないか観察する
- 冒険の書に記録: 公開URL+本番でやり直した受け入れテストの記録

(授業向け)クラスで互いのアプリを開き、注文し合ってください。他人の操作は最高のテスト係です。

演習14-2 当日運用計画を作る

- ミッション: 文化祭当日(企業読者は公開後1か月)の運用計画を1枚にまとめる
- 時間の目安: 30分
- クリア条件:
 1. 「誰が監視するか」が、時間帯ごとに名前を書いてある
 2. 「何が起きたら、誰に連絡するか」が3行以上あり、お客さんへの案内文が付いている
 3. バックアップを取る時刻が決まっていて、戻す練習を1回済ませた
 4. 終わったあとデータをどうするか(残す・消す・閉じる)が書いてある
- つまづいたら:
 - 「気づく→影響を止める→知らせる→復旧する→再確認する」の順に、一つずつ手順を書く
 - 連絡先で迷ったら、14.9の3分類(自分のアプリ/本部のAPI/大家)で分ける

- 裏ダンジョン: 計画どおりの「避難訓練」をする。わざとアプリを止め、案内文を掲示し、紙の伝票に切り替え、復旧までを通して演じる
- 冒険の書に記録: 運用手順書+バックアップ・障害対応表

この章のまとめ

- ビルドはアプリを届けられる形にまとめる作業、デプロイはそれを公開先へ運んで動かせる状態にする作業。URLが出ることと、そこで正しく動くことは別の話
- 設定と秘密は、環境ごとに別々にある。Gitに入れていない `.env` は公開先へ引き継がれないので、本番用の値は公開先のSecretsに登録する
- 公開先でだけ動かないときは、金庫の用意忘れ・依存部品の入れ忘れ・URLの違いの3類型を順に疑い、公開先のログで確かめる
- 公開後は、監視で異常に早く気づく。バックアップは「取ってあるか」ではなく「戻せるか」で確かめ、コードの戻しとデータの復元は分けて考える
- 料金は、枠を超えたら止まるのか課金されるのかをサービスごとに調べる。障害時の連絡先・止め方・案内文、引き継ぎと店じまいの手順は、前もって書いておく

理解度チェック

1. 「ビルド」と「デプロイ」の違いを、それぞれ1文で説明してください。
2. 公開した会計アプリで、画面は開くのに、会計すると「保存に失敗しました」と出ます。店長画面にもログインできません。ローカルでは全部動きます。まず何を疑い、どこで確かめますか。
3. 「開店前・昼・閉店後の3回、CSVでバックアップを取っている。だから売上データが消えても大丈夫」と言えますか。理由も教えてください。

解答のめやす

1. ビルドは、開発中のプロジェクトから届けるものだけを選んで、届けられる形にまとめる作業です。デプロイは、それを公開先へ運び込み、利用者が使える状態にする作業です(→14.1)
2. 本番の金庫が空であること、つまり保存先の接続設定や認証の設定が公開先に登録されていないことを疑います。公開先のログでエラーを確かめ、必要な環境変数を公開先のSecretsに登録します(→14.3、14.4)
3. 言えません。戻したことのないバックアップは、お守りにすぎないからです。検証用の保存先へ戻す練習をして、履歴と集計が一致するかを照合します。本体と一緒に失わない場所に置いているかも確かめます(→14.7)

60秒でクエスト報告

アプリは公開済みで、URLはQRコードにしてレジ横に貼ってあります。当日は、エラー通知を私のスマートフォンで受けて監視します。アプリの不具合なら私、イベントAPIの異常なら本部、と連絡先を決めてあります。止まっている間は紙の伝票で営業し、掲示する案内文も用意してあります。バックアップは開店前・昼・閉店後の3回で、戻せることは昨日確認しました。文化祭のあとは集計だけ残して明細を消します。来年の後輩には、運用手順書ごと渡します。

——これは口頭試問の第5問「壊れた場合、どこを確認し、どう戻しますか」へつながる答えです。誰の何を解決するか(Q1)、データの経路(Q2)、保存先(Q3)、確認した根拠(Q4)、復旧手順(Q5)。それぞれを、手元の成果物で説明できます。

レベルアップ!

Lv.14 称号: 届けた人

- 手に入れた装備: ビルド、デプロイ、ホスティング、ドメイン、本番環境、監視、クォータ、障害、保守、引き継ぎ
- 冒険の書: 公開URL+運用手順書+バックアップ・障害対応表

序章で作ったものは「デモ」でした。いまあなたの手元にあるものは、仕様を持ち、構造がわかり、テストされ、守られ、公開され、引き継げます。5つの壁は、すべてあなたの後ろ側にあります。

次の第15章は、卒業制作です。題材はもう、模擬店ではありません。あなた自身の困りごとです。

第15章 卒業制作

——自分の課題を解決するアプリを作る

クエスト15: 最後のクエスト——自分の地図を描く

文化祭は、終わった。ここからは、あなたの番。

この章のねらい

- 解決する困りごとを一文にし、MVPの3機能・扱うデータ・悪用のされ方を書き出せる
- 設計7点セットを書き、受け入れ条件で確かめながら、実装を段ごとに小さく頼める
- 検証の記録と手順書を添えて人に渡し、口頭試問の5問に成果物で答えられる

今回のクエスト

ここまでの14章、この本はいつも先に地図を渡してきました。題材はこちらで用意し、壊れる場所も、直し方も、先回りして書いてきました。

ここからは、その地図を、自分の題材で描きます。

題材を決めるのはあなたです。仕様を決めるのも、完成したと判定するのも、あなたです。この本の言葉で言えば、あなたが自分の地図を描く番です。それが卒業制作であり、この本の最後の課題です。

手元の開発ポートフォリオを開いてください。課題定義書、1ページ仕様書、構成図、テストケース表。題材が変わっても使える型が、そこにあります。まずは「誰の困りごとなら、明日、本人に話を聞けるか」から考えてみましょう。

この章で新しく学ぶことは、ほとんどありません。持っているものを全部使う。それだけの章です。

装備一覧——あなたがすでに持っているもの

#	成果物	手に入れた章	卒業制作で使う場面
1	課題定義書	第2章	ステップ1: 誰の何を解決するか
2	MVP範囲表	第3章	ステップ1: 機能を3つに絞る
3	1ページ仕様書	第3章	ステップ2: 設計の中心
4	AI開発ブリーフ	第4章	ステップ3: 最初の依頼
5	画面遷移図	第3章	ステップ2: 画面を数える
6	アプリ構成図	第5章	ステップ2: データの通り道
7	API接続仕様	第7章	ステップ2: 外部サービスの窓口
8	データベース設計図	第8章	ステップ2: 何をどう残すか
9	権限マトリクス	第9章	ステップ2・4: 誰に何を許すか
10	Gitコミット履歴	第10章	ステップ3: 動いたら記録
11	バグ報告書	第11章	ステップ4: 見つけて直した証拠
12	テストケース表	第12章	ステップ4: 確認した条件と根拠
13	セキュリティチェックリスト	第13章	ステップ4: 公開前の点検
14	運用手順書	第14章	ステップ5: 人に渡す準備
15	AI利用・判断記録	第15章(この章で作る)	ステップ5: 開発を理解している証拠

表の右端の列のとおり、15点はどれも、卒業制作のどこかのステップで使います。進め方は5つのステップです。課題を決め、設計し、小さく実装し、検証し、人に渡す。見覚えがあるはずです。この本全体の縮図であり、あなたが14章かけて一度通ってきた道です。

図 15-1

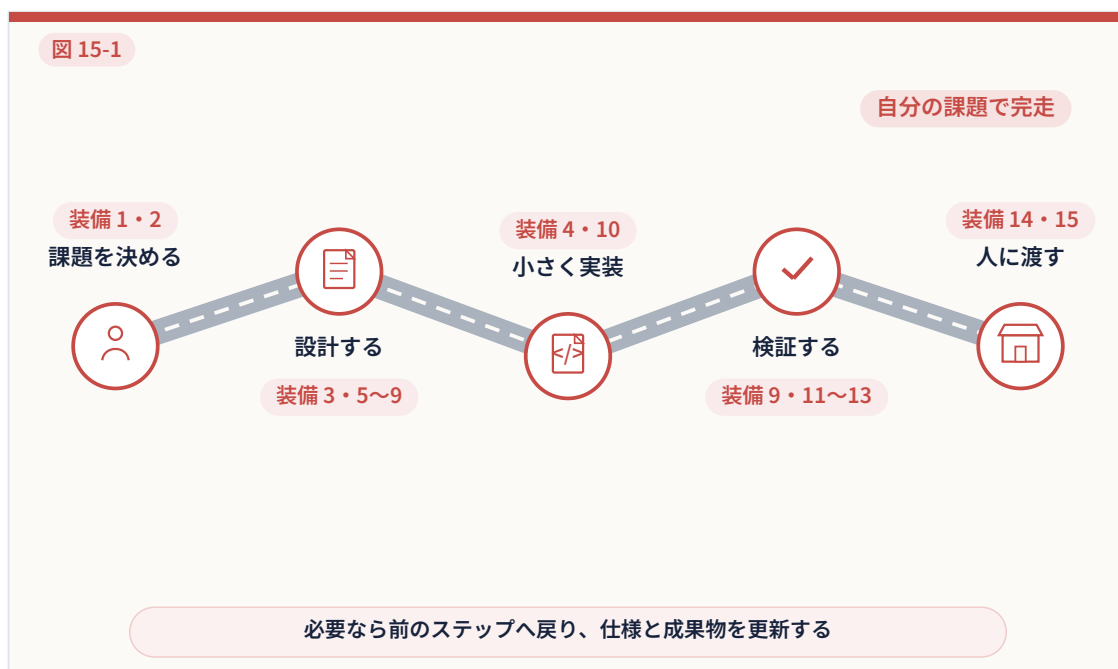


図15-1 卒業制作の5ステップ

図の各ステップに付いた「装備」の番号は、上の表の番号です。5ステップは、一度ずつ通るだけの道ではありません。実装して課題の理解が変われば、設計へ戻ってかまいません。ただし、何を確かめるために戻るかを決めます。迷ったときは、課題定義書の最初の一文が道しるべになります。

15.1 ステップ1 課題を決める

▼使う装備: 第2章「課題定義書」/第3章「MVP範囲表」の型/第13章「悪用を想像する」

何を作ってもよいと言われると、かえって手が止まるものです。題材が思いつかなければ、この一週間で二度以上繰り返した面倒な作業を探します。申し込み人数を数える、観察メモを探す、担当者へ同じ説明を送る。第2章と同じように、誰の、どんな困りごとを解決するのかを一文にします。

第2章の課題定義書がそのまま使えそうなら、それを育ててください。新しい題材を選び直してもかまいません。決まらない人のために、15.6に3つのコースを用意してあります。

本当にアプリが必要か

第2章の最後の問いを、もう一度自分に向けます。作らないという選択肢はないか。紙の表とホワイトボードで足りるなら、アプリは要りません。卒業制作だから作る、は理由になりません。

迷ったときの基準をひとつ渡します。「記録が増え続ける」「集計したい」「複数の人が使う」のどれかに当てはまるなら、アプリにする価値があります。どれにも当てはまらない題材は、動くけれど使われないアプリになりがちです。

MVPの範囲を切る

第3章の型で、機能を3つに絞ってください。基準も同じです。「それがないと、現場が回らないもの」だけを残す。落選した機能は捨てるのではなく、「今後改善する点」のリストに移します(ステップ5で使います)。

扱うデータと、利用上のリスク

このアプリに入力され、保存されるデータを全部書き出してください。そして1項目ずつ問います。これは個人情報か。

個人情報を持つ題材では、設計と運用の確認が増えます。名前、連絡先、健康状態、成績。誰が使い、誰が見られ、いつまで残すかを、作り始める前に決めます。健康状態など、とくに慎重な扱いが必要な情報を含む題材では、指導者や所属組織の担当者へ相談します。

人数の集計だけで足りるなら、氏名を集めずに済みます。これが、個人情報を「持たない設計」です。ニックネームに替えても、ほかの情報と照合して本人がわかれば個人情報になりうる点は、第13章で確認したとおりです。作る段階でも、最初は架空データで、必要な操作と保存だけを完成させる方法があります。

最後に、第13章の演習を自分の題材でやります。このアプリが悪用されるとしたら、どう使われるかを3通り。ここで出た答えは、ステップ2の権限設計と、ステップ4の検証にそのままつながります。

▼ステップ1のチェックリスト

- [] 「誰の、どんな困りごとを解決するか」が一文で書けている(第2章)
- [] アプリ以外の解決策と比べて、それでも作る理由が言える
- [] 機能を3つに絞り、落選した機能をリストに残した(第3章)
- [] 扱うデータを全部書き出し、個人情報を含むか判定した
- [] 個人情報を含む場合、「持たない設計」にできないか検討した
- [] 悪用のされ方を3通り書いた(第13章)

15.2 ステップ2 設計する

▼使う装備: 第3章「1ページ仕様書」/第5章「アプリ構成図」/第7章「API接続仕様」/第8章「データベース設計図」/第9章「権限マトリクス」

ひとりで設計できるだろうか——不安がいちばん大きいのは、実はこのステップです。でも、設計は7点セットを書く作業です。すべて既習の型で、書く順番も決まっています。

1. 1ページ仕様書(第3章)——要求、ユースストーリー、計算ルール、やらないことリスト
2. 画面遷移図(第3章)——画面を数える。3～5画面に収まらなければ、MVPが大きすぎます
3. アプリ構成図(第5章)——あなた(ブラウザ)・サーバー・データベース・外部APIの4役で描く
4. データベース設計図(第8章)——テーブルと列。ステップ1で書き出したデータがここに入る

5. API接続仕様(第7章)——外部APIを使うなら。使わないなら「なし」と書く。それも立派な設計です
6. 権限マトリクス(第9章)——ロールが「自分だけ」でも書く。「管理者=自分、利用者=閲覧のみ」の2行の表が、画面を隠すだけの権限制御を防ぎます
7. 受け入れ条件(第3章)——5つ以上。少なくとも2つには数字を入れる。これがステップ4のテストの種類になります

きれいに書く必要はありません。各1枚、合計7枚を1〜2日で書き切るのが目標です。埋まらない欄は空欄のままでかまいません。

第3章で覚えた技を使ってください。仕様書ごとAIに渡して、「空欄と、決まっていないことについて質問してください」と頼むのです。AIの質問は、あなたの設計の穴を映す鏡になります。

そしてもうひとつ、第3章の教訓を思い出してください。あなたの題材にも必ず、模擬店の「割引が重なったらどうする?」に当たる問いがあります。締め切りが重なったら。定員を超えたら。同じ人が二度登録したら。AIに聞いても答えが出ない、あなたにしか決められない1行——それを見つけて書けたら、設計は半分終わっています。

設計は実装中になんて変わってかまいません。ただし、変わったら仕様書を直してください。実物と仕様書がずれたまま進むと、第11章でやった切り分けができなくなります。壊れたとき、「正しい姿」を書いた紙がないと、何と比べればよいかわからなくなるからです。

▼ステップ2のチェックリスト

- ☐ 7点セットが(空欄を含んでよいので)全部ある
- ☐ 画面は5つ以内に収まっている
- ☐ 受け入れ条件が5つ以上あり、少なくとも2つに数字が入っている
- ☐ 「自分にしか決められない1行」がひとつ以上書けている
- ☐ 仕様書をAIに渡し、返ってきた質問に答えて設計を直した
- ☐ 権限マトリクスの行(ロール)が、ステップ1の「悪用3通り」と矛盾しない

15.3 ステップ3 小さく実装する

▼使う装備: 第4章「AI開発ブリーフ」/第10章「コミット」「ロールバック」

いよいよ実装です。はやる気持ちと、「本当に動くのか」という不安が、同時に来るところです。

ただし、一気に作らせないでください。第4章で学んだ通り、大きな依頼はAIの想像で埋まります。この本が推奨する積み上げの順番は、次の通りです。

図 15-2

検証環境・架空データで進める例

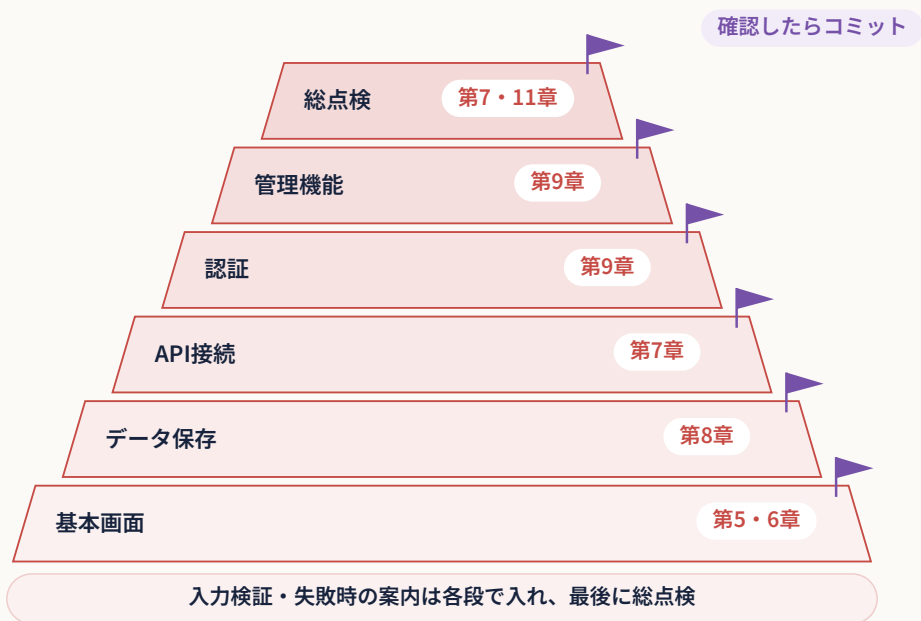


図15-2 実装を積む6段

1. 基本画面——データは仮でよいので、画面と操作を作る。第5章の四状態(表示中・読み込み中・データなし・エラー)を最初から意識する
2. データ保存——再読み込みしても残ることを確認する。序章の事件を、今度は自分で防ぐわけです
3. API接続——外部APIを使う場合のみ。まず一覧表示など読み取りから
4. 認証——ログインを自作せず、AI開発環境やサービスの認証機能を使う(第9章)
5. 管理機能——管理者だけの画面と操作。権限マトリクスの通りに
6. エラー処理の総点検——各段で入れた通信失敗や入力検証を、全体の流れでも確かめる

これは、検証用の環境と架空データで進める順番の例です。認証や認可が必要なアプリでは、実データを入れたり人に公開したりする前に、その対策を完成させます。入力検証や失敗時の案内も、それぞれの機能を作る段階で入れ、最後の段でつながりを総点検します。

最初にAI開発ブリーフと1ページ仕様書を渡し、あとは段ごと、機能ごとに小さく頼みます。そして、確かめられた変更をコミットします(第10章)。コミットの数を経る必要はありません。何を変え、何を確認したかが、履歴から読み取れるかを目安にしてください。

ここで、この本で何度も確認した約束をもう一度。AIの応答も、生成されるコードも、この本の例やあなたの前回の結果とは違うものになります。それで正常です。比べる相手はAIの出力ではなく、自分が書いた受け入れ条件です。各段で受け入れ条件が確認できれば、先へ進んでください。

行き詰まったときの手順も、すでに持っています。同じ指示を繰り返さない(第4章)。入力・処理・保存・表示のどこかを切り分ける(第11章)。30分たっても抜けられなければ、直前のコミットへロールバックして(第10章)、依頼をもっと小さくして出し直す。全部、既習です。

▼ステップ3のチェックリスト

- [] 最初の依頼で、AI開発ブリーフと仕様書を渡した(第4章)
- [] 再読み込みしても記録が残る(第8章)
- [] 確認できた変更をコミットし、変更内容と確認結果を記録した(第10章)
- [] ロールごとにログインすると、見える画面と操作が変わる(第9章)
- [] 通信失敗・データ0件・空欄入力で、画面が黙り込まない(第5章の四状態)
- [] 行き詰まったとき、ロールバックか切り分けで抜けた記録がある

15.4 ステップ4 検証する

▼使う装備: 第12章「テストケース表」/第9章「権限マトリクス」/第13章「セキュリティチェックリスト」/第11章「バグ報告書」

動きました。「もう提出していいのでは」——動いた直後は、誰でもそう思います。

ステップ4では、決めた条件で何を確認できたかを記録します。「一度動いた」だけでは、利用者に渡してよい根拠になりません。その根拠を増やす工程です。

まず、第12章の型で自分のアプリのテストケース表を書きます。正常系、異常系、そして境界値。境界値の探し方は模擬店と同じです。0件のとき。空欄のとき。上限を超えたとき。同じボタンを二度押したとき。そして日付や時刻の境目——模擬店ではタイムセール開始の13:59:59と14:00:00でした。

あなたのアプリの「境目」はどこですか。締め切り当日、定員ちょうど、月をまたぐ集計。そこにバグは住んでいます。

次に、権限の抜けを試します。権限マトリクスの全マス目を、実際に操作して確かめてください。とくに第9章で学んだ攻め方を自分のアプリに向けます。管理者用の画面のURLを、一般ロールのまま直接開く。ボタンが隠れているだけなら、それは権限制御ではありません。

3つめが、セキュリティ間違い探しの自分版です。第13章のチェックリストを、1項目ずつ自分のアプリに当てます。APIキーがコードに直書きされていないか。入力値を信用していないか。ログに個人情報が残っていないか。第13章でやった通り、別のAIにセキュリティレビューを頼むのも有効です。ただし最終判断は、レビューを読んだあなたがします。

最後の2つは、渡す直前の点検です。スマートフォンで全画面を操作すること。あなたのアプリの利用者は、たぶん机のパソコンの前にはいません。

そして公開環境での再テスト(第14章)。デプロイしたら、テストケース表の主要な行をもう一度、公開URLで流します。ローカルでは動くのに公開環境で動かない——その原因の多く(環境変数の設定漏れなど)は、ここで初めて姿を見せます。

見つけたバグは、第11章のバグ報告書の型で記録してから直してください。バグが出たことは減点ではありません。見つけて、直して、再確認した記録こそが、ステップ5の口頭試問での最強の答えになります。

▼ステップ4のチェックリスト

- [] テストケース表に正常系・異常系・境界値が揃っている(第12章)
- [] 自分のアプリの「境目」を2つ以上見つけてテストした
- [] 権限マトリクスの全マス目を実際に試し、URLの直接入力も試した(第9章)
- [] セキュリティチェックリストを全項目、自分のアプリに当てた(第13章)
- [] スマートフォンで全画面を操作した
- [] 公開環境で主要なテストを再実行した(第14章)
- [] 見つけたバグをバグ報告書に記録し、修正後に再確認した(第11章)

15.5 ステップ5 人に渡す

▼使う装備: 第14章「README」「運用手順書」/第4章「ロール分担」

作ったものを人に見せるのは、誰でも少しこわいものです。最後のステップは、あなたの手元からアプリを離す工程です。やることは4つあります。

1つめ、公開一式。デプロイして公開URLを取り、**README.md**と運用手順書を書きます。

README.mdには、何のアプリか・動かし方・環境変数の一覧(名前と置き場所だけ。値は書かない)。運用手順書には、バックアップ・障害時の連絡先・止め方。全部、第14章でやった通りです。自分のアプリ用に書き直すだけで済みます。

2つめ、3分デモ。発表の実演は、時間配分を決めて練習します。

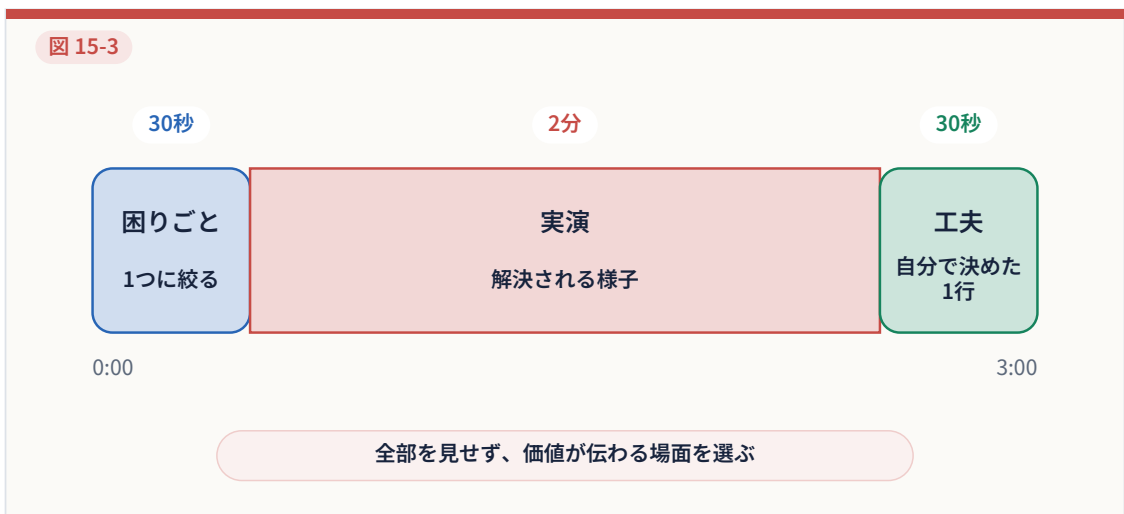


図15-3 3分デモの時間配分

コツは、機能を全部見せないことです。困りごとを30秒で語り、それが解決される様子を2分で見せ、最後の30秒で自分の工夫をひとつだけ話す。工夫とは、ステップ2で見つけた「自分にしか決められなかった1行」のことです。

デモは必ず2回、通しで練習してください。そして本番環境で見せる場合は、直前にもう一度動作確認を。万一のために、画面録画かスクリーンショットの保険を用意しておく、通信障害でも発表は成立します。

3つめが、この本の最後の成果物、AI利用・判断記録です。A4で1枚、4つの欄を埋めます。

- AIに任せた部分(例: 画面の実装、テストコードの生成、エラーの一次調査)
- 自分で決めた判断(例: 個人情報を持たない設計、料金の端数処理、MVPの3機能)
- AIの提案を採用しなかった箇所と、その理由
- AIの間違いに気づいて直した箇所

この記録は、開発をどう進めたかを次の人に伝えるためのものです。採用しなかった提案とその理由があれば、後から同じ判断を見直すときにも役立ちます。該当する出来事がなければ「該当なし」と書き、記録のために失敗談を作る必要はありません。

どこを任せ、何を自分で決めたかを言える人は、開発を理解しています。逆に、全部自分で書いても説明できなければ、理解しているとは言えません。AI利用・判断記録の1枚が、冒険の書の15点め、最後のページになります。

4つめ、今後改善する点を3つ。ステップ1で落選した機能や、検証で見つけたが今回は直しきれなかった点を、正直に書きます。これは未完成の言い訳ではありません。公開後のアプリに改善の計画があること自体が、第14章で学んだ「使い続けられる状態」の一部です。

▼ステップ5のチェックリスト

- [] 公開URLで、自分以外の人がスマートフォンから使えた(第14章)
- [] **README.md**と運用手順書が、自分のアプリの内容になっている
- [] 3分デモを2回通しで練習し、時間内に収まった
- [] AI利用・判断記録の4欄がすべて埋まっている
- [] 今後改善する点が3つ書けている

15.6 3つの制作コース

題材の候補として、3コースを用意しました。どのコースも最小構成と発展構成の2段階です。最小構成を完成させ、利用者に試してもらうところまでを、最初の到達点にします。

授業の8回版や2日研修では、最小構成が標準です(付録D)。迷ったら小さいほうを選んでください。小さく完成させた人が、結局いちばん遠くまで行きます。

コースA キャンパス・イベント(難易度: ふつう)

サークルの新歓、ゼミの発表会、学園祭の企画運営など。

段階	内容
最小構成	イベント一覧/参加登録(まず登録番号で足りるかを検討)/管理画面(登録数の集計)
発展構成	お気に入り登録/当日受付(チェックイン)/会場の地図表示/時間帯ごとの参加集計

参加登録では、何のために参加者を識別するかを先に決めます。人数だけなら登録番号で足りる場合もあります。ニックネームでも、本人がわかる情報と結び付くなら個人情報としての検討が必要です。練習は架空の参加者から始めてください。

コースB フィールドワーク・研究記録(難易度: ふつう)

インタビュー記録、観察メモ、文献ノートなど、研究活動の記録全般。

段階	内容
最小構成	記録の作成・一覧・検索/タグ付け/CSV出力(第8章)/閲覧できる人の限定(自分だけ)
発展構成	写真・資料の添付/共同研究者ごとの閲覧権限/AI要約(下の発展課題)

インタビューの発言には、名前を書かなくても個人を特定できる内容が含まれることがあります。だから最小構成でも、誰が閲覧できるかを決めます(はじめは自分だけ)。共同研究者と分け合う細かな権限は、発展構成で加えます。実際の研究記録を使う場合は、研究倫理や所属組織のルール、参加者への説明に沿って扱います。公開デモには架空の記録を使えます。

コースC 小さな店舗・業務改善(難易度: 高め)

本編で育てた模擬店の会計アプリの、いわば上級版です。文化祭では出てこなかった、実際の店舗の料金ルールに挑みます。

段階	内容
最小構成	メニュー登録(店側で変更できる)/人数入力/税・サービス料の計算/会計履歴
発展構成	特別利用者の料金ルール(会員割引・チャージ)/日次・月次の集計/スプレッドシート連携(第8章)/管理画面

なぜ難易度が高めなのか。チャージ、サービス料、税は、適用する順番で合計金額が変わるからです。サービス料は税の前に掛けるのか、後なのか。会員割引はサービス料にも効くのか。

第3章でセット割とタイムセールの併用に悩んだ、あの問題が3つ4つと重なって現れます。決めるのはAIではなく、あなた(と店の人)です。そのぶん見返りも大きく、決めたルールの1行1行が、そのままテストケース表の1行になります(第12章)。実在の店の業務を題材にするなら、店の人への短いインタビュー(第2章)から始めてください。

発展課題 AI連携(全コース共通・任意)

余力のある人は、アプリの中からAIのAPIを呼ぶ機能に挑戦できます。記録の自動要約(コースB)、イベントのおすすめ(コースA)、相談への応答など。APIの呼び方は第7章の型のままで、相手がAIになるだけです。

ただし、注意が2つ。第一に、個人情報を含むデータをAIに送ってよいか。これは、第13章で考えた「AIに社内データを入力してよいか」の判断そのものです。送る前に、送ってよいデータかを判定してください。

第二に、AIのAPIには従量課金のものがあります。料金と利用量を確認し、予算通知、サービス側の上限、アプリ側の回数制限を組み合わせます。通知だけなのか、利用が実際に止まるのかを確認してから公開します(第14章)。

AIの返答の誤りや待ち時間、失敗したときの表示も、テストケース表へ追加します。

発展課題は、卒業の条件ではありません。基本のアプリを「渡せる状態」にするのが先です。

スケジュール例

2週間版(授業と併走・1日1～2時間)

日	やること
1～2日目	ステップ1: 課題を決める
3～4日目	ステップ2: 設計7点セット
5～9日目	ステップ3: 実装(1日1段が目安)
10～11日目	ステップ4: 検証
12～13日目	ステップ5: 公開・資料・デモ練習
14日目	予備日+相互レビュー

4週間版(独習・週5～7時間)

週	やること
1週目	ステップ1～2: 課題と設計
2～3週目	ステップ3: 実装(週の終わりに必ず「動く状態」で止める)
4週目	ステップ4～5: 検証、公開、発表準備

共通の作戦をひとつだけ。遅れたら、機能を削ってください。スケジュールを延ばすのではなく、スコープを切り直すのです(第3章)。締め切りに間に合う小さなアプリは、間に合わない大きなアプリに常に勝ちます。

相互レビューのやり方

発表の前に、ペアで練習します。やり方は単純で、口頭試問5問(15.7)を出し合うだけです。

1. 一方が3分デモをする
2. もう一方が、5つの質問を順番にぶつける
3. 答えに詰まった質問をメモして、交代する

答えられなかった質問は、失点ではなく発表当日までに埋められる最高の宿題です。質問する側にも学びがあります。相手の構成図を見ながら「データはどこを通りますか」と聞くうちに、質問の目——第13章で言えばレビューの目が育ちます。

独習の人は、AIに面接官の役を頼んでください(第4章のロール分担の応用です)。「次の5つの質問を、私のアプリについて順番にしてください。答えが曖昧なら、さらに突っ込んでください」と頼めば、ひとりでも口頭試問の練習ができます。

15.7 発表と口頭試問——5つの質問に答える

卒業制作の締めくくりは、発表と口頭試問です。評価されるのは、完成度よりも説明です。3分デモのあと、次の5つの質問に自分の言葉で答えられれば、あなたの卒業制作は完成です。

5問には、それぞれ「答えの型」があります。そしてどの質問にも、対応する成果物がすでに手元にあります。

質問	答えの型	対応する章と成果物
Q1 このアプリは、誰の何を解決しますか。	「〈誰〉が〈場面〉で困っている〈こと〉を、〈機能〉で解決します」	第2・3章/課題定義書、1ページ仕様書
Q2 ボタンを押すと、データはどこを通りますか。	「画面から〈経路〉を通して〈処理と保存〉へ。構成図のこの矢印です」と、図を指でなぞりながら	第5～7章/アプリ構成図、API接続仕様
Q3 データはどこに保存されていますか。	「〈場所〉の〈テーブル〉に、この設計図の通りに入っています」	第8章/データベース設計図
Q4 正しく動くことを、どのように確認しましたか。	「受け入れ条件〈n〉件をテストし、見つけたバグを直して再確認しました。境界値は〈例〉です」	第11・12章/テストケース表、バグ報告書
Q5 壊れた場合、どこを確認し、どう戻しますか。	「〈ログ・画面〉で影響を確認します。コードは確認済みの公開版へ、データはバックアップからの復元を判断します。手順書のここです」	第10・11・14章/コミット履歴、運用手順書

答え方のコツは、暗記しないことです。成果物を机に並べ、見せながら話してください。Q2で構成図を指でなぞれる人と、そらで話そうとして詰まる人の差は、理解の差ではなく、道具を使う準備の差です。授業や研修では、この5問と成果物一式が採点ルーブリック(付録D)にそのまま対応しています。

そして、ここが大事なところです。この5問に答えられたなら——たとえコードの大半をAIが書いていたとしても、このアプリを開発したのは、あなたです。

第1章で、この本は「開発できる」を5つの動詞で定義しました。説明できる。変更できる。壊れても戻せる。正しさを確かめられる。人に渡せる。振り返ってください。Q1～Q3に答えたあなたは、説明できています。1変更=1依頼で積み上げ、コミットを刻んだあなたは、変更でき、戻せます。Q4に答えたあなたは、確かめられています。公開URLと手順書を差し出したあなたは、渡せています。5つとも、現実になりました。

最終成果物として提出するのは2つです。アプリ本体(公開URL)と、15点の記録を綴じた冒険の書。提出前の点検を、最後にもう一度どうぞ。

▼提出前チェック——全15点+アプリ本体

- ☐ 1 課題定義書(第2章)
- ☐ 2 MVP範囲表(第3章)
- ☐ 3 1ページ仕様書(第3章)
- ☐ 4 AI開発ブリーフ(第4章)
- ☐ 5 画面遷移図(第3章)
- ☐ 6 アプリ構成図(第5章)
- ☐ 7 API接続仕様(第7章)※外部APIなしの場合は「なし」の記載
- ☐ 8 データベース設計図(第8章)
- ☐ 9 権限マトリクス(第9章)
- ☐ 10 Gitコミット履歴(第10章)
- ☐ 11 バグ報告書(第11章)
- ☐ 12 テストケース表+実施記録(第12章)
- ☐ 13 セキュリティチェックリスト(第13章)
- ☐ 14 運用手順書+README(第14章)
- ☐ 15 AI利用・判断記録(第15章)
- ☐ アプリ本体の公開URL(スマートフォンで動作確認済み)

この章のまとめ

- 制作を一方向の工程にしない。検証用の環境と架空データで小さく作り、戻りながら仕様を更新する
- 作る前に、作らないという選択肢と比べる。個人情報扱う題材では、まず「持たない設計」にできないかを考え、誰が見られ、いつまで残すかを作り始める前に決める
- AIの出力は毎回違ってよい。比べる相手は自分が書いた受け入れ条件で、確かめられた変更ごとにコミットする
- 一度動いたことは、人に渡してよい根拠にならない。境界値、権限の抜け、スマートフォン、公開環境まで確かめ、見つけて直して再確認した記録を残す

- どこをAIに任せ、何を自分で決めたかを成果物で説明できれば、コードの大半をAIが書いていても、そのアプリを開発したのは自分だと言える

理解度チェック

1. 卒業制作で新しく作るAI利用・判断記録には、4つの欄があります。すべて挙げてください。
2. コースAで、新歓イベントの参加登録アプリを作ります。管理画面で必要なのは登録数の集計だけです。参加者の氏名を集める前に考えることを2つ挙げてください。
3. 「実装を全部終えてから、まとめて検証を始めればよい」と言えますか。理由も答えてください。

解答のめやす

1. AIに任せた部分、自分で決めた判断、AIの提案を採用しなかった箇所とその理由、AIの間違いに気づいて直した箇所(→15.5)
2. 次のうち2つ。氏名を集めない「持たない設計」にできないか(登録番号で足りるか)。ニックネームでも、ほかの情報と結び付けば個人情報になりうる。誰が見られ、いつまで残るか。練習は架空の参加者で始めること(→15.1、15.6)
3. 言えません。各段で受け入れ条件を確かめてからコミットし、入力検証や失敗時の案内も機能ごとに入れます。ステップ4は、その上に境界値・権限・公開環境の確認を重ねる工程です。途中で理解が変われば、設計へ戻って仕様書を直します(→15.2、15.3、15.4)

レベルアップ!

Lv.15 称号: 開発者

- 手に入れたもの: 自分の課題を、自分の判断で、人に渡せるアプリにした経験
- 冒険の書: 全15点、完成

序章では、まず動く画面を手にししました。いま手元にあるのは、誰の何を解決するかを説明でき、変更を記録し、確かめ方と戻し方を添えて人に渡せるアプリです。書いたコードの量よりも、判断と確認の積み重ねが、その違いを作りました。

称号「開発者」は、この本が贈るものではありません。ここまでの事実、あとから付いた名前です。序章の「冒険の記録」を開き、最後のマスに今日の日付を書き込んでください。

この本に、次の章はありません。最後に短い「おわりに」だけがあります。テーマは、この旅のあいだずっと底を流れていた一文——理解は外注できない、です。

おわりに——理解は外注できない

あなたが手に入れたもの

序章の3つのボタンを、覚えていますか。押すと記録が増え、再読み込みすると消える。短い体験から、「記録はどこにあるのか」という問いが生まれました。

いま、同じ画面を見たら、何を確かめるでしょうか。

メモリにだけ置かれていないか。保存処理は成功したか。読み出しに失敗していないか。ブラウザの中に保存するのか、サーバーのデータベースを使うのか。以前は1つに見えた「消えた」という出来事を、いくつかの問いに分けられます。この「分けて問える」ことが、本書で積み上げてきた力です。

価値、構造、変更、品質、運用。本書の5枚の地図です。序章の図0-3を、もう一度眺めてみてください。各地図の横に、自分が確かめたことを1つずつ書ければ、5枚の地図はもう抽象的な用語の一覧ではありません。自分のアプリに戻るための地図です。

冒険の書に集めたのは、課題定義書からAI利用・判断記録までの15点の記録です。書きかけのものがあっても、まず並べてみましょう。なぜ作ったか、どう作られているか、どう確かめたか、何をまだ確かめていないか。アプリの画面だけでは伝わらない判断が、そこに残っています。

序章の「冒険の記録」に空欄があれば、次に戻る章を1つ選んでください。すべてを覚えていることより、確かめる場所を知っていることが、次の制作を助けます。

コードは書けないままでいい、のか

正直に言います。本書を終えたあなたは、まだコードを「書けない」かもしれません。それでいいのか、と不安になる人もいるでしょう。

考えてみてほしいのです。本書であなたがやってきたことは、何だったか。

誰の困りごとかを決めた。仕様を決めた。割引の併用ルールを決めた。1円玉を扱わないと決めた。セールを操作できる人を決めた。何をテストするかを決めた。公開してよいかを判断した。文化祭が終わったあとのデータをどうするか、まで決めた。

どれも、現場の事情と、試した結果をもとに決める仕事です。AIは案を出し、比較し、コードやテストを書く手助けをしました。あなたは、その案を採用する理由を確かめてきました。

コードを読む力や書く力は、これからも役に立ちます。自分で直せる範囲が広がり、AIの変更も確かめやすくなります。本書で覚えた変数や関数を入口に、気になった処理から少しずつ学べばよいのです。

もう1つ、次の制作にも持っていきたい約束があります。

理解は外注できない

コードを書くことは、AIに外注できます。しかし、何を作ったのかを理解し、その結果に責任を持つことは、外注できません。

本書が「動いた!」をゴールにしなかった理由は、ここに 있습니다。動くだけなら、AIだけでも作れるのです。でも、そのアプリが誰かのお金を計算し、誰かの記録を預かり、誰かの信頼の上で動くのなら——それを理解している人間が、どこかに一人、必要です。あなたのアプリなら、それはあなたです。

口頭試問の5つの問いを思い出してください。誰の何を解決するのか。データはどこを通るのか。どこに保存されているのか。正しさをどう確かめたか。壊れたらどう戻すか。答えに詰まったところは、理解を深める次の入口です。アプリを見直し、必要なら専門家にも相談しながら、説明を更新していきます。

1人ですべてを抱える必要はありません。人に相談するときも、わかっていること、未確認のこと、起きていることを分けて伝える。そのために、図と記録を作ってきました。

ここから先の歩き方

本書の地図の先には、それぞれの方向に、深い大陸が広がっています。

- コードそのものを書けるようになりたい人は、プログラミング言語の学習へ。まず会計アプリの計算関数を、自分の手で1か所変えてみましょう。変更前後のテストが、学習の手がかりになります。
- 画面や体験を良くしたい人は、デザインへ。
- 集めたデータから何かを知りたい人は、データ分析へ。
- もっと大きなソフトウェアの世界を知りたい人は、ソフトウェア工学へ。本書の5枚の地図は、その世界の入口の看板を初心者向けに描き直したものです。本物の看板は、もっと詳しく、もっと面白い。

AIも、引き続き学習の相手になります。「私が作った会計アプリを例に説明してください」と頼めば、未知の言葉を知っている仕組みにつながられます。その説明を実際のコードや公式資料と照合するところまでが、AIへの教わり方です。

最後に。

あなたのアプリは、公開された日が完成の日ではありません。使われて、壊れて、直されて、引き継がれて——それが、アプリが生きているということです。開発は続きます。あなたの冒険の書には、まだ白いページが残っています。

次の困りごとを見つけたら、また地図を開いてください。

付録A ツール別スタートガイド

A.1 大原則——本書はツールに依存しません

最初に、いちばん大事なことを言います。本書の演習は、特定の製品を前提にしていません。

まず次の6つの機能が、どこにあるかを探します。序章は、このうちチャットとプレビューがあれば始められます。後半の章では、記録を消えないように残す保存先、ログインの仕組み、秘密の値の置き場所なども使います(第7～9章)。6つの機能が見つかったら、A.3の条件を、必要になる章の前に確かめてください。

1. AIとのチャット — 日本語で頼み、返事をもらう場所
2. ファイル — AIが作ったコードの置き場所を見渡す場所
3. ターミナル — コマンドで直接指示する場所
4. プレビュー — 作ったアプリを動かして確かめる場所
5. バージョン管理 — セーブ(コミット)と、戻る(ロールバック)の仕組み
6. 公開 — 自分以外の人がURLで使えるようにする仕組み

最初の準備は3ステップです。利用するサービスにログインし、練習用の新規プロジェクトを作り、チャットとプレビューを開きます。アカウント作成や料金の条件は、利用時点の公式案内で確認してください。授業では担当者が動作確認した環境を使うと、開始時の迷いを減らせます。

操作名が本書と異なる場合は、その環境の公式ヘルプで機能名を探します。この付録で名前を挙げる製品やサイトは、2026年時点の例です。練習用データと教材の用意のしかたは、A.6にまとめました。

A.2 6つの機能の対応表

新しい環境を前にしたら、この表で6つの機能を探してください。今日は使わない機能は、場所をメモしておくだけで十分です。

機能	本書での初出章	何に使うか	あなたの環境での確認方法
AIとのチャット	序章	作る・直す・質問する。本書のすべての対話	日本語で指示を送れる入力欄があるか
ファイル	第6章	AIが書いたコードを開いて読む。プロジェクトの構成を見渡す	ファイルの一覧(ツリー)が見えるか。クリックで中身が開くか
ターミナル	第10章	コマンドの実行、パッケージの導入、Gitの操作	文字でコマンドを打ち込む画面を開けるか
プレビュー	序章	作ったアプリを実際に動かして確かめる	変更がすぐ画面に反映されるか。プレビュー用のURLが出るか
バージョン管理	第10章	動いたらコミット。変更内容を見比べて戻す	Gitのコミット履歴を確認できるか。共有先を選べるか

機能	本書での初出章	何に使うか	あなたの環境での確認方法
公開	第14章	自分以外の人がスマートフォンからURLで使える状態にする	公開(デプロイ)のボタンや手順があるか

「この機能はどこ?」は、AIに聞いてもよい質問です。ただし、古い画面の説明が返る場合もあります。実際のメニューや公式ヘルプと見比べてください。

あなた → AI

この環境で、ファイルの一覧を見る方法と、ターミナルを開く方法を教えてください。

メニューを探し回るより、この一言のほうが早いことがよくあります。6つの機能それぞれについて、同じ聞き方が使えます。

第5章・第11章で使う「開発者ツール」は、AI開発環境の画面ではなく、アプリを表示しているブラウザ側の道具です。プレビューを独立したタブで開き、ブラウザのメニューから「開発者ツール」を探します。見つからなければ、ブラウザ名と「開発者ツールの開き方」で公式ヘルプを確認してください。

通信を見るときは「Network(ネットワーク)」を選び、開いた状態でアプリを再読み込みしてから対象の操作をします。「Console(コンソール)」ではエラーを確認できます。学校・社用PCで利用が制限されている場合は、管理者に相談します。

操作例(2026年時点。Chromeの場合): [Chrome公式ガイド・Networkで通信を調べる](#)

A.3 選び方チェックリスト——8つの条件

これから環境を選ぶ場合、また授業や研修で環境を指定する場合は、次の8つを確認してください。

1. ☐ 使う端末で準備できる(ブラウザ中心で始めるか、必要なソフトをインストールできる)
2. ☐ ファイル一覧が見える
3. ☐ コードを表示できる
4. ☐ ターミナルが使える
5. ☐ プレビューできる
6. ☐ Gitで履歴を残し、プロジェクトを書き出せる
7. ☐ サーバーでの処理・消えない保存・ログイン・秘密の値の保管を用意できる(外部サービスとの連携でもよい。第7～9章)
8. ☐ 公開機能がある(または公開サービスと連携できる)

補足を3つ。

- 序章ではチャットとプレビューがあれば始められます。第6章ではコードを開き、第7～9章では条件7、第10章では条件6、第14章では条件8を確かめます。料金、保存期間、書き出し制限も確認してください

- 8つすべてを1つのツールで満たす必要はありません。足りない機能は、別のサービスで補えます(例: 公開だけ別のホスティングサービスを使う)
- APIキーの置き場所は、「環境変数」という名前だけで安全と判断しません。ブラウザへ配信されず、サーバー側でだけ利用できる設定かを公式ヘルプで確認します(第9章)

A.4 作業環境の3つの選び方

製品名から迷い始める前に、どこで作業するかを決めましょう。同じ製品が複数の形に対応することもあるので、ここでは学習時の使い方と分けます。

作業の形	準備するもの	向いている場面	確かめること
PC上で作業する	AI対応エディタやターミナルで使うAIツール、実行環境、Git	ファイルや実行の仕組みを手元で確かめたい	インストール権限、必要な設定、外部への接続
ブラウザを主に使う	アカウント、クラウド上の作業場	インストールを減らして始めたい	コード閲覧、サーバー、保存、料金、書き出し機能
チャットと実行環境を組み合わせる	相談用AI、別のエディタ・実行環境	すでに使い慣れた環境がある	依頼内容と実ファイルの同期、実行結果の確認

選び分けの目安です。

- 授業・研修では、担当者が学内・社内ネットワークから事前に同じ演習を試します。アカウント作成、利用可能な機能、料金制限を含めて確認してください
- 卒業制作(第15章)まで見据えるなら、A.3の8条件を満たす組み合わせを選んでください。途中で環境を乗り換える場合は、コードに加え、データ・設定・秘密情報の移行方法も確認します(Gitで持ち運べる範囲は第10章)
- 迷ったら、序章をお試しにしてください。気分記録アプリが動けば最初の準備は完了です。後半に必要な機能はA.3で確認し、足りなければ早めに補います

A.5 スマートフォンでの表示とQRコード

序章0.4と第14章では、アプリをスマートフォンで表示します。手順の骨格はどのツールでも同じです。

- プレビューまたは公開のURLを、スマートフォンのブラウザで開く
- URLの受け渡しにはQRコードが便利です。これもAIに頼めます

あなた → AI

このアプリのURLを表示して、QRコードも生成してください。教室で他の人に読み取ってもらいます。

- **localhost**や**127.0.0.1**は、開いている端末自身を指します。PCのURLをスマートフォンへそのまま渡しても、PCにはつながりません。環境の共有プレビュー機能を確認します

- ログインを求められるなら共有範囲を、接続できないなら同じネットワークやURLの有効期限を確認します。QRコードはURLを渡す道具なので、アクセス権限は変わりません
- 開けなければ、PCの狭い画面幅で仮確認し、実機での確認は第14章までに行います。実機未確認であることを記録に残してください

A.6 練習用教材を用意する

授業・研修で教材が配布されていれば、その環境と版を確認して使います。配布がない場合も、本文の作例から練習用プロジェクトを用意できます。各章で使うものと、その作り方は次のとおりです。

教材	使う章	配布がない場合の用意のしかた
モックAPI	第7章	7.3のJSONを返す練習用の窓口を作り、7.8の失敗条件を切り替える
バグ診療所	第11章	自分の会計アプリのコピーで、本文の症状を1つずつ再現する
章の開始状態	各章	前章の動いた状態を保存し、再開点として使う
記録用テンプレート	各章	本文の表の見出しを文書へ写し、自分の観察と結果を埋める

第7章の準備では、7.3のイベント1件ぶんのJSON例を、次の依頼と一緒にAIへ貼り付けます。依頼文の404や429などの言葉は第7章で学ぶので、いまは意味がわからなくても、そのまま貼り付けてかまいません。

あなた → AI

この練習用プロジェクトに、**GET /mock/events**の窓口を作ってください。一緒に貼るJSON例をもとに、IDが重ならない2件の架空イベントを用意し、正常時は配列 [...] で返します。1件でも配列、0件は空配列 [] にしてください。外部サービスやAPIキーは使いません。正常、404、429、不正なJSON、5秒遅延を手動で切り替え、各モードの呼び方と確認方法を示してください。

AIが用意したら、正常時の一覧と、各モードの応答を自分で確かめます。壊すスイッチは練習用の環境だけに置き、本番公開には含めません。自分で窓口を実行できない環境では、JSONの読解と画面の状態切り替えまで進め、HTTP応答の確認はサーバーを使える環境へ持ち越します。

A.7 学校PC・社用PCで制限がある場合

インストールや設定変更ができないPCでも、あきらめる必要はありません。

- ブラウザ上の開発環境が使えるかを確認します。インストールを減らせますが、サービスやプランによって使える機能は違います。A.3で不足分を確かめてください
- ブラウザ自体の利用や特定サイトへの接続が制限されている場合は、管理部門に相談してください。その際は「何のサイトを、何の目的で使うか」を伝えます(A.3の8条件が、そのまま説明資料になります)
- 社用PCで学ぶ人へ。業務のデータをAIに入力してよいかは、別の問題です。先に第13章(とくに13.6)を読み、所属組織のルールを確認してください

- 利用制限の解除が必要ななら、用途と必要な機能を管理者へ説明します。設定を無断で回避するのではなく、許可された練習環境で進めます

付録B つまづき別チートシート

この付録は、最初から読むものではありません。困ったとき、症状から引いてください。

使い方は3ステップです。

1. B.1の表で、いまの症状に近い行を探す
2. 「まず見る場所」を自分の目で確認する
3. 「AIへの頼み方」の例文を型にして、確認した内容を添えて頼む

例文の「(全文を貼る)」には、エラーの手がかりを省略せずに添えます。ただし、秘密情報・個人情報
は先に伏せてください。伏せた項目名は残すと、意味を伝えやすくなります(例: `apiKey=[伏せた]`)。

B.1 症状から引く表

「くわしくは」の数字は、本文の節の番号です。例文で解決しないときは、その節を読み直してから
頼み直してください。

画面の症状

症状	まず見る場所	AIへの頼み方(例文)	くわしくは
画面が真っ白	開発者ツールのコンソール(赤いエラーが出ていないか)	「画面が真っ白になります。コンソールのエラーを貼ります: (全文を貼る)。原因の候補を挙げてください」	7.7、11.4、11.5
ボタンを押しても何も起きない	コンソールと、ネットワークタブ(押した瞬間に何か送られたか)	「〇〇ボタンを押しても反応がありません。押したときのコンソールとネットワークの記録を貼ります: (貼る)」	6.5、11.4、11.5
「読み込んでいます…」のまま止まる	ネットワークタブ(返事がそもそも来ているか)	「読み込み表示のまま進みません。一定時間で『時間をおいてお試しください』と案内する処理(タイムアウト)を追加してください」	5.6、7.7
スマートフォンで表示が崩れる	実機、または開発者ツールのスマートフォン表示モード	「画面幅が狭いとき〇〇が崩れます(重なる/はみ出す)。スマートフォンでの表示を直してください」	0.4、14.5
文字化けする	どこで化けるか(画面か、保存したデータか、CSVを開いたときか)	「〇〇を開くと文字化けします。例: (化けた文字列を貼る)。文字コードの扱いを確認して直してください」	8.8、11.5

データと計算の症状

症状	まず見る場所	AIへの頼み方(例文)	くわしくは
保存したはずのデータが消える	保存処理の成功、保存先、読み出し時のエラー	「再読み込みすると一覧が空になります。保存先と保存結果、読み出し処理を順に確認してください」	8.1、8.2、11.5
合計金額がおかしい	再現する注文の組み合わせを1つに絞る(期待した額と実際の額)	「〇〇を注文すると合計が〇円ですが、正しくは〇円です。仕様: (割引ルールを貼る)。計算の途中経過を一段ずつ表示してください」	3.6、11.2、11.4、12.3
同じデータが2件登録される	二重押し、通信の再試行、保存された注文ID	「同じ会計が2件登録されます。画面の連打防止とサーバーの重複排除を確認し、同じ注文の再送では1件だけになるようにしてください」	12.3、12.6
データベースに変な値が入っている	どの画面からの入力か。空欄や極端な値を入れて再現するか	「〇〇に空欄や巨大な数を入れると、そのまま保存されます。入力値の検証を追加してください」	8.7、13.3
特定の入力(0個・空欄・巨大な数)のときだけ止まる	エラーの1行目と、スタックトレースの最初の住所(ファイル名と行番号)	「0個のまま会計を押すと画面が止まります。エラー全文を貼ります: (貼る)。受け付けない入力には案内を出すようにしてください」	8.7、11.3

APIと通信の症状

症状	まず見る場所	AIへの頼み方(例文)	くわしくは
一覧が表示されない(あるはずのデータが出ない)	ネットワークタブ(ステータスコードの番号と、返事の中身)	「一覧が出ません。ネットワークタブの記録: ステータス〇〇、返ってきた中身: (貼る)。どこで止まっていますか」	7.4、7.7、11.4
「Not Found」(404)と出る	呼んでいるURL(エンドポイント)の綴り。仕様書と1文字ずつ比べる	「404が返ります。コードが呼んでいるURLと、この仕様書のURLを比べてください: (仕様書の該当部分を貼る)」	7.2、7.4、7.5
429と出る	短時間に何回呼んでいるか(再読み込みの連打、繰り返しの中での呼び出し)	「429が返ります。このアプリはAPIをどのタイミングで何回呼んでいますか。回数を減らす案を出してください」	7.4、7.6

公開と環境の症状

症状	まず見る場所	AIへの頼み方(例文)	くわしくは
ローカルでは動くのに公開先で動かない	開発と本番で違うもの(環境変数、URL、公開用の設定)	「ローカルでは動きますが公開先で〇〇が動きません。開発と本番で設定が違う項目を一覧にしてください」	14.3、14.4
昨日は動いていたのに今日は動かない	昨日から今日までの変更履歴(diff)	「昨日から変わったファイルと差分を一覧にしてください。最後に動いていたのは〇〇をした時点です」	10.5、10.7
店長画面が誰でも見られる	ログインしていない状態・別の利用者で、URLを直接開いてみる(画面だけでなくAPIも)	「店長用の画面とAPIに、店長以外がアクセスできないか確認し、サーバー側の権限チェックを追加してください」	9.5、13.4
店長でログインしたのに、店長画面だけ表示されない	ログイン中のロールと、店長画面を開いたときのネットワークの番号(403か200か)	「店長のアカウントで入っても店長画面が表示されません。ログイン中のロールと、ネットワークの記録を貼ります: (貼る)」	9.2、9.5
APIキーをうっかりコードに書いてしまった	公開・コミットの前か後か(対処が違う。B.2へ)	(B.2を見てから頼む)	9.7、10.5、13.4

AIとの対話の症状

症状	まず見る場所	AIへの頼み方(例文)	くわしくは
AIの返事が本と違う	演習のクリア条件(本と同じ画面かどうかは問題ではない)	「いまの状態、〇〇(クリア条件)は満たせていますか。確認する方法を教えてください」	0.2、4.3
エラー文が英語で読めない	項目名・エラーの行・直前の操作。秘密情報は伏せる	「このエラー文を日本語で説明し、次に確かめる場所を示してください: (全文を貼る)」	11.3
AIが同じ修正を繰り返して直らない	自分の直近の指示(同じ言い方を2回送っていないか)	(B.4のフローへ。3回目を送る前に止まる)	4.8、11.8

B.2 APIキーを書いてしまったとき——どこへ渡ったかを確認する

コードから消す前に、キーがどこへ渡ったかを確認めます。公開サイトだけでなく、AIチャット、ログ、共有フォルダー、Gitの履歴も対象です。公開前でも、秘密が自分の管理する範囲を出ていることがあります。

未共有のローカルファイルにだけあり、外部へ送っていないことを確認できる場合

- サーバー用のSecrets(秘密管理機能)へ移し、コードには設定名だけを残します(第9章)

- ・ 移した先がブラウザに配信されないこと、履歴やログに実値がないことを確認します

共有・送信・公開した場合、または外部へ出たか確かめられない場合

- ・ まず発行元でキーを失効・交換します。古いキーを使う機能は止まるため、影響するアプリと担当者を確認して対応します
- ・ 新しいキーをサーバー側へ設定し、動作と利用記録を確認します。コードから削除しただけでは、既に渡ったコピーを回収できません
- ・ 履歴の除去が必要かを検討します。共有済みのGit履歴の書き換えは共同作業へ影響するので、担当者と手順を合わせます
- ・ AIには実際のキーを渡さず、「キーがどこに入ったか」「共有の範囲」「利用するサービス名」を説明して手順を相談します

参照: [GitHub Docs「リポジトリからの機密データの削除」](#)。秘密の失効・交換を先に行い、履歴の変更は影響を確認して進めます。

B.3 ステータスコード早見表(第7章の再掲)

番号	意味	日常語で言うと
200	成功	「はい、どうぞ」
400	頼み方が変	「ご注文の意味がわかりません」
401	有効な認証情報がない	「会員証を確かめられません」
403	依頼を許可できない	「その注文はお受けできません」
404	対象が見つからない	「その窓口や品物はありません」
429	頼みすぎ	「少し間をあけてください」
500	向こうの故障	「厨房でトラブルが起きています」

先頭の数字が4なら頼む側(頼み方・対象・会員証・回数)を、5ならサーバー側の処理を調べる手がかかります。番号は出発点です。番号、返事の本文、API仕様書を合わせて読みます。

※厳密に言うと: 番号だけで原因の担当者は断定できません。404は、記録が削除された場合や、権限のない相手に存在を隠す場合にも返ります。200でも、返ってきた内容が期待どおりとは限りません。

参照: [MDN「HTTP response status codes」](#)。

B.4 AIループ脱出フロー

AIが同じ修正を繰り返しはじめたら、このフローに乗ってください。合図は明確です。同じ趣旨の指示を2回送った——それがループの入口です。

フローは上から順に進みます。各段で直らなければ、次の段へ下りてください。

同じ指示を2回繰り返した
↓

【止まる】3回目を送らない。同じ入力と同じ堂々めぐりを生む

↓

① エラー文と再現手順を全部貼る

「症状: ○○。再現手順: 1…2…3…。エラー全文: (貼る)。

これまでの修正で直っていません。原因を最初から調べ直してください」

↓ 直らない

② 最小構成で再現する

新しい小さなプロジェクトに、問題の機能だけを作らせて、同じ症状が出るか試す。

出る条件と出ない条件を比べる。再現しなくても、原因がないとは断定しない

↓ 直らない

③ 言い方を変える

原因を決めつけた指示(「○○を直して」)をやめ、症状と期待だけを伝える。

「○○のとき、△△になってほしい。いまは××になる」

↓ 直らない

④ ロールバックして、小さく作り直す

最後に動いていたコミットへ戻り(第10章)、今回の変更をひとまわり

小さな単位に分けて、1つずつ頼み直す

④は敗北ではありません。壊れた10行を直し続けるより、動く状態から小さく積み直すほうが早い——これはプロの現場でも日常的な判断です(11.8)。

B.5 困ったときの3原則

最後に、この付録全体をたたんで3行にします。迷ったら、ここに戻ってください。

1. 観察した結果を伝える — 秘密情報を伏せたエラー文と再現手順をセットにする
2. 1回の依頼は1つの変化 — まとめて頼むと、どれが効いたか(壊したか)わからなくなります
3. 動いたらセーブ(コミット) — 練習用コピーに戻れる地点を残す。保存データのバックアップは別に用意する

付録C 用語ミニ辞典

本書に登場した専門用語を、5枚の地図別にまとめました。

この辞典の使い方をひとつだけ。定義を暗記するための辞典ではありません。用語に出会うたび、自分のアプリに引きつけた問いに答えられるかを確かめてください。「サーバーとは何か」ではなく、「私のアプリでは、どれがサーバーで、止まると何が起きるか」。それが本書でいう「語彙を手に入れた」状態です。

意味を忘れたら、ここへ戻る。必要なときに調べ直し、自分のアプリで確かめられれば大丈夫です。

①価値の地図(第Ⅰ部)

「誰の、何をよくするか」から、最初に作る範囲を決めるための言葉です。

用語	日常語で言うと	くわしくは
Vibe Coding	AIとの対話を中心に作るスタイル。元の用法ではコードの細部を追わない姿勢を含む	第1章
デモ	見せるための試作	第1章
プロトタイプ	試して学ぶための試作	第1章
PoC	実現できるかの確認	第1章
MVP	利用者に価値を届け、仮説を確かめる最小限の製品	第1章
本番アプリ	実際の利用者に提供し、運用しているアプリ	第1章
ユーザー	使う人	第2章
課題	困りごと	第2章
利用場面	いつ・どこで使うか	第2章
価値仮説	「これで助かるはず」の一文	第2章
ペルソナ	代表ユーザーの人物像(作り込みすぎない)	第2章
要求	ほしい結果(願いごと)	第3章
仕様	作るものの決めごと(注文書)	第3章
ユーザーストーリー	「誰として、何をしたい。なぜなら」の一文	第3章
ユースケース	操作の物語(うまくいかない分岐つき)	第3章
画面遷移	画面から画面への移動	第3章
受け入れ条件	「これが確認できたら完成」の一覧	第3章
スコープ	今回作る範囲	第3章
プロンプト	AIへの依頼文	第4章
コンテキスト	AIに渡す前提情報	第4章
制約	守ってほしい条件	第4章

用語	日常語で言うと	くわしくは
完成条件	どうなったら完了か	第4章
タスク分割	大きな仕事を小さく割る	第4章

②構造の地図(第II部)

画面の向こうで、何が、どこへ、誰の権限で動くのかをたどるための言葉です。

用語	日常語で言うと	くわしくは
クライアント	頼む側(あなたの手元)	第5章
サーバー	リクエストを受けて処理を返すプログラムや、その実行環境	第5章
フロントエンド	画面側	第5章
バックエンド	向こう側の処理	第5章
リクエスト	注文	第5章
レスポンス	返事	第5章
ローカル	自分の環境の中	第5章
クラウド	借り物のコンピューター群	第5章
開発者ツール	ブラウザに隠れている観察窓	第5章
ソースコード	アプリの楽譜	第6章
変数	名前を付けた箱	第6章
型	箱に入るものの種類	第6章
配列	並んだ箱	第6章
オブジェクト	名前付きの箱のセット	第6章
条件分岐	もし〜なら	第6章
関数	名前を付けた手順(レシピ)	第6章
イベント	「押された」などの出来事	第6章
ステート	いまの状態	第6章
コンポーネント	画面の部品	第6章
コメント	コードに貼られた付箋(信用しすぎない)	第6章
API	アプリ同士の注文窓口	第7章
エンドポイント	窓口の住所	第7章
HTTP	注文の作法(の名前)	第7章
JSON	文字列・数・配列・オブジェクトなどを表すデータ形式	第7章
ステータスコード	HTTPの返事に付く、処理結果の種類を示す番号	第7章
APIキー	窓口の会員証(他人に見せない)	第7章

用語	日常語で言うと	くわしくは
レートリミット	頼みすぎ防止の回数制限	第7章
モック	本物そっくりの練習相手	第7章
永続化	実行が終わっても読み戻せる場所へ保存すること	第8章
データベース	みんなで使える頑丈な専用ノート	第8章
テーブル	ノートの1ページ(1種類の記録を集めた表)	第8章
主キー	行の背番号	第8章
スキーマ	ノートの罫線(列の名前と型の決め)	第8章
CRUD	記録の基本4動作(作る・読む・変える・消す)	第8章
クエリ	データへの質問文	第8章
ローカルストレージ	同じブラウザ内でサイトごとに保存する領域。端末間共有はしない	第8章
CSV	表の共通語	第8章
論理削除	消さずに「取消」の印を付ける消し方	第8章
認証	「あなたは誰か」の確認(部員証)	第9章
認可	「何をしてよいか」の確認(スタッフパス)	第9章
ロール	権限を決めておく役割の札	第9章
セッション	「ログイン中」を覚えてもらう入場スタンプ	第9章
トークン	持ち歩ける通行証	第9章
最小権限	必要な人に、必要なだけ	第9章
Secrets(秘密管理機能)	秘密の値を保管する金庫。プログラムへの受け渡しは環境変数などで行う	第9章

③変更の地図(第III部)

プロジェクトの部品と変更を管理し、戻れる状態を残すための言葉です。コードを読む言葉は、前の「構造の地図」を参照してください。

用語	日常語で言うと	くわしくは
ライブラリ	できあいの部品集	第10章
パッケージ	部品が届く箱(取り寄せの単位)	第10章
フレームワーク	アプリの骨組み	第10章
ランタイム	コードを動かすエンジン	第10章
依存関係	借りている部品のリスト	第10章
ターミナル	コンピューターと文字で話す窓口	第10章

用語	日常語で言うと	くわしくは
環境変数	実行環境からプログラムへ渡す設定値。仕組み自体は金庫ではない	第9・10章
Git	変更履歴を記録する道具	第10章
GitHub	リポジトリの預け先(共有サービス)	第10章
リポジトリ	過去の姿ごと覚えていた保管庫	第10章
コミット	Gitで対象にしたファイルの状態と変更の説明を残す記録	第10章
diff	比較した2つの状態の差分	第10章
ブランチ	本体に触らずに試すための枝	第10章
マージ	枝を本体に合流させること	第10章
ロールバック	変更を取り消し、以前の状態へ戻すこと。コード・公開版・データで手順が違う	第10章

④品質の地図(第III部)

「動いた」という観察から一歩進み、正しさと安全性を確かめるための言葉です。

用語	日常語で言うと	くわしくは
エラー	処理を正常に終えられなかった状態や、その通知	第11章
バグ	仕様どおりに動かない原因となる不具合。エラーを起こすこともある	第11章
再現手順	同じ故障をもう一度起こすための手順書	第11章
スタックトレース	故障現場の住所と、そこへ至る経路	第11章
ログ	アプリが自分で書き残す航海日誌	第11章
切り分け	犯人のいる区間を絞り込む捜査	第11章
テストケース	確かめごと1件ぶんのカード	第12章
正常系	晴れの日の確認	第12章
異常系	雨の日の確認	第12章
境界値	切り替わりの崖っぷちの値	第12章
単体テスト	部品の検査	第12章
結合テスト	つなぎ目の検査	第12章
E2Eテスト	通し稽古	第12章
回帰テスト	ぶり返しの検査(直したら、動いていたものが壊れていないか)	第12章
入力検証	入りの口(入ってくる値を信用しない)	第13章
脆弱性	守りの穴(攻める側から見た破れ目)	第13章
個人情報	個人を識別できる情報。他の情報との組み合わせも考える	第13章

用語	日常語で言うと	くわしくは
ライセンス	「貸すけれど、この約束は守ってね」の紙	第13章
アクセシビリティ	入り口の段差をなくすこと(色だけに頼らない)	第13章

⑤運用の地図(第IV部)

利用者へ届けたあとも、状態を知り、問題から回復するための言葉です。

用語	日常語で言うと	くわしくは
ビルド	開店準備の梱包	第14章
デプロイ	店舗への引っ越し	第14章
ホスティング	場所を貸してくれる大家	第14章
ドメイン	看板に出す屋号	第14章
開発環境	練習場	第14章
検証環境	リハーサル室	第14章
本番環境	本番の舞台	第14章
監視	店の見回り当番	第14章
バックアップ	復元に必要なデータなどの控え。戻せる範囲と手順も確認する	第8・14章
クォータ	利用量などの割り当て・上限。無料枠や請求の上限とは限らない	第14章
障害	店を開けられないトラブル	第14章
保守	店の手入れ	第14章
引き継ぎ	店を次の代に渡すこと	第14章

巻末の問い

辞典を閉じる前に、5枚の地図から1語ずつ選んで、自分のアプリで答えてみてください。

1. あなたのアプリのMVPには、何が入っていて、何を外しましたか(価値)
2. あなたのアプリでは、どれがサーバーで、止まると何が起きますか(構造)
3. 最後のコミットはいつですか。いま壊れたら、どこまで戻れますか(変更)
4. いちばん意地悪な境界値は何ですか(品質)
5. あなたが卒業したあと、誰がこのアプリを保守しますか(運用)

答えが変わるのは、アプリが育ったときです。次の変更のあとにも、同じ5問で見直してみてください。

付録D 授業・研修シラバス

この付録は、本書を授業や研修で使う先生・研修担当者の方のためのものです。大学15回版、文系向け8回版、企業2日研修版の3つの進行表と、採点ルーブリック、口頭試問の運用方法をまとめました。そのままシラバスや研修設計書に転記して使えます。

D.1 評価の考え方——完成画面に、説明と証拠を添える

AIは、初級プログラミング課題の多くを解けます。完成した画面やコードだけを評価すると、「AIに頼んだだけの受講者」と「開発を理解した受講者」を区別できません。

そこで本書の評価は、次の3点セットを対象にします。

- **成果物:** 各章で蓄積する開発ポートフォリオ「冒険の書」(全15点)
- **説明:** 口頭試問5問に、自分の言葉と図で答えられるか
- **検証の証拠:** テスト実施記録、バグ報告書、AI利用記録

言いかえると、評価するのは「作れたか」ではありません。「説明できる・変更できる・戻せる・確かめられる・渡せる」の5つの卒業条件です。完成度が低くても、失敗の原因と対処を記録し説明できる受講者は高く評価してください。逆に、完璧に動くのに説明できない提出物こそ、注意が必要です。

D.2 大学15回版(90分授業)

本書の章と授業回の対応をもとに、各回の運営情報を一覧にしました。「宿題」は次回までに行うものの、「回収する成果物」は冒険の書に綴じる提出物です。

回	到達目標	扱う章	授業内演習	宿題(次回までに)	回収する成果物
1	AIでアプリが動く体験をし、デモとMVPの違いを言える	序章+第1章	気分記録アプリを20分で作る	自作アプリのデモ/MVP/本番判定メモ	最初のミニアプリ+疑問リスト
2	機能ではなく困りごとから出発できる	第2章	身の回りの不便10個出し→1つ選定	対象者への短い観察・インタビュー	課題定義書
3	願いを決めごと(仕様)に変換できる	第3章	会計アプリの仕様化(割引・端数・権限)	仕様書の清書	1ページ仕様書+MVP範囲表
4	仕様・制約・完成条件を含む依頼が書ける	第4章	曖昧な依頼と構造化した依頼の比較	自分の題材でAI開発ブリーフ作成	AI開発ブリーフ+タスクリスト
5	自分のアプリの構成図を描ける	第5章	ボタン→結果の流れを矢印で描く	構成図の清書	アプリ構成図+データフロー図
6	生成コードの計算・表示・データを指させる	第6章	コードの色分け+仕様変更の差分説明	対応表の完成	日本語・図・コード対応表

回	到達目標	扱う章	授業内演習	宿題(次回までに)	回収する成果物
7	APIの成功と失敗を画面で扱える	第7章	イベントAPI表示+失敗演習(404・429・壊れたJSON)	エラー処理表の完成	API接続仕様+エラー処理表
8	データを永続化し、集計できる	第8章	消えるアプリの保存化+売上集計	データベース設計図の清書	データ項目一覧+データベース設計図
9	認証と認可を区別し、権限漏れを直せる	第9章	権限漏れアプリの修正	権限マトリクスの完成	権限マトリクス+秘密情報一覧
10	変更前にコミットし、壊れたら戻せる	第10章	意図的に壊して以前のコミットへ戻す	卒業制作の課題選定(付録E)※併走開始	リポジトリ+コミット履歴+README
11	エラーの原因を自分で切り分けられる	第11章	バグ診療所(5症状の診断)	卒業制作の1ページ仕様書	バグ報告書
12	境界値を含むテストを設計できる	第12章	割引計算のテスト(13:59:59と14:00:00)	卒業制作のテストケース表	テストケース表+テスト実施記録
13	公開前の危険を列挙し、対処できる	第13章	セキュリティ間違い探し+悪用5通り	卒業制作のセキュリティ点検	セキュリティチェックリスト
14	アプリを公開し、運用計画を書ける	第14章	公開+スマートフォン確認。卒業制作の中間チェック	卒業制作の仕上げと発表準備	公開URL+運用手順書
15	口頭試問5問に自分の言葉で答えられる	第15章	発表会・口頭試問	—	アプリ本体+冒険の書(全15点)

制作時間を厚くしたい場合は、第12・13章の読解とテスト表の下書きを事前課題に回します。授業中は実行結果と公開前の点検に時間を取り、確認項目そのものは省かないようにします。

理解度チェックの使い方。各章末の理解度チェック(3問)は、次の回の冒頭5～10分の振り返り小テスト、グループでの答え合わせ、宿題に使えます(8回版・2日研修でも同じです)。答え合わせでは解答のめやすを読み上げて終わらせず、2問目・3問目を「自分のアプリではどうか」に置き換え、構成図や記録を指して説明させます。AIに聞けば模範解答はすぐに得られるので、見るのは解答の暗記ではなく、この説明の具体さです。成績に含めるかは任意で、含める場合もD.5の配点は変わらず、説明の根拠を確かめる材料の一つとして使ってください。

卒業制作の併走(第10回～)

卒業制作は第15回だけの活動ではありません。第10回到課題選定を課外で開始し、第11～13回の宿題を「卒業制作の該当成果物を作る」に切り替えます。授業で学んだ章の成果物を、その週のうちに自分の題材で作ります。第14回の中間チェックでは、次の3点だけを確認します。

- ・ 1ページ仕様書に受け入れ条件と「やらないことリスト」があるか
- ・ 動く最小構成(MVP)が公開できる状態か
- ・ 個人情報・秘密情報の扱いに危険がないか

第15回(発表会・口頭試問)の運営

1人あたりの時間配分例です。

- デモ3分+口頭試問2分+交代1分=1人6分
- 90分で12人程度。冒頭10分で評価方法を再説明し、残りを発表に充てます

受講者が13人を超える場合の選択肢です。

- 口頭試問は5問から2問を当日抽選にし、残り3問は冒険の書の記述で評価する
- 2グループ並行にする(TAまたは相互評価シートを併用)
- デモを3分の事前録画にして、当日は口頭試問に集中する

口頭試問はQ1～Q5(D.6参照)から出題します。相互評価を入れる場合も、成績への反映は小さくし、主評価はループリック(D.5)で行ってください。

D.3 文系向け8回版

課題発見から限定的な公開までを8回で通す構成です。第6章は、計算や保存に関わる短いコードを読む活動に絞ります。第9章の権限と第10章の戻し方は、独立した回がなくても必要な演習の前に扱います。

回	内容	扱う章	備考
1	ガイダンス+20分体験+課題発見	序章・第2章	序章の成功体験を必ず全員に
2	仕様化	第3章	会計アプリ演習をそのまま使用
3	AIへの依頼	第4章	自分の題材でブリーフ作成まで
4	API	第7章	冒頭15分で第5章の構成図のみ導入
5	データ保存と集計	第8章	消えるアプリ→保存化を体験
6	デバッグとテスト	第10～12章(抜粋)	保存済みの練習コピーで2症状を診断し、戻し方も確認
7	権限・セキュリティ・公開	第9・13・14章(抜粋)	閲覧と更新の権限を確認し、限定公開で点検
8	発表・口頭試問	第15章	1人5分。試問は2問抽選

- 参照章: 第1・5・6・9・10章。必要になった場面で該当節を参照させます
- 卒業制作: コースAまたはB(付録E)から選び、最小構成(ハーフサイズ)を標準とします
- 初めての卒業制作では模擬データの題材を選びます。共有データを更新する場合は個人情報の有無にかかわらず権限が必要です。権限を用意できなければ、共有書き込みを公開しない構成にします

D.4 企業2日研修版

- Day1: 序章+第1～5章(課題・MVP・仕様・依頼・構造)

- Day2: 第7～14章から必要箇所を選択(API・データ・権限・履歴・検証・限定公開)
- 修了条件: 「業務課題を検証できる、安全な試作品+仕様書」。本番利用可能なシステムではありません

タイムテーブル例(9:00～17:00)

時刻	Day1	Day2
9:00～9:30	オリエンテーション(評価方針・AI利用ルール・社内データの扱い)	Day1の振り返り+試作品の状況共有
9:30～10:30	序章: 気分記録アプリの20分体験	第7章: API接続と失敗演習
10:30～11:15	第1章: デモ/MVP/本番の判定	第8章: データ保存と集計
11:15～12:00	第2章: 業務の困りごと出し(各自3つ)	第11章: バグ診療所(2症状)
12:00～13:00	昼休憩	昼休憩
13:00～14:15	第3章: 仕様化演習(自分の業務課題で)	第12章: 境界値テスト(自分の計算ルールで)
14:15～15:15	第4章: AI開発ブリーフ作成と依頼実践	第13章: セキュリティチェックリスト実施
15:15～15:30	休憩	休憩
15:30～16:30	第5章: 構成図を描く+試作開始	第14章: 公開+試作品の仕上げ
16:30～17:00	振り返り(理解度チェックを自分の業務課題で答える)+宿題(仕様書の仕上げ)	成果発表(1人3分)+修了確認

- Day1の宿題は任意ですが、仕様書が仕上がっているとDay2の密度が大きくなります
- 社用PCの制限がある場合に備え、ブラウザだけで完結する演習ルートを事前に確認してください
- 模擬データだけで演習する取り決め(第13章)を、初日の冒頭で合意します。Day2の保存演習には権限の確認、診断演習には履歴の保存と戻し方を組み込みます

D.5 採点ルーブリック

合計100点の評価例です。Aを配点の100%、Bを60%、Cを20%、未提出・確認不能を0%として計算します。説明は最後の5点だけでなく、各項目の根拠を確認するためにも使います。

評価対象(配点)	A	B	C
課題とユーザー価値(15)	誰の・どんな場面の困りごとかが具体的で、観察やインタビューの根拠がある	課題は書かれているが、対象者や場面が一般的なまま	「あったら便利」の機能アイデアだけで、困る人が特定されていない
仕様とMVP(15)	受け入れ条件が観察・判定できる形で書かれ、やらないことが明確	条件はあるが曖昧(「使いやすい」「速い」など)	機能の羅列のみ

評価対象(配点)	A	B	C
構成図・データ設計 (15)	図が実際のアプリと一致し、データの通り道と保存場所を指で示せる	図はあるが実物とずれている、または登場人物が欠けている	図がない、または説明できない借り物の図
アプリの動作(15)	MVPの受け入れ条件を指定された提供環境で満たす	一部の条件が未達で、範囲を説明できる	主要機能が動かない
テストとデバッグ記録 (15)	境界値・異常系を含むテスト表と、原因まで書かれたバグ報告がある	正常系のテストのみ、または記録が結果だけ	「何回か触って大丈夫だった」の申告のみ
セキュリティ・プライバシー(10)	チェックリストを実施し、秘密情報の置き場所と権限の設計を説明できる	実施はしたが、なぜ安全かを説明できない	APIキーの露出や権限漏れが残っている
公開・運用・引き継ぎ (10)	公開URL・README・運用手順(バックアップと停止方法)が揃う	公開はしたが、手順書が不完全	公開されていない
自分の言葉による説明 (5)	出題された問いすべてに、自分の言葉と図で答える	一部を資料の読み上げに頼る	「AIがやってくれた」以上の説明が出ない

運用上の注意です。

- ・ 「動作15点」は全体の15%です。他の項目も、提出された記録が実物と一致するかを短い確認で判定します。記録の体裁だけで高得点にしません
- ・ 公開範囲は授業の条件で定めます。学内の限定公開でも、指定した利用者が使えて運用を説明できれば評価できます。外部公開の有無だけで能力を判定しません
- ・ 未完成でも、バグ報告書と切り分けの記録があれば「テストとデバッグ記録」で評価できます。この点を初回に伝え、失敗の隠蔽が減ります

D.6 口頭試問5問の運用

5問それぞれについて、良い答えの特徴と、注意すべき危険信号をまとめます。試問は「知識の暗唱」ではなく、自分のアプリに引きつけて答えられるかを見るものです。

問	良い答えの特徴	危険信号
Q1 このアプリは、誰の何を解決しますか	具体的な一人と利用場面を語り、「作らない」選択と比較している	「みんなに便利」「あると嬉しい」で止まる
Q2 ボタンを押すと、データはどこを通りますか	自分の構成図を指し、端末内か通信を伴うかを含めて実際の経路を説明する	実物の経路を示せず、図と挙動の違いを説明できない
Q3 データはどこに保存されていますか	保存場所を特定し、消えたら何が起きるか、バックアップの有無まで言える	「クラウドのどこか」で止まる。個人情報情報の保存に気づいていない

問	良い答えの特徴	危険信号
Q4 正しく動くことを、どのように確認しましたか	境界値の具体例(締切時刻の前後、0件、二重押しなど)を挙げ、記録を示せる	「何回か触って大丈夫でした」。テスト表と実施記録が一致しない
Q5 壊れた場合、どこを確認し、どう戻しますか	エラーの見る場所→切り分け→ロールバック手順の順で答える	「AIにエラーを貼って直してもらっただけ。戻し方を知らない」

- 危険信号が出たら、1段やさしい追い質問をしてください。例: Q2で詰まったら「保存ボタンを押した瞬間、あなたのスマートフォンの外で何かが起きていますか」。それでも答えられない場合のみC判定にします
- Q2とQ5は差がつきやすい問です。時間がないときはこの2問を優先してください

D.7 AI利用の申告ルール例

本書の立場は明快です。AIの使用は減点しません。評価するのは、AIに何を頼み、何を確認し、何を自分で決めたかの記録です。次のルール例をシラバスに転記して使えます。

AI利用に関するルール(例)

- AIの利用は全課題で認めます。使用を減点しません。
- 提出物には**AI利用・判断記録**(冒険の書の成果物15)を必ず添付します。主要な依頼文、AIの提案を採用・却下した判断、自分で決めた仕様を記録してください。
- 口頭試問で提出物を説明できない場合、その部分は評価しません。
- AI利用記録のない提出、および記録と実物が食い違う提出は、減点の対象とします。
- 他人のアプリ・記録の提出は不正行為として扱います。

このルールの狙いは監視ではありません。「AIに書かせたことを隠す」インセンティブを消し、判断の記録という開発者の本来の仕事に受講者の力を向けることです。

なお、この付録のルーブリックと時間配分は出発点の例です。クラスの数・受講者層に合わせて調整し、最終的な成績評価の設計は各機関の規程に従って先生ご自身の判断で確定してください。

付録E 卒業制作テーマカタログ

第15章の卒業制作で使える30のテーマを、3つのコース別にまとめました。どれもそのまま使えますが、いちばん良いテーマは、あなた自身の「困った」から生まれたものです。このカタログは、その呼び水として使ってください。

テーマの選び方 3か条

1. 自分が現場を知っているものを選ぶ。本編の第3章で「1円玉を扱いたくない」という現場知識が仕様の1行になったのを思い出してください。現場を知らないテーマでは、AIの想像に反論できません。
2. 個人情報を扱う題材は、難易度がひとつ上がる。名前・連絡先・相談内容などを預かるテーマは、第9章(権限)と第13章(プライバシー)の比重が一気に増えます。★の数にかかわらず、初めての卒業制作では「個人情報を持たない設計にできないか」をまず考えてください。
3. 迷ったら、狭いほうを選ぶ。「全学部で使える履修管理」より「うちのゼミの発表順決め」。ユーザーの顔が浮かぶ狭さが、良い仕様と良い発表を生みます。

難易度の目安: ★=本編の会計アプリより簡単/★★=同等/★★★=発展(本編+αの決めごとが必要)。

表の読み方です。まず、困りごとが自分に近いテーマを探します。最初の到達点は最小構成の3機能です。注意の欄には、個人情報や権限の扱いと、本編のどの章が活きるかを書いています。

コースA キャンパス・イベント

学園祭・サークル・授業まわりの題材。工学部1年生・文系学生の最初の卒業制作に向いています。

テーマと困りごと	最小構成と発展	難易度・設計のポイント
A1 学園祭イベント一覧+当日受付。来場者が「次にどこで何があるか」わからない	最小: イベント一覧/詳細表示/当日の受付チェック。発展: お気に入り登録、混雑表示	★★ 本編第7章のモックAPIの発展形
A2 サークル出欠とりまとめ。幹事がチャットの返事を数えるのに疲れた	最小: 予定登録/出欠回答/集計表示。発展: リマインド表示、会費連動	★ ニックネームでも個人と結び付く。閲覧・変更できる人を決める
A3 ゼミ発表タイムキーパー。発表が毎回時間超過する	最小: 持ち時間設定/残り時間表示/ベル(色変化)。発展: 発表順のくじ引き、記録の保存	★ 保存なしでも成立する入門向け
A4 落とし物掲示板。教室の落とし物が持ち主に戻らない	最小: 落とし物の登録(写真・場所)/一覧・検索/受け渡し済み処理。発展: 学内地図連携	★★ 写真の氏名・顔・番号を確認。受け渡し相手の確認方法も決める
A5 教室・部室の空き状況ボード。空き教室を探して歩き回っている	最小: 利用中・空きの切り替え/一覧表示/時間帯メモ。発展: 予約機能	★★ 更新者の権限は最初から必要。予約では同時操作の重複も扱う
A6 部室の備品貸出ノート。誰が何を借りているかわからなくなる	最小: 備品一覧/貸出・返却の記録/借りっぱなし表示。発展: 貸出履歴の集計、点検リスト	★★ 本編のCRUDがそのまま使える

テーマと困りごと	最小構成と発展	難易度・設計のポイント
A7 新歓スタンプラリー。新入生がブースを回ってくれない	最小: ブース一覧/スタンプ付与/達成表示。発展: 景品抽選、達成率集計	★★ スタンプ付与の権限設計が学びどころ
A8 履修相談の質問箱。新入生が先輩に聞く場がない	最小: 質問投稿/回答表示/よくある質問一覧。発展: タグ検索、匿名運用	★★★ 相談内容=個人情報になりうる。匿名設計を第9・13章で検討
A9 学食混雑投票。昼の学食がどこも行列	最小: 混雑度の投票/現在の混雑表示/時間帯グラフ。発展: 曜日別の傾向表示	★ 集計(第8章)の練習に最適
A10 文化祭シフト表。誰がいつ店番か、直前にわからなくなる	最小: シフト登録/自分のシフト表示/交代依頼メモ。発展: 会計アプリ(本編)との連携	★★ 本編の会計アプリと世界観がつながる

コースB フィールドワーク・研究記録

文系学生・研究活動向け。「集めて、整理して、探せるようにする」型の題材です。

テーマと困りごと	最小構成と発展	難易度・設計のポイント
B1 インタビュー記録+タグ検索。聞き取りメモがノートに埋もれて探せない	最小: 記録の登録/タグ付け/タグ・キーワード検索。発展: AI要約(発展課題)	★★ 語り手の個人情報・公開範囲に注意(第13章)
B2 フィールドノート(写真+位置メモ)。現地写真がどこで撮ったかわからなくなる	最小: 写真+メモ登録/場所タグ/時系列一覧。発展: 地図表示	★★ 人物が写る写真の扱いに注意
B3 文献メモカード。読んだ論文の内容を忘れる	最小: 文献登録/引用メモ/著者・年・タグ検索。発展: 引用形式の書き出し	★★ CSV出力(第8章)が活きる
B4 アンケート集計ビューア。紙アンケートの集計が手作業	最小: 回答入力/単純集計/グラフ表示。発展: クロス集計	★★ 練習は模擬回答で。実データでは利用目的と個人を識別できる組み合わせを確認
B5 観察記録の時系列ビューア。実習・観察の記録が時系列で見返せない	最小: 記録登録(時刻自動)/日別表示/キーワード検索。発展: 写真添付、CSV出力	★ 本編の履歴画面の応用
B6 方言・語彙採集帳。採集した言葉が整理できない	最小: 語の登録(地域・話者世代)/一覧・検索/地域別集計。発展: 音声メモ添付	★★ 話者の特定につながる情報は持たない設計に
B7 練習記録(部活・楽器)。練習の積み上げが見えず続かない	最小: 練習登録/累計表示/週間グラフ。発展: 目標設定、仲間と共有(→権限)	★ 共有まで作ると★★

テーマと困りごと	最小構成と発展	難易度・設計のポイント
B8 読書会の論点ボード。議論した論点が次回に引き継がれない	最小: 論点登録/賛否・コメント/回ごとのアーカイブ。発展: 論点の関連づけ	★★ 発言者名の扱いを決めてから
B9 実験ノート。条件と結果の対応があとでわからない	最小: 実験条件の登録/結果記録/条件検索。発展: 失敗記録のタグ化	★★ スキーマ設計(第8章)の良い練習
B10 地域史聞き書きアーカイブ。聞き書きが個人のノートで眠っている	最小: 記録登録/年代・地区タグ/検索。発展: 公開版と研究版の権限分け	★★★ 公開の同意管理が本体。第9・13章を総動員

コースC 小さな店舗・業務改善

企業の非エンジニア・社会人向け。本編の会計アプリの発展形を含む、実務直結の題材です。

テーマと困りごと	最小構成と発展	難易度・設計のポイント
C1 会計アプリ本格版(チャージ・サービス料・税)。席料や税・サ料のある店の伝票計算が煩雑	最小: メニュー・チャージ設定/税・サ料込みの計算/会計履歴。発展: 特別会員の料金ルール、月次集計	★★★ 練習は仮の計算規則で。実務の税・表示・端数処理は適用される規則を確認して仕様化
C2 シフト表。紙のシフト表の差し替えが追いつかない	最小: シフト登録/日別・人別表示/交代記録。発展: 希望収集、労働時間集計	★★ 氏名を扱う。社内ルール確認(第13章)
C3 在庫メモ。発注のたびに倉庫を数えに行く	最小: 品目登録/入出庫記録/残数表示。発展: 発注点アラート表示	★★ 論理削除(第8章)が活きる
C4 予約受付。電話予約の聞き間違い・二重予約	最小: 予約登録/日別一覧/重複チェック。発展: 顧客メモ連携(→★★★★)	★★ 顧客名を持つ場合は最小限に
C5 日報。日報がメールで流れて蓄積されない	最小: 日報登録/日別・人別表示/キーワード検索。発展: 週次サマリー	★ 本編の履歴+検索の応用
C6 顧客メモ。常連さんの好みを担当者しか知らない	最小: 顧客登録/対応メモ/検索。発展: 来店履歴の集計	★★★★ 個人情報のかたまり。アクセス権限と「集めすぎない」(第9・13章)が本体
C7 備品点検チェックリスト。点検の実施漏れ・記録漏れ	最小: 点検項目一覧/実施チェック/未実施の表示。発展: 月次の実施率集計	★ 初めての業務アプリに最適
C8 会議室予約。ホワイトボードの予約表が取り合いに	最小: 部屋一覧/予約登録/重複チェック。発展: 定例予約、利用集計	★★ A5の職場版
C9 経費の事前チェック。経費の申請ミスで差し戻しが多い	最小: 費目と上限の一覧/事前チェック入力/判定表示。発展: 申請メモの出力	★★ 社内規定を仕様に翻訳する練習。金額計算のテスト(第12章)が活きる
C10 朝礼連絡板。朝礼の連絡が非番の人に届かない	最小: 連絡登録/日別表示/既読チェック。発展: 重要フラグ、検索	★ 既読の権限設計が小さな学びどころ

発展課題: AI連携アプリについて

どのコースでも、発展構成として「AIを内蔵したアプリ」(要約、分類、相談への応答など)に挑戦できます。第7章で学んだ通り、AI自体もAPIで呼べるからです。ただし、次の3点が本編よりも重くなります。

1. 入力への扱い: 利用者の入力を外部AIへ送る範囲、保存や再利用の条件、組織のルールを確認する(第13章)
2. AI出力の評価: 第12章のテストを土台に、正解が一意でない出力の判定基準と失敗例を用意する。結果が変わることも含めて検証する
3. 利用量と料金: APIごとの課金単位、使用量の通知、停止条件を決める。クォータがそのまま請求上限になるとは限らない(第14章)

この3点に答えを用意できるなら、挑戦する価値があります。答えられないうちは、AIなし版で完成させてから足しましょう。